

Programmazione Orientata agli Oggetti

Introduzione al Linguaggio di
Programmazione Java

Sommario

- Breve riassunto della storia del linguaggio Java
 - Motivazioni del linguaggio e scelte progettuali
 - Versione di Riferimento
- Macchina virtuale JVM
- La portabilità e la piattaforma Java
- Librerie Standard Java
- Java vs C
- Java vs C++
- Controllo del Flusso
- Tipi di dato primitivi
 - Un tipo molto particolare: **String**
- Errori a tempo di compilazione e di esecuzione
- Diagnostica in Java vs in C
- Limiti del Paradigma Procedurale

Breve Storia (1)

- Java è un linguaggio di programmazione sviluppato dalla Sun Microsystems e pubblicato nella sua versione 1.0 nel 1995
- Lo sviluppo di Java iniziò nei primi anni '90 sotto la guida di James Gosling
 - Inizialmente il nome del linguaggio era Oak (quercia)
 - ed era pensato per sistemi embedded (televisori, sportelli bancomat, palmari, ecc...)
- Verso il 1993 l'interesse verso i dispositivi embedded, per cui Java era stato pensato, crollò
 - Java aveva (ben presto!) perso il suo originale e principale dominio di applicazione!

Breve Storia (2)

- Tuttavia, sempre nel 1993, la NCSA rilasciò *Mosaic*
 - Uno dei primi browser con interfaccia grafica
 - Rese più accessibile Internet
- Il team di Java decise allora di dedicarsi allo sviluppo di un sistema embedded per i browser: le **applet**
 - Le applet sono/erano piccole applicazioni che potevano essere eseguite all'interno del browser
 - Lo scopo era quello di arricchire le pagine web tramite l'uso di contenuto interattivo
- Perché il concetto era rivoluzionario? il codice veniva scaricato da remoto ed eseguito (in maniera sicura) localmente. *Javascript* non c'era ancora...
- Ci fu un significativo effetto traino. Ma oggi, quanti sanno cosa sia un'applet?

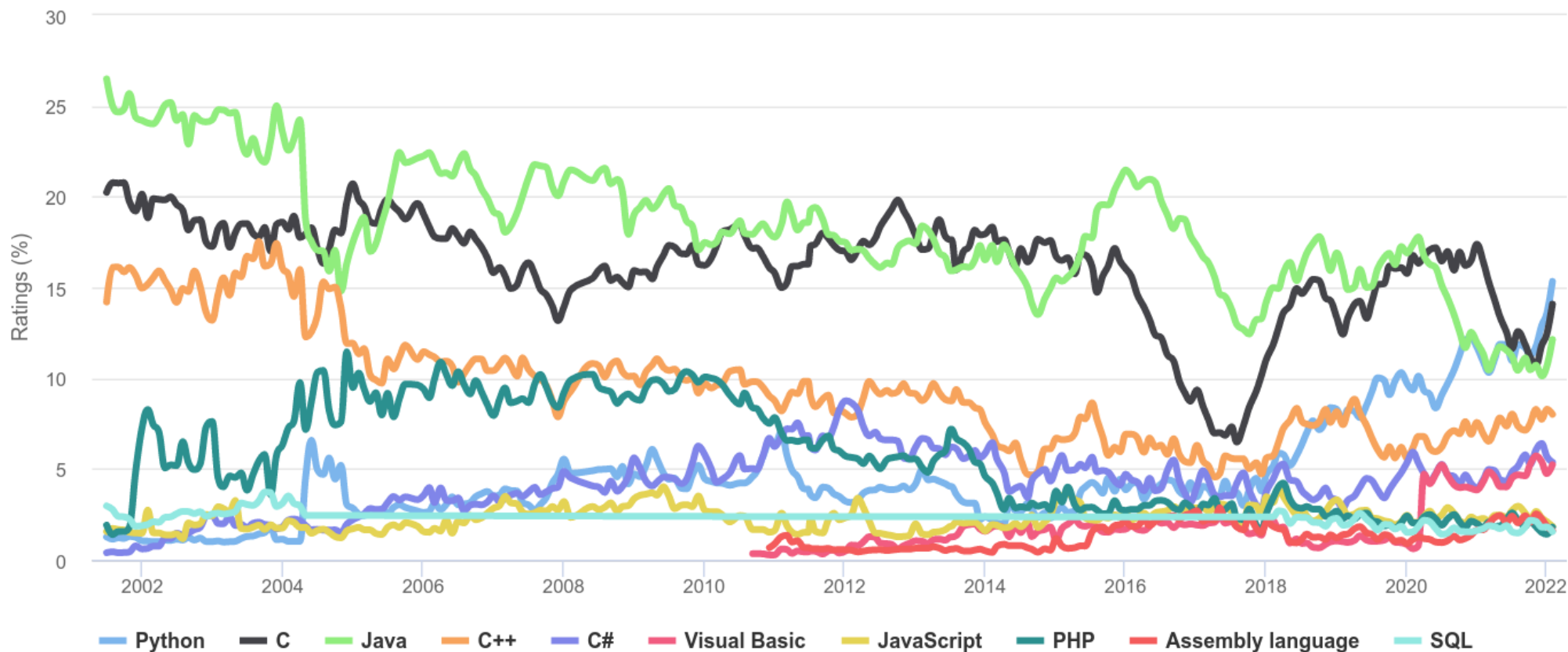
Breve Storia (3)

- Le applet sono ormai obsolete
 - Ma cavalcando gli anni della diffusione di massa di internet, hanno sicuramente contribuito a rendere Java uno dei linguaggi più popolari tra gli sviluppatori dell'epoca
- La popolarità e diffusione di Java sono dovuti a molti fattori
- Tra i principali sicuramente l'incremento di produttività dei programmatori che lo adottarono
- Java semplifica alcuni aspetti rispetto al C/C++
 - possiede un gran numero di strumenti facenti parte della *piattaforma* che semplificano grandemente lo sviluppo
 - riduce significativamente il livello di *expertise* necessario ad un programmatore per essere produttivo
 - In sintesi, Java ampliò la disponibilità di manodopera (>>)

Breve Storia (4)

- Oltre 25 anni dopo la nascita e la sua diffusione di massa, rimane molto adottato sia nell'industria che nell'accademia
- Talvolta forse addirittura con eccessivo entusiasmo (RM3!)
- Ha conosciuto nuovi ed importanti ambiti di utilizzo
 - Applicazioni su Web
 - Sistemi embedded
 - App Android
 - E molti altri ancora (*"3 Billion Devices Run Java"*)
- Vari motivi dietro la *durata* di questo successo (>>)
 - pur avendo accumulato molti difetti, ha saputo innovarsi senza rompere la retro-compatibilità, che rimane un obiettivo dogmatico nonostante il succedersi di numerose nuove versioni, talune anche molto innovative

Quanto è Popolare Java?



<https://www.tiobe.com/tiobe-index/>

Da Java 1 a Java 17 in ~25 Anni

- Java 1.0: 1995, la prima versione di Java
 - Java 1.1: 1997, Inner Classes e Reflection
 - Java 1.2: 1998, Just In Time Compiler introdotto nella JVM di Sun,
Java Collections Framework
 - Java 1.3: 2000
 - Java 1.4: 2002
 - Java 1.5 / 5: 2004, Generics, Auto boxing/unboxing, Enum, Varargs, for each loop
 - Java 1.6 / 6: 2006
 - Java 1.7 / 7: 2011
-
- Java 1.8 / 8: 2014, “Deriva funzionale”, lambda expressions
 - Java 9: 2017, “Moduli”
 - Java 10: 2018
 - Java 11: 2019
 - ... (Cambio politica dei rilasci, ora accelerati)
 - Java 17!: 2021
 - ... e presto seguiranno altre: rilasci semestrali

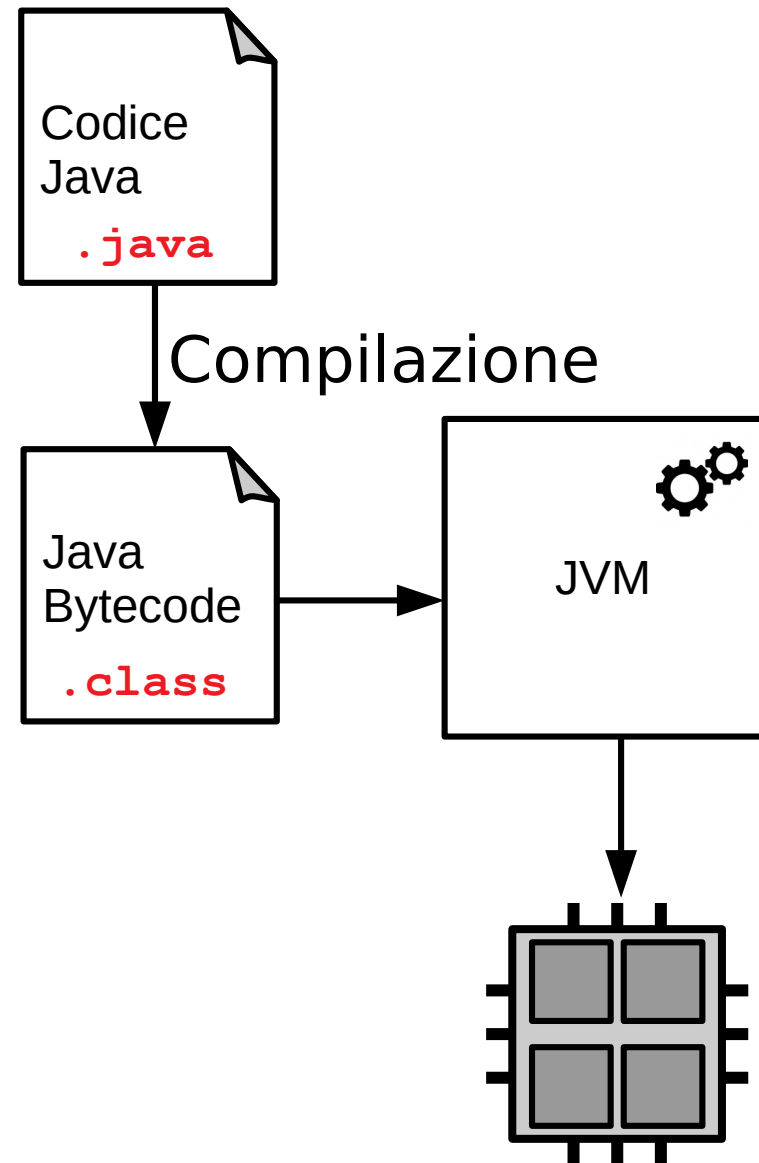
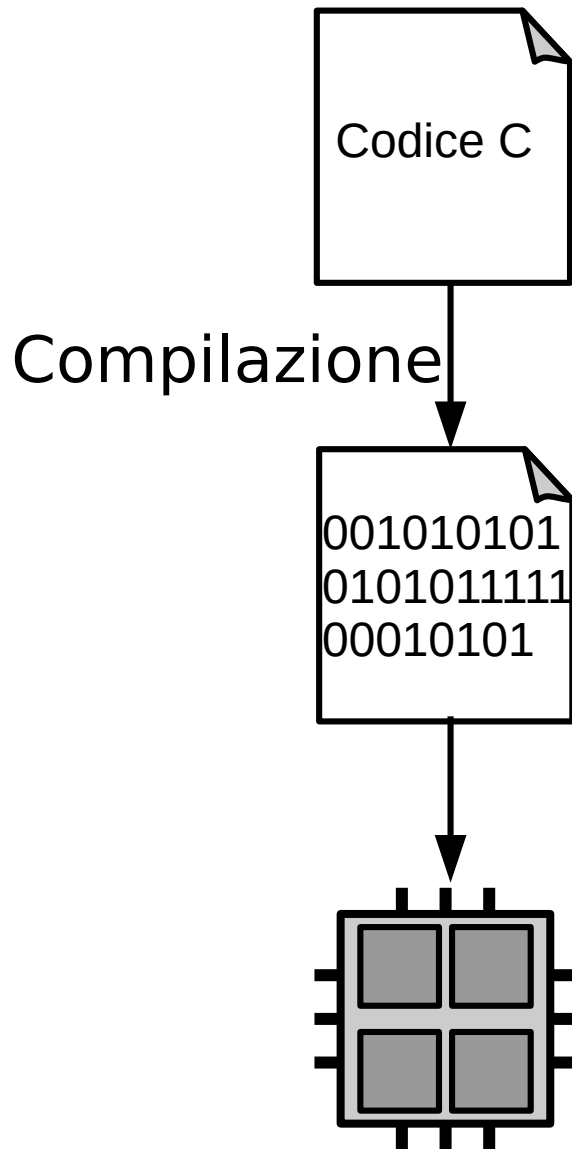
La Portabilità di Java

- Java ha rivoluzionato il concetto di portabilità
- Uno dei principi fondamentali di Java
 - *Write Once Run Anywhere*
 - Un programma Java può essere eseguito su tutte le piattaforme che supportano Java senza dover ricompilare il codice per quella specifica piattaforma
- Java è infatti nato come un linguaggio interpretato
 - Il risultato della compilazione è il cosiddetto *Bytecode*: codice oggetto che può essere eseguito da un processore virtuale noto come *Java Virtual Machine (JVM)*

JVM vs Codice Nativo

- Una JVM è, banalizzando, un processore “software”
- Programma in grado di interpretare ed eseguire codice oggetto Java (bytecode) letto da file di estensione **.class**
 - I dispositivi che supportano Java forniscono una implementazione della JVM per il sottostante hardware
- Il Bytecode può essere eseguito su qualunque computer che disponga di una implementazione della JVM
 - Al contrario, i compilatori C generano codice *nativo* eseguibile *solo* su un particolare hardware
 - Un programma C deve essere *ricompilato* per ogni piattaforma su cui si desidera eseguirlo

Compilazione Nativa vs JVM



Java e Portabilità

- Java è supportato praticamente da tutti i sistemi operativi più diffusi (Mac OS, Windows, Linux)
- E da altrettante piattaforme hardware
 - Lo stesso programma Java può essere eseguito su tutte le piattaforme che dispongono di una JVM
 - Senza bisogno di ricompilarlo!
 - Basta spostare i file .class
- Java permette di scrivere programmi altamente portabili
 - Senza preoccuparsi di quali operazioni siano supportate da quali piattaforme
 - N.B. Tuttavia le piattaforme sottostanti sono diverse, e talvolta rimane possibile osservarne le differenze anche programmando in Java
 - Ad es. nomi dei file (>>)

Java e Portabilità: Dov'Eravamo Rimasti? Il Linguaggio C...

- Ad esempio, il seguente codice in C ha un comportamento indefinito

```
int i = 127;
```

```
i = i++;
```

- In C la rappresentazione dei tipi predefiniti non possiede una dimensione fissata da uno standard
 - ✓ Un `int` è grande quanto la parola del microprocessore usato. Rappresentazione degli interi a 8, 16 o 32 bit? Dipende dalla piattaforma: possibile overflow!
 - Se `++` viene eseguito prima dell'assegnazione allora il valore in `i` sarà 1 (incrementa `i` a 2 ma il risultato, assegnato a `i`, è 1)
 - Se `++` è eseguito dopo l'assegnazione allora il valore in `i` sarà 2.
- Il comportamento dipende *anche* dalla precedenza data dal compilatore alle diverse operazioni

Portabilità del Codice Java

- In Java la sintassi e la semantica del linguaggio è ben definita e coerente su tutte le implementazioni della JVM
- Sono descritte in un documento noto come *Java Language Specification*
 - Eventuali ambiguità residuali sono progressivamente identificate, quindi rimosse e/o accettate
- Le istruzioni non dipendono da una particolare piattaforma
 - rimangono le stesse per ogni piattaforma hardware

Librerie di Java

- Altro principale motivo del successo di Java: distribuzione congiunta linguaggio e librerie
Piattaforma Java = Linguaggio + Compilatore + Librerie +...
- In C la libreria standard permette di gestire
 - la memoria (`malloc`, `free`, ...)
 - l'I/O (`FILE`, `printf`, `scanf`, ...)
 - le stringhe (`strcpy`, `strcat`, ...)
 - ecc. ecc....
- Ma non esiste il supporto per liste, insiemi, array associativi ecc. ecc.
- Tutto si può aggiungere ma...

Librerie Standard di Java

- In Java quasi tutto quello di cui si ha bisogno è già presente nella libreria del linguaggio distribuita assieme al compilatore ed all'ambiente di esecuzione
 - Le collezioni sono presenti in Java da anni
 - Librerie spesso sviluppate da esperti del settore
 - Usate e testate da migliaia di utenti
- Rarissimo che si trovino ancora bug di significativo impatto pratico
- Queste librerie sono *>sempre<*, *invariabilmente*, meglio delle nostre implementazioni "artigianali"

Filosofia Java vs C (1)

- Al giorno d'oggi alcune differenze tra il linguaggio C ed il linguaggio Java (che pure deriva dal primo), si possono interpretare correttamente solo considerando le differenze nella filosofia di fondo da cui scaturirono i due linguaggi
- A sua volta queste derivano dalle differenze negli obiettivi
- A sua volta queste differenze negli obiettivi nascono dalle differenze nel contesto in cui i due linguaggi sono nati
 - C: nato in cui contesto in cui le risorse di calcolo erano scarse
 - Java: era già chiaro che le risorse di calcolo sarebbero state sempre meno un problema per gli applicativi ordinari
 - Il vero problema (per l'industria) erano i programmatori!
- C: linguaggio di *medio* livello di astrazione
 - Ideato per scrivere parti dei S.O.
- Java: linguaggio di *alto* livello di astrazione

Filosofia Java vs C (2)

- Java nasce cercando di correggere in tal senso i più evidenti difetti dei linguaggi C/C++ rispetto agli obiettivi all'epoca sentiti come più nuovi ed urgenti
- C: prestazioni e controllo anzitutto
 - meglio veloce/compatto che semplice e portabile
 - non fare nulla se non è richiesto esplicitamente dal programmatore a cui si lascia il controllo diretto e quasi totale dell'esecuzione
- Java: semplice e portabile anzitutto
 - evitiamo gli errori più comuni anche se farlo costa, in generale, e non serve sempre: semplicità piuttosto che prestazioni e portabilità piuttosto che controllo totale
 - togliamo al programmatore alcune responsabilità, soprattutto quelle più faticose da gestire correttamente (es. gestione memoria)

Java vs C++

- Alcune volte Java sembra veramente limitarsi a *rimuovere* alcune possibilità aggiuntive offerte dal linguaggio di programmazione C++
 - Il vero competitor iniziale di Java, più del linguaggio C...
 - Perché nasce per supportare lo stesso paradigma (>>)
 - Java rimuove (e non rimpiazza) i costrutti del linguaggio C++ più discutibili rispetto agli obiettivi della piattaforma Java stessa
 - I costrutti rimossi non sono strettamente indispensabili e rendono il linguaggio più utile solo per pochissimi esperti
 - Contemporaneamente, però, finiscono per renderlo più difficile da utilizzare correttamente da parte di tutti gli altri
- Ad es. in Java non esiste
 - Ereditarietà multipla delle implementazioni (sino Java 7)
 - Operator overloading
 - Costruttore di copia
 - Passaggio di parametri per variabile/riferimento
 - Puntatori/Aritmetica dei puntatori...

Un Programma Java

- La sintassi è simile al C perché ideata per esserlo
 - con lo scopo di attrarre il parco programmatori del linguaggio Java, che non furono “sorpresi” dalla nuova sintassi
 - Java è il più famoso linguaggio *C-like* ma ne esistono altri (C++)
- Ogni programma inizia con l'esecuzione del cosiddetto *metodo* statico `main()` esattamente come in C l'esecuzione comincia dalla *funzione* statica `main()`
- Curiosamente i nuovi linguaggi (tra tutti Scala) che cercano di “scalzare” Java, usano tattiche simili, se non addirittura più efficaci
 - Compatibilità con la JVM

Programma in C vs Java (1)

- Un programma che stampa tutti gli argomenti passati da linea di comando
- In C:

```
#include<stdio.h>

int main(int argc, char *argv[]) {
    int i = 0;
    for (i=0; i<argc; i++)
        printf("%s\n", argv[i]);

    return 0;
}
```

Programma in C vs Java (2)

- In Java (scriviamo un file `JavaApp.java`):

```
public class JavaApp {  
    public static void main(String[] args) {  
        for (int i=0; i<args.length; i++)  
            System.out.println(args[i]);  
    }  
}
```

Per eseguirlo (da linea di comando):

```
$ javac JavaApp.java
```

```
$ java JavaApp ciao mondo
```

```
$ ciao mondo
```

Compilazione
in ByteCode

Esecuzione
tramite la JVM

Metodo `main()`

- Ogni programma Java inizia con l'esecuzione del `main()`
 - La dichiarazione del metodo *deve* essere preceduta dalla parola chiave `static`
 - N.B. usiamola per questo scopo ma dimentichiamoci della sua esistenza per le lezioni a venire (>>)
- Il `main()`
 - Non restituisce alcun valore
 - Il suo tipo di ritorno è `void`
 - Prende in ingresso un array di stringhe
 - `String[] args`
 - argomento passato da linea di comando a sostituire la coppia di parametri in C
 - `char *argv[]`
 - `int argc`

Java: Produrre Stampe

- Per stampare sullo schermo è possibile usare `System.out.println( <valore> )`
- `println` stampa `<valore>` e si posiziona su una nuova riga
- Il tipo del parametro attuale di `println()` può essere qualsiasi:
 - `int, boolean, float, double, String, ...`

Tipi di Dato Primitivi Java

- N.B. a differenza che nel linguaggio C, la dimensione della rappresentazione di questi tipi di dato primitivi non dipendono dalla piattaforma, ma sono fissati (per sempre) nella JLS
- Java supporta i seguenti tipi primitivi
 - **int**
interi a 32 bit (da -2^{31} a $2^{31} - 1$)
 - **long**
interi a 64 bit (da -2^{63} a $2^{63} - 1$)
 - **char**
caratteri in codifica UTF-16 (alfabeto latino, arabo, greco, cinese, giapponese, ecc....)
 - **float**
numeri in virgola mobile a 32 bit
 - **double**
numeri in virgola mobile a 64 bit
 - Altri tipi di dato primitivi meno usati ...

Dichiarazione di Variabili

- Per dichiarare una variabile si usa la seguente sintassi: `<tipo> <nome>;`

```
int eta = 18;
```

- Java è case sensitive quindi `eta != Eta != eTa != ETA != ...`
- Per evitare ambiguità (differentemente dal linguaggio C), ad ogni variabile deve essere assegnato un valore prima del suo utilizzo
- ✓ In caso contrario la compilazione non andrà a buon fine

Tipo Booleano in Java

- A differenza di quanto accade nel linguaggio C, Java ha un tipo dedicato per le variabili booleane:
 - **boolean**
variabili di tale tipo possono avere solo due valori: **true** e **false**
- Java offre i gli operatori logici *and* e *or* con la seguente sintassi:
 - **And: &&**
 - **Or: ||**

Sistema dei Tipi Java

- Ogni variabile ha un certo tipo e può assumere solo valori appartenenti al *dominio* di quel tipo
 - Il tipo di una variabile deve essere specificato al momento della sua dichiarazione

```
int eta = 18;
```
 - Se ad una variabile si assegna un valore non appartenente al dominio del suo tipo si genera un errore durante la compilazione

```
eta = false;
```
- Il comportamento di Java e C sono simili
- Sono entrambi linguaggi *staticamente* tipati
 - ✓ già durante la compilazione deve essere noto il tipo di ogni dato utilizzato

Il Tipo di Dato String

- Java introduce il tipo di dato **String** per rappresentare stringhe
 - **String** non è un tipo di dato primitivo (come ad es. **int**, **double**, ...)
 - Le stringhe, per diffusione ed importanza, hanno da subito meritato un supporto molto particolare dal linguaggio (e dal compilatore)
 - si volevano attirare programmatori non esperti facilitando l'utilizzo delle stringhe
 - È possibile dichiarare una variabile di tipo **String** ed assegnargli un valore come segue:

```
String nome = "Bob";
```

```
System.out.println(nome); // stampa Bob
```

String

- I letterali di tipo stringa sono compresi tra doppi apici
 - `"una stringa"`
- A differenza di C, è possibile assegnare più volte ad una variabile di tipo `String` un letterale di stringa

```
String nome = "Bob";
```

```
nome = "Alice"; // ok
```

```
System.out.println(nome); // Stampa Alice
```

- Molte operazioni sulle stringhe offerte direttamente dalle librerie Java distribuite con la piattaforma (>>)

Controllo del Flusso

- Le istruzioni per il controllo del flusso sono le stesse del C
 - La sintassi è esattamente la stessa
- Sono quindi presenti
 - `if e else`
 - `switch, break`
 - `for, while, do while`
 - `break e continue`

Controllo del Flusso

- Essendo presente il tipo `boolean`, le condizioni sono sempre espresse su espressioni di tipo `boolean`

```
int i = 0;  
if (i) { ... }
```

- In Java NON compila
- In C è prassi comune eseguire controlli direttamente su espressioni di tipo `int`

```
char stringa[] = "null terminated";  
int c = 0;  
while (stringa[c]) {  
    printf("%c\n", stringa[c]);  
    c++;  
}
```

Errori: Tempo di Compilazione vs Tempo di Esecuzione

- Due momenti ben distinti in cui si può riscontrare un errore in un programma
 - A tempo di compilazione
 - A tempo di esecuzione
- La rapida risoluzione degli errori in tempi brevi, è importante per la produttività
- La diagnostica dovrebbe sempre riportare il motivo dell'errore e la posizione esatta in cui si è verificato
- La diagnostica del C talvolta risulta “vaga”
 - Soprattutto per gli errori a tempo di esecuzione

Segmentation fault

Diagnostica

- La diagnostica è decisamente uno dei principali punti forti della piattaforma Java
- Permette di trovare gli errori di entrambi i tipi per risolverli tempestivamente, così riducendo:
 - i tempi di ricerca degli errori (debugging)
 - i costi
- Buona parte di questi vantaggi (soprattutto a tempo dinamico) derivano dall'adozione di un processore virtuale (JVM)
- ✓ Ottenere le stesse funzionalità con un processore fisico è decisamente più ostico
- Consiglio:

Abituarsi fin da subito ad un uso ossessivo della diagnostica

Errori a Tempo di Compilazione

```
public class Prova {  
    public static void main(String[] args) {  
        System.out.println("hi")  
    }  
}
```

Exception in thread "main" java.lang.Error:
Unresolved compilation problem:

Syntax error, insert ";" to complete
BlockStatements

Dove si è
verificato

Il problema

at Prova.main(Prova.java:3)

Errori a Tempo di Esecuzione (1)

- Gli errori a tempo di esecuzione sono decisamente più temibili
 - Si verificano durante l'esecuzione del codice, non sono né riconoscibili né prevedibili dal compilatore
 - In Java, gli errori a tempo di esecuzione sono spesso anche detti *runtime-exception*
 - prendendo in prestito il nome del costrutto del linguaggio pensato per gestirli (>>)
- Anche in questo caso la diagnostica è molto precisa

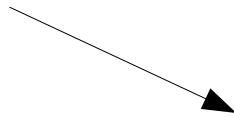
Errori a Tempo di Esecuzione (2)

```
public class Prova {  
    public static void main(String[] args) {  
        int infinito = 1/0;  
        System.out.println(infinito);  
    }  
}
```

La compilazione va a buon fine ma durante l'esecuzione si ha

Exception in thread "main"
java.lang.ArithmeticException: / by zero

Dove si è
verificato



Il problema

at Prova.main(Prova.java:3)

Paradigma di Programmazione

- Il *paradigma di programmazione* influenza decisamente il modo di programmare
- Non esiste una definizione comunemente accettata di *paradigma di programmazione* (e forse neanche serve...)
- Con un pò di esperienza inter-paradigma risulta facile apprezzare come un linguaggio di programmazione rende un paradigma più semplice da esprimere rispetto ad altri linguaggi
- Il vostro corso di studi ne tiene conto!
 - Procedurale: FdI - I anno (Linguaggio C)
 - Oggetti: POO - II anno (Linguaggio Java)
 - Funzionale: PF - III anno (Linguaggio OCaml)
 - Paradigma Ibrido Oggetti/Funzionale (Scala)

Il Paradigma Procedurale

- Il paradigma procedurale è stato il primo a diffondersi
- La divisione tra dati ed operazioni è tipica dell'hardware moderno (memoria/CPU) e dell'architettura di Von Neumann
 - Tale divisione impone che i dati siano sempre separati dalle operazioni
 - La correlazione tra dati ed il codice che li lavora non è *esplicita*
- Nel paradigma procedurale un programma è specificato come una sequenza di comandi che cambiano lo stato dell'esecuzione, passo dopo passo
- Il C è un linguaggio di programmazione che rende naturale esprimere il paradigma procedurale
- In effetti è un linguaggio “vicino alla macchina”

Limiti del Paradigma Procedurale (1)

- Uno dei maggiori limiti del paradigma procedurale è proprio la *netta* separazione tra dati ed operazioni sugli stessi
 - lo stato è ben distinto e separato dalle operazioni
- Dati e funzioni che li lavorano, sono comunque separati nel codice

```
struct Persona {  
    const char* nome;  
    int eta;  
};
```

```
int isMaggiorenne(struct Persona persona) {  
    return ( persona.eta >= 18 );  
}
```

Limiti del Paradigma Procedurale (2)

- È del tutto legittimo avere operazioni sugli stessi dati in luoghi anche *molto distanti* del codice
- Addirittura incentivata(!) la ripartizione dei due aspetti in file distinti
- La definizione di un tipo di dato in C specifica solo i dati componenti ma non le operazioni che vi si possono applicare
 - queste sono definite altrove e non fanno parte della definizione del tipo di dato

Limiti del Paradigma Procedurale (3)

File: Persona.h

```
.....  
  
struct Persona {  
    const char* nome;  
    int eta;  
};
```

Dati

File: Persona.c

```
.....  
  
void invecchia(struct Persona *p) {  
    p->eta = p->eta + 1;  
}  
  
int isMaggiorenne(struct Persona p)  
{  
    return ( p.eta >= 18 );  
}
```

Operazioni

- Il tipo di dato **Persona** non definisce le operazioni che possono essere eseguite sul tipo stesso

Programmazione Orientata agli Oggetti

Introduzione al Paradigma di
Programmazione Orientato
agli Oggetti

Sommario

- Paradigmi di Programmazione
 - Il Paradigma Orientato agli Oggetti
 - Classi
 - Oggetti
 - Esercizio con Eclipse
 - Gli oggetti in *Rete*
- La notazione puntata
- Stato degli oggetti
 - Variabili di istanza
 - Inizializzazione
 - Campo d'Azione (Scope)
- Comportamento degli oggetti
 - Metodi

Il Paradigma Orientato agli Oggetti (1)

- La divisione tra codice e operazioni è tipica dell'hardware moderno (memoria/CPU) secondo l'architettura di Von Neumann
- Ma gli esseri umani ragionano allo stesso modo?
- Ovvero, quando pensiamo ad un *oggetto* pensiamo solo al suo stato od anche alle operazioni che vi si possono compiere
- Ad esempio... un *televisore*...!?

Il Paradigma Orientato agli Oggetti (2)

- Un problema è più naturalmente modellabile se pensato come popolato da una pluralità di *oggetti* che possiedono uno stato (*spento, acceso, sintonizzato* su un certo canale)

Televisore, Persona, ecc...

e che interagiscono ciascuno secondo i messaggi che sanno naturalmente interpretare:

fabio.accende(tvInSalone)

- Gli oggetti conoscono ed interagiscono con altri esemplari (anche dello stesso *tipo*)
 - Un oggetto **n11** di tipo **Stanza** conosce altri oggetti **Stanza**, ad esempio **n12**, nelle immediate adiacenze a formare un oggetto **Labirinto**

Il Paradigma Orientato agli Oggetti (3)

- Ognuno di questi oggetti ha delle proprietà
 - *Peso, Nome, Altezza, ...*
- Ognuno di questi oggetti può svolgere delle azioni
 - Un oggetto **Macchina** può **accendersi()**
 - Un oggetto **Persona** può **salutare()**
- Risulta più naturale programmare se programmiamo in maniera più vicina a come naturalmente pensiamo

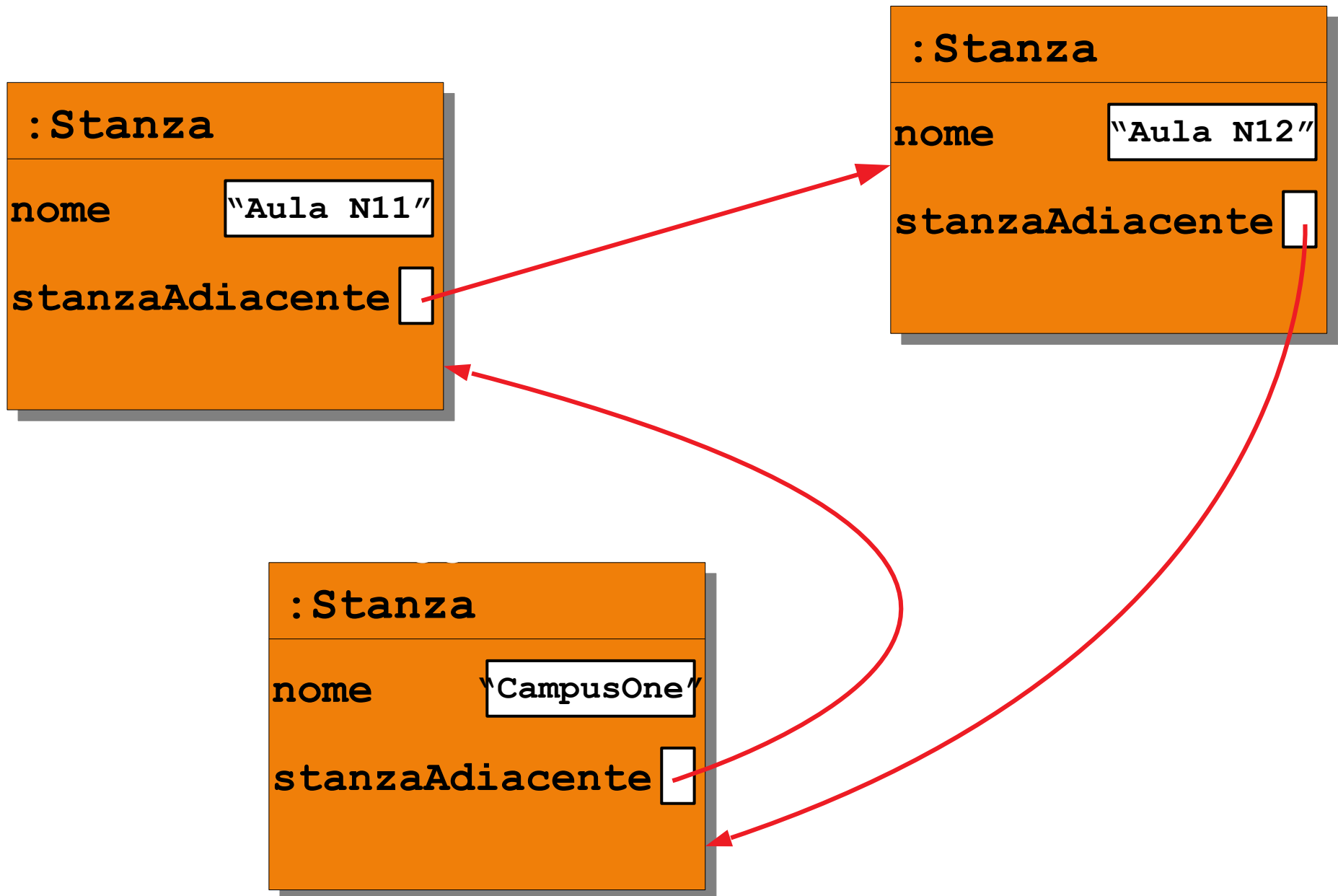
Il Paradigma Orientato agli Oggetti (4)

- E' infatti possibile definire classi di oggetti che
 - mantengono uno stato
 - offrono operazioni che lo modificano/interrogano in maniera logicamente coesa
- La realizzazione di un programma consiste nella definizione di opportune classi di oggetti che si scambiano messaggi che sanno come interpretare
- Java è solo uno dei tanti linguaggi ideati per supportare la programmazione secondo il paradigma orientato agli oggetti
- Anzi... a ben vedere esistono fonti che spiegano come programmare in maniera orientata agli oggetti anche in linguaggio C (>>)

La Rete di Oggetti (1)

- L'esecuzione di un programma Java si risolve nello scambio di messaggi tra oggetti che aggiornano ed interrogano il proprio stato
- Ad esempio, se si volesse rappresentare la topologia delle aule della nostra università, useremmo degli oggetti **Stanza** (**n10**, **n11**, **campusOne**, ...)
 - Per potersi scambiare messaggi, questi oggetti devono conoscersi tramite dei *riferimenti*
 - si crea una *Rete di Oggetti*

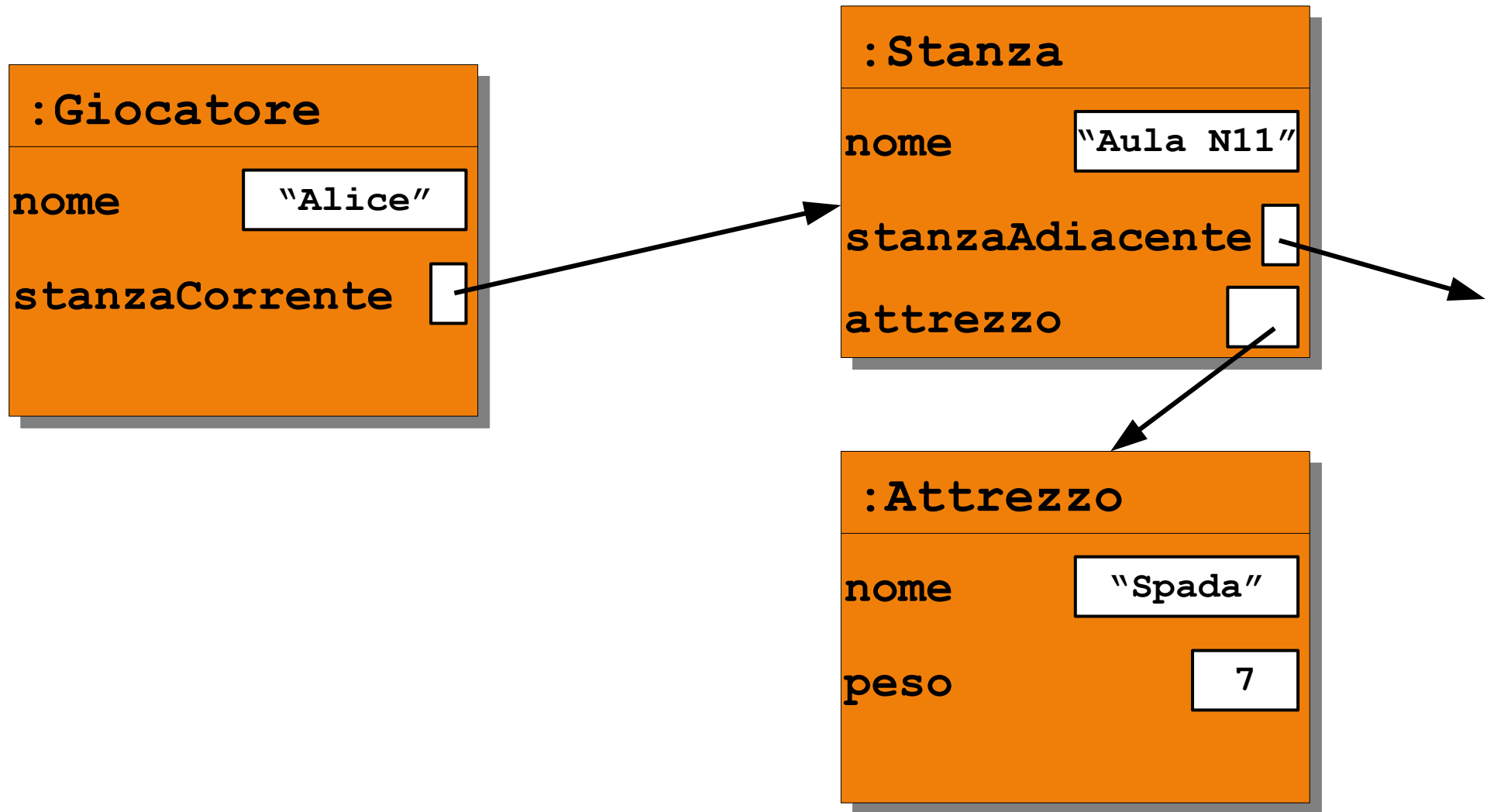
La Rete di Oggetti (2)



La Rete di Oggetti (3)

- Grazie ad un riferimento è possibile rappresentare l'oggetto **Stanza** corrente di un ipotetico oggetto **Giocatore**
- Grazie ai collegamenti tra oggetti è possibile spostarsi all'interno di un oggetto **Labirinto** che *conosce* certamente le stanze
- Un oggetto **Stanza** può “contenere” un oggetto **Attrezzo**
- L'oggetto **Giocatore** potrebbe “chiedere” all'oggetto **Stanza** corrente quale oggetto **Attrezzo** possiede, usando operazioni come **prendiAttrezzo()** e **rimuoviAttrezzo()**
- Allo scopo deve scambiare messaggi con l'oggetto **Stanza**

La Rete di Oggetti (4)



Oggetti: Stato + Comportamento (1)

- Secondo il paradigma OO l'esecuzione di un programma avviene con la creazione di un rete di oggetti che si scambiano messaggi, aggiornano ed interrogano il proprio stato durante l'esecuzione
- Ogni oggetto
 - possiede uno stato interno
 - offre operazioni agli altri oggetti
- Gli oggetti sono dotati di:
 - *Stato*: informazioni memorizzate in **variabili di istanza** (o campi o attributi)
 - *Comportamento*: **metodi** che si possono invocare sull'oggetto
 - *Identità*: un oggetto può essere distinto dagli altri (anche e soprattutto da *esemplari* della stessa tipologia)

Oggetti: Stato + Comportamento (2)

- Ad esempio l'oggetto `Giocatore` possiede come stato il suo nome e la stanza in cui si trova
- È possibile chiedere all'oggetto `Giocatore` di compiere delle azioni
 - ad es. `getStanzaCorrente()` per ottenere l'oggetto che rappresenta la stanza corrente
 - ad es. `setStanzaCorrente()` per spostarsi nella prossima stanza
- Può servire quindi interrogare l'oggetto `Stanza`
 - ad es. `getStanzaAdiacente()` per ottenere l'oggetto che rappresenta la stanza adiacente a quella considerata *corrente*

Classe (1)

- Possono esistere vari oggetti dello stesso tipo
 - diverse stanze; attrezzi; giocatori
- Tutti gli oggetti di un certo *tipo* possiedono le informazioni dello stesso *tipo*
 - tutte le stanze hanno un `nome` ed una o più `stanzaAdiacente`
- Tutti gli oggetti di un certo *tipo* offrono le stesse operazioni
 - A tutti i giocatori è possibile chiedere di spostarsi nella prossima stanza o di raccogliere un oggetto `Attrezzo`

Classe (2)

- È necessario un meccanismo per costruire oggetti:
 - 1) Possiedono una propria identità ed autonomia
 - 2) Possiedono informazioni e dati specifici...
 - 3) ...ma sembrano chiaramente rispondere anche ad altri tratti comuni a tutti gli oggetti della stessa tipologia
- Serve una sorta di *fabbrica specializzata* che definisca come tutti gli oggetti di un certo *tipo* siano fatti lasciando la libertà di variare alcuni aspetti
- Si pensi ad un fabbrica di *Orologi* tutti uguali
 - 1) Ciascun esemplare è distinto dagli altri
 - 2) Ciascun esemplare ha un numero di serie ed un colore diverso...
 - 3) ...ma tutti offrono la possibilità di leggere l'ora
- Nella programmazione orientata agli oggetti queste fabbriche sono dette **classi**

Classe (3)

- Per ogni tipo di oggetto che si vuole rappresentare esiste una *classe*
 - Una per **Giocatore**, una per **Stanza**, ecc...
- Tutti gli oggetti sono costruiti a partire dalla definizione di una classe
- La classe astrae e definisce
 - lo stato di un oggetto di un certo tipo
 - il comportamento degli oggetti di un certo tipo

Classe, un Esempio

- Tralasciamo per ora le stanze, gli attrezzi e i giocatori... Torneranno più tardi (>>)
- Più semplice utilizzare forme geometriche in un piano cartesiano con coordinate intere per un primo esempio di classe
 - per rappresentare una forma è necessario conoscerne la posizione
 - servono le coordinate di un punto sul piano cartesiano: **x**, **y**
 - si definisce una classe **Punto**

La Classe Punto (1)

```
public class Punto {
```

```
    private int x;
```

```
    private int y;
```

} Stato

```
    public void setX(int posX) {
```

```
        x = posX;
```

```
    }
```

```
    public void setY(int posY) {
```

```
        y = posY;
```

```
    }
```

```
    public int getX() {
```

```
        return x;
```

```
    }
```

```
    public int getY() {
```

```
        return y;
```

```
    }
```

```
}
```

} Operazioni, per impostare e leggere lo stato

La Classe Punto (2)

```
public class Punto {  
    private int x;  
    private int y;  
  
    public void setX(int posX) {  
        x = posX;  
    }  
    public void setY(int posY) {  
        y = posY;  
    }  
    public int getX() {  
        return x;  
    }  
    public int getY() {  
        return y;  
    }  
}
```

Ogni oggetto di tipo **Punto** ha una *x* e una *y*
variabili d'istanza

Inoltre, dispone di una serie di operazioni per leggere e impostare il suo stato

metodi

Creazione di Oggetti

- La creazione di un oggetto a partire da una classe avviene utilizzando l'operatore **new**:
`Punto origine = new Punto();`
- Restituisce un *referimento* ad un oggetto appena creato
- Ora è presente un oggetto in memoria ed è possibile chiedergli di svolgere delle operazioni per tramite del riferimento, qui conservato nella variabile locale **origine**
- Ad esempio per impostare la ascissa ed ordinata
`origine.setX(0);`
`origine.setY(0);`

Concetti Correlati ma Ben Distinti!

Questa semplice istruzione

```
Punto origine = new Punto();
```

già nasconde almeno tre concetti
chiaramente distinti sebbene correlati

I. Variabile locale

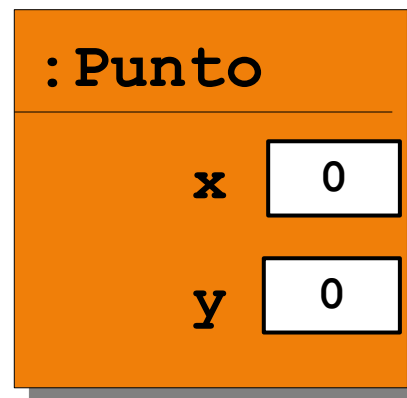
II. Riferimento

III. Oggetto

Variabile locale `origine`

riferimento

oggetto



Ci torneremo più volte sopra!

La Notazione Puntata

- Per richiedere un'operazione ad un oggetto si usa la cosiddetta *notazione puntata*

`<referimento-oggetto>.<metodo>(<parametri-attuali>);`

- Ad esempio:
 - `origine.setX(0);`
- Anche solo questa notazione mostra come i dati siano stati *avvicinati* alle operazioni sugli stessi
dati.operazione(parametri)

Classi e Oggetti

- A partire da una classe possiamo creare molte *istanze*, ognuna dotata di uno stato autonomo

```
Punto unoZero = new Punto();
```

```
unoZero.setX(1);
```

```
unoZero.setY(0);
```

Un nuovo oggetto viene creato in memoria; è possibile chiedergli di svolgere operazioni attraverso il riferimento conservato nella variabile locale **unoZero**

- L'oggetto creato in precedenza (ovvero quello il cui riferimento è conservato dentro **origine**) risulta invariato dopo tali operazioni e non viene modificato dall'invocazione di metodi tramite il riferimento conservato dentro **unoZero** perché le operazioni sono svolte su un oggetto distinto

Terminologia

- Nella programmazione orientata agli oggetti si utilizza la seguente terminologia:
 - **Oggetti o istanze:** istanze di una certa classe, presenti in memoria, mantengono lo stato di un oggetto
 - **Metodi:** specificano le operazioni che determinano il comportamento degli oggetti
 - Es.: `setX()`, `getY()`, ...
 - **Variabili di Istanza:** consentono di memorizzare le informazioni di ciascun oggetto (lo stato dell'oggetto)
 - **Riferimento** ad oggetto: un riferimento ad un oggetto in memoria (>>)

Classi e File

- In Java, una classe di nome **xyz** *DEVE* essere contenuta all'interno di un file di nome **XYZ.java**
 - esistono alcune eccezioni a questa regola che tralasciamo per il momento
classi nidificate (>>)
- Da qualsiasi altra classe del nostro programma sarà possibile far riferimento ad una classe specificandone il suo nome
 - In Java *non* è necessario importare un file esplicitamente per poter usare il codice in esso contenuto
 - Ma bisogna mettere il compilatore (e la JVM durante l'esecuzione) in condizione di trovarlo a partire dal suo nome (>>)

Esercizio: Eclipse (0)

- JDK (Standard Edition)
 - Scaricare ed installare dal sito della ORACLE l'ultima versione disponibile di Java 8
https://www.java.com/download/java8_update.jsp
 - Recommended Version 8 Update 321
 - Oppure Java 11
- Documentazione JDK
 - Scaricare ed installare dal sito della ORACLE
- Strumenti di sviluppo
 - Un IDE professionale: Eclipse (4.X)
- Scaricare **Eclipse IDE for Java Developers** da
<https://www.eclipse.org/downloads/eclipse-packages/>
(è la versione usata anche in sede d'esame)
- Configurare Eclipse per compilare **Java 7**

Esercizio: Eclipse (1)

- Realizzare la classe **Punto** usando l'IDE eclipse
 - *File > New > Java Project*
 - *Project Name: Forme > click su 'Finish'*
 - Selezionare il progetto appena creato
 - *File > New > Class*
 - *Name: Punto > click su 'Finish'*
- Oltre al codice mostrato prima, aggiungere il metodo `public void trasla(int dx, int dy)` che sposta il punto di `dx` sull'asse x e `dy` sull'asse y

Esercizio: Eclipse (2)

Realizzare la classe **MainForme** usando l'IDE eclipse

- Creare una classe di nome **MainForme**
- Aggiungere il metodo **main()**

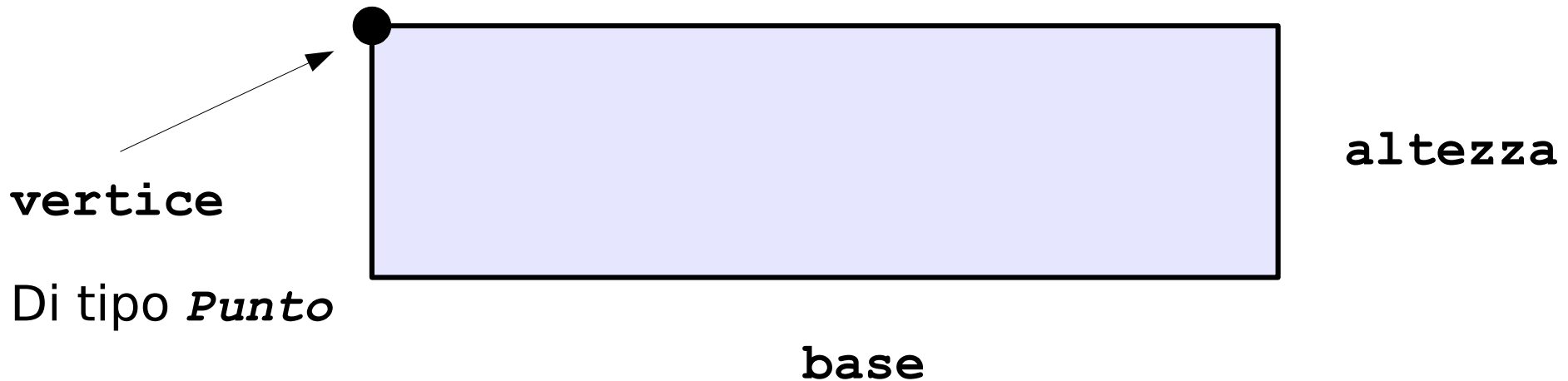
```
public class MainForme {  
    public static void main(String[] args) {  
        Punto origine = new Punto();  
        origine.setX(0);  
        origine.setY(0);  
        System.out.println(origine.getX());  
        System.out.println(origine.getY());  
        origine.trasla(1, 1);  
        System.out.println(origine.getX());  
    }  
}
```

Gli Oggetti in *Rete* (1)

- La *definizione* di un programma “orientato agli oggetti” consiste nella definizione di diverse *classi* di oggetti
- L'*esecuzione* di un programma orientato agli oggetti avviene orchestrando lo scambio di messaggi tra un pluralità di oggetti istanza delle classi definite nel programma
 - gli oggetti devono “conoscersi”
 - un oggetto può possedere *riferimenti* verso gli altri oggetti
 - gli oggetti possono inviare messaggi ad altri oggetti dei quali possiedono un *riferimento*
- Si vuole realizzare la classe **Rettangolo**
 - Stato composito:
 - base
 - altezza
 - posizione del vertice in alto a sx

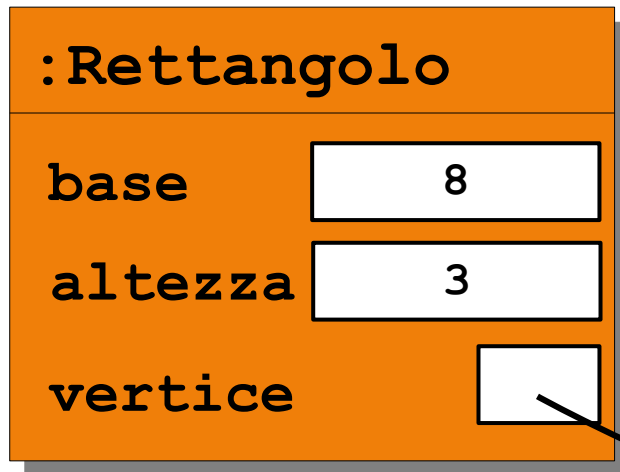
Gli Oggetti in *Rete* (2)

- Il vertice in alto a sinistra è un oggetto istanza della classe **Punto**
 - di coordinate (x, y)
- Ogni oggetto della classe **Rettangolo** deve conoscere un oggetto della classe **Punto** che rappresenta il suo vertice in alto a sinistra



Gli Oggetti in *Rete* (3)

oggetto



*riferimento
ad oggetto*

oggetto



(Con Eclipse) La Classe Rettangolo

- Nello stesso progetto della classe Punto

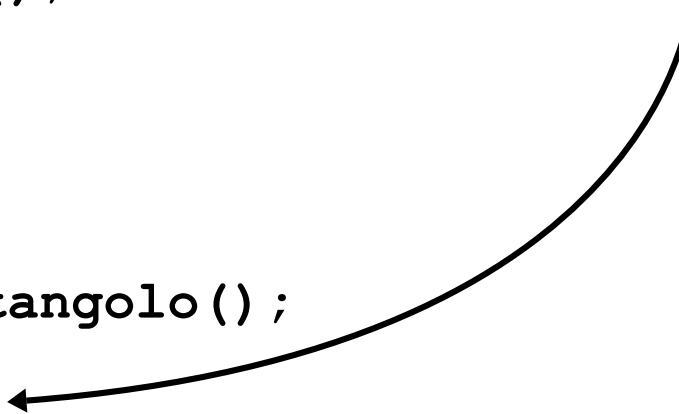
```
public class Rettangolo {  
    private int base;  
    private int altezza;  
    private Punto vertice;  
  
    public void setBase(int b) { base = b; }  
    public void setAltezza(int a) { altezza = a; }  
    public void setVertice(Punto v) { vertice = v; }  
  
    public int getBase() { return base; }  
    public int getAltezza() { return altezza; }  
    public Punto getVertice() { return vertice; }  
}
```

La Classe Rettangolo (2)

- Il `main()` può essere modificato come segue

```
public class MainForme {  
    public static void main(String[] args) {  
        Punto origine = new Punto();  
        origine.setX(0);  
        origine.setY(0);  
  
        Rettangolo rect = new Rettangolo();  
        rect.setVertice(origine);  
        rect.setBase(8);  
        rect.setAltezza(3);  
        // ...  
        // Seguono stampe per verificarne il funzionamento  
    }  
}
```

Dopo questa istruzione l'oggetto istanza di **Rettangolo** conosce l'oggetto istanza di **Punto**



Messaggi tra Oggetti (1)

- Si vuole implementare il metodo `sposta(int deltaX, int deltaY)` nella classe `Rettangolo`
 - Per traslare il suo vertice il rettangolo può chiedere al suo stesso vertice di spostarsi: scambio di messaggi tra oggetti

```
public class Rettangolo {  
    // ... Come prima ...  
    public void sposta(int deltaX, int deltaY) {  
        vertice.trasla(deltaX, deltaY);  
    }  
}
```


Messaggi tra Oggetti (2)

- E se la classe `Punto` non disponesse del metodo `trasla()`?

Messaggi tra Oggetti (3)

```
public class Rettangolo {  
    // ... come prima ...  
    public void sposta(int deltaX, int deltaY) {  
        int xVertice = vertice.getX();  
        int yVertice = vertice.getY();  
        vertice.setX(xVertice + deltaX);  
        vertice.setY(yVertice + deltaY);  
    }  
}
```

- Il comportamento desiderato è comunque ottenibile utilizzando i metodi `setX()` e `setY()` ma il codice risulta “meno pulito” rispetto alla soluzione basata sulla disponibilità del metodo `trasla()` già all’interno della classe `Punto`

Variabili di Istanza e Metodi

- Una classe definisce sia delle variabili di istanza sia dei metodi; rappresentano, rispett.
 - *lo stato* degli oggetti istanza di quella classe
 - *il comportamento* di tali oggetti
- La definizione di una classe segue questa sintassi difatti:

```
public class <NomeClasse> {  
    <definizione di variabili di istanza>  
    <definizione di metodi>  
}
```

Variabili di Istanza (1)

- Le variabili di istanza memorizzano informazioni che rappresentano lo stato di un oggetto

```
public class Rettangolo {  
    private int base;  
    private int altezza;  
    private Punto vertice;  
    ...  
}
```

Variabili di Istanza (2)

- Nella definizione di una variabile di istanza:
 - Modificatore di visibilità (>>)
 - Tipo
 - Nome della variabile

private int base;



Modificatore di visibilità

Tipo

Nome della variabile

- Il modificatore di visibilità specifica se una certa variabile è visibile dall'esterno
 - Per il momento si usa **private**: la variabile è visibile solo all'interno della classe in cui è dichiarata
 - Per accedervi dall'esterno si usano i metodi *getter* e *setter*

Variabili Istanza: Inizializzazione

- Ci sono diversi modi per inizializzare lo stato di un oggetto
- Le variabili di qualsiasi genere (ovvero di istanza, locali ed altro >>) vengono sempre inizializzate, esplicitamente od implicitamente
- In Java non esiste il problema delle variabili accidentalmente rimaste non inizializzate tipico del linguaggio C
- Per le variabili di istanza: se non viene specificato alcun valore iniziale, assumono un valore di default:
 - per le variabili di un tipo numerico (`int`, `float`...) è 0

```
Rettangolo rect = new Rettangolo();  
System.out.println(rect.getBase()); // Stampa 0
```

- Stampa (sempre e prevedibilmente) 0 nonostante non sia stato invocato il metodo `setBase(0)` ;

Variabili di Istanza: Campo d'Azione (o *Scope*)

- Le variabili di istanza sono visibili all'interno della sola classe in cui sono dichiarate
 - Ogni metodo può referenziarle semplicemente per nome

```
public class Rettangolo {  
    private int base;  
    // ...  
    public int getBase() {  
        return base;  
    }  
    // ...  
}
```

Qui 'base' fa
riferimento alla
variabile di
istanza **base**



Variabili di Istanza

- Una variabile di istanza ha un valore come parte dello stato di uno specifico oggetto istanza della sua classe
- Si usa dire che una variabile di istanza *“appartiene ad un oggetto”* anche se è *definita* nella sua classe
- Un oggetto, tramite le proprie variabili di istanza, possiede un proprio stato *autonomamente* rispetto a tutti gli altri oggetti istanza della sua stessa classe

```
Rettangolo rect1 = new Rettangolo();  
rect1.setBase(10);
```

```
rect1.base; // NON COMPILA
```

```
Rettangolo rect2 = new Rettangolo();  
rect2.setBase(20); /* N.B. la base del secondo oggetto cambia;  
                    Quella del primo rimane invariata */
```


Un Parallelismo con il Linguaggio C (1)

- Pare abbastanza naturale associare una variabile di istanza di una classe Java ad un campo di una **struct** di C

```
typedef struct {  
    int base;  
    ...  
} Rettangolo;
```

```
Rettangolo *r = malloc(sizeof(Rettangolo)) ;  
r->base = 15;  
Rettangolo *r2 = malloc(sizeof(Rettangolo)) ;  
r2->base = 30;  
free(r1) ;  
free(r2) ;
```

Un Parallelismo con il Linguaggio C (2)

- I metodi definiscono le operazioni che si possono svolgere su un oggetto di una certa classe
- Viene naturale associare un metodo di una classe Java ad una funzione C che opera su una **struct**

```
typedef struct {  
    int base;  
  
    ...  
} Rettangolo;
```

```
void setBase(Rettangolo *this, int base) {  
    this->base = base;  
}
```

Invocazione dei Metodi

- L'invocazione dei metodi è alla base della programmazione orientata agli oggetti come meccanismo per lo scambio di messaggi tra oggetti
- I metodi mettono in comunicazione diretta l'oggetto che invoca il metodo con quello su cui il metodo viene invocato
- Ad esempio:
 - Per impostare od ottenere la base di un rettangolo abbiamo invocato dei metodi della classe **Rettangolo**
 - Per spostare un oggetto istanza della classe **Rettangolo** il suo metodo **sposta()** ha invocato dei metodi della classe **Punto** una cui istanza ne rappresenta il vertice

Metodo `main()`

- L'esecuzione di un programma inizia sempre con l'invocazione di un particolare e specifico metodo
- Per convenzione (eredità dal linguaggio C) tale metodo si chiama `main()`
 - Questo metodo “scatena” l'esecuzione invocando a sua volta altri metodi
- Tranne che per il metodo `main()` da cui comincia l'esecuzione, per ogni invocazione di metodo esiste sempre un metodo *invocante* ed un metodo *invocato*

Metodo Invocante e Invocato

Invocazione di metodo

*Metodo
invocante*

```
public class Main {  
    public static void main(String args[]) {  
        Rettangolo rect = new Rettangolo();  
        rect.setBase(22);  
    }  
}
```

*Metodo
invocato*

Definizione di Metodo

- I metodi sono dichiarati all'interno della definizione di una classe e definiscono il comportamento di tutti gli oggetti appartenenti a quella classe
- La dichiarazione di un metodo comprende due parti:
 - Intestazione
 - Modificatore di accesso/visibilità
 - Tipo valore restituito
 - Nome del metodo
 - Lista dei parametri formali
 - Corpo
 - Definizioni di variabili locali
 - Istruzioni

Intestazione *Corpo*

```
public void setX(int x) { ... }
```

Metodi: Valore Restituito

- I metodi possono comunicare verso l'esterno restituendo un valore
 - Il metodo *invocato* comunica con il metodo *invocante*
 - esattamente come per le funzioni in C
- Se un metodo non ritorna nessun valore al momento della dichiarazione del tipo di ritorno si utilizza la parola chiave **void**

Metodi e Aggiornamenti di Stato

- Conviene, per diversi motivi (>>), distinguere sempre i metodi che
 - **interrogano** (solamente) lo stato dell'oggetto su cui sono invocati
 - Solo *lettura* dello stato
 - **aggiornano** lo stato dell'oggetto su cui sono invocati
 - Anche *scrittura* dello stato
- Ad esempio (nella classe **Punto**)

```
public int getX() {  
    return x;           // interroga lo stato  
}  
  
public void trasla(int dx, int dy) {  
    x += dx;             // aggiorna lo stato  
    y += dy;             // aggiorna lo stato  
}
```


(Esercizio con Eclipse)

Variabili di Istanza e Metodi

- Realizzare la classe **Attrezzo**
 - Con le variabili di istanza
 - **nome** di tipo **String**
 - **peso** di tipo **int**
 - aggiungere i relativi metodi *getter & setter*
- Realizzare la classe **Stanza**
 - Con le variabili di istanza
 - **nome** di tipo **String**
 - **stanzaAdiacente** di tipo **Stanza**
 - **attrezzoContenuto** di tipo **Attrezzo**
 - aggiungere i relativi metodi *getter & setter*

Modificare lo Stato di un Oggetto (1)

- Lo stato di un oggetto può essere cambiato

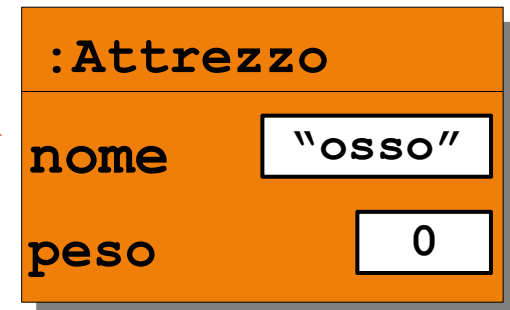
```
public class MainStanzeAttrezzi {  
    public static void main(String[] args) {  
        Attrezzo spada = new Attrezzo();  
        spada.setNome("spada");  
        spada.setPeso(7);  
  
        Attrezzo osso = new Attrezzo();  
        Osso.setNome("osso");  
        Osso.setPeso(1);  
  
        Stanza n11 = new Stanza();  
        n11.setNome("N11");  
  
        n11.setAttrezzo(spada);  
  
        n11.setAttrezzo(osso);  
    }  
}
```



Modificare lo Stato di un Oggetto (2)

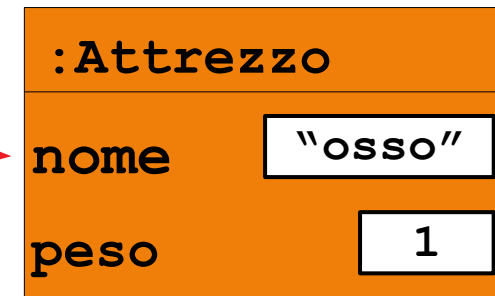
- Lo stato di un oggetto può essere cambiato

```
public class MainStanzeAttrezzi {  
    public static void main(String[] args) {  
        Attrezzo spada = new Attrezzo();  
        spada.setNome("spada");  
        spada.setPeso(7);  
  
        Attrezzo osso = new Attrezzo();  
        osso.setNome("osso");  
        osso.setPeso(1);  
  
        Stanza n11 = new Stanza();  
        n11.setNome("N11");  
  
        n11.setAttrezzo(spada);  
  
        n11.setAttrezzo(osso);  
    }  
}
```



Modificare lo Stato di un Oggetto (3)

```
public class MainStanzeAttrezzi {  
    public static void main(String[] args) {  
        Attrezzo spada = new Attrezzo();  
        spada.setNome("spada");  
        spada.setPeso(7);  
  
        Attrezzo osso = new Attrezzo();  
        osso.setNome("osso");  
        osso.setPeso(1);  
          
        Stanza n11 = new Stanza();  
        n11.setNome("N11");  
  
        n11.setAttrezzo(spada);  
  
        n11.setAttrezzo(osso);  
    }  
}
```

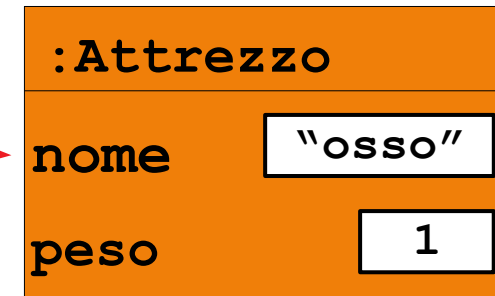


Modificare lo Stato di un Oggetto (4)

```
public class MainStanzeAttrezzi {  
    public static void main(String[] args) {  
        Attrezzo spada = new Attrezzo();  
        spada.setNome("spada");  
        spada.setPeso(7);
```



```
        Attrezzo osso = new Attrezzo();  
        Osso.setNome("osso");  
        Osso.setPeso(1);
```



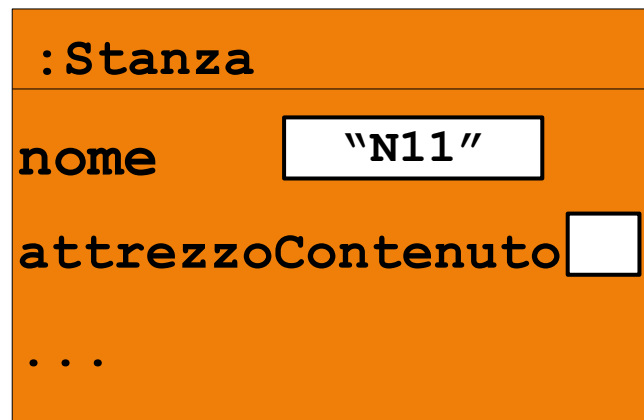
```
        Stanza n11 = new Stanza();  
        n11.setNome("N11");
```

```
        n11.setAttrezzo(spada);
```

```
        n11.setAttrezzo(osso);
```

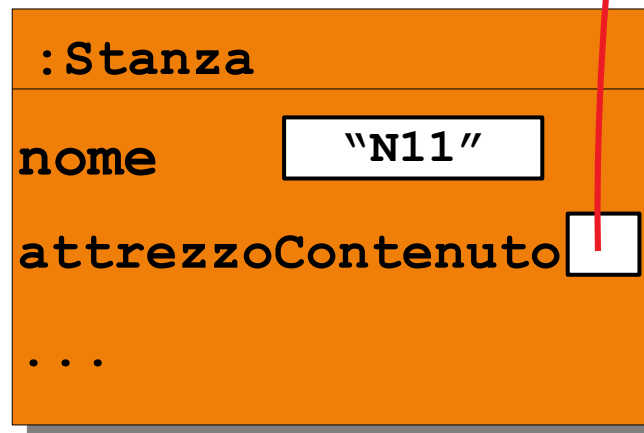
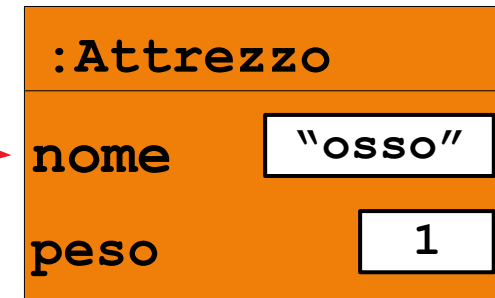
```
    }
```

```
}
```



Modificare lo Stato di un Oggetto (5)

```
public class MainStanzeAttrezzi {  
    public static void main(String[] args) {  
        Attrezzo spada = new Attrezzo();  
        spada.setNome("spada");  
        spada.setPeso(7);  
  
        Attrezzo osso = new Attrezzo();  
        Osso.setNome("osso");  
        Osso.setPeso(1);  
  
        Stanza n11 = new Stanza();  
        n11.setNome("N11");  
  
        n11.setAttrezzo(spada);  
  
        n11.setAttrezzo(osso);  
    }  
}
```



Modificare lo Stato di un Oggetto (6)

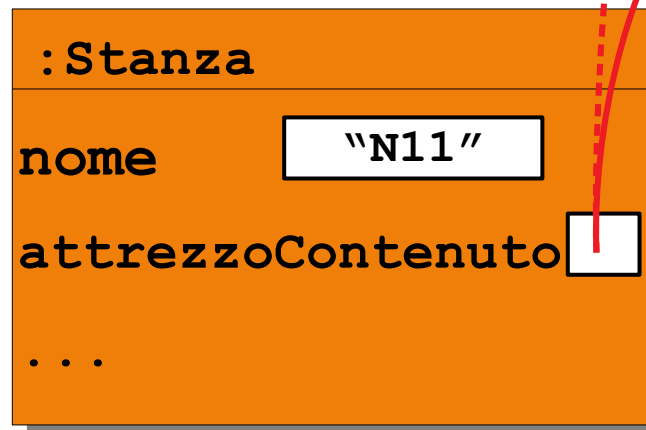
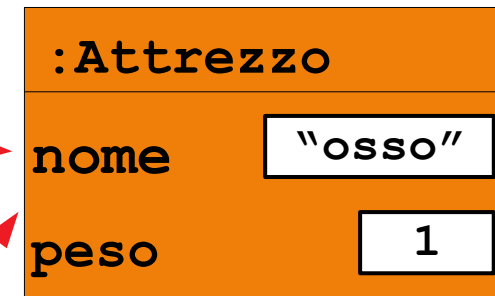
```
public class MainStanzeAttrezzi {  
    public static void main(String[] args) {  
        Attrezzo spada = new Attrezzo();  
        spada.setNome("spada");  
        spada.setPeso(7);
```

```
        Attrezzo osso = new Attrezzo();  
        Osso.setNome("osso");  
        Osso.setPeso(1);
```

```
        Stanza n11 = new Stanza();  
        n11.setNome("N11");
```

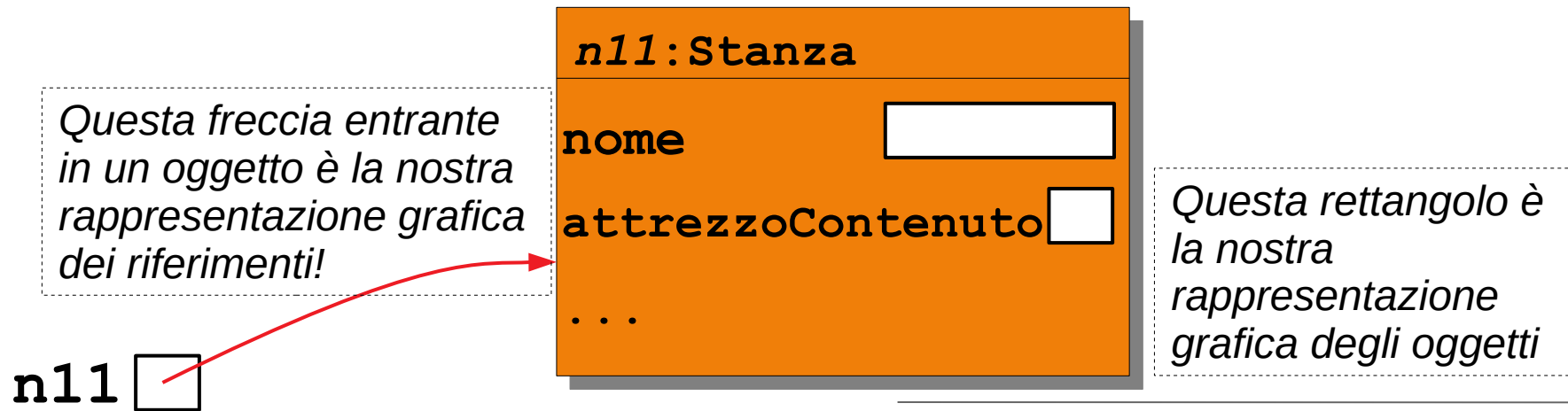
```
        n11.setAttrezzo(spada);
```

```
        n11.setAttrezzo(osso);  
    }  
}
```



Riferimenti ad Oggetti (continua...)

- La creazione di un nuovo oggetto in memoria avviene tramite l'operatore **new**
- L'operatore **new** restituisce un *riferimento ad un oggetto* appena creato
- Ad esempio: `Stanza n11 = new Stanza();`
- La variabile locale **n11** **NON** contiene l'oggetto creato, ma bensì un *riferimento ad esso* (>>)



Programmazione Orientata agli Oggetti

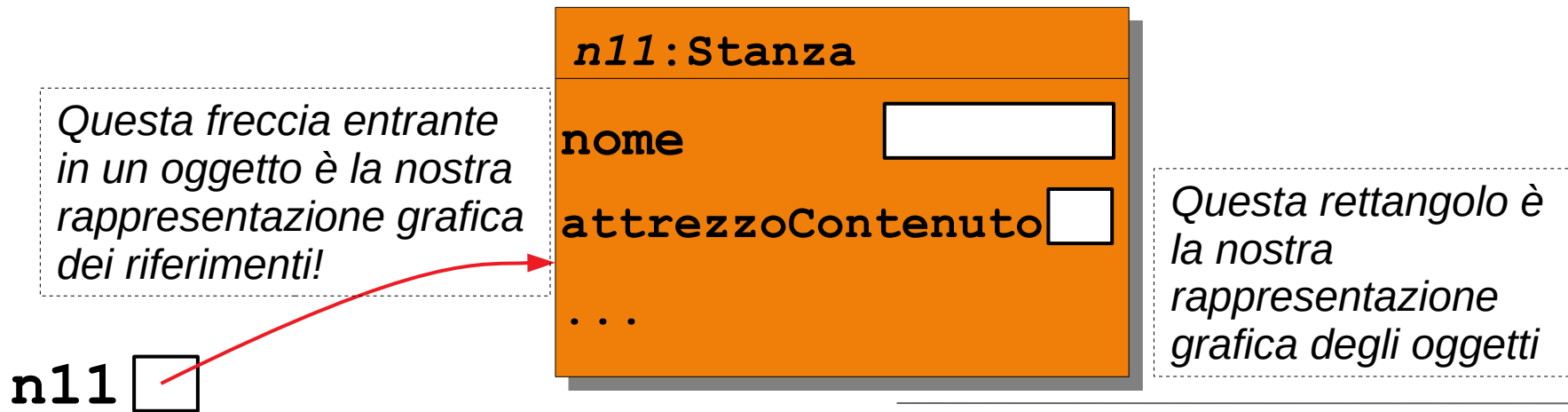
Oggetti e Riferimenti

Sommario

- Riferimenti ad Oggetti
 - Molteplici riferimenti verso lo stesso oggetto
 - Riferimenti ed *Effetti Collaterali*
- Riferimenti e passaggio dei parametri per valore
- Metodi che restituiscono riferimenti
- Riferimento *null* e **NullPointerException**
- Campo d'Azione delle variabili e *Shadowing*
- La parola chiave **this**
- Convenzioni di Stile

Riferimenti ad Oggetti (1)

- La creazione di un nuovo oggetto in memoria avviene tramite l'operatore **new**
- L'operatore **new** restituisce un *riferimento ad un oggetto* appena creato
- Ad esempio: `Stanza n11 = new Stanza();`
- La variabile locale **n11** **NON** contiene l'oggetto creato, ma bensì un *riferimento* ad esso (>>)



Riferimenti ad Oggetti (2)

- Proviamo a stampare il valore di variabili che contengono riferimenti

```
public class MainRiferimenti {  
    public static void main(String[] args) {  
        Stanza n11 = new Stanza();  
        System.out.println(n11);  
    }  
}
```

- Stampa: **Stanza@15db9742**
 - Ma ovviamente dipende dalla particolare esecuzione
 - Possiamo per il momento semplificare il significato di questa stampa: è *[un numero che dipende dal]l'indirizzo in memoria dell'oggetto referenziato*
 - In realtà non è esattamente così, ma per i nostri presenti scopi questa semplificazione fa molto comodo (>>)

Riferimenti ad Oggetti (3)

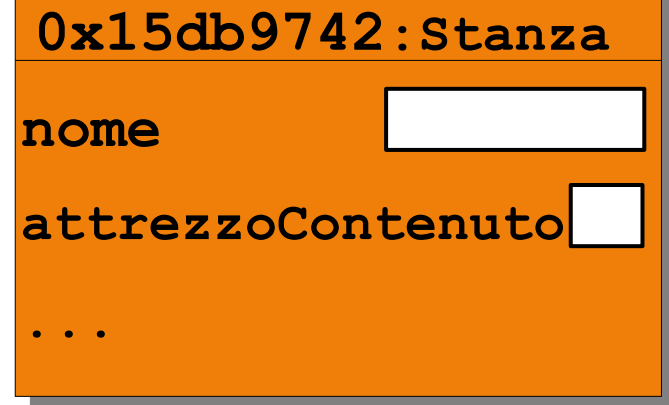
- Una *stessa* variabile può contenere, in momenti diversi, riferimenti ad oggetti distinti dello stesso tipo. Ad esempio:

```
public class MainRiferimenti {  
    public static void main(String[] args) {  
        Stanza n11 = new Stanza();  
        System.out.println(n11); // Stampa Stanza@15db9742  
  
        n11 = new Stanza();  
        System.out.println(n11); // Stampa Stanza@6d06d69c  
    }  
}
```

Riferimenti ad Oggetti (4)

```
public class MainRiferimenti {  
    public static void main(String[] args) {  
        Stanza n11 = new Stanza();  
        System.out.println(n11); // stampa Stanza@15db9742  
  
        n11 = new Stanza();  
        System.out.println(n11); // stampa Stanza@6d06d69c  
    }  
}
```

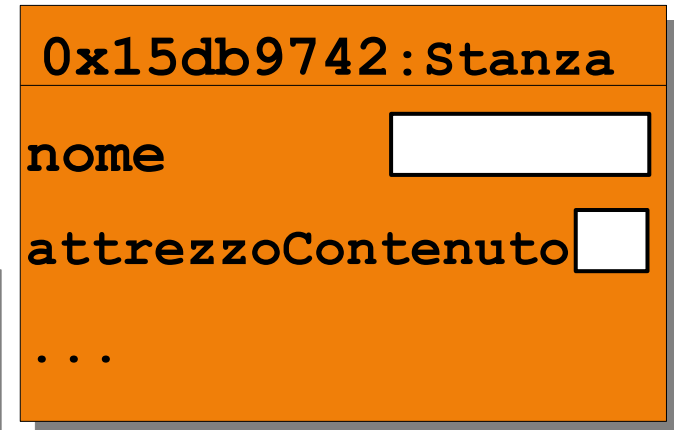
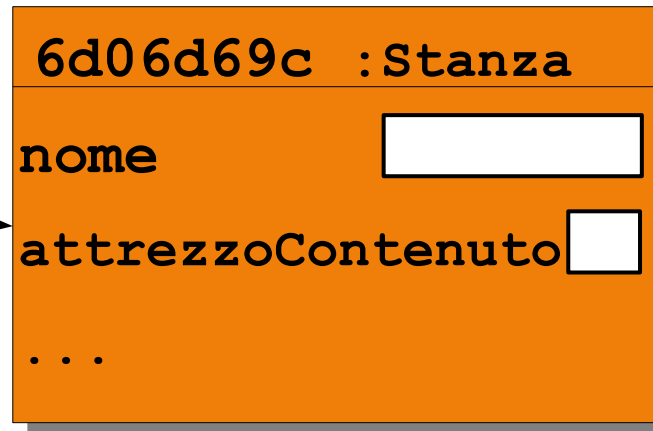
n11 ☐



Riferimenti ad Oggetti (5)

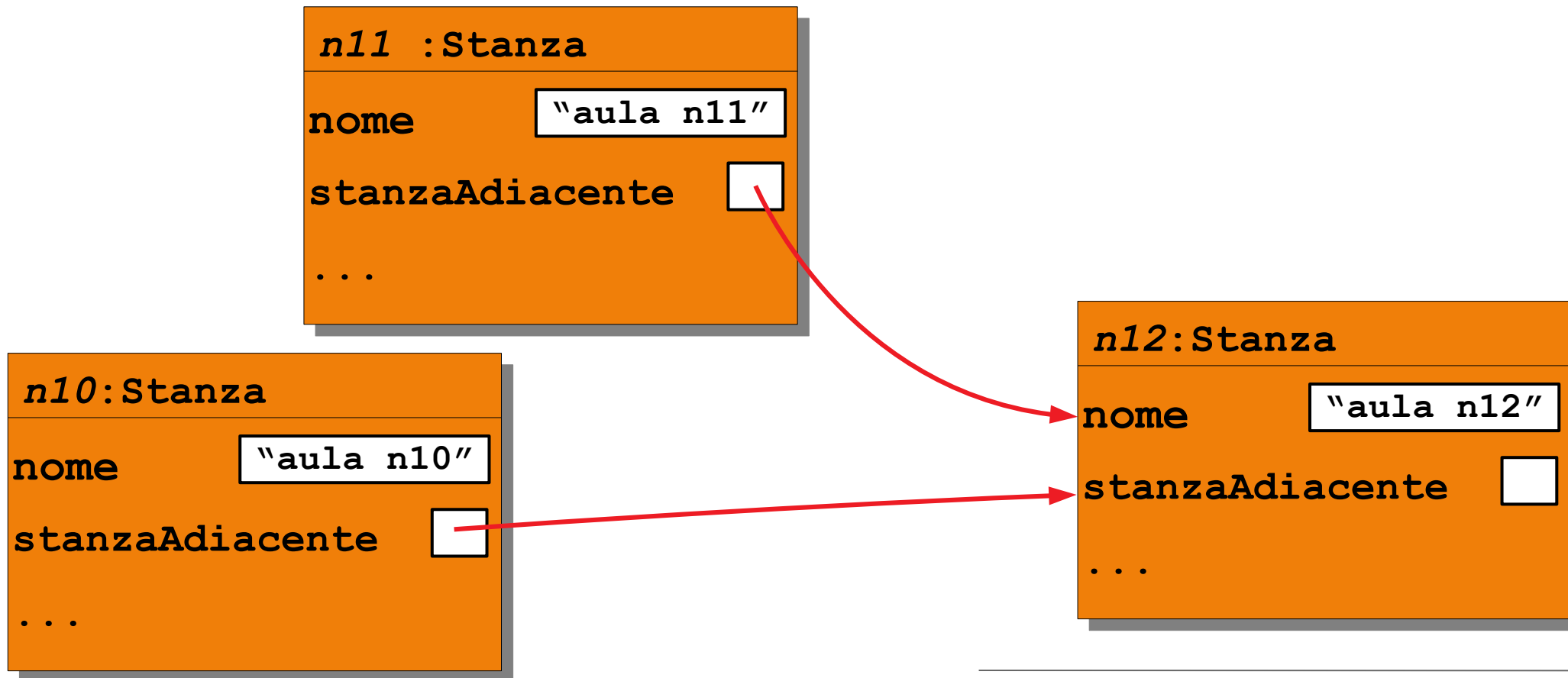
```
public class MainRiferimenti {  
    public static void main(String[] args) {  
        Stanza n11 = new Stanza();  
        System.out.println(n11); // stampa Stanza@15db9742  
  
        n11 = new Stanza();  
        System.out.println(n11); // stampa Stanza@6d06d69c  
    }  
}
```

n11 ☐



Molteplici Riferimenti verso lo Stesso Oggetto (1)

- In alcuni casi più variabili contengono un riferimento allo stesso oggetto
- Ad esempio due stanze adiacenti la medesima:



Molteplici Riferimenti verso lo Stesso Oggetto (2)

- La configurazione appena vista si può ottenere con il seguente codice

```
public class MainStanzeRiferimenti {  
    public static void main(String[] args) {  
        Stanza n12 = new Stanza();  
        n12.setNome("aula n12");  
  
        Stanza n11 = new Stanza();  
        n11.setNome("aula n11");  
        n11.setStanzaAdiacente(n12);  
  
        Stanza n10 = new Stanza();  
        n10.setNome("aula n10");  
        n10.setStanzaAdiacente(n12);  
    }  
}
```

Molteplici Riferimenti verso lo Stesso Oggetto (3)

- Un altro esempio:

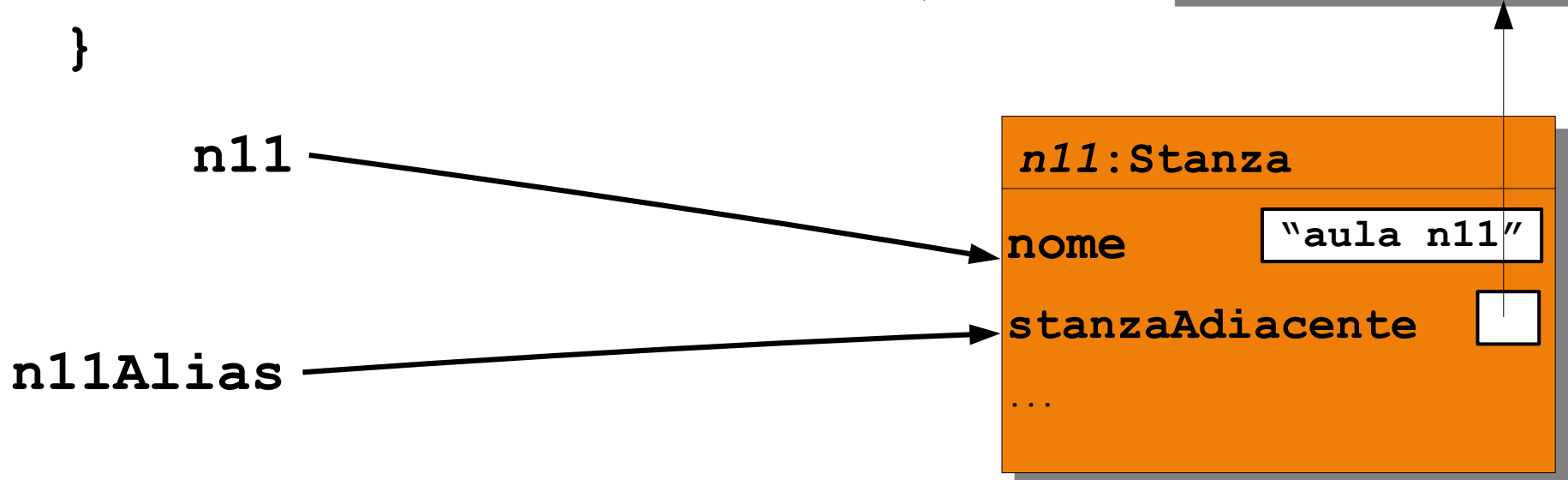
```
public class MainStanzeRiferimenti {  
    public static void main(String[] args) {  
        Stanza n12 = new Stanza();  
        n12.setNome("aula n12");  
  
        Stanza n11 = new Stanza();  
        n11.setNome("aula n11");  
        n11.setStanzaAdiacente(n12);  
  
        Stanza n11Alias = n11;  
    }  
}
```

- Ora sia `n11` sia `n11Alias` fanno riferimento allo stesso oggetto

Molteplici Riferimenti verso lo Stesso Oggetto (4)

```
public class MainStanzeRiferimenti {  
    public static void main(String[] args) {  
        Stanza n12 = new Stanza();  
        n12.setNome("aula n12");  
  
        Stanza n11 = new Stanza();  
        n11.setNome("aula n11");  
        n11.setStanzaAdiacente(n12);  
  
        Stanza n11Alias = n11;  
    }  
}
```

Stanza n11Alias = n11;



Più Riferimenti & Side-Effect (1)

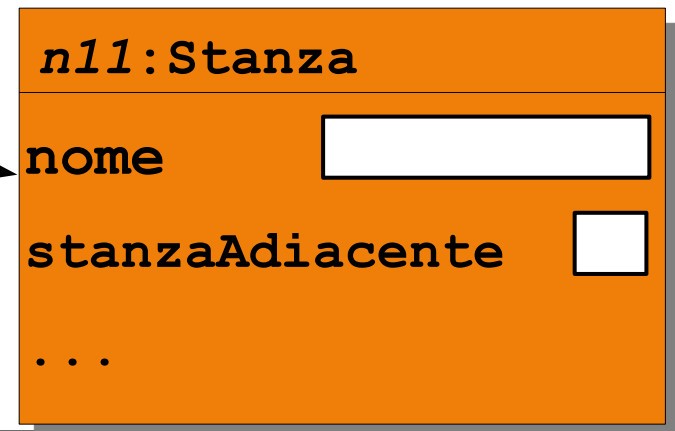
- Qual è l'output del seguente programma?

```
public static MainRiferimentiSideEffect {  
    public static void main(String[] args) {  
        Stanza n11 = new Stanza();  
  
        Stanza n11Alias = n11;  
  
        n11.setNome("N11");  
        n11Alias.setNome("aula N11");  
  
        System.out.println(n11.getNome());  
    }  
}
```

Più Riferimenti & Side-Effect (2)

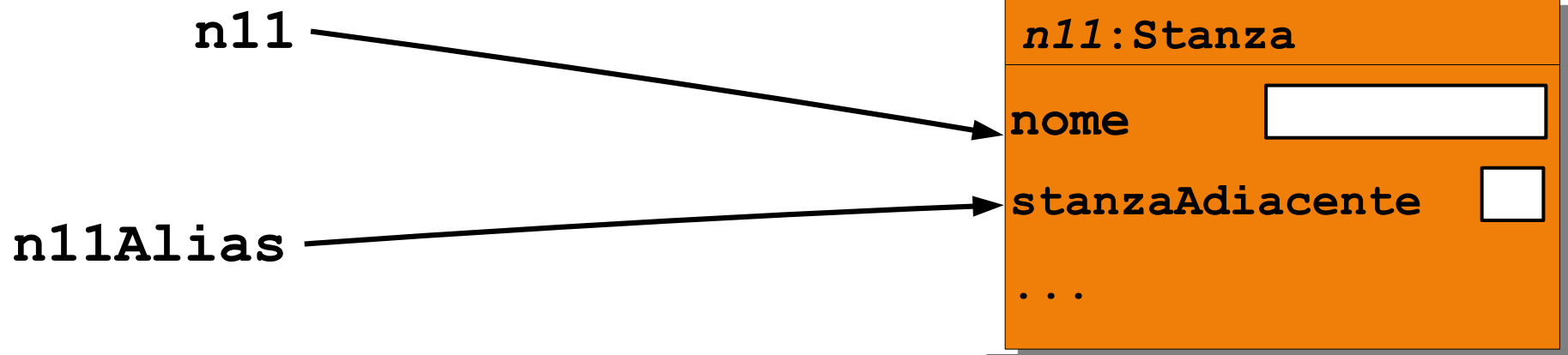
```
public static MainRiferimentiSideEffect {  
    public static void main(String[] args) {  
        Stanza n11 = new Stanza();  
        ───────────────────────────────────  
        Stanza n11Alias = n11;  
  
        n11.setNome("N11");  
        n11Alias.setNome("aula N11");  
  
        System.out.println(n11.getNome());  
    }  
}
```

n11



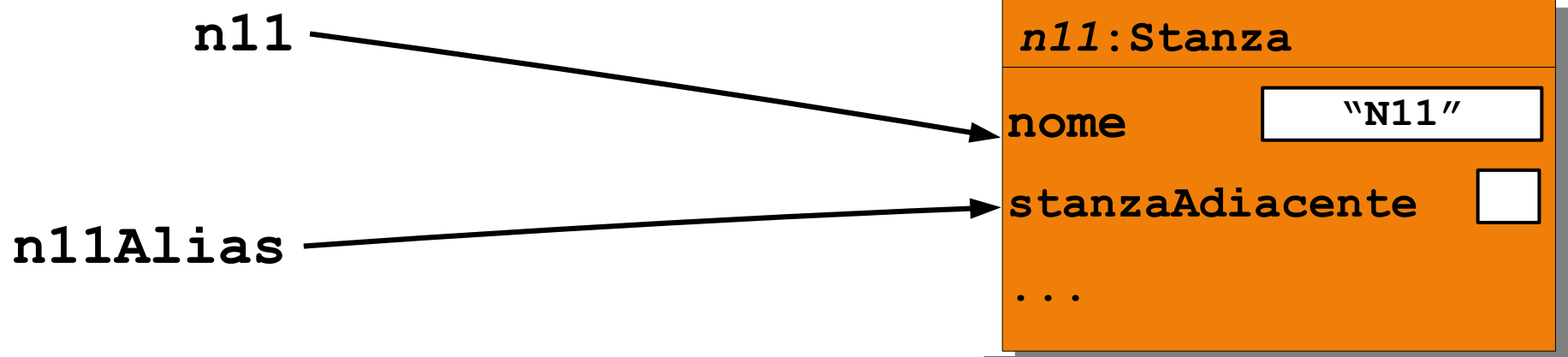
Più Riferimenti & Side-Effect (3)

```
public static MainRiferimentiSideEffect {  
    public static void main(String[] args) {  
        Stanza n11 = new Stanza();  
  
        Stanza n11Alias = n11;  
          
        n11.setNome("N11");  
        n11Alias.setNome("aula N11");  
  
        System.out.println(n11.getNome());  
    }  
}
```



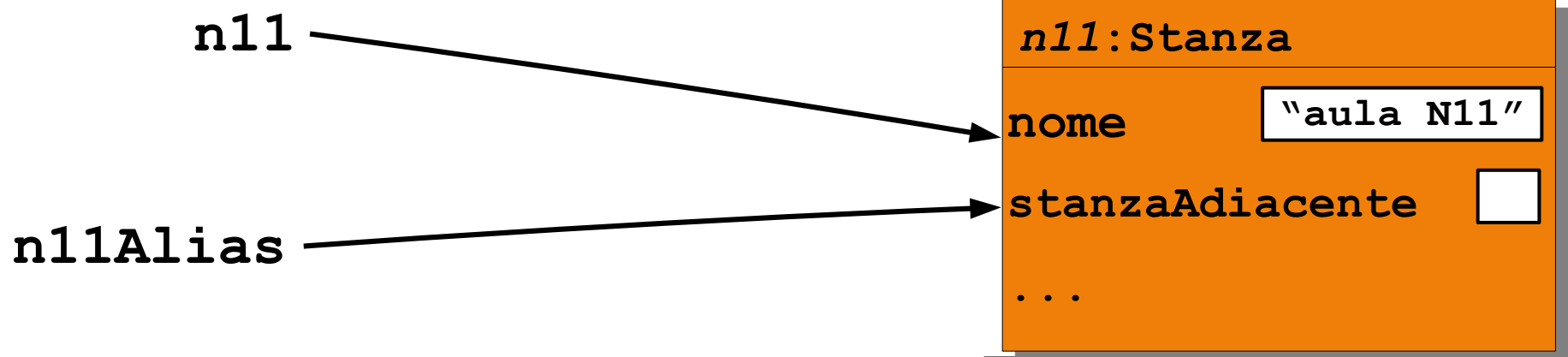
Più Riferimenti & Side-Effect (4)

```
public static MainRiferimentiSideEffect {  
    public static void main(String[] args) {  
        Stanza n11 = new Stanza();  
  
        Stanza n11Alias = n11;  
  
        ► n11.setName("N11"); ◀  
        n11Alias.setName("aula N11");  
  
        System.out.println(n11.getNome());  
    }  
}
```



Più Riferimenti & Side-Effect (5)

```
public static MainRiferimentiSideEffect {  
    public static void main(String[] args) {  
        Stanza n11 = new Stanza();  
  
        Stanza n11Alias = n11;  
  
        n11.setNome("N11");  
        n11Alias.setNome("aula N11");  
        System.out.println(n11.getNome());  
    }  
}
```



Più Riferimenti & Side-Effect (6)

- L'output è “aula N11”
- Sorprendente per chi aveva creato l'oggetto e lo aveva chiamato semplicemente “n11”?
- Questo tipo di comportamenti spesso vengono indicati con il nome di

Effetti Collaterali (Side-Effect)

- un'azione genera effetti visibili ben al di fuori dell'ambito in cui è avvenuta
- Sia la variabile **n11** che **n11Alias** fanno riferimento allo stesso oggetto
 - una modifica effettuata a tale oggetto tramite uno dei due riferimenti è visibile *anche* usando l'altro

Riferimenti per Valore (1)

- Quando una variabile contenente un riferimento è passata come argomento ad un metodo
 - il passaggio è per valore
 - viene copiato *il riferimento* contenuto nell'argomento
 - l'oggetto a cui fa riferimento *NON* viene copiato
- Tramite due copie distinte ma identiche dello stesso riferimento si finisce per accedere (e modificare) lo stesso oggetto
- Il cambio di stato operato ad un oggetto all'interno del *metodo invocato* è visibile (come effetto collaterale) anche al livello *metodo invocante* che effettuato la chiamata passando per valore un riferimento all'oggetto
 - Passando per valore riferimenti si possono quindi ottenere effetti simili a quelli che in altri linguaggi si ottengono mediante il cosiddetto *passaggio dei parametri per variabile*

Riferimenti per Valore (2)

```
typedef struct {  
    int base; ...  
} Rettangolo;
```

```
void setBase(struct Rettangolo *this, int b) {  
    this->base = b;  
}
```

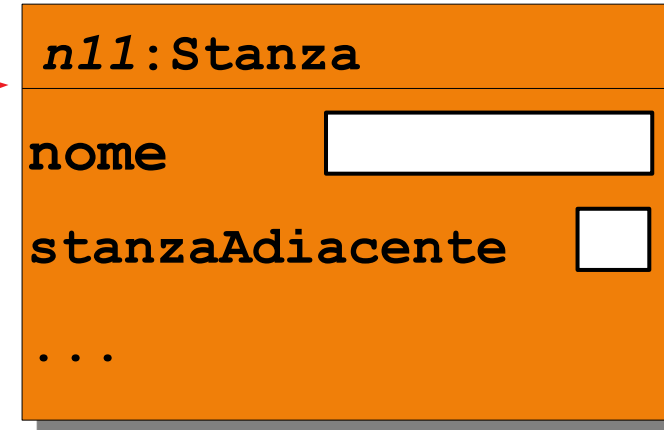
```
Rettangolo *r = malloc(sizeof(Rettangolo));
```

```
setBase(r, 15);
```

- Similarmente a quanto avviene in C, passando (per valore) il **puntatore** ad un'area di **memoria** allocata con **malloc**
 - è possibile cambiare il contenuto della **memoria** il cui **indirizzo** è fornito come argomento
 - non è possibile cambiare il contenuto della variabile che ospita tale **indirizzo** al momento dell'invocazione
- anche in Java, passando un **riferimento** ad un **oggetto**
 - è possibile cambiare lo stato dell'**oggetto** il cui **riferimento** è fornito come argomento
 - non è possibile cambiare il contenuto della variabile che ospita tale **riferimento** al momento dell'invocazione

Passaggio di Riferimenti (1)

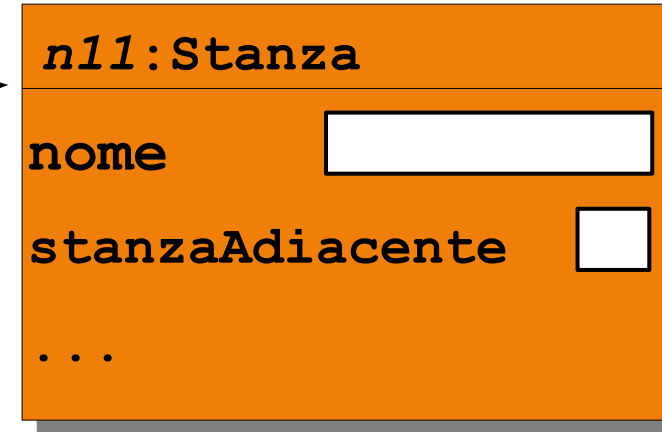
```
public class MainPassRef {  
    public static void main(String[] args) {  
        Stanza n11 = new Stanza();  
        Stanza n12 = new Stanza();  
        n11.setStanzaAdiacente(n12);  
    }  
}
```



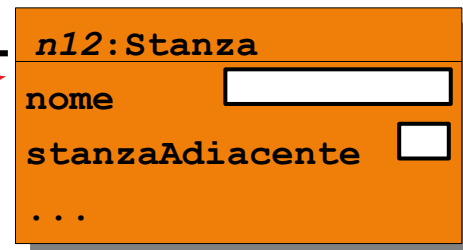
```
public class Stanza {  
    // ...  
    public void setStanzaAdiacente(Stanza stanza) {  
        this.stanzaAdiacente = stanza;  
    }  
}
```

Passaggio di Riferimenti (2)

```
public class MainPassRef {  
    public static void main(String[] args) {  
        Stanza n11 = new Stanza();  
  
        Stanza n12 = new Stanza();  
        n11.setStanzaAdiacente(n12);  
    }  
}
```

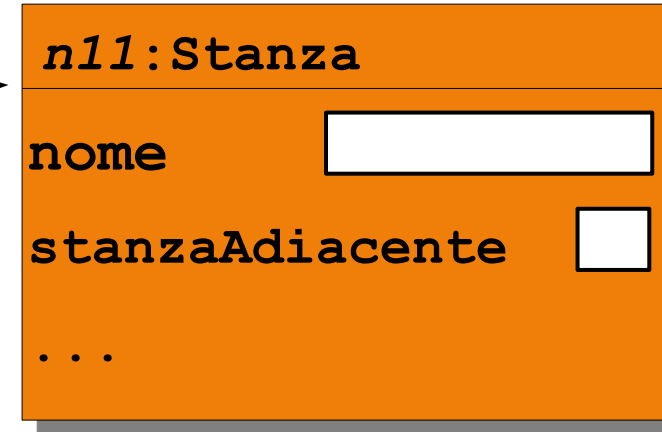


```
public class Stanza {  
    // ...  
  
    public void setStanzaAdiacente(Stanza stanza) {  
        this.stanzaAdiacente = stanza;  
    }  
}
```

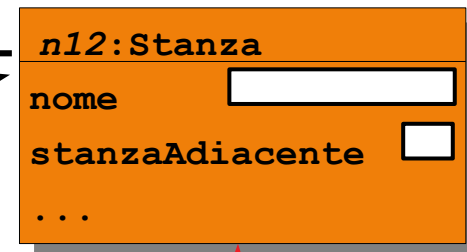


Passaggio di Riferimenti (3)

```
public class MainPassRef {  
    public static void main(String[] args) {  
        Stanza n11 = new Stanza();  
  
        Stanza n12 = new Stanza();  
        n11.setStanzaAdiacente(n12);  
    }  
}
```

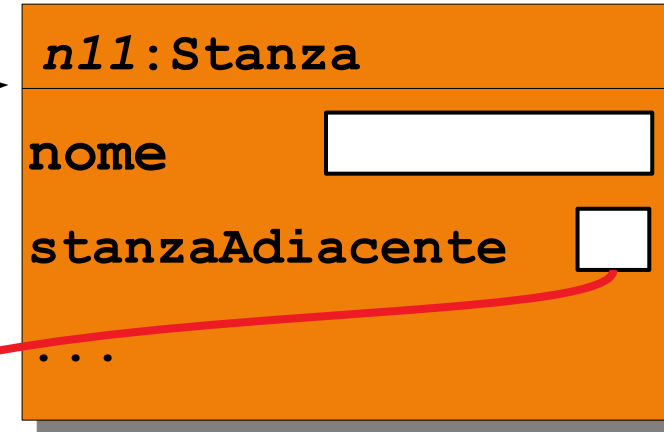


```
public class Stanza {  
    // ...  
    public void setStanzaAdiacente(Stanza stanza) {  
        this.stanzaAdiacente = stanza;  
    }  
}
```

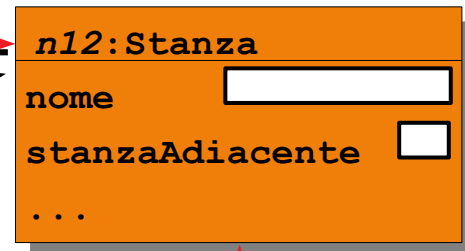


Passaggio di Riferimenti (4)

```
public class MainPassRef {  
    public static void main(String[] args) {  
        Stanza n11 = new Stanza();  
  
        Stanza n12 = new Stanza();  
        n11.setStanzaAdiacente(n12);  
    }  
}
```



```
public class Stanza {  
    // ...  
    public void setStanzaAdiacente(Stanza stanza) {  
        this.stanzaAdiacente = stanza;  
    }  
}
```



Riferimenti & Valore Restituito

- Quando un riferimento viene *restituito* da un metodo
 - Viene restituita una *copia del riferimento*
 - L'oggetto a cui si riferisce *NON* viene copiato

```
public class Stanza {  
    // ...  
    public Stanza getStanzaAdiacente() {  
        return stanzaAdiacente;  
    }  
}
```


Esercizio (*con Eclipse*)

- Assumiamo che:
 - la classe **Rettangolo** *non* disponga del metodo **sposta()**
 - la classe **Punto** invece disponga del metodo **trasla()**
- Trovare un modo alternativo per spostare gli oggetti **Rettangolo**
- Vediamo due soluzioni:
N.B. nessuna delle due è raccomandabile (>>)

Esercizio (2)

- Prima soluzione

```
public static void main(String[] args) {  
    Punto origine = new Punto();  
    origine.setX(0);  
    origine.setY(0);  
    Rettangolo rect = new Rettangolo();  
    rect.setVertice(origine);  
  
    origine.trasla(1, 1);  
}
```

- Se spostiamo l'oggetto istanza della classe **Punto** che utilizziamo come vertice dell'oggetto istanza della classe **Rettangolo**, spostiamo, *come effetto collaterale*, il rettangolo stesso

Esercizio (3)

- Seconda soluzione

```
public static void main(String[] args) {  
    Punto origine = new Punto();  
    origine.setX(0);  
    origine.setY(0);  
    Rettangolo rect = new Rettangolo();  
    rect.setVertice(origine);  
  
    Punto verticeRect = rect.getVertice();  
    verticeRect.trasla(1, 1);  
}
```

- Si ottiene *una copia del riferimento* all'oggetto istanza della classe **Punto** che figura come vertice dell'oggetto istanza della classe **Rettangolo** che spostato produce, *ancora come effetto collaterale*, lo spostamento del rettangolo stesso

Esercizio (4)

- Entrambe le soluzioni sono poco raccomandabili, perché non rendono affatto evidente la reale intenzione di spostare il rettangolo
- Nessuna *invocazione diretta* di un metodo della classe **Rettangolo** induce a pensare che lo si sta spostando
- Il rettangolo viene spostato solamente come risultato dell'effetto collaterale dello spostamento di un punto (il vertice) di cui conservava un riferimento
- E' preferibile dotare la classe **Rettangolo** di un apposito metodo **trasla()** la sua implementazione può fare affidamento ad un metodo (anche con lo stesso nome) di **Punto**
- Gli effetti collaterali risultano difficili da tracciare

Riferimento *Null*

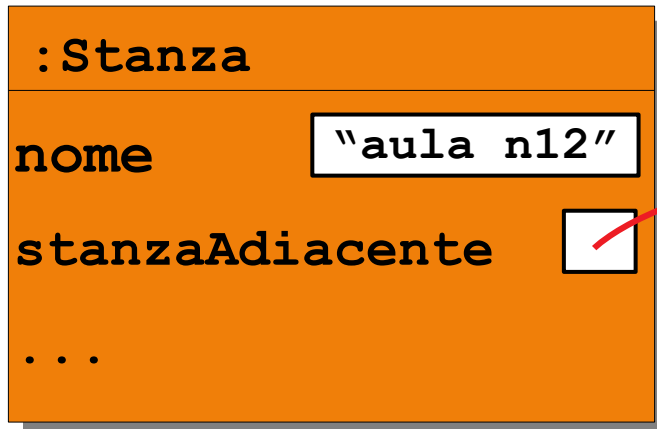
- In Java esiste un solo letterale di tipo riferimento ad oggetto: `null`
- Un valore speciale e distinto da tutti gli altri valori, il riferimento *null*
- Indica l'assenza di un reale riferimento ad un oggetto esistente
 - N.B. In Java non esiste alcuna relazione particolare tra il valore `null` e `0` (letterale di tipo `int`)
 - In C, la macro `NULL` è invece un *alias* per il valore `0`
- Un po' come già accadeva per i booleani, la tipizzazione Java è più *stringente*

Utilizzo del Riferimento Nullo

- Il riferimento nullo è utile
 - come valore speciale restituito da un metodo per segnalare un caso speciale. Ad es.:
`Persona cercata = rubrica.trova("Alice") ;`
restituisce `null` se non esiste alcuna persona di nome `"Alice"` nella rubrica
 - per fornire un valore di default a variabili che contengono riferimenti ad oggetti

Uso di null: Esempio

```
public class MainNull {  
    public static void main(String[] args) {  
        Stanza n12 = new Stanza();  
        n12.setNome("aula n12");  
  
        n12.setStanzaAdiacente( null );  
    }  
}
```



Si intende
rappresentare che
la stanza `n12` *NON*
possiede stanze
adiacenti

NullPointerException (1)

- Cosa succede se si invoca un metodo su un riferimento nullo?

```
public class MainNullPointerException {  
    public static void main(String[] args) {  
        Stanza n12 = new Stanza();  
        n12.setNome("aula n12");  
  
        n12.setStanzaAdiacente( null );  
  
        Stanza adiacenteN12 = n12.getStanzaAdiacente();  
  
        System.out.println(adiacenteN12.getNome());  
    }  
}
```


NullPointerException (2)

- `null` rappresenta l'assenza di un riferimento ad un oggetto
 - Si genera un errore a tempo di esecuzione, un'eccezione (*runtime-exception* >>)

NullPointerException

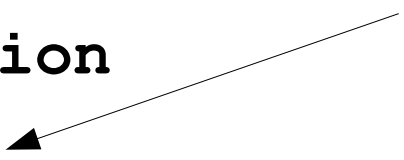
```
$ java MainNullPointerException
```

```
Exception in thread "main"
```

```
java.lang.NullPointerException
```

```
at MainNullPointerException.main(MainNullPointerException.java:10)
```

*Tipologia
eccezione*



Numero di linea del codice in cui si è verificata



NullPointerException: Diagnostica

- Java, ancora una volta, fornisce una diagnostica efficace
- Cosa accadrebbe utilizzando il linguaggio di programmazione C?

```
struct Punto {  
    int x;  
    int y;  
};  
int main() {  
    struct Punto *origine = NULL;  
    origine->x = 0;  
}
```

Segmentation fault!

Inizializzazione delle Variabili di Istanza e `null`

- Il compilatore forza l'inizializzazione di tutte le variabili di istanza
- Quelle dichiarate come contenenti un riferimento ad oggetto sono inizializzate a `null`

```
Rettangolo rect = new Rettangolo();
```

```
System.out.println(rect.getBase()); // 0
```

```
System.out.println(rect.getVertice()); // null
```

```
System.out.println(rect.getVertice().getX());
```

- ✓ Il valore restituito da `getVertice()` è `null`, invocando un metodo sul suo risultato si genera una `NullPointerException`

Evitare NullPointerException

- Se è noto che una funzione può ritornare `null` come valore speciale è necessario predisporre un controllo sul valore restituito

...

```
Persona cercata = rubrica.trova("Alice");  
if (cercata!=null)  
    System.out.println(cercata.getEta());  
else  
    System.out.println("non trovato");
```

- In C: `if (cercata!=0)`
 - In Java non compilerebbe

Campo d'Azione delle Variabili e dei Parametri

- Se il metodo `setX()` venisse così dichiarato?

```
public class Punto {  
    private int x;  
    private int y;  
    public void setX(int x) {  
        x = x;  
    }  
    ...  
}
```

```
Punto unoUno = new Punto();  
unoUno.setX(1);  
System.out.println(unoUno.getX()); // Stampa 0
```

Shadowing

- Si è verificato il cosiddetto *shadowing*:
 - Il parametro formale **x** ha lo stesso nome della variabile di istanza **x**
 - Il parametro formale ha però uno scope (>>) più ristretto e quindi ha precedenza
 - Nel contesto del corpo del metodo, l'identificatore '**x**' viene considerato un riferimento al parametro formale (e *non* alla var. di istanza)
 - Si dice anche che il parametro formale offusca ("fa ombra") la variabile di istanza
 - **x = x;** è un'espressione che assegna al parametro formale **x** il suo stesso valore (inutile!)
- ✓ Alcuni IDE moderni (come Eclipse) possono essere configurati per segnalare il problema

La Parola Chiave `this` (1)

- All'interno di ogni metodo è possibile ottenere un riferimento all'oggetto *corrente*
- La parola chiave `this`
 - Conserva un riferimento all'oggetto sul quale il metodo in corso di esecuzione è stato invocato
 - Tramite questo riferimento è quindi possibile:
 - modificare le variabili di istanza dell'oggetto
 - fare invocazioni di metodo nidificate sullo stesso oggetto
 - passare un riferimento all'oggetto corrente come argomento di altre invocazioni di metodo...

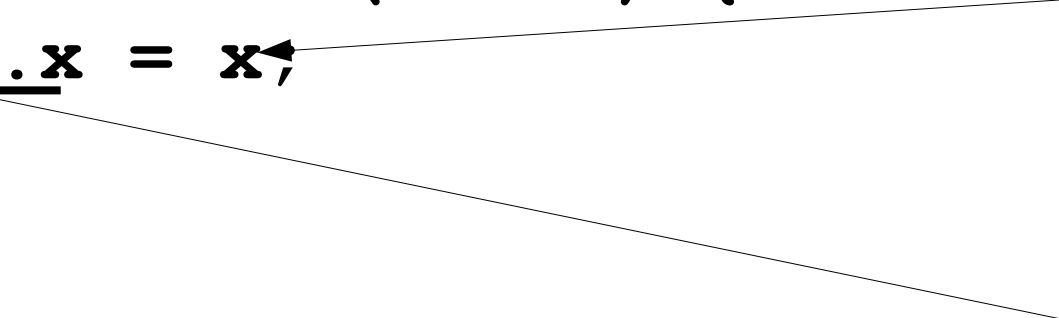
La Parola Chiave `this` (2)

- Si risolve anche il problema dello *shadowing*

```
...  
public void setX(int x) {  
    this.x = x;  
}  
...
```

Parametro

Variable di istanza



```
Punto unoUno = new Punto();  
unoUno.setX(1);
```

```
System.out.println(unoUno.getX()); // Stampa 1
```

- All'interno del corpo del metodo `setX()`, `this` è un riferimento allo stesso oggetto a cui si riferisce anche `unoUno`

this in C?

- Talvolta può far comodo pensare a **this** come ad un parametro aggiuntivo passato automaticamente (ed implicitamente) ad ogni metodo
- Ad esempio il *metodo* `setX()` verrebbe tradotto in C con la seguente *funzione*

```
void setX(struct Punto *this, int x) {  
    this -> x = x;           // Codice C  
}
```

- Quindi l'invocazione di `unoUno.setX(1)`; diverrebbe:

```
struct Punto unoUno;    // Codice C  
setX(&unoUno, 1);      // Codice C
```

Accedere le Variabili di Istanza con `this`

- `this` può essere usato per accedere alle variabili di istanza ma può anche essere omesso in assenza di ambiguità

```
...  
    public int getX() {  
        return this.x;  
    }
```

...

- Equivale a

```
...  
    public int getX() {  
        return x;  
    }
```

...

- Il compilatore risolve l'identificatore `x` come variabile di istanza dell'oggetto su cui il metodo è stato invocato
- In un certo senso, è come se aggiungesse `this.` automaticamente

this: Convenzione di Stile

- Adottiamo comunque la convenzione di usare *sempre e comunque* **this** per referenziare variabili di istanza
- Con le seguenti motivazioni:
 - si evita lo *shadowing*
 - si favorisce la leggibilità del codice
 - ✓ si favorisce l'apprendimento di questi concetti di base

Invocare Metodi Mediante `this` (1)

- La parola chiave `this` può essere usata per invocare metodi sullo stesso oggetto su cui il metodo corrente è stato invocato
 - Se `this` viene omissso il compilatore lo considera comunque presente
- Ad esempio è possibile scrivere il metodo `setXY()` usando gli altri due metodi della classe `Punto`: `setX()` e `setY()`

Invocare Metodi Mediante `this` (2)

```
public void setXY(int x, int y) {  
    this.setX(x) ;  
    this.setY(y) ;  
}
```

- Anche per *alcune* invocazioni di metodi è consigliabile usare `this` per aumentare la leggibilità
- Ad esempio quando si vuole evidenziare l'utilizzo di altri metodi della stessa classe nella scrittura di un primo metodo (passo *top-down*)

Esercizio

Fare le verifiche disponibili sul sito del corso:

- `Studiante.java`
- `Tesi.java`
- `Sommatore.java`

Programmazione Orientata agli Oggetti

Gestione della Memoria

Sommario

- Allocazione di Oggetti
 - L'operatore **new**
 - Costruttori
 - Costruttore di default
- Stack e Heap
 - Stack e Record di Attivazione
 - Stack-Overflow
 - Heap
- Equivalenza di oggetti ed identità dei riferimenti
- Riferimenti in Java vs Puntatori in C
- Gestione della Memoria
 - *Garbage Collection*

Operatore new (1)

- La creazione di oggetti in Java è permessa dall'operatore **new**
 - richiede la specifica della classe di cui si vuole creare una nuova *istanza*
 - ovvero, più precisamente, di uno dei *costruttori* di tale classe
 - restituisce un *riferimento* all'oggetto appena creato

- `Punto origine = new Punto();`

```
public class Punto {  
    private int x;  
    private int y;  
    public Punto() {  
        this.x = 0;  
        this.y = 0;  
    }  
    ...  
}
```

*Definizione del Costruttore
(sintassi: stesso nome della classe)*

Corpo del costruttore {...}

File `Punto.java`

- Attenzione: `origine` contiene un *riferimento*; NON contiene l'oggetto appena creato

Operatore new (2)

- Solo la JVM può creare oggetti
 - l'operatore `new` consente di chiederne i servizi
- Per inizializzare nuovi oggetti, sinora:
 - invocazione del cosiddetto *costruttore senza argomenti*
`new Punto()` ;
 - invocazione dei vari metodi *setter*
- Ad esempio:
`Punto zeroUno = new Punto();`
`zeroUno.setX(0);`
`zeroUno.setY(1);`
- Si possono definire *costruttori* che ricevono parametri

Costruttori

- Ogni classe ha *sempre* (almeno) un costruttore che viene eseguito ogni volta che un oggetto di tale classe viene creato
 - ✓ altrimenti non sarebbe possibile creare oggetti
- Il corpo del costruttore costruisce lo stato iniziale di un nuovo oggetto
 - riceve informazioni sullo stato iniziale da costruire tramite i parametri
 - fissa i valori iniziali delle variabili di istanza

Sintassi dei Costruttori

- I costruttori si denotano con lo stesso nome della classe in cui compaiono
- A differenza dei metodi, NON possono né devono specificare il tipo del valore restituito
 - N.B. NON è nemmeno possibile usare `void`

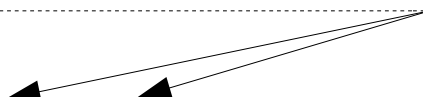
```
public class Punto {  
    private int x;  
    private int y;  
    public Punto(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
}
```

- ✓ N.B. I costruttori NON sono niente affatto metodi
 - Le differenze non sono limitate al solo tipo restituito

Invocazione dei Costruttori

- Attraverso l'operatore **new** è possibile invocare il costruttore e specificare degli argomenti
- Ad esempio:

Parametri attuali del costruttore



```
Punto origine = new Punto(0, 0);
```

```
System.out.println(origine.getX()); // Stampa 0
```

Oggetti in Corso di Costruzione

- Il corpo del costruttore è adibito alla costruzione dello stato iniziale dell'oggetto creato
- Dal momento dell'invocazione dell'operatore per la creazione di un nuovo oggetto
 - `new Punto()`
al momento in cui l'esecuzione del corpo del costruttore è completamente terminata, l'inizializzazione dello stato non è completa
- Durante la costruzione l'oggetto transita per una serie di stati intermedi, ovvero è *inconsistente*
 - è *buona regola* non farsi mai “sfuggire” riferimenti ad oggetti in uno stato potenzialmente inconsistente (>>)

Esempio (*con Eclipse*)

- Implementare il costruttore della classe **Rettangolo**
 - I parametri servono a specificare
 - vertice
 - base
 - altezza
- dell'oggetto appena creato

Esempio (2)

```
public class Rettangolo {  
  
    private Punto vertice;  
    private int base;  
    private int altezza;  
  
    public Rettangolo(Punto v, int base, int altezza) {  
        this.vertice = v;  
        this.base = base;  
        this.altezza = altezza;  
    }  
    // soliti getter  
}
```


Costruttore di Default (1)

- Tutte le classi devono avere almeno un costruttore, *sempre*
- Se non viene esplicitamente dichiarato, ne viene aggiunto uno *implicitamente*
- E' un costruttore senza parametri (*costruttore no-args*)
- Anche senza dichiarare esplicitamente il costruttore senza argomenti di **Punto**:
`Punto punto = new Punto() ; //COMPILA`

Costruttore di Default (2)

- Appena viene definito esplicitamente un costruttore, il *costruttore no-args* non viene più generato automaticamente e non è più possibile invocarlo
- Ad esempio:
 - *Dopo aver dichiarato un primo costruttore (con parametri) nella classe Rettangolo:*

```
public class MainNoArgs {  
    public static void main(String[] args) {  
        Rettangolo rect = new Rettangolo(); //NON COMPILA  
    }  
}
```

Exception in thread "main" java.lang.Error: Unresolved compilation problem:

The constructor Rettangolo() is undefined
at MainNoArgs.main(MainNoArgs.java:4)

Variabili di Istanza: Valore di default e `null`

- È fortemente consigliato l'uso di un costruttore che inizializzi le variabili di istanza di tipo riferimento
 - Onde evitare `NullPointerException`
 - Se non specificato altrimenti, le variabili di istanza di tipo riferimento sono inizializzate a `null`
- Altro possibile costruttore per la classe `Rettangolo`:

...

```
public Rettangolo(int base, int altezza) {  
    this.vertice = new Punto(0, 0);  
    this.base = base;  
    this.altezza = altezza;  
}
```

...

Costruttori Alternativi

- Se il vertice non viene specificato, questo costruttore suppone che sia nell'origine
 - è poi possibile usare la variabile di istanza `vertice` senza sollevare `NullPointerException`
- Spesso si definiscono molteplici costruttori con diversi parametri
 - Utili per costruire oggetti
 - a partire da informazioni diverse. Ad es. per i rettangoli
 - base, altezza, vertice alto a sx
 - vertice alto a sx, vertice basso a dx
 - senza essere costretti a specificare tutti i parametri
 - Per ora: unico costruttore, il più generico possibile (>>)

Gestione della Memoria: Stack e Heap

- Durante l'esecuzione, un programma ha accesso ad almeno due distinte aree di memoria
- *Stack*
 - Per la memorizzazione delle informazioni necessarie per l'esecuzione dei metodi, anche nidificati
 - Conserva:
 - Stato dell'esecuzione
 - Variabili locali e loro valore
- *Heap*
 - Per la memorizzazione di oggetti creati tramite l'operatore `new`
 - Conserva:
 - Oggetti e loro stato

Stack

- Area di memoria assegnata dalla JVM all'inizio dell'esecuzione
 - dimensione massima prefissata (qualche *mb*)
- Adibita a conservare quanto serve per mantenere lo stato dell'esecuzione (tranne lo stato degli oggetti >>)
 - È una struttura dati gestita secondo una disciplina LIFO
 - *Last In First Out*
 - Per gestire le invocazioni di metodo, comunque nidificate, a cominciare dal `main()`
- Contiene i *Record di Attivazione (RDA)*
 - Struttura dati che contiene tutte le informazioni necessarie all'invocazione ed all'esecuzione dei metodi

Record di Attivazione

- Ogni volta che un metodo viene invocato, il corrispondente RDA viene creato ed inserito in cima allo stack
- Quando il metodo termina, il suo RDA viene rimosso
- La gestione dello stack avviene in modo automatico e trasparente da parte della JVM

Metodi e Record di Attivazione (1)

```
public static void main(String[] args) { ←
```

```
...
```

```
    oggetto1.metodo1();
```

```
}
```

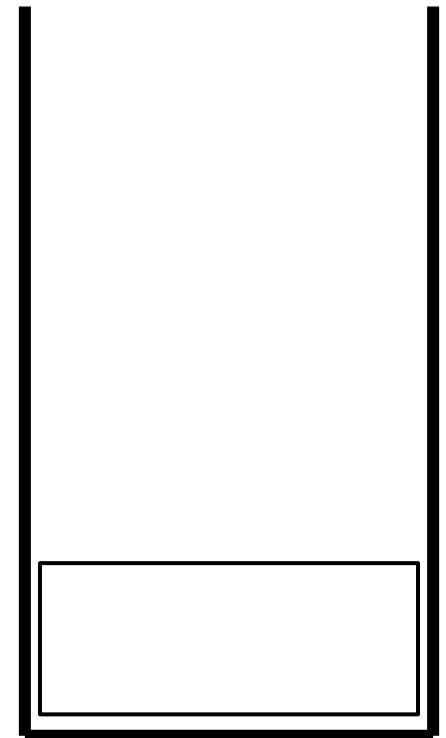
```
// ... (nella classe di "oggetto1")
```

```
public void metodo1() {  
    oggetto2.metodo2();
```

```
}
```

```
...
```

RDA main



Stack

Metodi e Record di Attivazione (2)

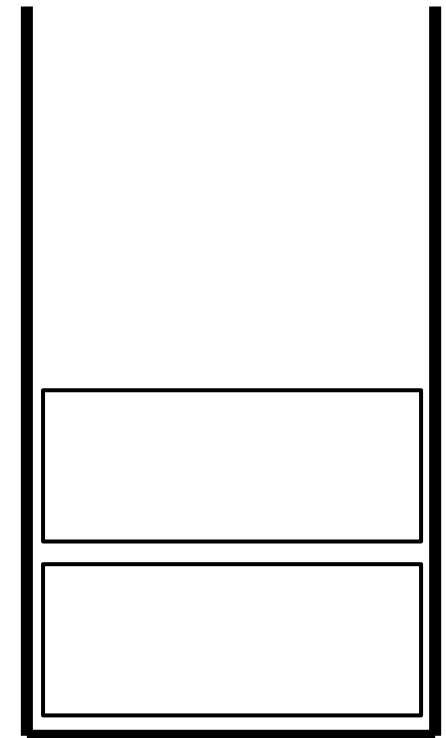
```
public static void main(String[] args) {  
    ...  
    oggetto1.metodo1(); ←  
}
```

// ... (nella classe di "oggetto1")

```
public void metodo1() {  
    oggetto2.metodo2();  
}  
...
```

RDA metodo1

RDA main



Stack

Metodi e Record di Attivazione (3)

```
public static void main(String[] args) {  
    ...  
    oggetto1.metodo1();  
}
```

// ... (nella classe di "oggetto1")

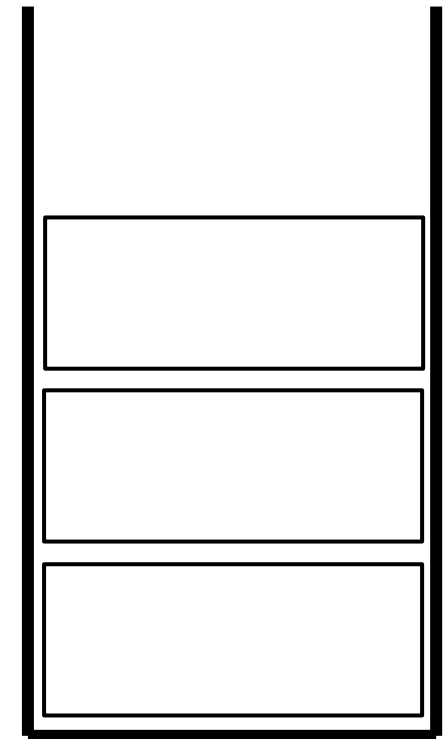
```
public void metodo1() {  
    oggetto2.metodo2();  
}
```

...

RDA metodo2

RDA metodo1

RDA main



Stack

Metodi e Record di Attivazione (4)

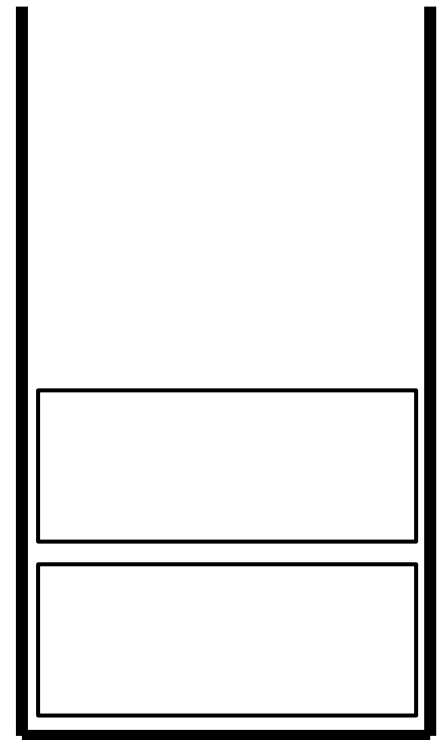
```
public static void main(String[] args) {  
    ...  
    oggetto1.metodo1();  
}
```

// ... (nella classe di "oggetto1")

```
public void metodo1() {  
    oggetto2.metodo2();  
    ←  
}  
...
```

RDA metodo1

RDA main



Stack

Metodi e Record di Attivazione (5)

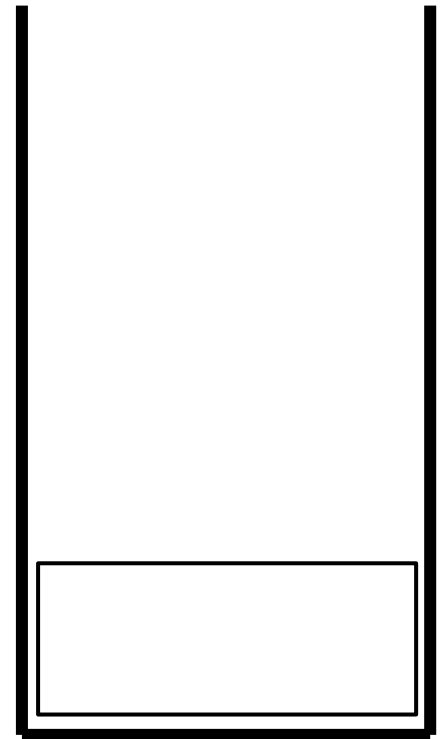
```
public static void main(String[] args) {  
    ...  
    oggetto1.metodo1();  
}
```



// ... (nella classe di "oggetto1")

```
public void metodo1() {  
    oggetto2.metodo2();  
}  
...
```

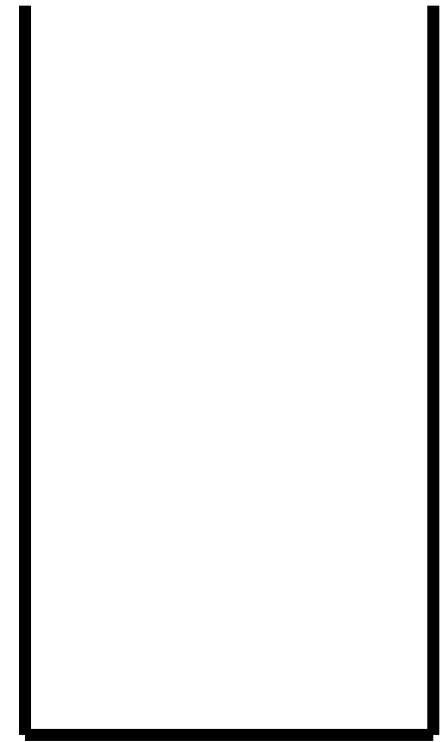
RDA main



Stack

Metodi e Record di Attivazione (6)

```
public static void main(String[] args) {  
    ...  
    oggetto1.metodo1();  
}  
// ... (nella classe di "oggetto1")  
  
public void metodo1() {  
    oggetto2.metodo2();  
}  
...
```



Stack

Contenuto di un *RDA*

- Un *Record di Attivazione* contiene:
 - parametri attuali
 - eventuale riferimento all'oggetto corrente per invocazione (`this`)
 - variabili locali
 - valore di ritorno del metodo (se non è `void`)
 - il punto di ritorno dell'invocazione di metodo: l'istruzione successiva all'invocazione nel metodo chiamante

Variabili Locali dentro il RDA

- Il record di attivazione ospita anche le variabili locali di ogni metodo
- Lo scope delle variabili locali è il metodo in cui sono definite, ed il ciclo di vita è chiaramente quello dell'invocazione di metodo
- L'identificatore di una variabile locale è un *alias* (gestito dal compilatore e dall'ambiente di esecuzione) di un indirizzo di memoria relativo (sullo stack) in cui è conservato il suo valore
 - Attenzione: Nulla a che fare con i riferimenti
 - Gli identificatori (ad es. *origine*) sono decisamente più facili da ricordare rispetto ad un indirizzo di memoria!

Stack & Variabili Locali (1)

```
public static void main(String[] args) { ←
```

```
...
```

```
    oggetto1.metodo1();
```

```
}
```

```
// ... (nella classe di "oggetto1")
```

```
public void metodo1() {
```

```
    int contat = 3;
```

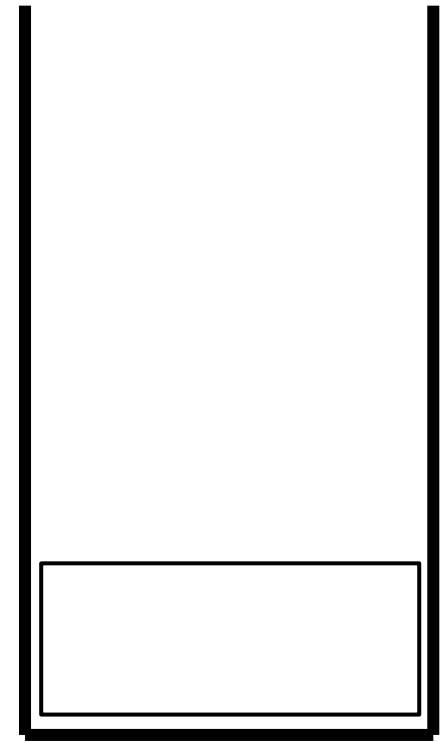
```
    oggetto2.metodo2();
```

```
    contat = 5;
```

```
}
```

```
...
```

RDA main



Stack

Stack & Variabili Locali (2)

```
public static void main(String[] args) {
```

```
...
```

```
    oggetto1.metodo1();
```



```
}
```

```
// ... (nella classe di "oggetto1")
```

```
public void metodo1() {
```

```
    int contat = 3;
```

```
    oggetto2.metodo2();
```

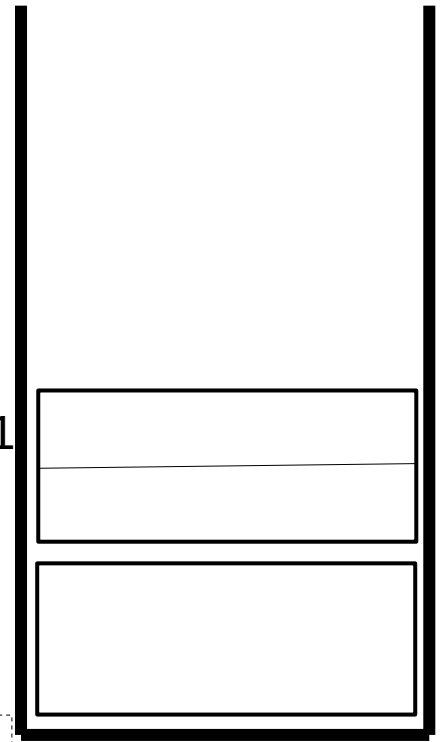
```
    contat = 5;
```

```
}
```

```
...
```

RDA metodo1
contat: 0x34fa09b1

RDA main



contat è in realtà un alias per un indirizzo di memoria relativo nel *record di attivazione*

Stack

Stack & Variabili Locali (3)

```
public static void main(String[] args) {  
    ...  
    oggetto1.metodo1();  
}
```

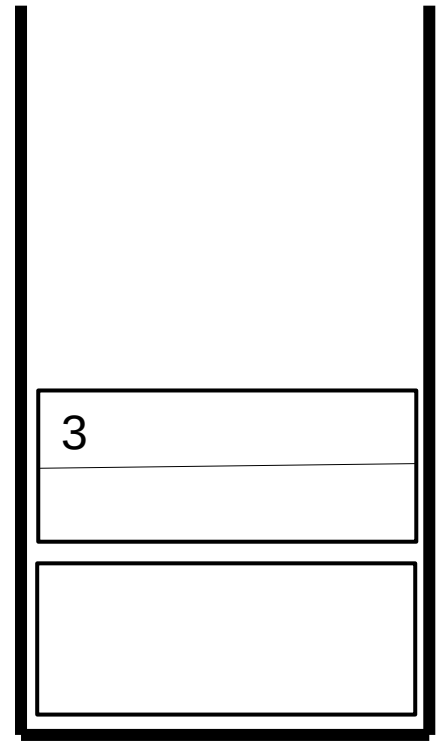
// ... (nella classe di "oggetto1")

```
public void metodo1() {  
    int contat = 3;  
    oggetto2.metodo2();  
    contat = 5;  
}  
...
```



RDA metodo1
contat:

RDA main



Stack

Stack & Variabili Locali (4)

```
public static void main(String[] args) {  
    ...  
    oggetto1.metodo1();  
}
```

// ... (nella classe di "oggetto1")

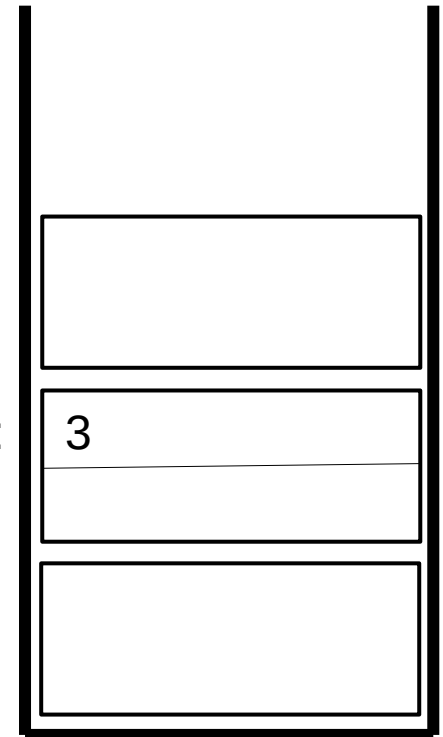
```
public void metodo1() {  
    int contat = 3;  
    oggetto2.metodo2();  
    contat = 5;  
}
```

...

RDA metodo2

RDA metodo1
contat:

RDA main



Stack

Stack & Variabili Locali (5)

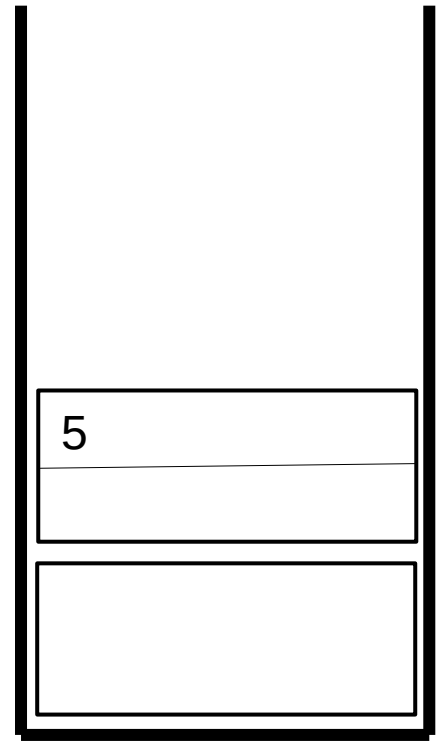
```
public static void main(String[] args) {  
    ...  
    oggetto1.metodo1();  
}
```

// ... (nella classe di "oggetto1")

```
public void metodo1() {  
    int contat = 3;  
    oggetto2.metodo2();  
    contat = 5;  
}  
...
```

RDA metodo1
contat:

RDA main

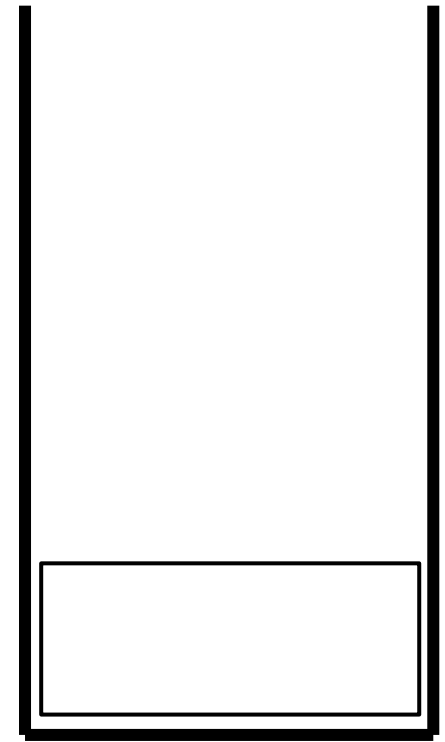


Stack

Stack & Variabili Locali (6)

```
public static void main(String[] args) {  
    ...  
    oggetto1.metodo1();  
}  
  
// ... (nella classe di "oggetto1")  
  
public void metodo1() {  
    int contat = 3;  
    oggetto2.metodo2();  
    contat = 5;  
} ←  
...
```

RDA main



Stack

Stack & Variabili Locali (7)

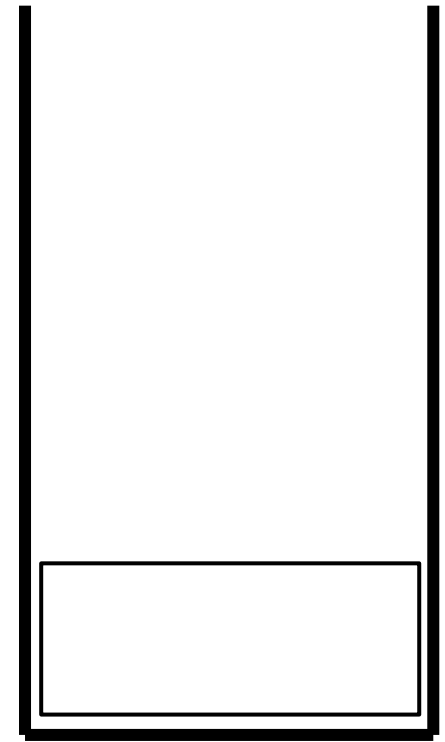
```
public static void main(String[] args) {  
    ...  
    oggetto1.metodo1();  
}
```



// ... (nella classe di "oggetto1")

```
public void metodo1() {  
    int contat = 3;  
    oggetto2.metodo2();  
    contat = 5;  
}  
...
```

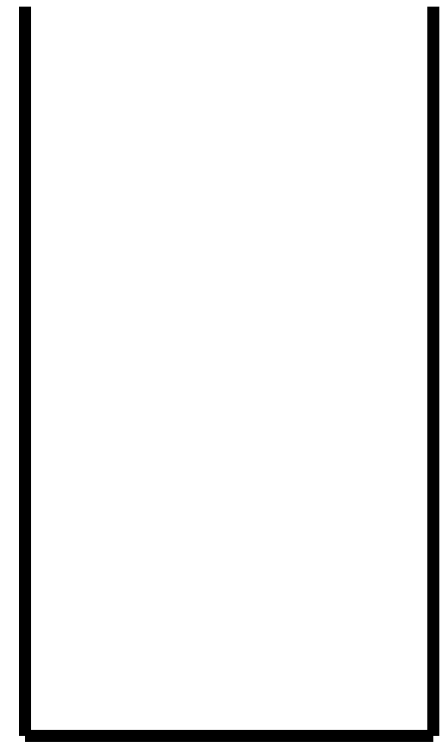
RDA main



Stack

Stack & Variabili Locali (8)

```
public static void main(String[] args) {  
    ...  
    oggetto1.metodo1();  
}  
←  
// ... (nella classe di "oggetto1")  
  
public void metodo1() {  
    int contat = 3;  
    oggetto2.metodo2();  
    contat = 5;  
}  
...
```



Stack

Scope delle Variabili Locali

- Il primo record di attivazione è sempre quello del metodo `main()`
- Quando un *RDA* viene rimosso dallo stack le variabili locali e i parametri non sono più utilizzabili
 - Lo scope di queste informazioni è limitato al solo metodo di appartenenza

Stack Overflow (1)

- Lo stack ha una dimensione limitata
 - non può contenere troppi RDA
- Ad esempio

```
public class SommatoreRicorsivo {  
    public int sommaDaZeroA(int n) {  
        return n + sommaDaZeroA(n-1);  
    }  
}
```

// ERRORE! manca il caso base!

Stack Overflow (2)

- L'esecuzione del metodo `sommaDaZeroA(1)` genera il seguente errore (indipendentemente dal suo argomento)

Exception in thread "main"

`java.lang.StackOverflowError`

```
at SommatoreRicorsivo.sommaDaZeroA(SommatoreRicorsivo.java:3)
at SommatoreRicorsivo.sommaDaZeroA(SommatoreRicorsivo.java:3)
at SommatoreRicorsivo.sommaDaZeroA(SommatoreRicorsivo.java:3)
at SommatoreRicorsivo.sommaDaZeroA(SommatoreRicorsivo.java:3)
...
at SommatoreRicorsivo.sommaDaZeroA(SommatoreRicorsivo.java:3)
at SommatoreRicorsivo.sommaDaZeroA(SommatoreRicorsivo.java:3)
at SommatoreRicorsivo.sommaDaZeroA(SommatoreRicorsivo.java:3)
at SommatoreRicorsivo.sommaDaZeroA(SommatoreRicorsivo.java:3)
at SommatoreRicorsivo.sommaDaZeroA(SommatoreRicorsivo.java:3)
at SommatoreRicorsivo.sommaDaZeroA(SommatoreRicorsivo.java:3)
at SommatoreRicorsivo.sommaDaZeroA(SommatoreRicorsivo.java:3)
...
```

N.B. La diagnostica riporta nomi delle classi e dei metodi i cui RDA sono nello stack al momento del *trabocco*

Heap

- Tutti gli oggetti creati con l'operatore **new** possiedono uno stato che viene conservato in un'area di memoria denominata *Heap*
 - Letteralmente “*mucchio*”, si tratta di un'area di memoria assegnata dalla JVM
 - NON è gestito come lo stack (LIFO)
 - la sua dimensione può crescere e decrescere dinamicamente anche durante l'esecuzione
 - gli oggetti sono creati e deallocati in ordine sparso
 - spesso e volentieri l'occupazione dell'heap risulta molto frammentata

new VS malloc

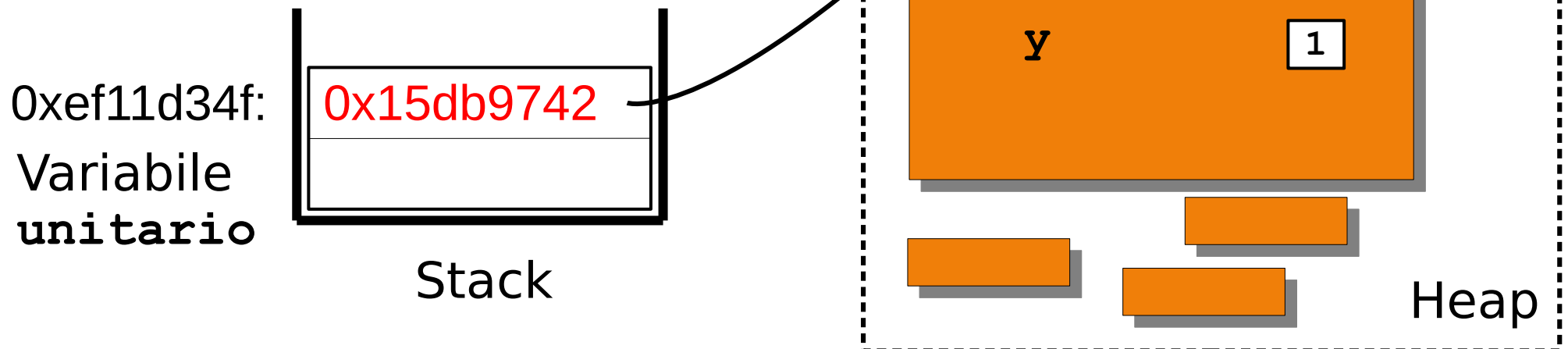
- **new** sta ad una classe (quasi) come **malloc** sta ad una **struct**
 - **malloc**
 - **alloca memoria nello heap**
 - **restituisce l'indirizzo di tale area di memoria**
 - **new**
 - **alloca memoria nello heap**
 - **restituisce il riferimento all'oggetto appena allocato**
 - **invoca un costruttore**

Heap e Stack (1)

- Il valore restituito dall'operatore `new` è il riferimento all'oggetto appena creato

```
Punto unitario = new Punto(1, 1)
```

- La variabile locale `unitario` contiene il riferimento all'oggetto creato, *NON* l'oggetto stesso
- `unitario` è un alias per l'indirizzo `0xef11d34f`



Heap e Stack (2)

- È possibile stampare anche il contenuto di una variabile che contiene un riferimento ad oggetto usando il metodo `println()`

```
System.out.println(unitario);
```

- `println()` stampa una stringa che dipende solamente dall'indirizzo di memoria e dalla classe di appartenenza

Stampa: **Punto@15db9742**

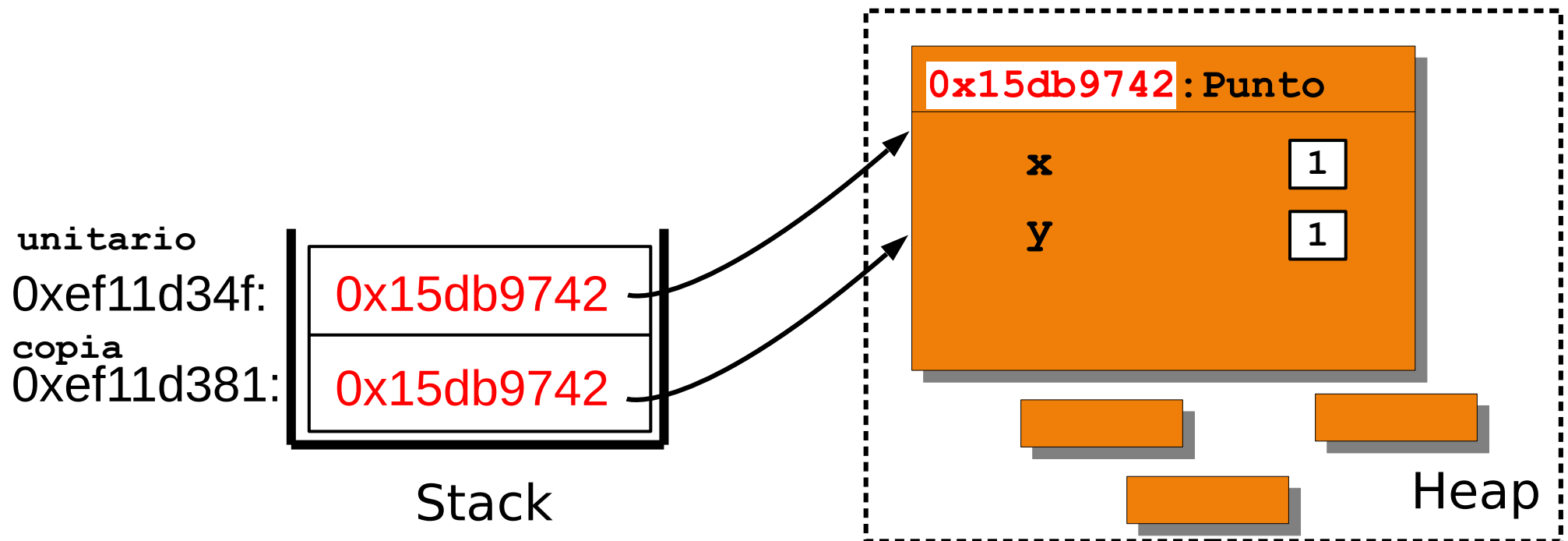
(è possibile alterare questo comportamento>>)

Assegnazione di Riferimenti (1)

- L'assegnazione di una variabile che contiene un riferimento copia **il riferimento**
 - **NON** si crea un nuovo oggetto

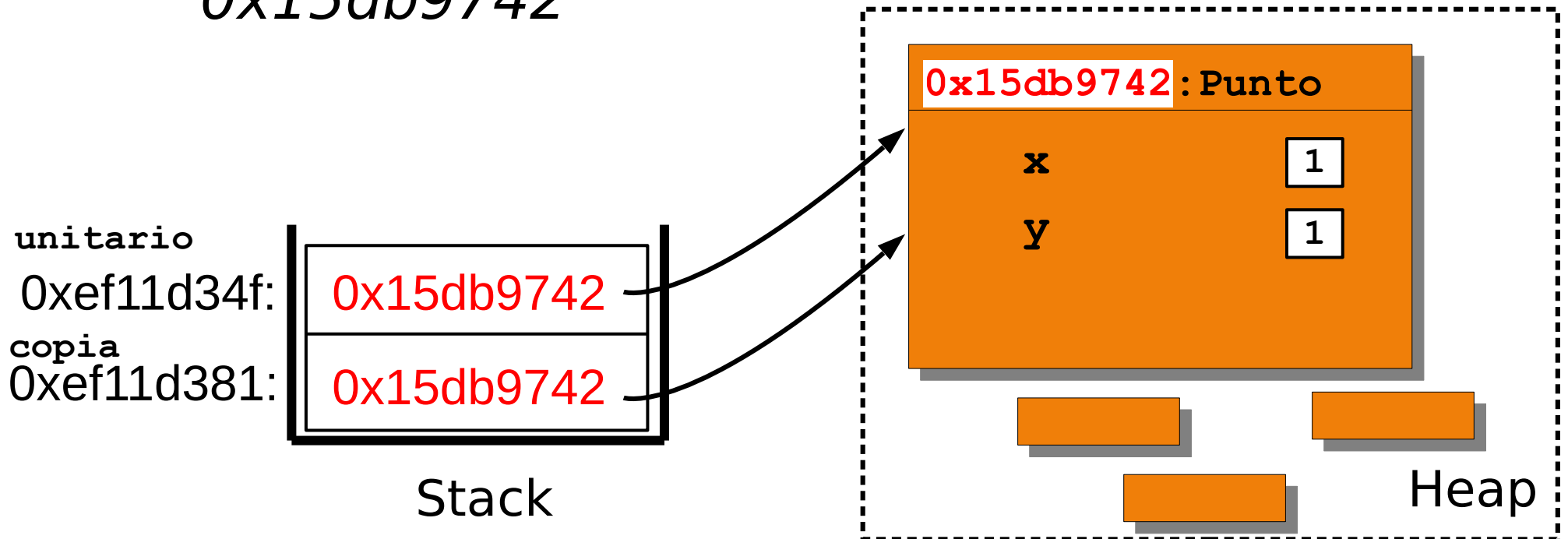
```
Punto unitario = new Punto(1, 1);
```

```
Punto copia = unitario;
```



Assegnazione di Riferimenti (2)

- **unitario** e **copia** sono due variabili locali distinte
- Possiedono valori in posizioni distinte sullo *Stack*...
 - *0xef11d34f* e *0xef11d381*
- ...ma fanno riferimento allo stesso oggetto dentro l'*heap*, quello di indirizzo
 - *0x15db9742*



Molteplici Riferimenti Stesso Oggetto

- Se due variabili contengono un riferimento al medesimo oggetto, ogni modifica operata ad un oggetto tramite un riferimento è anche visibile tramite l'altro riferimento
- Ad esempio:

```
Punto unitario = new Punto(1, 1);  
Punto copia = unitario;
```

```
System.out.println(copia.getX()); // Stampa 1
```

```
copia.setX(2);
```

```
System.out.println(unitario.getX()); // Stampa 2
```

Side-Effect e Metodi

- I metodi che aggiornano lo stato di oggetti di cui ricevono un riferimento (tra i parametri) producono effetti collaterali (*side-effect*)

```
public class ModificatoreDiPunti {  
    public void azzera(Punto p) {  
        p.setX(0); p.setY(0);  
    }  
}  
  
...  
  
public static void main(String[] args) {  
    ModificatoreDiPunti m = new ModificatoreDiPunti();  
    Punto unitario = new Punto(1, 1);  
    m.azzera(unitario);  
    System.out.println(unitario.getX()); // Stampa 0  
}
```

Equivalenza tra *Oggetti* vs Identicità dei *Riferimenti*

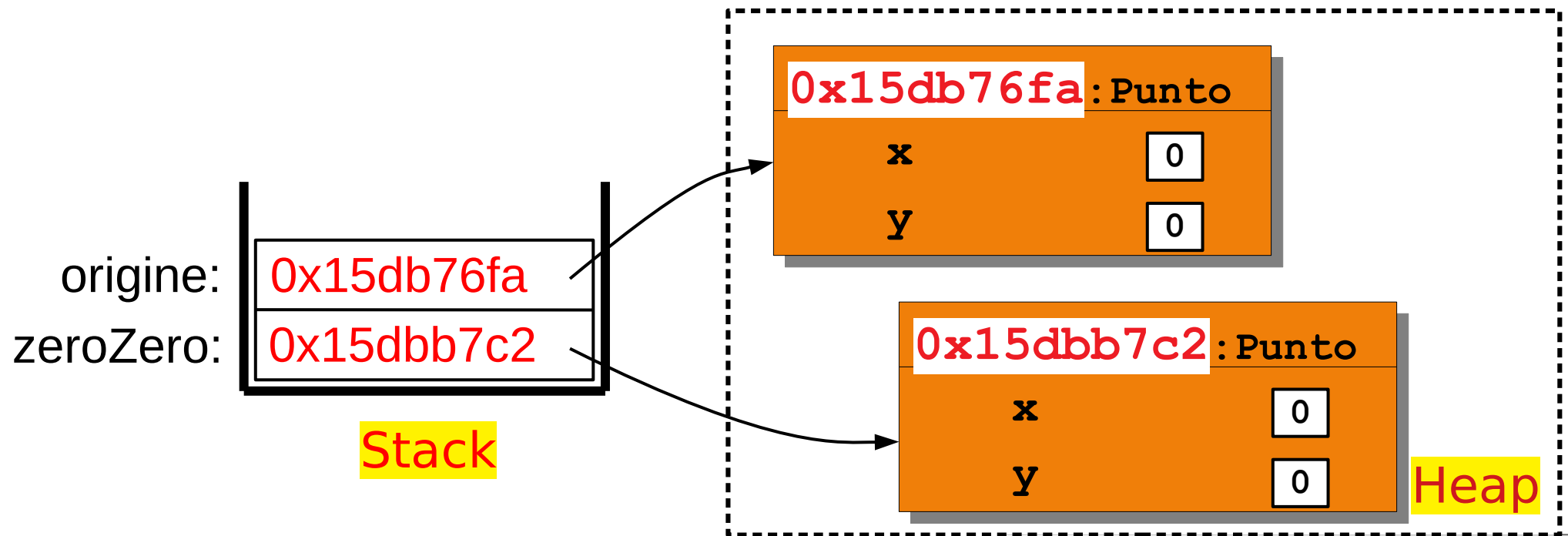
```
Punto origine = new Punto(0, 0);  
Punto zeroZero = new Punto(0, 0);
```

```
if (origine == zeroZero)  
    System.out.println("uguali");  
else  
    System.out.println("diversi");
```

- Stamperà "uguali" o "diversi"?

Identicità dei Riferimenti (1)

- L'operatore **==** verifica se il contenuto delle due variabili è identico
 - **origine** e **zeroZero** contengono però riferimenti diversi verso due oggetti distinti (con diversi indirizzi di memoria)
 - Sono quindi considerati diversi



Identicità dei Riferimenti (2)

- L'operatore `==`, quando applicato a variabili che contengono riferimenti ad oggetti, verifica se i riferimenti raggiungono lo stesso oggetto

```
Punto origine = new Punto(0, 0);
```

```
Punto copiaDelRif = origine;
```

```
if (origine == copiaDelRif)
```

```
    System.out.println("uguali");
```

```
else
```

```
    System.out.println("diversi");
```

- Può essere applicato anche ad espressioni di tutti i tipi primitivi, con la semantica più naturale: identità di valori
- Invece, per verificare se due oggetti distinti sono equivalenti in base al loro stato (*non* in base ai riferimenti) è necessario implementare un *criterio di equivalenza* tra oggetti di un certo tipo (>>)

Equivalenza fra Stringhe

- Molte classi offrono un metodo `equals()` che definisce un criterio di equivalenza basato sullo stato
 - verifica l'uguaglianza con un altro oggetto passato come parametro
 - segnatura niente affatto scontata ($>>$)
- Per il momento conviene vedere esempi di criteri di equivalenza già definiti nelle librerie standard, come per la classe `String`
- In Java le stringhe sono oggetti a tutti gli effetti
 - è possibile confrontarle con il loro metodo `equals()`
 - equivalenza *carattere-per-carattere*

```
String nome1 = new String("alice");  
String nome2 = new String("alice");
```

```
System.out.println(nome1 == nome2);           // Stampa: false  
System.out.println(nome1.equals(nome2));      // Stampa: true
```

Metodo equals()

- Il metodo `equals()` si usa quando si vuole verificare l'*equivalenza* tra due oggetti distinti secondo un criterio definito dal programmatore (e dipendente dalla classe)
 - In effetti vedremo che il metodo `equals()` è offerto, sempre, da tutte le classi (>>)
 - Ma se non viene esplicitamente implementato (>>) ha la stessa semantica dell'operatore `==` per il confronto tra riferimenti
- Se lo scopo è quello di controllare se due variabili fanno riferimento allo stesso oggetto meglio usare, sempre e comunque, il più esplicito operatore `==`

Uguaglianza con null

- L'uguaglianza di un riferimento ad oggetti con il valore `null` si verifica tramite l'operatore `==`
 - `null` è un valore speciale che può essere utilizzato per rappresentare l'assenza di un oggetto (di un qualsiasi tipo)
- Tipica istruzione condizionale:

```
if (varRif!=null) {  
    <operazioni su varRif non null>  
}
```
- Attenzione: valutando `varRif.equals(null)` si ottiene
 - `false` se `varRif!=null`
 - `NullPointerException` se `varRif==null`

Null Simmetria dell'Equivalenza

- ✓ Quindi: `a.equals(b)` non è, in generale, pari a `b.equals(a)`
- Comportamento atteso se `a` e `b` possono anche assumere valori `null`
- Se invece sia `a` sia `b` sono
 - non nulle
 - riferimenti ad oggetti dello stesso tipo
- È necessario che valgano le proprietà tipiche delle relazioni di equivalenza, come:
 - `a.equals(a)` (*riflessiva*)
 - `( a.equals(b) ) == ( b.equals(a) )` (*simmetrica*)
 - (altre seguiranno >>)

Riferimenti in Java vs Puntatori C

- Nel linguaggio C il concetto più simile a quello di *riferimento Java* è sicuramente quello di *puntatore ad una cella di memoria*
- N.B. sono e rimangono comunque concetti molto diversi, i cui costrutti, nei due linguaggi, supportano un insieme di operazioni ben diverse
 - In C tale insieme di operazioni è molto più ampio
 - In Java non esiste l'aritmetica dei puntatori come in C o C++
 - In Java non è possibile referenziare *nulla* che non sia un oggetto

Diagnostica: *Riferimenti vs Puntatori*

- Per colpa di un errore di programmazione, spesso si finisce per produrre puntatori “impazziti” in C
 - Questi a loro voltano, causano errori a tempo di esec.
 - In buona parte i puntatori sono alla base della pessima diagnostica a tempo di esecuzione: non si può prevedere a cosa si finisce per accedere
 - ✓ risulta difficile risalire alla vera causa di un problema se gran parte degli errori viene diagnosticato con un generico *Segmentation fault* !

Riferimenti vs Puntatori

- In Java i riferimenti sono stati progettati proprio per superare i limiti ed i difetti tipici dei puntatori in C
- In Java è stato deciso di evitare alla radice i rischi tipici dei puntatori
 - Non è possibile referenziare nulla che non sia un oggetto
- Si “nasconde” al programmatore l'esistenza stessa del concetto di puntatore sostituendolo con il ben più “sicuro” riferimento ad oggetto

Gestione della Memoria

- In Java l'operatore **new** (che serve ad allocare oggetti) è il costrutto più simile alla funzione (C) **malloc**
 - entrambi allocano un'area di memoria nell'heap
- Ad ogni chiamata della funzione **malloc** deve però corrispondere una chiamata ad una funzione **free** per deallocare l'area di memoria occupata nell'heap e restituirla ai successivi utilizzi
- In Java NON è necessario
 - finora non è stata scritta neanche una operazione per liberare la memoria occupata con le **new**
- La deallocazione della memoria a carico della JVM
 - specificatamente di un componente, il *Garbage Collector*

Deallocazione della Memoria in C

- Deallocazione della memoria a carico del programmatore
- Possibili errori che ne conseguono, ad es.:
 - **malloc** senza corrispondente **free**
Memory leak: perdita di memoria utilizzabile
 - **free** senza porre il puntatore a **NULL**
Dangling Pointer: rimane disponibile un puntatore verso un'area di memoria non più disponibile
 - N.B. è possibile avere la stessa problematica ancor più semplicemente (senza **malloc**) anche creando puntatori ad aree di memoria sullo stack non più in uso!

```
struct Punto *origine;  
origine = malloc(sizeof(struct Punto));  
free(origine);  
origine->x = 0; // Comportamento indefinito
```

Deallocazione della Memoria in Java

- La piattaforma Java solleva il programmatore dalla responsabilità di deallocare esplicitamente la memoria
 - La JVM si occupa, a tempo di esecuzione, di trovare gli oggetti inutilizzati e non più utilizzabili e recupera la loro memoria
 - Il componente della JVM che si occupa di questo compito si chiama *Garbage Collector*
- Alcuni esperti ritengono che questa sia la differenza tra i linguaggi C/Java che da sola, ed in assoluto, contribuisce in maggiore misura all'incremento di produttività dei programmatori Java

Attenzione!

- Per prevenire fraintendimenti, meglio precisare subito che la gestione della memoria, ed un suo utilizzo appropriato, rimane comunque tra le più importanti responsabilità anche di un programmatore Java
- Il fatto che non è necessario fare esplicitamente le chiamate alla **free** non significa affatto che non si stia occupando memoria!

Il Garbage Collector (1)

- Il ruolo del garbage collector è quello di identificare gli oggetti non utilizzati e non più utilizzabili
 - Questi oggetti devono essere marcati come *reclamabili*
 - la loro memoria recuperata per fare spazio all'allocazione di nuovi oggetti
- Come può succedere che un oggetto diventa *reclamabile*? Semplice:

`Punto origine = new Punto(0,0) ;// occupa mem.`

`origine = null;`

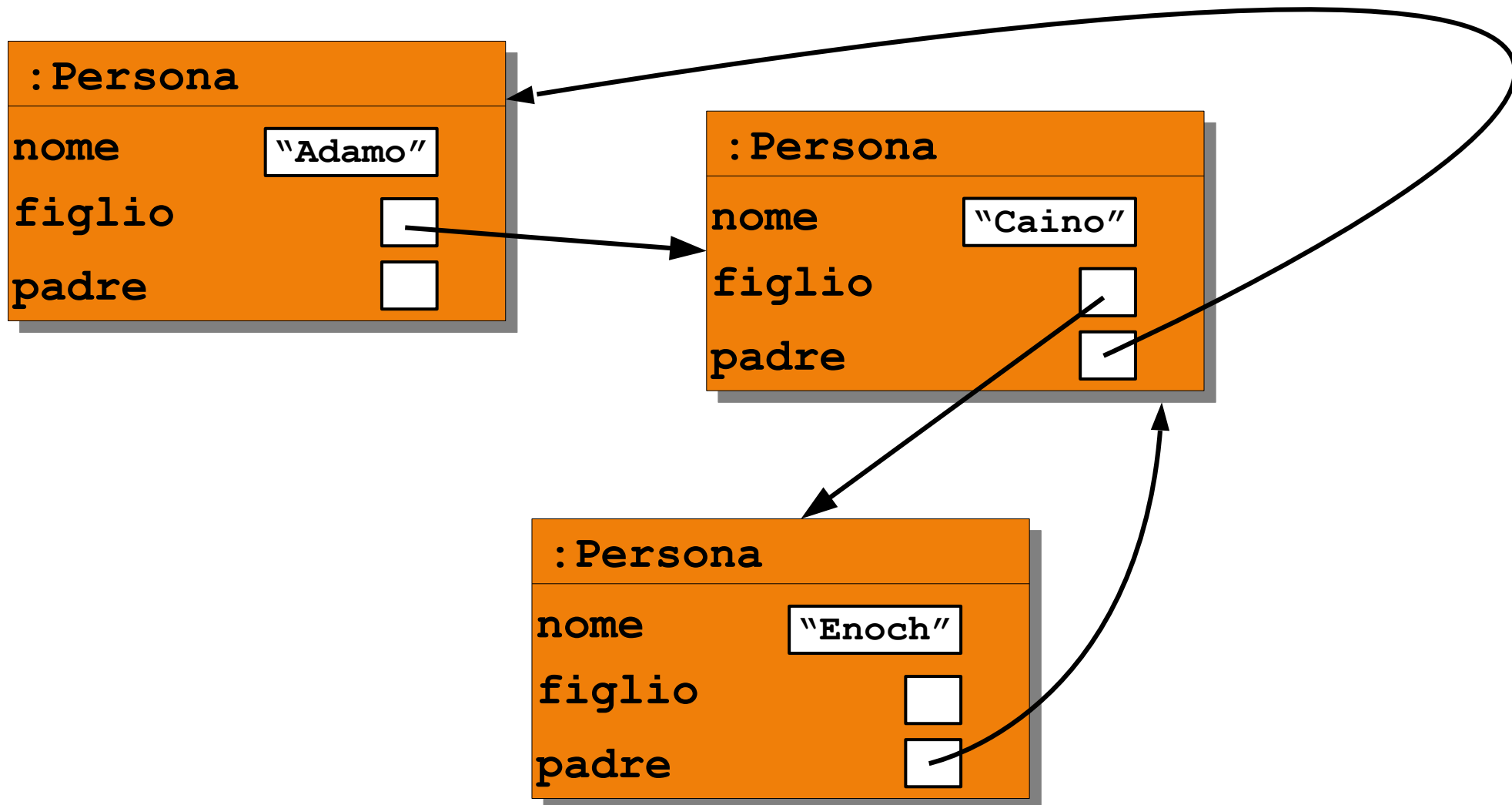
// da qui in poi la memoria dell'oggetto può essere recuperata perché l'oggetto appena creato non è più raggiungibile

Il Garbage Collector (2)

- Dal punto di vista del Garbage Collector, un oggetto può essere:
 - *IN_USO*: raggiungibile tramite una *catena di riferimenti*
 - *RECLAMABILE*: oggetto non più raggiungibile, non esiste alcuna *catena di riferimenti* che vi arrivi
- *Catena di riferimenti*: sequenza di riferimenti che parte dallo stato dell'esecuzione corrente e conduce ad un oggetto
 - eventualmente passando attraverso diversi oggetti intermedi
- Come di creano queste catene?
- Da dove iniziano?

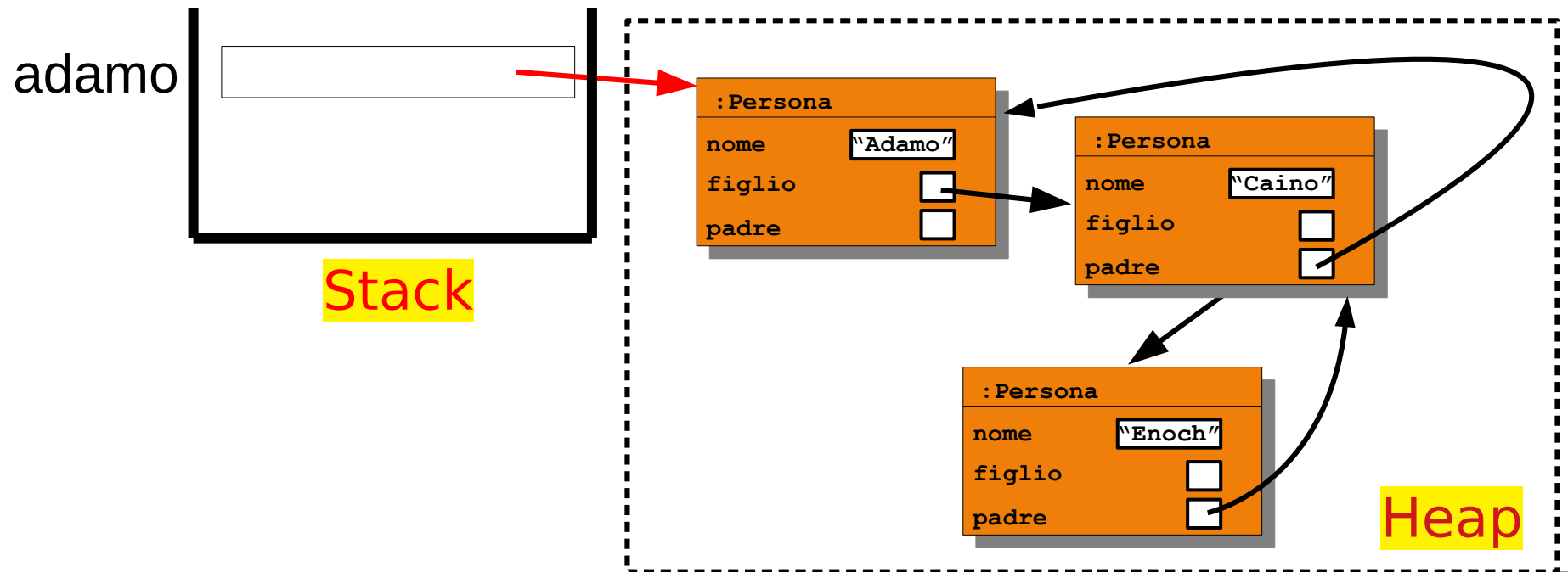
Il Garbage Collector: Catene di Riferimenti (1)

- I riferimenti costituiscono un grafo orientato
 - Si pensi ad una possibile classe **Persona**



Il Garbage Collector: Catene di Riferimenti (2)

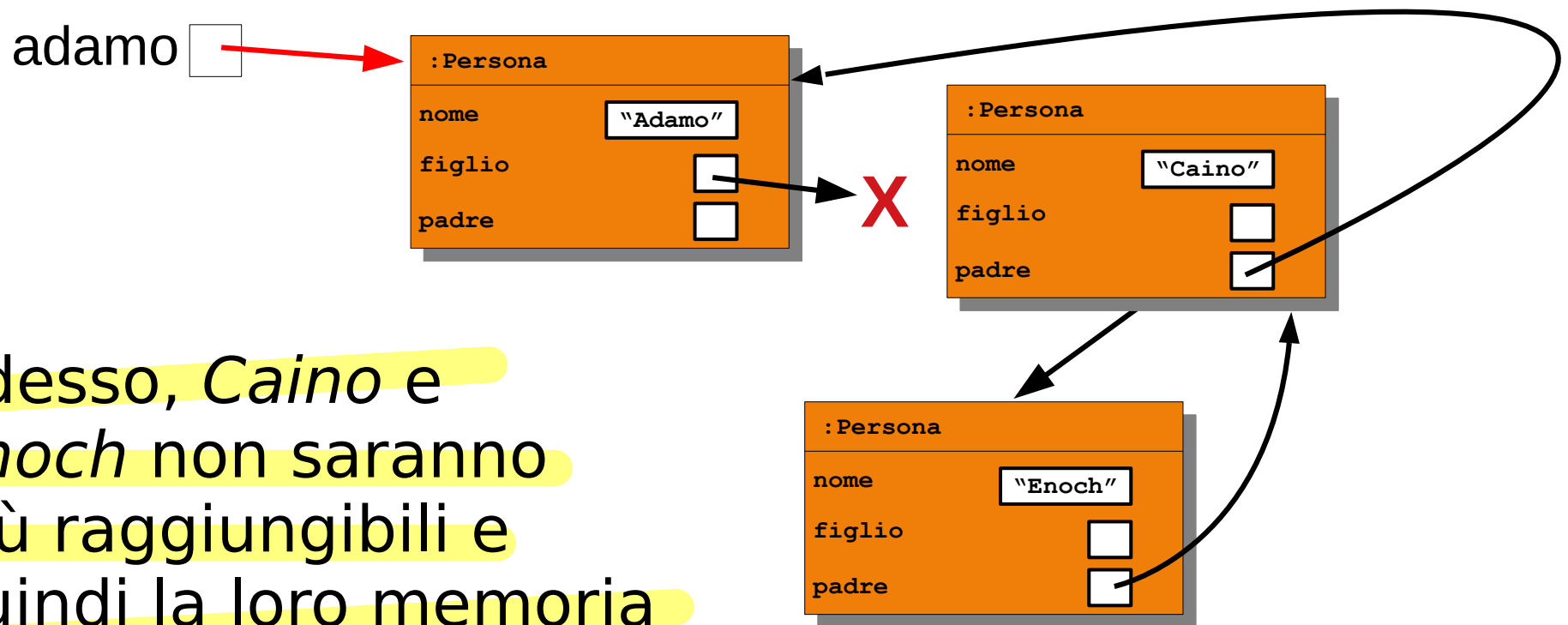
- Se la prima persona (Adamo) è raggiungibile tutte le altre mostrate nello schema sono allora raggiungibili (indirettamente tramite la prima)
- Riferimento *iniziale* (>>): 
- Se la prima persona non fosse raggiungibile, nessuna delle altre lo sarebbe



Il Garbage Collector: Catene di Riferimenti (3)

- Per interrompere una catena di riferimenti è sufficiente porre a **null** una variabile (di istanza o locale) che contiene il riferimento ad un oggetto

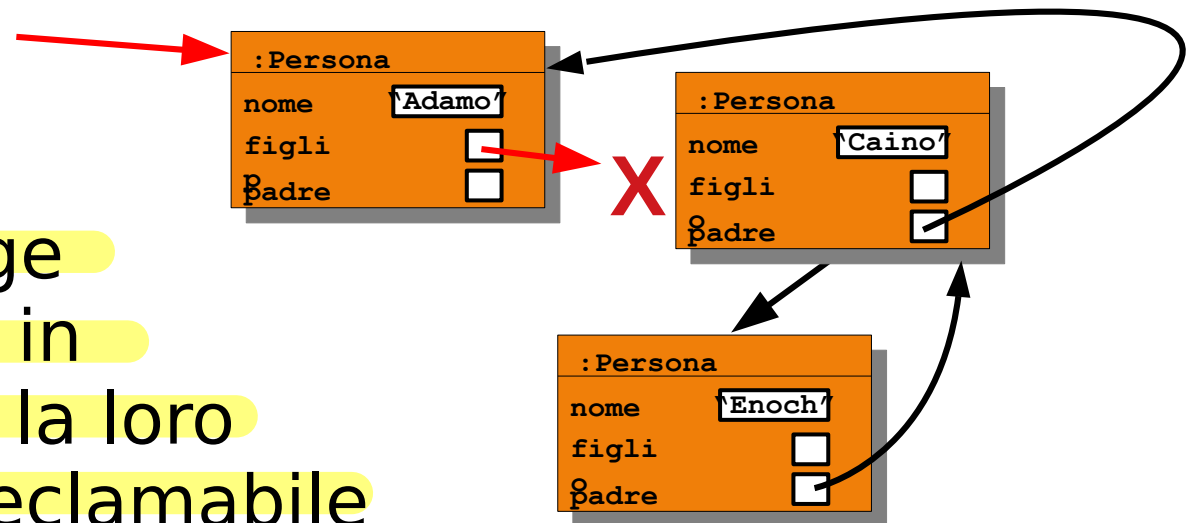
`adamo.setFiglio(null);`



- Adesso, *Caino* e *Enoch* non saranno più raggiungibili e quindi la loro memoria può essere recuperata

Il Garbage Collector: Catene di Riferimenti (4)

- In questo modo il Garbage Collector rileva che gli oggetti *Caino* ed *Enoch* non saranno più utilizzati
 - verranno marcati come *reclamabili*
- Mettendo a `null` il riferimento verso *Caino* non è più possibile raggiungerli



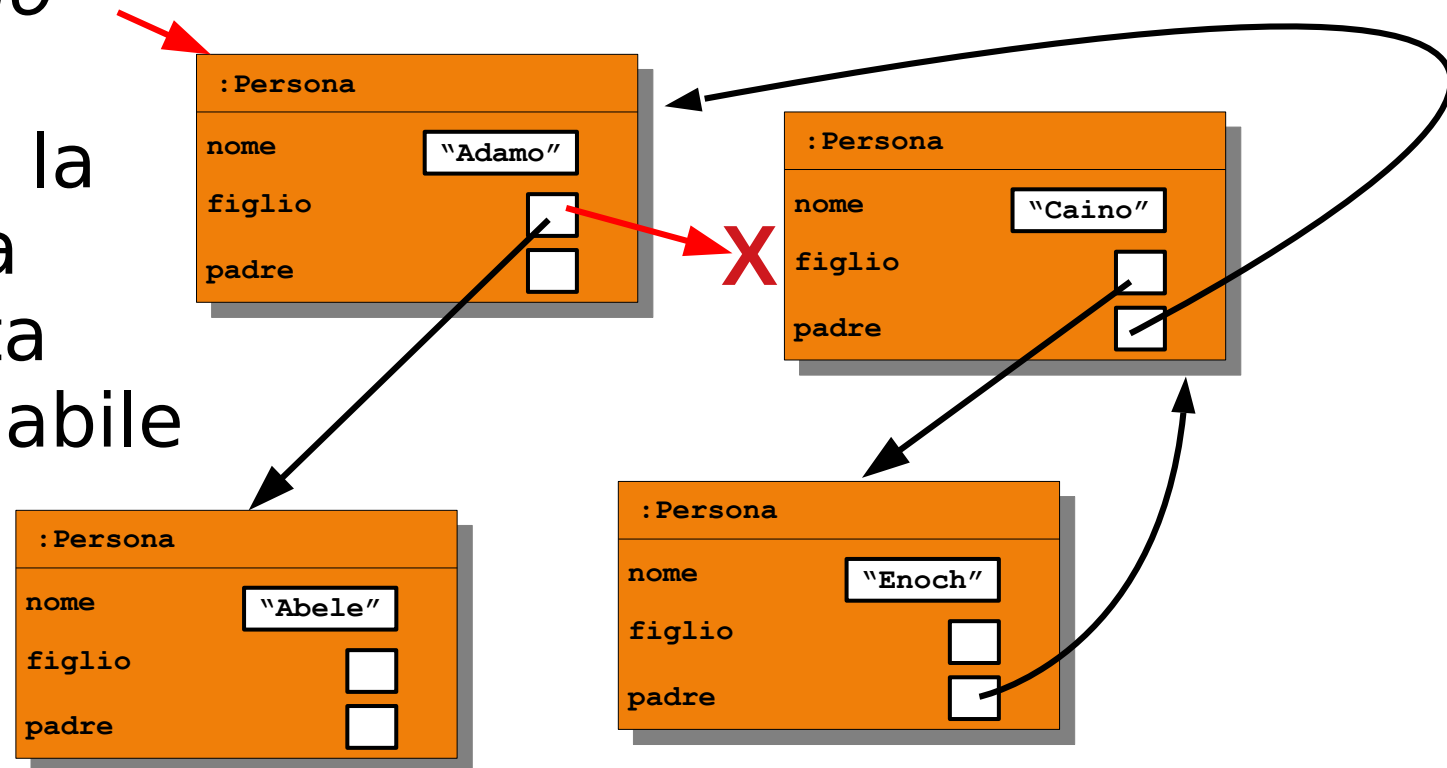
- Quando il Garbage Collector entrerà in azione marcherà la loro memoria come reclamabile

Il Garbage Collector: Catene di Riferimenti (5)

- Un oggetto può diventare irraggiungibile anche *sovrascrivendo* un riferimento

```
Persona abele = new Persona("Abele");  
adamo.setFiglio(abele);
```

- Enoch e Caino* non sono più raggiungibili, la loro memoria verrà marcata come reclamabile



Inizio delle Catene di Riferimenti

- Sinora si supponeva che il primo oggetto *Adamo* fosse raggiungibile
 - è necessario avere un primo riferimento per iniziare le catene
 - sia gli oggetti sia le variabili di istanza contenenti i riferimenti si trovano nell'heap
- Quali oggetti sono però raggiungibili da fuori dell'heap, e con quali riferimenti iniziali?
 - Quelli referenziati dalle variabili locali o dai parametri dei metodi i cui *RDA* sono nello Stack

Garbage Collection (1)

- L'algoritmo lavora inseguendo i riferimenti a cominciare da quelli contenuti dentro i RDA (var. locali e parametri attuali) nello Stack
 - esplora il grafo dei riferimenti dentro l'Heap
 - continua con quelli conservati nelle var. di istanza degli oggetti conservati nell'Heap
 - se un oggetto è
 - *Raggiungibile*: allora viene mantenuto in memoria
 - *Non raggiungibile*: la sua memoria viene marcata *reclamabile*

Garbage Collection (2)

- Successivamente (quando serve memoria e non necessariamente subito) la memoria reclamabile viene recuperata per poter allocare nuovi oggetti
- Abbiamo discusso solo i problemi ed i concetti basilari di una semplice implementazione dell'Algoritmo di *Garbage Collection*
 - Enorme interesse per questi algoritmi
 - Quelli utilizzati sono il risultato di decenni di ricerca (accademica ed industriale)
 - Una delle classi di algoritmi più studiate in assoluto: sempre più veloci ed ottimizzati

Garbage Collection: Vantaggi e Svantaggi

- Facilitano lo sviluppo rapido di applicazioni
- Tuttavia esistono alcuni importanti svantaggi:
 - La visita del grafo dei riferimenti è un'operazione onerosa che limita le prestazioni
 - Il garbage collector è fuori dal controllo diretto del programmatore
 - può intervenire in momenti non facilmente prevedibili e talvolta inopportuni
 - non adatto in tutte le situazioni in cui i tempi di risposta devono essere prevedibili con certezza
- Un programmatore esperto può sicuramente ottenere prestazioni migliori gestendo la memoria manualmente ma specificatamente per la sua applicazione

Programmazione Orientata agli Oggetti

Overloading, Costruttori,
Stringhe e Array

Sommario

- Overloading
- Costruttore primario e costruttori secondari
- La classe **String**
 - Il metodo `toString()`
- Diagramma degli oggetti
- Array
- Costanti
- Variabili e metodi di classe

Overloading (1)

- Caratteristica che permette ad un linguaggio di programmazione di definire molteplici metodi/funzioni/procedure con lo stesso nome
- Una classe Java può ospitare due o più *versioni* dello stesso metodo (ovvero, con lo stesso nome)
- Tale metodo si dice *sovraccarico*
- Le versioni sovraccariche dello stesso metodo devono comunque risultare distinguibili
- Devono differire per la lista di parametri formali
 - per numero di parametri, e/o
 - per il tipo di uno o più parametri, e/o
 - per l'ordine dei parametri

Overloading (2)

- La classe **Sommatore** può avere più metodi di stesso nome per sommare due interi o tre interi

```
public class Sommatore {  
    public int add(int a, int b) { return a + b; }  
    public int add(int a, int b, int c) {  
        return a + b + c;  
    }  
}
```

- Il tipo e l'ordine dei parametri è significativo:

```
public double add(int a, double b) {  
    return a + b;  
}
```

... è un metodo distinto da:

```
public double add(double a, int b) {  
    return a + b;  
}
```

Overloading: in C?

- Il linguaggio C non supporta l'overloading
 - Non si possono definire due funzioni con lo stesso nome
- A conferma che questa possibilità offerta dal linguaggio Java è avvertita come una viva esigenza da molti programmatori, basta osservare la convenzione adottata da molti programmatori C per gestire situazioni simili, ovvero:
 - `double add_id(int a, double b)`
 - `double add_di(double a, int b)`
- ✓ I nomi sono resi sintatticamente diversi, con un prefisso comune che rende evidente l'esistenza di una radice comune

Risoluzione Chiamate in Overloading

- Per ciascuna invocazione di metodo (sovraccarico) si confrontano:
 - tutte le *signature* del metodo, ed in particolare
 - tipo e numero dei parametri formali
 - con la lista dei parametri attuali
- In presenza di una lista di parametri formali in perfetto accordo con la signature di un metodo (per numero e tipo di parametri), la scelta è semplice:
 - Si utilizza la versione di un metodo sovraccarico che segue fedelmente il numero ed il tipo dei parametri attuali

Chi Risolve le Chiamate Sovraccariche?

- Quando viene operata la scelta?
 - a tempo statico (durante la *compilazione*)
oppure
 - a tempo dinamico (durante l'*esecuzione*)
- E' il **compilatore** a decidere quale versione di un metodo sovraccarico invocare
 - Già durante la compilazione (a tempo statico)
 - La scelta è definitiva, scritta nei **.class** generati
 - Operata esclusivamente sulla base dell'analisi dei tipi della lista di parametri attuali (nell'invocazione di metodo) effettuata durante la compilazione

Overloading

```
public class ProvaOverloading {  
    public void metodo(int a) {  
        System.out.println("parametro int");  
    }  
    public void metodo(double a) {  
        System.out.println("parametro double");  
    }  
}
```

```
ProvaOverloading prova = new ProvaOverloading();  
prova.metodo(3);      // Stampa "parametro intero"  
prova.metodo(3.0d);  // Stampa "parametro double"
```

Overloading e Tipo Restituito

- Versioni sovraccariche dello stesso metodo possono differire anche per il tipo del valore restituito:

```
int add(int a, int b);
```

```
double add(double a, int b);
```

- NON è però possibile distinguere due metodi con gli stessi parametri formali (stesso numero, tipo e ordine dei parametri) usando solamente il tipo di ritorno

```
int f(int a, int b);    // Non Compila
```

```
double f(int a, int b); // Non Compila
```

- ✓ Non compila: il compilatore non sarebbe in grado di capire quale versione si dovrebbe invocare (in generale)...

Si usa anche dire che

il tipo restituito non fa parte della segnatura di un metodo

Esempio (con Eclipse)

- All'interno della classe `Rettangolo` scrivere i metodi
 - `void scala(int fattore)`
deve modificare l'oggetto `Rettangolo` di modo che
`base = base * fattore`
`altezza = altezza * fattore`
 - `void scala(int fattoreBase, fattoreAltezza)`
deve modifica l'oggetto `Rettangolo` di modo che
`base = base * fattoreBase`
`altezza = altezza * fattoreAltezza`

Esempio (2)

```
public class Rettangolo {  
    private int base;  
    private int altezza;  
    // ...  
    public void scala(float fattore) {  
        this.scala(fattore, fattore);  
    }  
    public void scala(float fattoreBase,  
                      float fattoreAltezza) {  
        this.base *= fattoreBase;  
        this.altezza *= fattoreAltezza;  
    }  
}
```

Esempio (3)

- Notare come il metodo
`scala(float fattore)`

faccia uso del metodo

`scala(float fattoreBase, float fattoreAltezza)`

...per evitare la duplicazione del codice (>>)

Risoluzione Metodi Sovraccarichi (1)

- In assenza di una corrispondenza perfetta tra
 - lista dei parametri attuali di una invocazione
 - segnatura di un metodo

il compilatore applica un algoritmo di risoluzione che prima di fermare la compilazione cerca di capire se semplici conversioni di tipo permettono la chiamata

 - Ad es. `int` → `float`
- L'algoritmo è intriso di dettagli per coprire l'ampia casistica; in pratica basta quasi sempre limitarsi a ricordare che:
 - sono ammesse semplici promozioni di tipo se necessarie a rendere un'invocazione confacente con una delle signature disponibili
 - sono preferite le promozioni più “conservative”, ovvero quelle al tipo immediatamente più grande

Risoluzione Metodi Sovraccarichi (2)

- Più nel dettaglio:
 - Argomento di tipo intero
 - se esiste un metodo che prende come parametro formale `int`, allora viene usato quel metodo
 - altrimenti viene promosso al tipo di dato più piccolo tra quelli disponibili (ma “capaci di contenere” un `int`): `long`, `float`, `double`
 - Argomento in virgola mobile
 - Si cerca il metodo che ha come parametro formale un `double`
 - Argomento di tipo carattere
 - Se non trova una corrispondenza con `char`, si prova la promozione a `int`

Risoluzione Metodi: Esempio

Motivo di confusione è la promozione *implicita* di tipo per i tipi primitivi. Ad esempio:

```
public class Prova {  
    void f(long i)    { System.out.println("long");    }  
    void f(float i)   { System.out.println("float");   }  
    void f(double i) { System.out.println("double"); }  
    public static void main(String[] args) {  
        f(5); // ???  
    }  
}
```

```
public class Prova {  
    void f(long i)    { System.out.println("long");    }  
    void f(float f)   { System.out.println("float");   }  
    void f(double i) { System.out.println("double"); }  
    public static void main(String[] args) {  
        f(5); // ???  
        f(5f); // ???  
    }  
}
```

Con Riferimenti: Cosa Stampa?

```
public class Boo {  
  
    void f(String n) {  
        System.out.println("stringa");  
    }  
  
    void f(int n) {  
        System.out.println("intero");  
    }  
  
    public static void main(String[] args) {  
        Boo b = new Boo();  
        String s = new String("pppp");  
        int i = 0;  
        b.f(s);  
        b.f(i);  
    }  
}
```

Overloading di Operatori

- L'overloading è una caratteristica comoda ma usata in maniera decisamente controllata in Java
- Anche l'operatore `+` è sovraccarico: così come già accade in C, risulta più agevole la manipolazione di tipi numerici
 - Somma interi (`int`, `long`)
 - Somma numeri in virgola mobile (`float`, `double`)
- Invece specifica del linguaggio Java è la sua applicazione ad una classe con un supporto particolare: **`String`**
 - Concatenazione di stringhe; Ad es.:
 - `System.out.println("Ciao"+" "+"Mondo");`
- In C++ (ed in Scala) c'è anche la possibilità di sovraccaricare tutti gli altri operatori, in Java (fortunatamente) NO
 - per il pessimo rapporto costi/benefici

Costruttori ed Overloading

- Anche i costruttori possono essere sovraccarichi
- Ad esempio la classe `Rettangolo` potrebbe avere i seguenti costruttori
 - Un costruttore *no-args*. Quindi
 - `base = 0; altezza = 0, vertice = (0, 0)`
 - Un costruttore che ha come parametri base e altezza. Quindi:
 - `vertice = (0, 0)`
 - Un costruttore generico
 - `vertice`
 - `base`
 - `altezza`

Esempio (*con Eclipse*)

- Realizzare i costruttori della classe **Rettangolo** appena descritti

Esempio (2)

```
public class Rettangolo {  
    private int altezza;  
    private int base;  
    private Punto vertice;  
    public Rettangolo(Punto vert, int base, int altezza) {  
        this.vertice = vert;  
        this.base = base;  
        this.altezza = altezza;  
    }  
    public Rettangolo(int base, int altezza) {  
        this.vertice = new Punto(0, 0);  
        this.base = base;  
        this.altezza = altezza;  
    }  
    public Rettangolo() {  
        this.vertice = new Punto(0, 0);  
        this.base = 0;  
        this.altezza = 0;  
    }  
}
```

Esempio (3)

```
public class MainOverloading {  
    public static void main(String[] args) {  
        // base 0, altezza 0, vertice in (0, 0)  
        Rettangolo r1 = new Rettangolo();  
  
        // base 3, altezza 5, vertice in (0, 0)  
        Rettangolo r2 = new Rettangolo(3, 5);  
  
        Punto vertice = new Punto(4, 9);  
        Rettangolo r3 = new Rettangolo(vertice, 3, 5);  
    }  
}
```


Costruttore “Primario”

- Costruttori definiti ripetendo molto codice, come appena fatto sono... *sconsigliabili!*
- Le ripetizioni nel codice causano problemi (>>)
- Meglio eleggere un costruttore al ruolo di “*primario*”, il più generico possibile
- Eccolo per la classe **Rettangolo**:

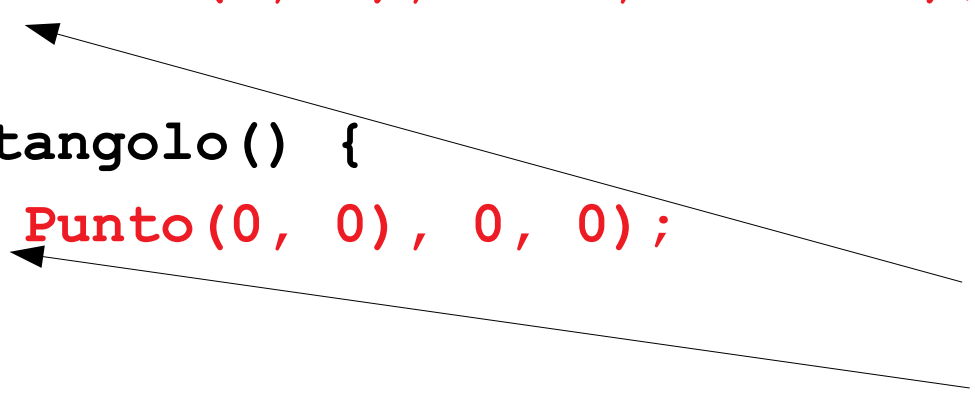
```
public Rettangolo(Punto vertice, int base, int altezza) {  
    this.vertice = vertice;  
    this.base = base;  
    this.altezza = altezza;  
}
```

Costruttori “Secondari”

- A questo punto, gli altri costruttori (“secondari”) possono affidarsi a quello più generico
- Per invocare un costruttore da un costruttore:
`this(<lista di argomenti>)`
 - L'invocazione di un altro costruttore *deve essere la prima istruzione nel corpo del costruttore secondario*
 - Altrimenti: errore di compilazione
- I costruttori secondari finiscono per fissare dei valori predefiniti per tutti i parametri che non ricevono esplicitamente
- In Scala esiste una sintassi apposita per distinguere costruttori *primari* dai *secondari*, in Java NO

Costruttori Chiamati dai Costruttori

```
public class Rettangolo {  
    private int altezza;  
    private int base;  
    private Punto vertice;  
    public Rettangolo(Punto vert, int base, int altezza) {  
        this.vertice = vert;  
        this.base = base;  
        this.altezza = altezza;  
    }  
    public Rettangolo(int base, int altezza) {  
        this(new Punto(0, 0), base, altezza);  
    }  
    public Rettangolo() {  
        this(new Punto(0, 0), 0, 0);  
    }  
}
```



Crea un nuovo oggetto **Punto** ed invoca un altro costruttore entro un'unica (la prima) riga di codice

La Classe String (1)

- In Java esiste la classe `String` per rappresentare sequenze di caratteri *immutabili*
 - Le stringhe sono oggetti, `String` è la classe a cui appartengono le sue istanze
- Una variabile dichiarata di tipo `String` contiene quindi un *referimento* ad un oggetto istanza della classe `String`
- `String favorita = new String("Sono la favorita!");`
- Pur essendo una classe come tutte le altre, spesso si ha la tentazione di pensare che non lo sia affatto...
- Il motivo è che ha un supporto *molto* particolare sia nel linguaggio che da parte del compilatore

La Classe String (2)

- Per rendere il linguaggio più semplice ed attraente per nuovi sviluppatori, all'epoca si decisero trattamenti decisamente “di favore” per questa classe
- Prima di Java 5 era (>>)
 - l'unica classe che possiede dei letterali appositi
 - l'unico tipo di oggetto che si può creare senza fare una **new** esplicita
- `String favorita = "Sono la favorita!";`
equivale a
- `String favorita = new String("Sono la favorita!");`
- L'inserimento dell'invocazione della **new** è operata direttamente dal compilatore
- Anche il fatto che l'operatore `+` sia sovraccarico per gestire la concatenazione conferma il trattamento di favore...
- Sicuramente tutto questo “opacizza” il codice (>>)

Equivalenza di Stringhe (1)

- Dato che le stringhe sono oggetti *veri* e propri, l'equivalenza deve essere valutata mediante il metodo appositamente previsto dalla classe `equals()`

Infatti:

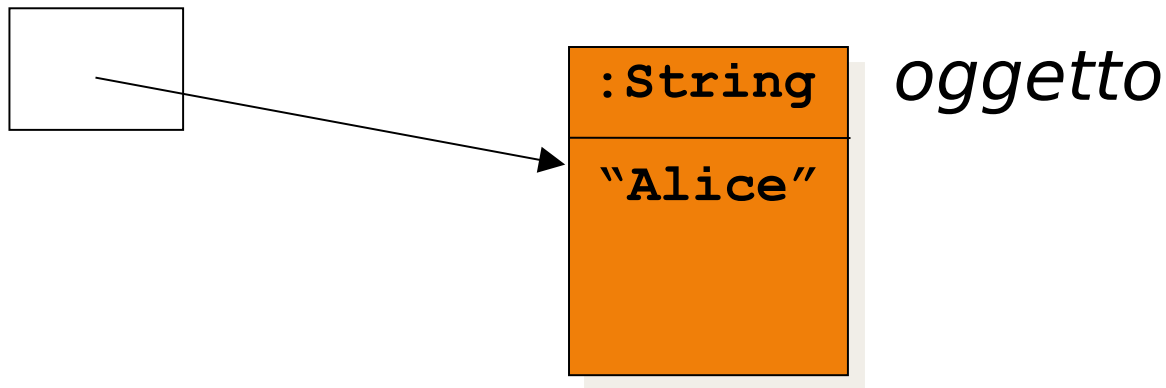
- `String nome = new String("Alice");`
- `String omonimo = new String("Alice");`

```
System.out.println(nome == omonimo);  
// Stampa false
```

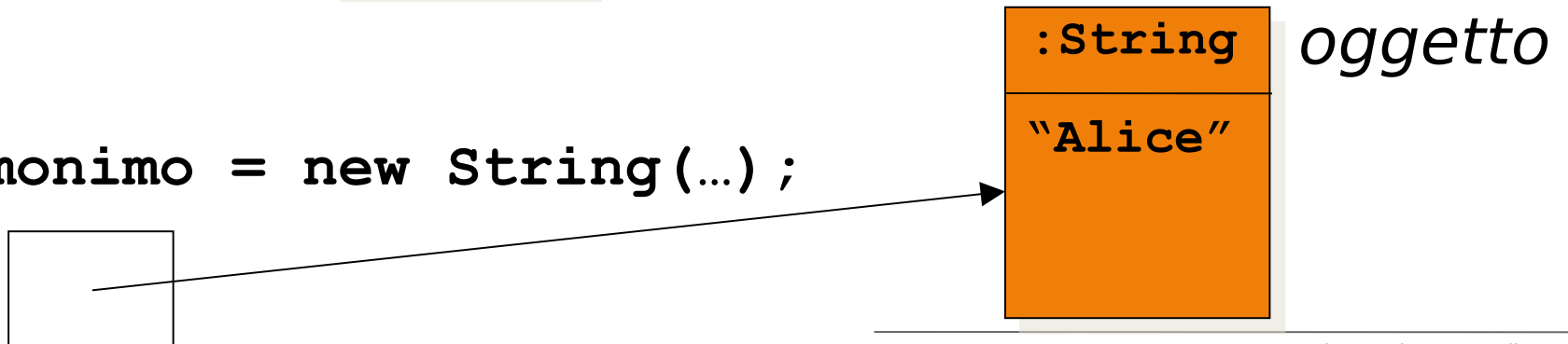
Equivalenza di Stringhe (2)

- Si sta in realtà verificando che i *referimenti* risultino identici
 - *falso*: `nome` e `omonimo` contengono riferimenti diversi, ovvero restituiti da due distinte invocazioni della `new`

```
String nome = new String(...);
```



```
String omonimo = new String(...);
```



Equivalenza di Stringhe (3)

- Invece, utilizzando:

```
String nome = new String("Alice");  
String omonimo = new String("Alice");
```

```
System.out.println(nome.equals(omonimo));  
// stampa true
```

il metodo `equals()` controllerà l'equivalenza della sequenza di caratteri che compone le stringhe, carattere per carattere

Immutabilità della Classe String

- Gli oggetti `String` usati per rappresentare stringhe in Java sono *immutabili*
 - Non è possibile modificare i caratteri all'interno di una stringa una volta creata
 - Non si possono aggiornare: bisogna sempre crearne di nuove per memorizzare la modifica
- Ad esempio per concatenare due stringhe:

```
String s1 = "ciao "; // = new String("ciao ");  
String s2 = "mondo"; // = new String ("mondo");  
s1 = s1 + s2; // → Si sta creando un nuovo oggetto  
// String e si sta sovrascrivendo il vecchio  
riferimento
```

```
System.out.println(s1); // ciao mondo
```

Metodi della Classe String (1)

- `String s = "POO";`
 - Lunghezza della stringa
`s.length()` ; restituisce **3**
 - Ottenere un carattere in una certa posizione
`s.charAt(0)` ; restituisce '**P**'
 - Indicizzazione base 0
 - come per gli array il primo carattere ha indice 0

Metodi della Classe String (2)

- `String s = "una stringa";`
 - Indice di un carattere
`s.indexOf('s');` restituisce 4
 - Indice di una stringa
`s.indexOf("ring");` restituisce 6
- Il metodo `indexOf()` è sovraccarico
- Restituisce -1 se non trova il carattere o la stringa cercata
`s.indexOf('z');` restituisce -1

Metodi della Classe String (3)

- `String s = "una stringa";`
 - Rimpiazzare caratteri
- `String s2 = s.replace("stringa", "stringa lunga");`

```
System.out.println(s); // Stampa 'una stringa'
```

```
System.out.println(s2); // Stampa 'una stringa lunga'
```

- Le stringhe sono immutabili: in realtà si crea un nuovo oggetto `String` il cui riferimento finisce in `s2`
- *molti e molti altri metodi ancora....*

<https://docs.oracle.com/javase/7/docs/api/java/lang/String.html>

Stampare Descrizioni di Oggetti

- Se si esegue `println()` su un riferimento ad un oggetto verrà stampato il valore di tale riferimento

```
// un attrezzo di nome spada e peso 7 kg
```

- `Attrezzo spada = new Attrezzo("spada", 7);`

```
System.out.println(spada);
```

- Stampa, ad es.: `Attrezzo@70dea4e`
 - Non descrive affatto il contenuto e lo stato dell'oggetto, ma il riferimento

Il Metodo toString() (1)

- È possibile cambiare questo comportamento implementando di segnatura:

```
public String toString()
```

per tutte le classi in cui si vuole specificare come trasformare gli oggetti in stringhe

- Il metodo `toString()` restituisce la rappresentazione dell'oggetto sotto forma di stringa
- Addirittura fondamentale per rendere agevole il *debugging* ed il *tracing*

Il Metodo toString() (2)

- Ad esempio in **Attrezzo**

```
public class Attrezzo {  
    private String nome;  
    private int peso;  
    public Attrezzo(String nome, int peso) {  
        this.nome = nome; this.peso = peso;  
    }  
    // getter  
    public String toString() {  
        return "Attrezzo di nome " + this.getNome() +  
            ". Peso: " + this.getPeso();  
    }  
}
```

Il Metodo toString() (3)

```
public class MainToString {  
    public static void main(String[] args) {  
        • Attrezzo spada = new Attrezzo("spada", 7);  
  
        System.out.println(spada);  
    }  
}
```

- Stampa

Attrezzo di nome spada. Peso: 7

Il Metodo toString() (4)

- Stesso comportamento concatenando un riferimento ad un oggetto con un letterale stringa:

```
public class MainToString {  
    public static void main(String[] args) {  
        Attrezzo spada = new Attrezzo("spada", 7);  
        String descr = "attrezzo posseduto: " + spada;  
        System.out.println(descr);  
    }  
}
```

Stampa:

attrezzo posseduto: Attrezzo di nome spada. Peso: 7

`println()` & `toString()`

- Il metodo `toString()` (come `equals()`), anche è presente in tutti gli oggetti che creiamo (>>)
- Tuttavia:
 - Se non esplicitamente implementato stampa [un numero che dipende dal]l'indirizzo di memoria dell'oggetto su cui è invocato
 - L'invocazione del metodo `toString()`, è inserita direttamente dal compilatore, nelle istruzioni di stampa (anche se non esplicitamente richiesta!):
 - `System.out.println(oggRef)`
stampa il risultato di `oggRef.toString()`

Diagramma delle Classi

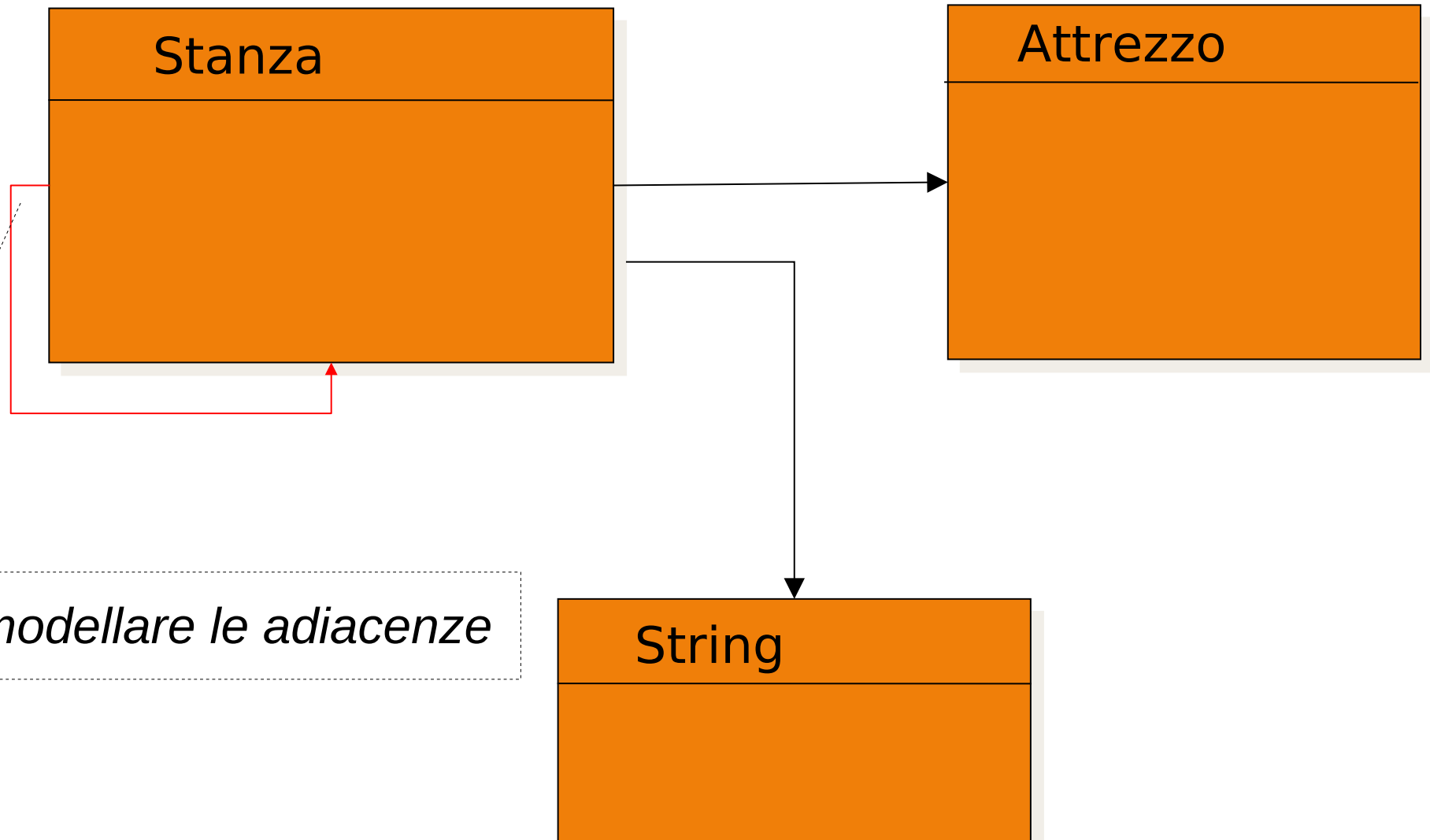
- Diagrammi UML
 - Ne esistono diversi tipi
- Sono un utile e comodo strumento a supporto (e non in sostituzione) della comunicazione
 - Tempo dinamico: abbiamo già utilizzato rappresentazioni diagrammatiche degli oggetti
 - Tempo statico: diagrammi delle classi
- Il diagramma delle classi illustra le caratteristiche principali (variabili di istanza, costruttori e metodi) delle classi che compongono la applicazione
- L'enfasi è sulla relazione tra le classi, quindi sugli aspetti “statici”

Esempio

- Supponiamo

```
public class Stanza {  
    private String nome;  
    private Attrezzo attrezzo;  
  
    public Stanza(String nome) {  
        this.nome = nome;  
    }  
  
    public void setAttrezzo(Attrezzo attrezzo) {  
        this.attrezzo = attrezzo;  
    }  
    ... altri metodi  
}
```

Diagramma delle Classi: Esempio



Per modellare le adiacenze

Diagramma degli Oggetti (1)

- Per avere un'idea della evoluzione di un programma è utile rappresentare lo stato delle istanze: a tal fine usiamo una rappresentazione grafica, chiamata **diagramma degli oggetti**
- Il diagramma degli oggetti mostra gli oggetti istanziati in memoria durante l'esecuzione dell'applicazione
- L'enfasi è sullo stato interno degli oggetti e sugli aspetti dinamici
 - ogni oggetto ha un indirizzo di memoria
 - le variabili di tipo riferimento ad oggetto memorizzano l'indirizzo dell'oggetto referenziato
 - una rappresentazione grafica efficace basata sull'uso di frecce...

Esempio

...

```
public static void main(String[] args) {  
    Attrezzo spada = new Attrezzo("spada", 10);  
    Stanza n10 = new Stanza("Aula N10");  
    n10.addAttrezzo(spada);
```

```
// < ---
```

```
/* disegnare il diagramma degli oggetti  
in questo punto dell'esecuzione */
```

```
}
```

Diagramma degli Oggetti: Esempio (1)

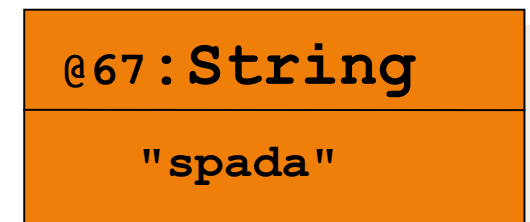
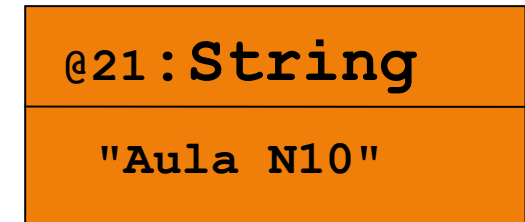
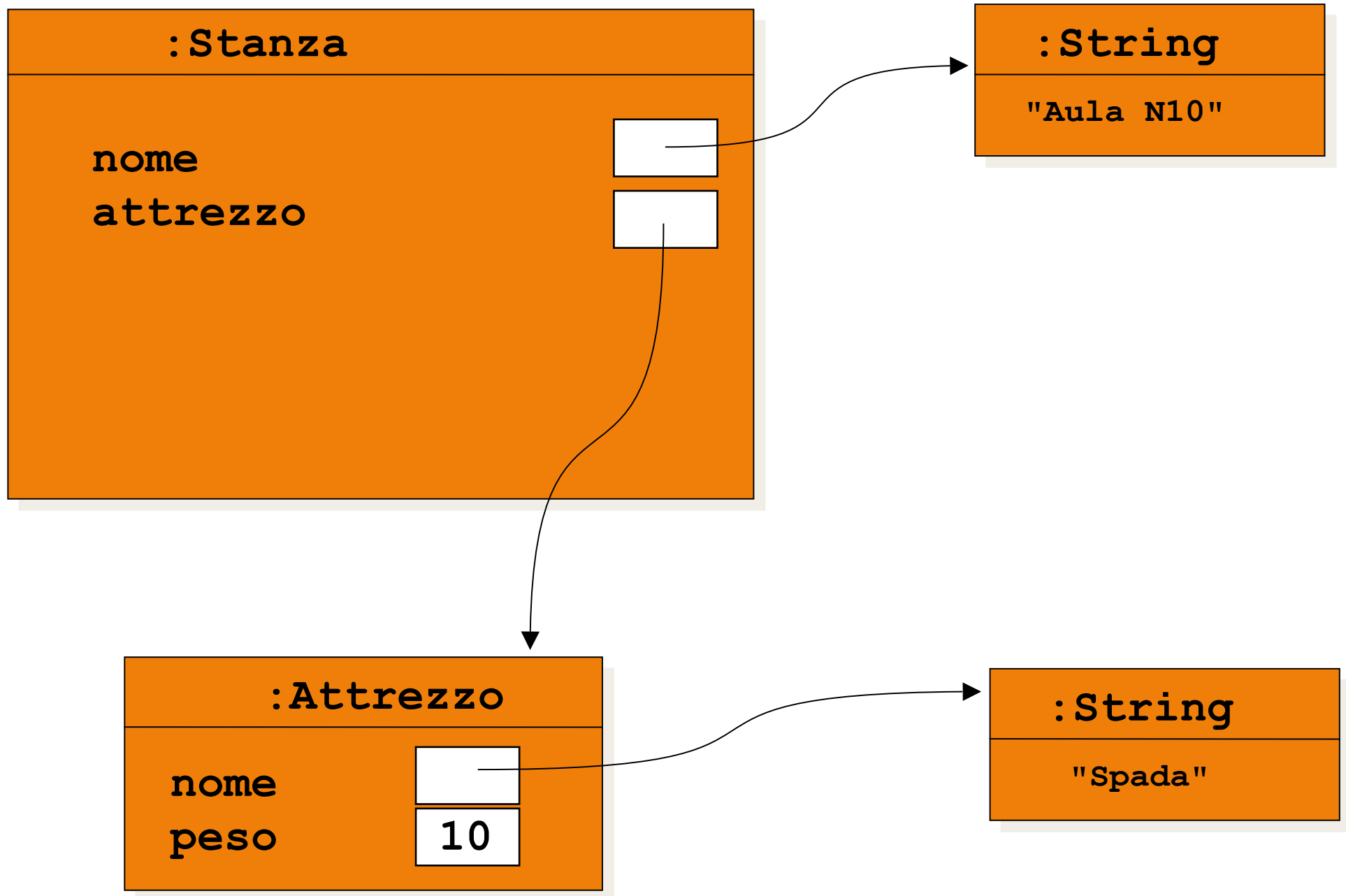


Diagramma degli Oggetti (2)

- Una rappresentazione grafica efficace dei valori memorizzati nelle variabili riferimento prevede l'uso di frecce che collegano
 - la variabile riferimento
 - all'oggetto referenziato

Diagramma degli Oggetti: Esempio (2)



Tipi Primitivi e Oggetti

- Dal diagramma degli oggetti notiamo che le variabili (di istanza) possono memorizzare
 - tipi primitivi
 - riferimenti ad oggetti

Tipi Primitivi in Java

boolean	vero (true) o falso (false)
char	caratteri Unicode 2.1 (16-bit)
byte	interi a 8 bit (con segno e in C2)
short	interi a 16 bit (con segno in C2)
int	interi a 32 bit (con segno in C2)
long	interi a 64 bit (con segno in C2)
float	numeri in virgola mobile a 32-bit
double	numeri in virgola mobile a 64-bit

Tipi Primitivi e Oggetti (1)

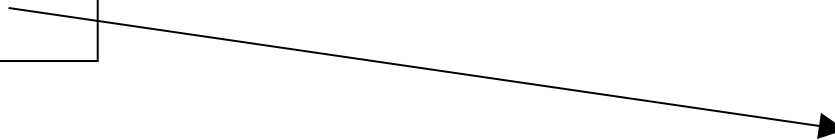
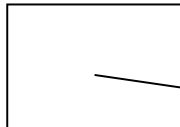
Tipo primitivo

```
int i;
```



Riferimento ad Oggetto

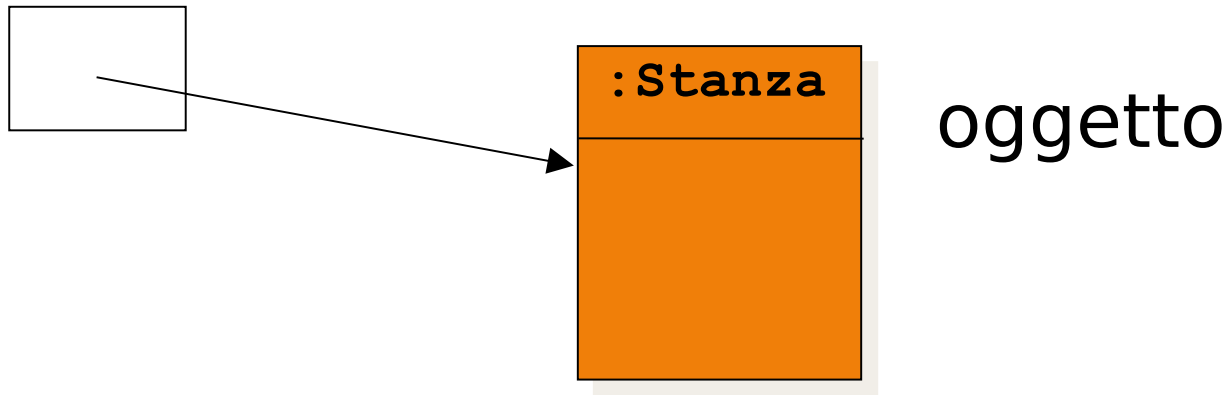
```
Stanza s;
```



Oggetto

Tipi Primitivi e Oggetti (2)

```
Stanza s = new Stanza (...);
```



- La variabile *s* *non* memorizza un oggetto (nell'esempio, una istanza della classe `Stanza`), ma un *riferimento* all'oggetto

Tipi Primitivi e Oggetti (3)

```
int i;
```

32

Tipo primitivo

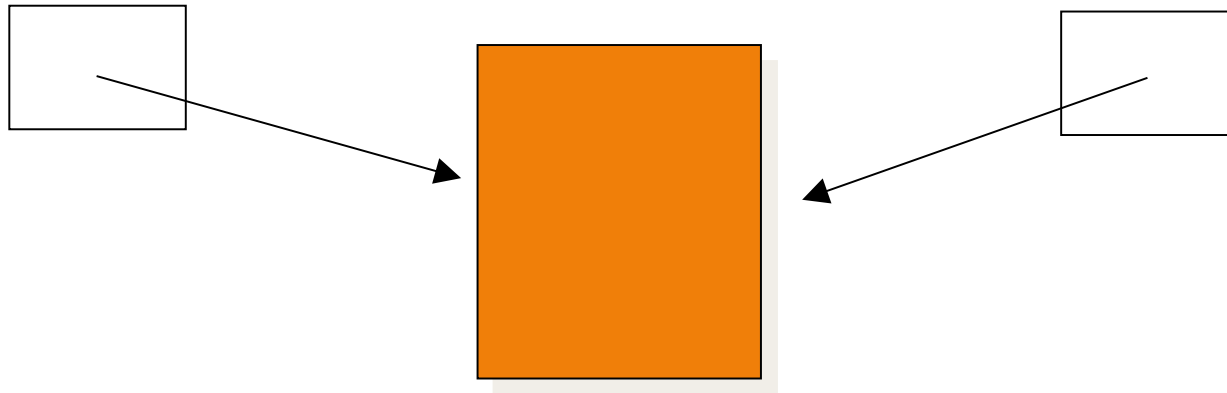
- Nel caso dei tipi primitivi, il valore è memorizzato direttamente nella variabile

Tipi Primitivi e Oggetti (4)

Oggetti

`MiaClasse a;`

`MiaClasse b;`



`a = b;`

Tipi primitivi

`int a;`

32

`int b;`

32

Tipi Primitivi e Oggetti (5)

```
int i1 = 0;
```

```
int i2 = 5;
```

```
i1 = i2;
```

i1

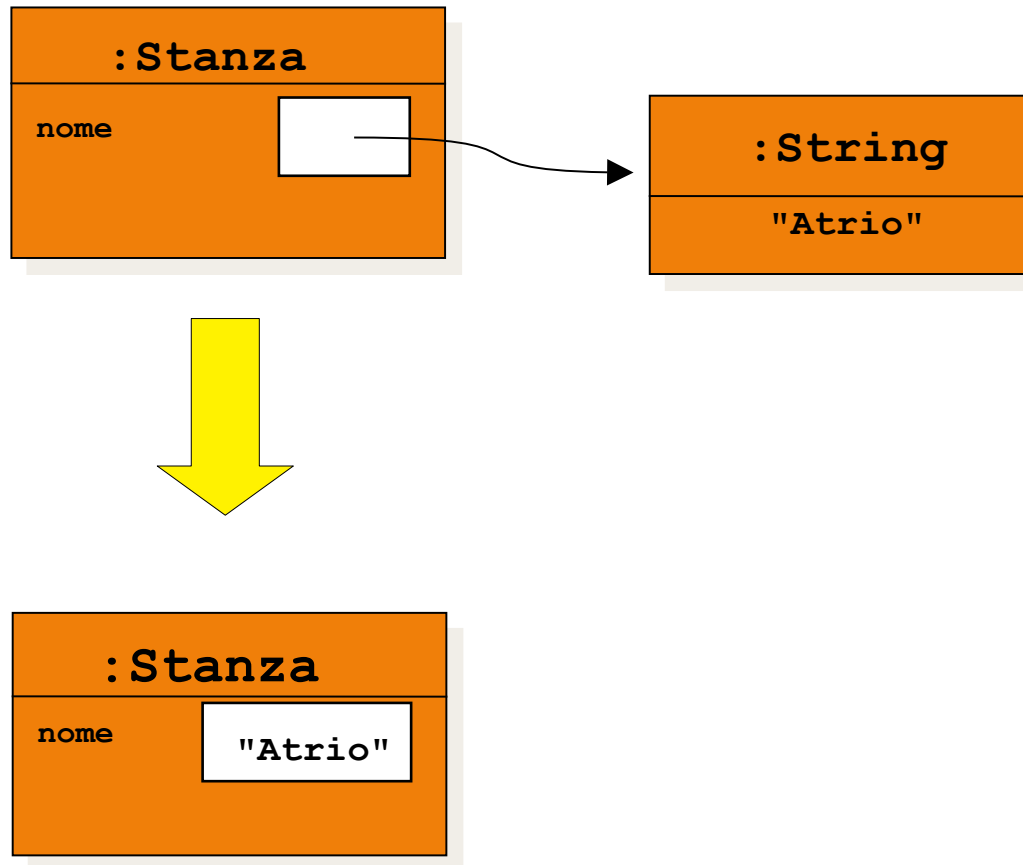
5

i2

5

String nei Diagrammi ad Oggetti

- Come il compilatore Java, anche noi riserviamo un trattamento di favore agli oggetti istanza della classe `String`



Tipi Primitivi e Oggetti (6)

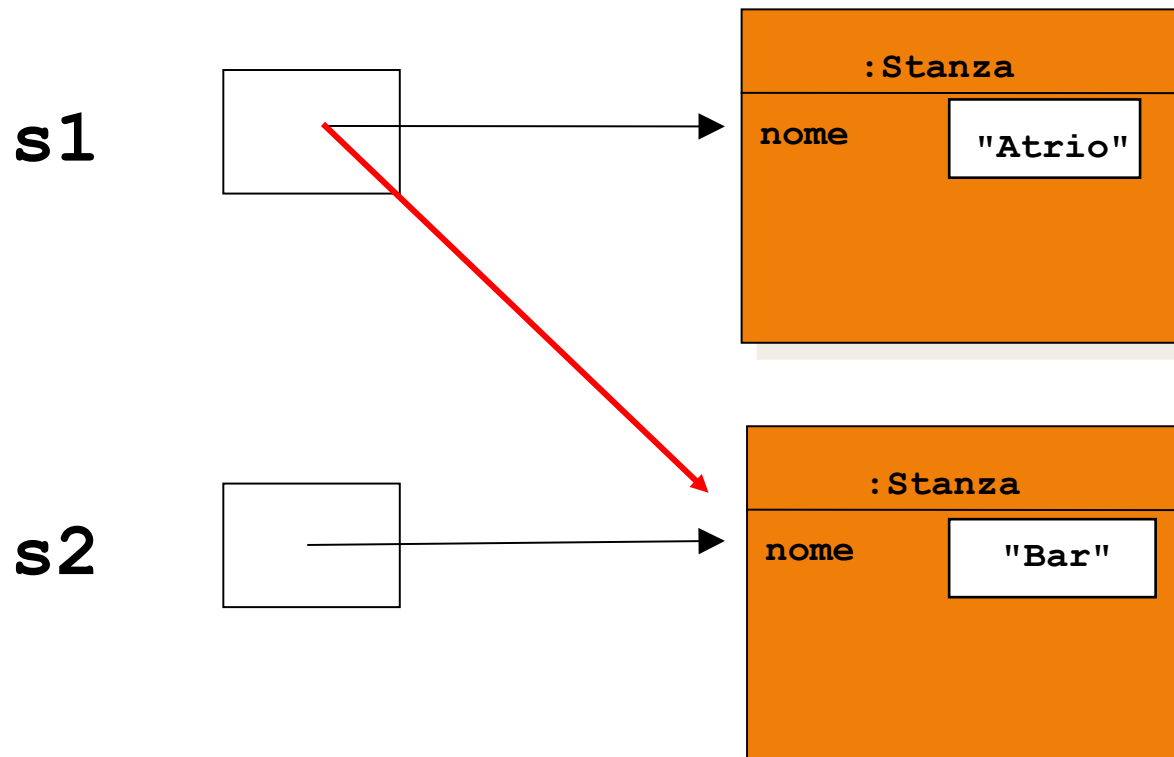
```
Stanza s1;
```

```
s1 = new Stanza("atrio");
```

```
Stanza s2;
```

```
s2 = new Stanza("bar");
```

```
s1 = s2;
```



Esercizio

- Disegnare il diagramma degli oggetti che rappresenta lo stato degli oggetti referenziati dalle variabili **a**, **b**, **c** del seguente programma al termine della esecuzione della istruzione in linea 3

```
1. Attrezzo a = new Attrezzo("spada", 40);  
2. Attrezzo b = new Attrezzo("scudo", 30);  
3. Attrezzo c = new Attrezzo("lancia", 10);  
4. a = b;
```

Esercizio

- Disegnare il diagramma degli oggetti che rappresenta lo stato degli oggetti referenziati dalle variabili **a**, **b**, **c** del seguente programma al termine della esecuzione della istruzione in linea **4**

```
1. Attrezzo a = new Attrezzo("spada", 40);  
2. Attrezzo b = new Attrezzo("scudo", 30);  
3. Attrezzo c = new Attrezzo("lancia", 10);  
4. a = b;
```

Esercizio

- Disegnare il diagramma degli oggetti che rappresenta lo stato del seguente programma al termine della esecuzione della istruzione in linea 5
- Quale valore ha il peso dell'attrezzo che si trova nella stanza referenziata dalla variabile **s** al termine della istruzione 4? E al termine della istruzione 5?

```
1. Attrezzo a = new Attrezzo("spada", 40);  
2. Attrezzo b = new Attrezzo("lancia", 10);  
3. Stanza s = new Stanza("N10");  
4. s.setAttrezzo(a);  
5. a = b;  
6. System.out.println(s.toString()); //quale attrezzo è  
7.                               //presente nella stanza s?
```

Array (1)

- Un array definisce una struttura di dati che memorizza un insieme di valori dello stesso tipo
- Dichiarazione di una variabile array:

```
int[] a;
```
- L'oggetto array va creato, specificando il numero di elementi

```
a = new int[10];
```

- Un array può essere inizializzato esplicitamente al momento della creazione:

```
int[] a = {21,12,23,34,15,21,7,80,1,-21};
```

- In ogni caso verrà inizializzato con dei valori di default

Array (2)

- Le sintassi
`int[] array` e `int array[]`
sono equivalenti.
- `int[] array` forse più esplicita:
 - già dal prefisso della dichiarazione si capisce che la variabile seguente è un array

Array (3)

- È possibile accedere a ciascun valore dell'array mediante un indice

```
int[] a;  
int i;  
a = new int[10];  
i = a[4];  
a[6] = 3*i;
```

- Attenzione: gli array in Java (come in C) usano *base-0*: l'indice del primo elemento è 0
- Per avere la dimensione di un array: **.length**
- Scansione degli elementi di un array:

```
for (int i = 0; i < a.length; i++)  
    System.out.println(a[i]);
```

Array (4)

- A partire da Java 5 è stata introdotta una **variante** del costrutto **for** (chiamata *for-each*) che consente di scorrere gli elementi di un array (e, come vedremo, anche di altre tipologie di collezioni) **senza gestire esplicitamente l'indice di iterazione:**

```
int a[];  
a = new int[100];  
for (int elemento : a)  
    System.out.println(elemento);
```

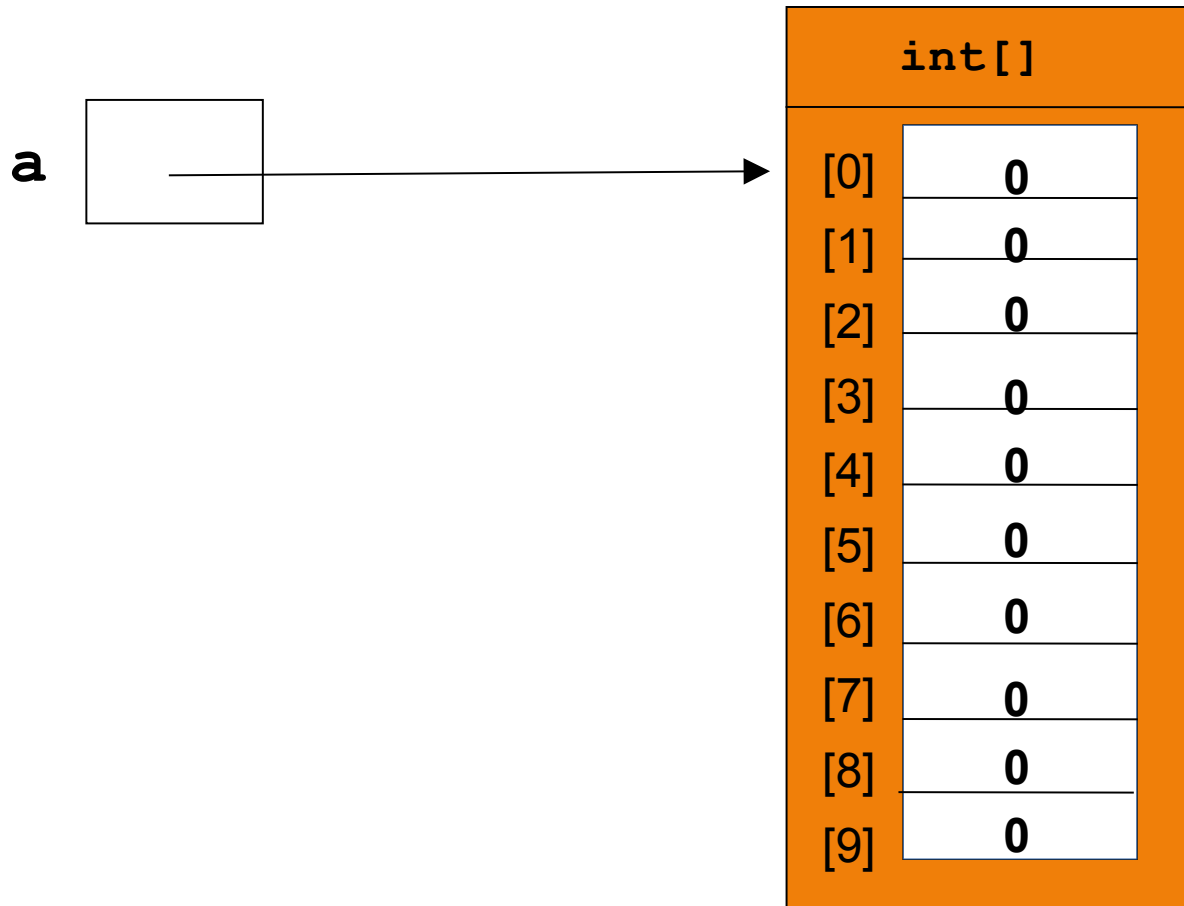
equivale a:

```
for (int i=0; i<a.length; i++) {  
    int elemento = a[i];  
    System.out.println(elemento);  
}
```

Array: Diagramma degli Oggetti

```
int[] a;
```

```
a = new int[10];
```

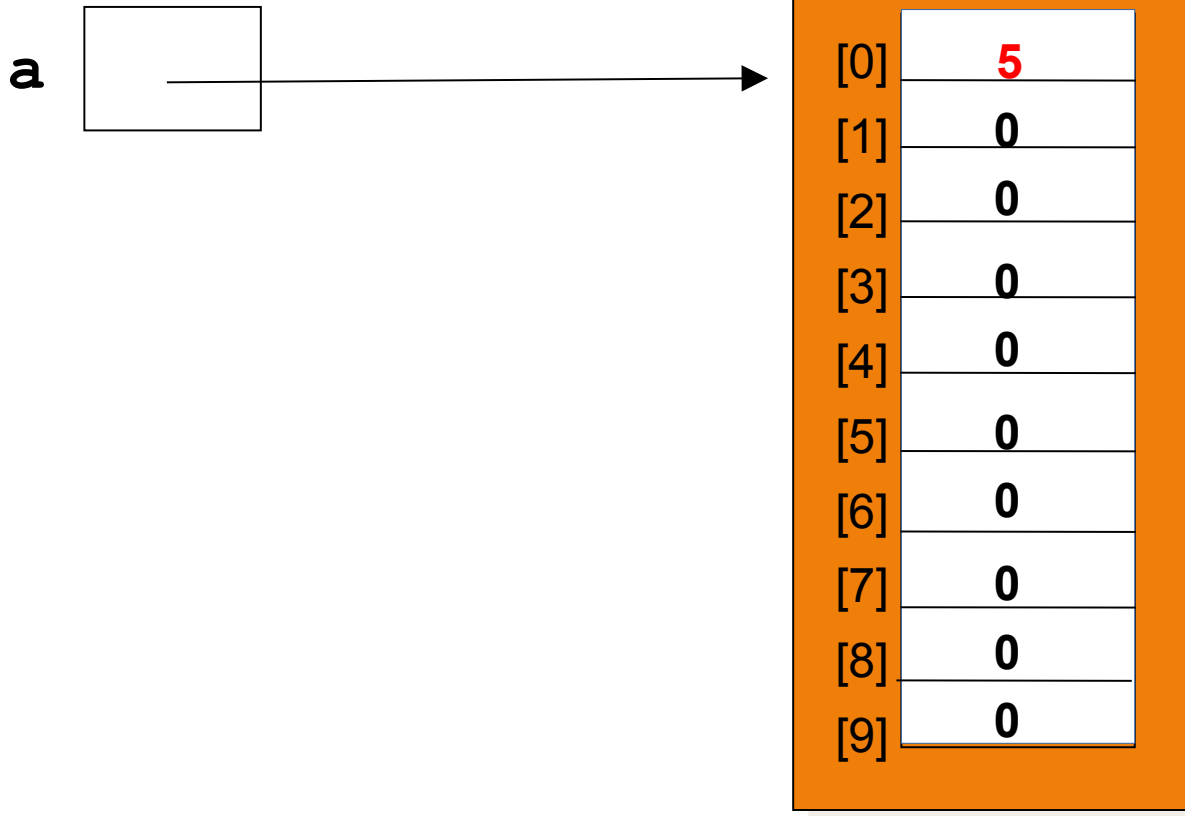


Array: Diagramma degli Oggetti

```
int[] a;
```

```
a = new int[10];
```

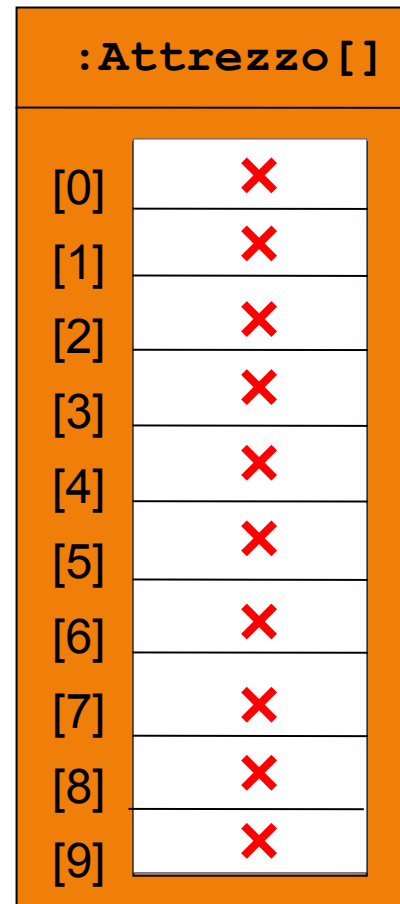
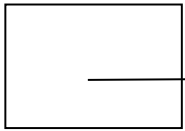
```
a[0] = 5;
```



Array: Diagramma degli Oggetti

```
Attrezzo attrezzi[];  
attrezzi = new Attrezzo[10];
```

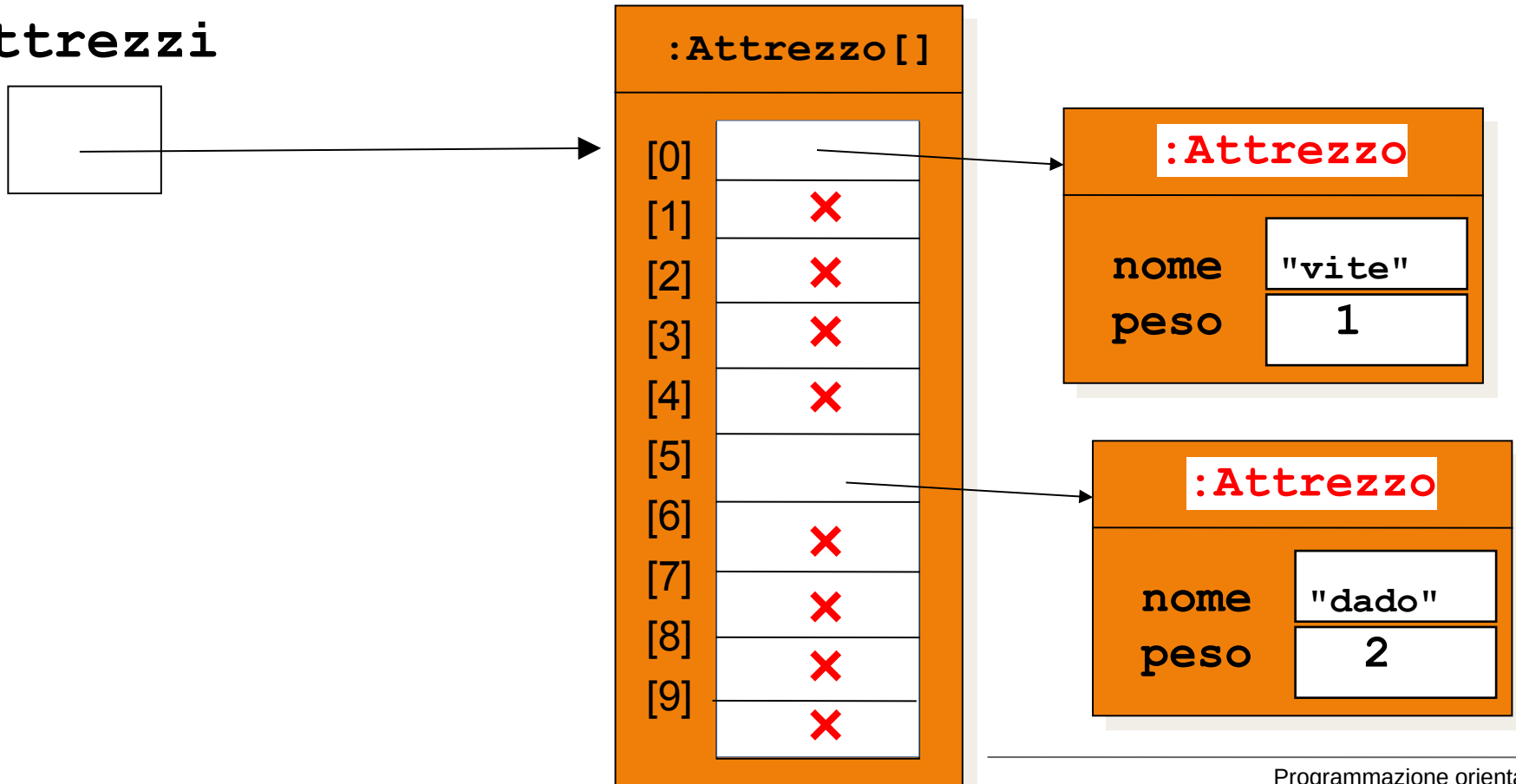
attrezzi



Array: Diagramma degli Oggetti

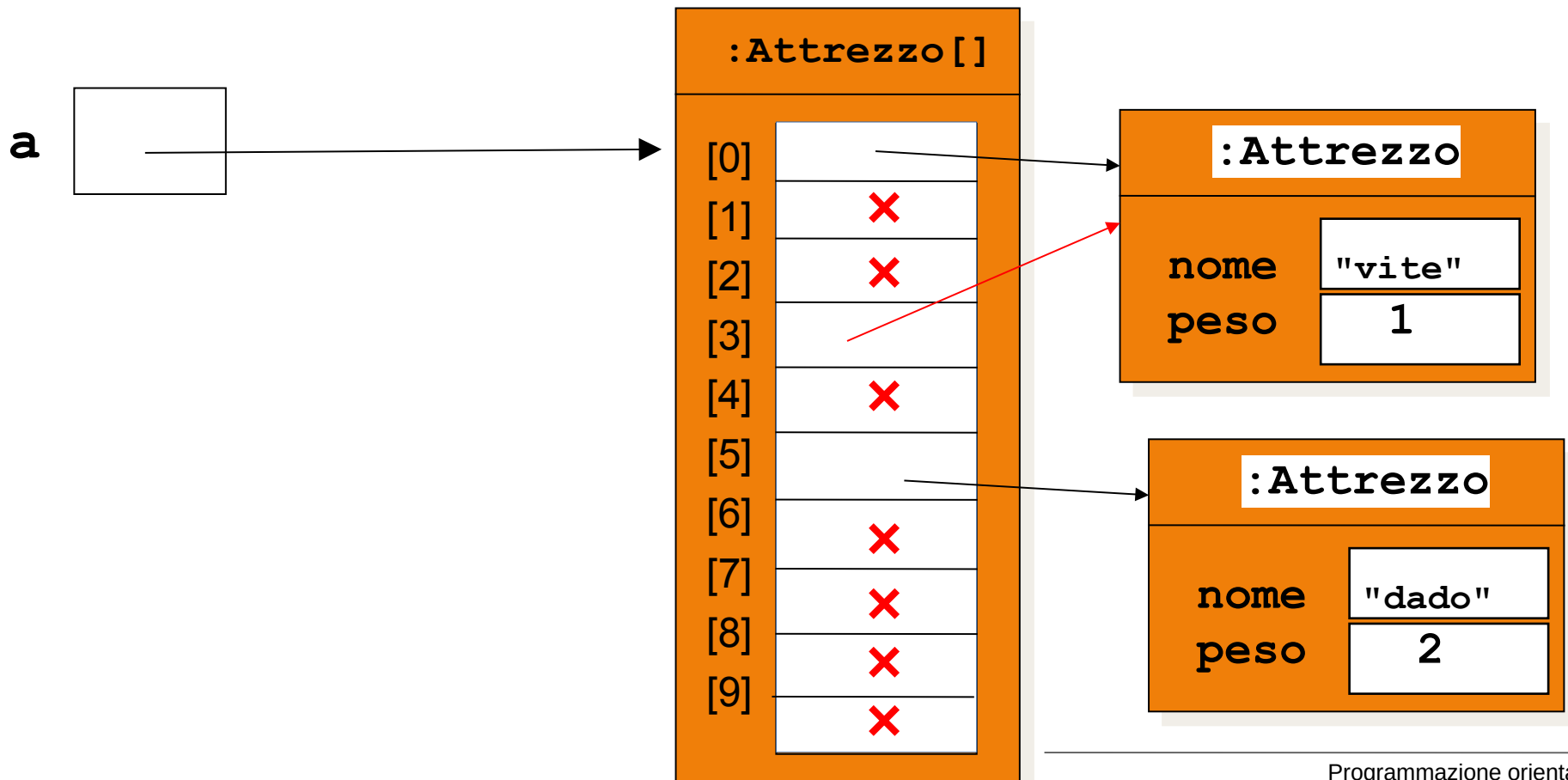
```
Attrezzo attrezzi[];  
attrezzi = new Attrezzo[10];  
attrezzi[0] = new Attrezzo("vite",1);  
attrezzi[5] = new Attrezzo("dado",2);
```

attrezzi



Array: Diagramma degli Oggetti

```
Attrezzo[] attrezzi;  
attrezzi = new Attrezzo[10];  
attrezzi[0] = new Attrezzo("vite",1);  
attrezzi[5] = new Attrezzo("dado",2);  
attrezzi[3] = attrezzi[0];
```



Esercizio

- Disegnare il diagramma degli oggetti che rappresenta lo stato del seguente programma al termine della esecuzione della istruzione in linea 7
- Quale valore ha il peso dell'attrezzo referenziato dalla variabile con indice 0 dell'array al termine della istruzione 6? E al termine dell'istruzione 7?

```
1. Attrezzo[] attrezzi;  
2. attrezzi = new Attrezzo[5];  
3. Attrezzo a = new Attrezzo("spada", 40);  
4. Attrezzo b = new Attrezzo("scudo", 30);  
5. attrezzi[0] = a;  
6. attrezzi[1] = b;  
7. a = b;
```


Esercizio

- Disegnare il diagramma degli oggetti che rappresenta lo stato del seguente programma al termine della esecuzione della istruzione in linea 7
- Quale valore ha il peso dell'attrezzo referenziato dalla variabile con indice 0 dell'array al termine della istruzione 6? E al termine della istruzione 7?

```
1.  Attrezzo[] attrezzi;  
2.  attrezzi = new Attrezzo[5];  
3.  Attrezzo a = new Attrezzo("spada", 40);  
4.  Attrezzo b = new Attrezzo("scudo", 30);  
5.  attrezzi[0] = a;  
6.  attrezzi[1] = b;  
7.  attrezzi[0] = attrezzi[1];
```

Esercizio (*con Eclipse*)

- Riscrivere la classe **Stanza** in modo che contenga un array di **Attrezzi**
 - Implementare quindi il metodo:
`boolean addAttrezzo(Attrezzo attrezzo)`
aggiunge un attrezzo nella stanza.
 - Se c'è spazio restituisce **true**; altrimenti restituisce **false**
 - quindi aggiungere il metodo
`boolean hasAttrezzo(String nomeAttrezzo)`
Controlla che nella stanza ci sia un attrezzo di nome `nomeAttrezzo`:
 - Se presente restituisce **true**; altrimenti **false**

Esercizio

```
public class Stanza {
    private Attrezzo[] attrezzi;
    private int numeroAttrezzi;
    private String nome;
    public Stanza(String nome) {
        this.nome = nome;
        this.attrezzi = new Attrezzo[10]; // per ora solo 10 attrezzi
        this.numeroAttrezzi = 0;
    }
    public boolean addAttrezzo(Attrezzo attrezzo) {
        if (this.numeroAttrezzi < 10) { // massimo 10 attrezzi
            this.attrezzi[numeroAttrezzi] = attrezzo;
            this.numeroAttrezzi++;
            return true;
        } else {
            return false;
        }
    }
    // ... continua ...
}
```

Esercizio

```
...  
  
public boolean hasAttrezzo(String nomeAttrezzo) {  
    boolean trovato;  
    trovato = false;  
    for (Attrezzo attrezzo : this.attrezzi) {  
        if (attrezzo.getNome().equals(nomeAttrezzo))  
            trovato = true;  
    }  
    return trovato;  
}  
  
} // fine classe Stanza
```

- L'equivalenza di stringhe viene controllata con il metodo `equals()`

Esercizio

- Scrivere il metodo

`Attrezzo getAttrezzo(String nomeAttrezzo)`

che restituisce l'attrezzo di nome `nomeAttrezzo` se esiste, `null` altrimenti

- Scrivere il metodo

`String toString()`

che restituisca una descrizione della stanza compresa una lista di tutti gli oggetti in essa contenuti

- Conviene scrivere un metodo `toString()` anche nella classe `Attrezzo`

Costanti

- Talvolta, vogliamo imporre che i valori di alcune variabili di istanza non possano essere cambiati
 - definiamo valori costanti
- A tal fine si usa la parola chiave `final`

```
final double NUMERO_MASSIMO_DIREZIONI = 4;  
NUMERO_MASSIMO_DIREZIONI = 20; // ERR. COMPILAZIONE
```

- Convenzione di stile: gli identificatori delle costanti si scrivono in maiuscolo

Variabili di Classe

- Talvolta è utile avere variabili che devono essere condivise da tutti gli oggetti della stessa classe: variabili di classe
- A tal fine si usa la parola chiave **static**

```
private static int perTutti;
```

- ATTENZIONE: Esistono importanti motivazioni didattiche per limitarne il più possibile l'uso
- Per il momento limitiamoci ad usare questa parola chiave solo ed esclusivamente per
 - dichiarare il metodo **main()**
 - definire costanti

Costanti e Variabili di Classe

- È ragionevole che una costante sia anche un variabile di classe. Perché?
- Per questo le dichiarazioni delle costanti di solito sono come segue:

```
final static double NUMERO_MASSIMO_DIREZIONI = 4;
```


Metodi di Classe (Statici)

- Attenzione: nella POO i metodi di classe sono una vera e propria *anomalia*
 - Durante l'apprendimento del paradigma OO sono quasi “pericolose”!
- Un metodo di classe corrisponde ad una operazione che può essere svolta senza utilizzare lo stato dell'oggetto (oppure usando solo variabili di classe, condivise da tutti gli oggetti)
- Tipicamente si usano per realizzare funzioni pure (es. i metodi di `java.lang.Math`)
- Sono come il freno a mano di un'automobile:
 - Molto utile da fermi...
 - ...ma non usatelo mai per frenare in corsa!!

Ancora su final (1)

- Attenzione:

```
final int a = 10;  
a = 3; // ERRORE DI COMPILAZIONE
```

```
final int a = 10;  
int b = 4;  
a = b; // ERRORE DI COMPILAZIONE
```

Ancora su final (2)

- Attenzione:

```
final Stanza ds1 = new Stanza("Aula DS1");  
Stanza n7 = new Stanza("Aula N7");  
  
ds1 = n7; // ERRORE DI COMPILAZIONE
```

```
final Stanza ds1 = new Stanza("Aula DS1");  
Stanza n7 = new Stanza("Aula N7");  
Attrezzo v = new Attrezzo("vite",1);  
  
ds1.setAttrezzo(v); // LECITO! PERCHE' ?
```

Ancora su `final` (3)

- In sostanza, quando `final` si usa su
 - primitivi: rende costante la variabile
 - riferimenti: rende costante il riferimento, ma non il contenuto dell'oggetto referenziato, che rimane libero di cambiare

Studio di Caso (1)

- Il gioco di ruolo diadico
 - In questa versione iniziale ci concentriamo su poche classi
 - La prima versione, oltre ad essere molto semplice, ha un codice scritto piuttosto male:
 - Ci sono errori (a tempo di esecuzione)
 - È di difficile manutenzione
 - È poco riutilizzabile
 - Il primo aspetto (errori) lo verificherete immediatamente eseguendo il programma
 - Capire come ovviare agli altri due aspetti è uno degli obiettivi formativi del corso

Studio di Caso (2)

- Scaricare lo studio di caso versione base:
 - <https://sites.google.com/view/rm3-poo/materiale-didattico>
- Eseguire il metodo `main()` nella classe `DiaDia`
- Digitare alcuni comandi
 - `aiuto` per avere un elenco dei comandi
 - `vai nord|sud|est|ovest` per cambiare stanza
- Spostandoci da una stanza all'altra noteremo presto errori (uno porta alla terminazione del programma)

Esercizio

- Studiare a fondo il codice della classe `Stanza`
 - a che cosa serve la variabile `numeroDirezioni`?
 - che cosa fanno i metodi `impostaStanzaAdiacente(String, Stanza)` e `getStanzaAdiacente(String)`?

Esercizio

- Scrivere una classe `StanzaTest1` con il metodo `main()` con le istruzioni per:
 - Definire due oggetti `Stanza`: *bar* e *mensa*
 - Impostare le uscite dei due oggetti affinché:
 - L'uscita nord del *bar* porti nella *mensa*
 - L'uscita sud della *mensa* porti nel *bar*
 - Stampare la descrizione della stanza dietro la porta nord del *bar*
 - Stampare la descrizione della stanza dietro la porta sud della *mensa*
- ✓ Controllare che le stampe siano quelle attese

Esercizio

- Scrivere una classe **StanzaTest2** con un metodo **main()**:
 - Definire due oggetti **Stanza**: *bar* e *mensa*
 - Definire due oggetti **Attrezzo**: *tazzina* e *piatto*
 - Impostare le uscite dei due oggetti **Stanza** affinché:
 - L'uscita nord del *bar* porti nella *mensa*
 - L'uscita sud della *mensa* porti nel *bar*
 - Aggiungere nel *bar* l'oggetto **Attrezzo** *tazzina*
 - Aggiungere nella *mensa* l'oggetto **Attrezzo** *piatto*
 - Stampare il nome e il peso dell'attrezzo presente nella stanza dietro la porta nord del *bar*
 - Stampare il nome e il peso dell'attrezzo presente nella stanza dietro la porta sud della *mensa*
 - ✓ Controllare che le stampe siano quelle attese

Programmazione Orientata agli Oggetti

Qualità del codice:
Java Base Library
Documentazione
Package

Sommario

- Java Base Libraries
- Consultare la documentazione
- Produrre la documentazione delle proprie classi
- Package

Java Base Libraries

- Migliaia di classi
- Decine di migliaia di metodi
- Molte classi utili che ci semplificano drasticamente la vita
- Un programmatore competente deve essere in grado di lavorare con le librerie

Java Base Libraries

- Un programmatore competente dovrebbe saper costruire le proprie classi, ma anche sapere quando è inutile scriverne di nuove
- Per poter usare efficacemente una libreria, bisogna:
 - Conoscere alcune sue importanti classi per nome
 - Sapere come trovare e usare altre classi
- Importante:
 - Serve conoscere solo l'interfaccia, non l'implementazione

Documentazione di Librerie (1)

- La documentazione delle librerie Java è in formato HTML
 - javadoc
- Si può leggere agevolmente con un browser
- Class API: *Application Programmers' Interface*
 - Descrizione delle interfacce per tutte le classi della libreria

Documentazione di Librerie (2)

- *La documentazione include*
 - Il nome della classe
 - Una descrizione generale della classe
 - Una lista dei costruttori e dei metodi
 - Valori di ritorno e parametri per costruttori e metodi
 - Una descrizione dello scopo di ciascun costruttore e di ciascun metodo
- *Come si usa la classe: tutto quello che serve al programmatore-utilizzatore*

Documentazione di Librerie (3)

- *La documentazione non include*
 - Campi privati
(tutti i campi dovrebbero essere privati)
 - Metodi privati
 - Il corpo dei metodi e dei costruttori
- Dettagli sull'implementazione: non servono al *programmatore-utilizzatore*
- Anzi: potrebbe risultare controproducente doverli necessariamente conoscere

Produrre Documentazione (1)

- Le classi che progettiamo dovrebbero essere documentate come le classi della libreria
- Altri programmatori devono essere in grado di usare le nostre classi senza conoscere l'implementazione di dettaglio

Elementi della Documentazione

- *La documentazione di una classe dovrebbe includere*
 - Il nome della classe
 - Un commento che descriva lo scopo e caratteristiche generali della classe
 - Un numero di versione
 - Il nome degli autori
 - Riferimenti ad altre classi
 - Documentazione per ciascun costruttore e per ciascun metodo

Documentare Costruttori e Metodi

- *La documentazione di ciascun costruttore / metodo dovrebbe includere*
 - Nome e tipo di ciascun parametro
 - Una breve descrizione di ciascun parametro
 - Una descrizione dello scopo e della funzione del costruttore/metodo
 - Il nome del metodo
 - Il tipo di ritorno
 - Una descrizione del valore ritornato

Produrre Documentazione (2)

- Un'idea tanto semplice quanto efficace
 - Documentazione in formato ipertestuale
 - pagine web
 - La documentazione viene generata dai commenti immersi direttamente nel codice
 - Basta usare una semplice sintassi pensata allo scopo
 - e l'utility `javadoc`

Commenti di Documentazione

- Tutti i comandi javadoc si trovano solo entro commenti `/** ... */`
- Speciali marcatori (immersi nei commenti) permettono di definire aspetti specifici della documentazione
- Per l'elenco completo dei marcatori javadoc si consulti la documentazione:

<http://docs.oracle.com/javase/1.5.0/docs/tooldocs/solaris/javadoc.html#javadoctags>

Documentazione delle Classi

- *Forma generale*

```
/**  
 * Nome-classe: commento che descrive  
 * scopo e caratteristiche generali della classe  
 *  
 * @author nome-autore  
 * @see riferimento ad altra classe  
 * @see riferimento ad altra classe  
 * @version versione  
 */  
public class Nome-classe {
```

Esempio

```
/**
 * Una semplice classe che modella un attrezzo.
 * Gli attrezzi possono trovarsi all'interno delle stanze
 * del labirinto.
 * Ogni attrezzo ha un nome ed un peso.
 *
 * @author    docente di POO
 * @see Stanza
 * @version 0.9
 *
 */
public class Attrezzo {
    ...
}
```

Documentazione dei Costruttori

- *Forma generale*

```
/**  
 * Commento che descrive scopo e caratteristiche  
 * generali del costruttore  
 *  
 * @param nome-parametro breve descrizione  
 */  
Nome-classe(...) {
```


Esempio

```
/**
 * Crea un attrezzo
 * @param nome il nome che identifica l'attrezzo
 * @param peso il peso dell'attrezzo
 */
public Attrezzo(String nome, int peso) {
    this.peso = peso;
    this.nome = nome;
}
```

Documentazione dei Metodi

- *Forma generale*

```
/**  
 * Commento che descrive scopo e caratteristiche  
 * generali del metodo  
 *  
 * @param nome-parametro breve descrizione  
 * @return valore di ritorno, breve descrizione  
 */  
public type nome-metodo(...) {
```

Esempio

```
/**
 * restituisce il nome identificatore dell'attrezzo
 * @return identificatore dell'attrezzo
 */
public String getNome() {
    return this.nome;
}
```

```
/**
 * restituisce il peso dell'attrezzo
 * @return peso dell'attrezzo
 */
public int getPeso() {
    return this.peso;
}
```

Generare Documentazione

- Si usa il tool **javadoc**

- Per dettagli

javadoc -h

- Per generare la documentazione

javadoc *.java

Package

- Le classi sono raggruppate in package
- Questo raggruppamento consente di:
 - Mantenere assieme classi concettualmente e logicamente correlate
 - Creare spazi di nomi che evitino conflitti
 - Definire un dominio di protezione (cfr modificatori di accesso)

Import

- Una classe può usare tutte le classi dello stesso package e tutte le classi *pubbliche* di altri package
- Si può accedere alle classi pubbliche di un altro package in due modi
 - Usando il nome completamente qualificato di una classe, cioè anteponendo il nome del pacchetto alla classe:

```
java.util.Scanner s =  
    new java.util.Scanner(input) ;
```

- Importando la classe e scrivendone direttamente il nome

```
import java.util.Scanner;  
  
...  
Scanner s = new Scanner(input) ;
```

Package

- È bene organizzare il proprio codice organizzando le classi in package
- Le classi che appartengono ad un package devono dichiarare la propria appartenenza al package tramite la dichiarazione

`package nome-package;`

- La dichiarazione di appartenenza ad un package deve comparire all'inizio del file
- Una classe può appartenere al più ad un package

Package, Convenzioni sui Nomi

- Il nome di un package deve essere univoco.
- A tal fine **di solito** il nome del package comprende il nome del dominio Internet dell'organizzazione, scritto in ordine inverso
`package it.uniroma3.diadia;`

Package e Classi Pubbliche

- Una classe può essere usata al di fuori del package solo se è dichiarata **pubblica**

- Esempio:

```
package it.uniroma3.diadia;  
public class Stanza {  
    ...  
}
```

la classe **Stanza** può essere usata al di fuori del package `it.uniroma3.diadia` (importandola)

- Se invece scrivessimo:

```
package it.uniroma3.diadia;  
class Stanza {  
    ...  
}
```

la classe **Stanza** non potrebbe essere usata fuori dal package `it.uniroma3.diadia`

Package

- Il nome di un package possiede una struttura gerarchica
- Tale struttura deve trovare corrispondenza diretta nel file system
- Ad esempio le classi del package

`it.uniroma3.diadia`

devono essere memorizzate nella cartella

`it/uniroma3/diadia`

Compilazione ed Esecuzione di Classi nei Package

- Per compilare le classi di un package si deve far riferimento alla gerarchia fisica
- Con un buon IDE (ad es. Eclipse: cfr. *source folders*) il processo è quasi completamente trasparente
- Da riga di comando questo può essere un po' articolato:
 - Supponiamo di mettere tutto il nostro codice nella directory `c:\src`
 - La versione *base* di `diadia` è nel package:
`it.uniroma3.diadia`
quindi le classi Java sono nella directory
`src/it/uniroma3/diadia`
 - Per **compilare** una classe (ad esempio la classe `stanza.java`) del package:
 - dalla radice del package, cioè dalla directory che contiene la directory `it` (supponiamo `c:\src`)
`javac it/uniroma3/diadia/Stanza.java`
 - oppure, dalla directory in cui si trovano le classi da compilare
`javac -classpath "C:\src\" Stanza.java`
(supponendo che la directory `it` sia nella directory `src` del volume C:)
 - Per **eseguire** una classe di un package si deve far riferimento alla gerarchia logica
 - dalla radice del package (cioè dalla directory che contiene la directory `it`)
`java it.uniroma3.diadia.DiaDia`
 - oppure da una qualunque directory
`java -classpath "C:\src" it.uniroma3.diadia.DiaDia`

Ricapitolazione

- Java offre un insieme estremamente vasto e ricco di librerie
- Le librerie sono documentate in un formato standard
- Nella definizione delle nostre classi è possibile creare automaticamente documentazione standard
 - Usando opportunamente i commenti `javadoc`
- Le librerie (e le applicazioni) sono organizzati in package
 - Creazione di uno spazio univoco dei nomi
 - Raggruppamento delle classi
 - Definizione di un nuovo livello di visibilità

Esercizio

- Mettere tutte le classi dello studio di caso (versione base) in un package diadia
- Compilare ed eseguire il programma

Programmazione Orientata agli Oggetti

Qualità del codice:
Coesione e Accoppiamento

Contenuti

- Accoppiamento
- Coesione
- Introduzione a:
 - Testing
 - Refactoring

Il Software Evolve

- Il software evolve in continuazione
- ...inevitabilmente...
- Viene esteso, corretto, mantenuto, portato su altre piattaforme, adattato, ...
- Molte persone, in tempi diversi partecipano a questo processo
- Se il costo dell'evoluzione è troppo alto, il software viene gettato

Qualità del Codice

- La qualità del codice dipende da due fattori importanti:
 - Accoppiamento (coupling)
 - Coesione (cohesion)

Accoppiamento

- Due o più unità di un programma si dicono accoppiate quando è impossibile modificare una senza dover modificare anche le altre
- L'accoppiamento si riferisce ai legami tra unità separate e distinte di un programma
- Se due classi dipendono strettamente e per molti dettagli l'una dall'altra, diciamo che sono *strettamente accoppiate*
- Per un codice di qualità dobbiamo puntare ad un *basso accoppiamento*

Duplicazione del Codice

- “Codice Copia e Incolla”
- La duplicazione del codice
 - è sintomo di un cattivo progetto
 - porta facilmente alla propagazione di errori durante lo sviluppo
 - rende difficile la manutenzione
 - porta inevitabilmente alla introduzione di errori nelle attività di manutenzione
- E' una forma elementare di accoppiamento

Basso Accoppiamento

- Un basso accoppiamento permette di:
 - Capire il codice di una classe senza leggere i dettagli delle altre
 - Modificare una classe senza che le modifiche comportino conseguenze sulle altre classi
- Quindi un basso accoppiamento migliora la manutenibilità del software

Coesione

- La coesione fa riferimento al numero e alla eterogeneità dei compiti di cui una singola unità è responsabile
- Se ciascuna unità è responsabile di un singolo compito, diciamo che tale unità possiede una *alta coesione*
- La coesione si applica alle classi e ai metodi (ed anche ai package!)
- Perseguiamo l'alta coesione

Alta Coesione

- Un'alta coesione favorisce:
 - La comprensione dei compiti di una classe o di un metodo
 - L'utilizzo di nomi appropriati, efficaci, comunicativi
 - Il riuso delle classi e dei metodi

Coesione

- Coesione dei metodi
 - Un metodo dovrebbe essere responsabile di un solo compito ben definito
- Coesione delle classi
 - Ogni classe dovrebbe rappresentare un singolo concetto ben definito

Coesione ed Accoppiamento

- Sono le due facce della stessa medaglia
 - L'alta coesione di una classe si ottiene perseguendo lo scarso accoppiamento di quella classe verso altre classi
 - Lo scarso accoppiamento di una classe verso le altre classi si ottiene perseguendo la sua alta coesione

Localizzare le Modifiche

- Basso accoppiamento e alta coesione portano ad una localizzazione delle modifiche
- Quando è necessario operare una modifica, il minor numero possibile di classi dovrebbero essere coinvolte
- La qualità del proprio codice si osserva proprio quando sorge l'esigenza di fare modifiche

Pensare in Avanti

- Quando progettiamo una classe dovremmo sforzarci di pensare a
 - quali cambiamenti potranno essere richiesti in futuro
 - come verrà usata la nostra classe dal programmatore-utilizzatore
- Le nostre scelte iniziali potrebbero facilitare l'evoluzione futura (su questo aspetto faremo esperienza nello studio di caso ed in altri esercizi)

Qualità Interna vs Qualità Esterna del Codice

- La qualità del codice può essere osservata da molti punti di vista ben distinti
- Due particolarmente interessanti per noi
 - Quello degli utilizzatori finali: *qualità esterna*
 - Quello degli sviluppatori: *qualità interna*
- Gli utilizzatori finali possono fornire un giudizio di merito sulla capacità di un applicativo di rispondere ai requisiti ed alle proprie esigenze
- Solo gli sviluppatori possono valutare la manutenibilità del codice nel momento in cui nasce la necessità di modificarlo

“Refactoring”

- La manutenzione del software spesso richiede l'aggiunta di nuovo codice
- Le classi e i metodi tendono così a diventare più lunghi, a perdere in coesione, ad aumentare l'accoppiamento verso altre porzioni di codice
- A seguito delle modifiche, per mantenere un'alta coesione ed un basso accoppiamento, classi e metodi dovranno essere riorganizzate
- Questo processo di riorganizzazione del codice viene definito “*refactoring*”
- ✓ E' il principale strumento che uno sviluppatore possiede per controllare la qualità **interna** del codice
 - Modifiche del codice che non alterano il funzionamento
 - Ma preparano il codice ad accogliere le modifiche future

Refactoring e Testing

- Quando modifichiamo il codice, è necessario isolare le conseguenze del refactoring da altri fattori
- Quindi la riorganizzazione delle classi e dei metodi (il refactoring) deve essere effettuata prima di introdurre nuove funzionalità
- Come contrastare la naturale paura di fare modifiche su un base di codice che si considerava funzionante?
- Per assicurarci di non aver introdotto nuovi errori, eseguiamo i test prima e dopo ogni azione di refactoring
 - Vedremo come organizzare metodicamente refactoring e testing

Linee Guida (1)

- Domande comuni
 - Quanto dovrebbe essere lunga una classe?
 - Quanto dovrebbe essere lungo un metodo?
- Possiamo rispondere in termini di coesione e accoppiamento

Linee Guida (2)

- Un metodo è troppo lungo se è responsabile di più di un compito logico
- Una classe è troppo complessa se rappresenta più di un concetto, se ha più di una responsabilità
- **Nota:** queste sono *linee guida* – solo attraverso l'esperienza si riescono a concretizzare

Caso di Studio: le classi **Diadia** e **Partita**

- Le classi **Diadia** e **Partita** sono particolarmente lunghe
- Se guardiamo il loro codice ci accorgiamo che hanno diverse (troppe!) responsabilità
 - **Diadia** implementa la logica di tutti i possibili comandi
 - **Partita** gestisce lo stato del gioco e crea il labirinto
- Questa miriade di responsabilità è indice di poca coesione

Aumentiamo la Coesione

- Possiamo iniziare a migliorare la coesione della classe **Partita** togliendole qualche responsabilità
- Iniziamo a togliere la responsabilità di creare e gestire il labirinto
- Affidiamo questa responsabilità ad una nuova classe appositamente introdotta: **Labirinto**

Esercizio

- Creare la classe **Labirinto** e modificare la classe **Partita** affinché non abbia la responsabilità della creazione del labirinto
 - Un labirinto ha una entrata (stanza di ingresso) ed una uscita (stanza vincente)
 - La classe **Labirinto** ha un metodo privato **init()** che inizializza il labirinto
- Provare ad eseguire il codice del gioco prima e dopo le modifiche e verificare che il comportamento sia rimasto invariato

Refactoring ...

- Un programmatore OO deve avere forte senso critico, evidenziare i limiti delle proprie soluzioni sia in termini di funzionalità che di qualità del codice per risolverli attraverso disciplinati passi di refactoring
 - N.B. con un pizzico di attenzione al pericolo della sovra-ingegnerizzazione
 - ✓ bisogna risolvere i problemi di oggi del progetto! Poi, se c'è la possibilità, anche quelli che si prevedono
 - ✓ mai compromettere le soluzioni di **oggi** per bisogni che **forse** avremo in futuro

Refactoring ...

- Concludiamo citando alcune problematiche risolvibili efficacemente solo dopo aver studiato il **polimorfismo**
 - Forte accoppiamento tra l'insieme dei comandi disponibili e la classe **Diadia**: risolvendo il primo homework è evidente che ogni volta che aggiungiamo/modifichiamo un comando dobbiamo agire su questa classe
 - Anche dopo i passi di refactoring precedenti la classe **Diadia** continua ad addossarsi diverse (troppe!) responsabilità
 - Implementa la logica del gioco
 - Implementa la logica di tutti i possibili comandi
 - Ma è possibile che la struttura del labirinto sia cablata all'interno della classe **Labirinto**? E pensando ad un gioco a più livelli di difficoltà con diversi labirinti?
 - Se volessi cambiare labirinto o comunque riutilizzare un oggetto istanza della classe **Partita** con diversi labirinti, potrei farlo?

Ricapitoliamo

- I programmi SW sono in continua evoluzione
- E' importante facilitare e prevedere questa evoluzione
- La qualità del codice richiede molto più del corretto funzionamento di un programma in un preciso momento
 - “funziona!”... condizione necessaria, ma non sufficiente
 - in altri termini:
 - se funziona non è detto che vada bene
 - se non funziona certamente non va bene
- Il codice deve essere comprensibile e manutenibile

Ricapitoliamo

- Codice di buona qualità: “no copia e incolla”, alta coesione, basso accoppiamento
- Anche lo stile di codifica (identificatori, indentazione, spaziatura) è fondamentale (il programma è uno strumento di comunicazione)
 - codice ben scritto non ha bisogno di commenti!
- Il costo di manutenzione del software dipende fortemente dalla qualità del codice ed è ormai da tempo noto essere la voce di costo preponderante sul medio/lungo termine

Programmazione Orientata agli Oggetti

Qualità del Codice:
Introduzione Unit-Testing



Sommario

- Errori nel software
 - errori di compilazione vs bug
 - località dei bug
- Testing
 - motivazioni
 - le tre fasi di un test
- Unit-Testing
- Introduzione a JUnit
- Qualità dei test
- Testing Continuo
- TDD

Software ed Errori

- I primi errori con i quali ci scontriamo di solito sono *errori di sintassi*
 - Ci vengono indicati dal compilatore
- Successivamente incorriamo in *errori logici*
 - Il compilatore non ci può aiutare
 - Sono noti anche come “*bug*” (*bachi*)
- Alcuni errori logici non si manifestano immediatamente
 - Il software è estremamente complesso
 - Anche il software commerciale non è affatto privo di errori

Errori di Compilazione

- Il compilatore ci dà indicazioni precise e molto utili a correggere l'errore
- Il messaggio di errore del compilatore VA LETTO E CAPITO

LINEA DI CODICE IN CUI E' STATO RISCONTRATO L'ERRORE

ERRORE RISCONTRATO

```
Diadia.java:27:invalid method declaration;  
return type required  
    private creaStanze() {  
                ^
```

1 error

Errori a Tempo di Esecuzione

- Anche in questo caso abbiamo informazioni molto precise (dalla macchina virtuale)

```
Exception in thread "main"  
java.lang.NullPointerException
```

```
    at Diadia.vaiNellaStanza(Gioco.java:176)  
    at Diadia.processaComando(Gioco.java:117)  
    at Diadia.gioca(Gioco.java:71)  
    at Diadia.main(Gioco.java:209)
```

Motivazioni del Testing

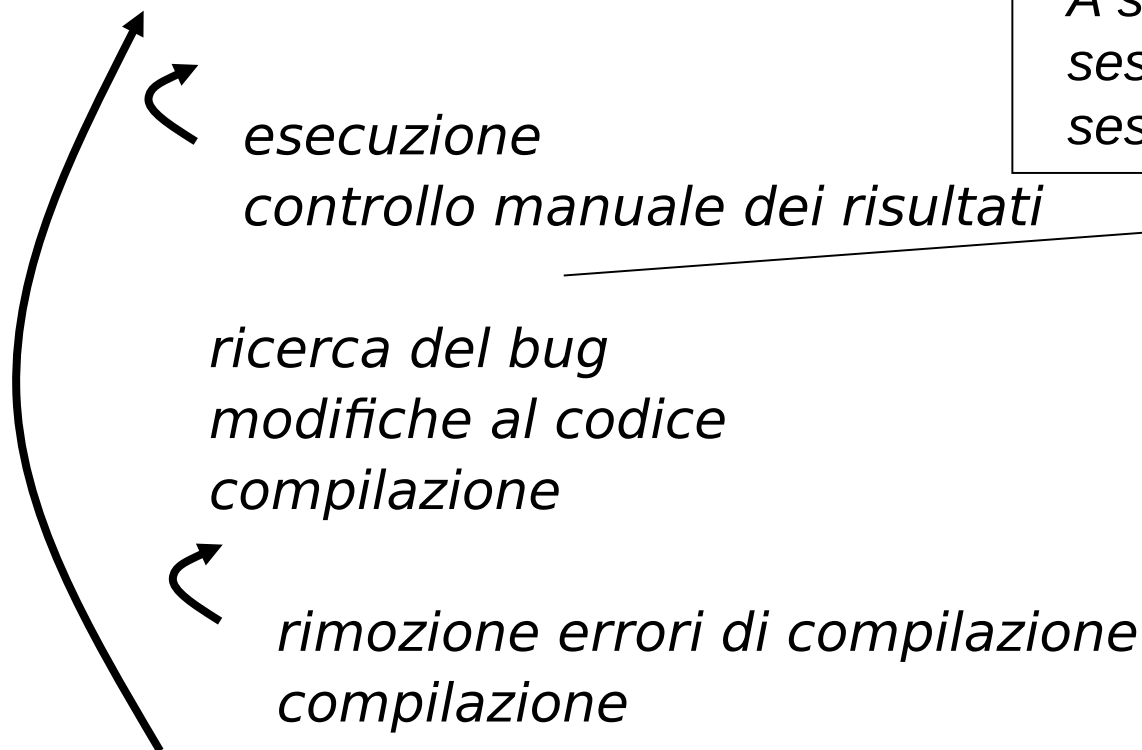
- I programmi sono descrizioni “statiche” a cui possono corrispondere molteplici esecuzioni “dinamiche”
- I compilatori moderni sono in grado di indicare esattamente posizione e motivo degli errori di compilazione
 - addirittura già mentre si scrive! (compilazione *incrementale*)
- Al contrario i compilatori NON possono prevedere come evolverà l'esecuzione di un programma e non sono in grado di individuare gli errori dei programmatori (né possono sapere cosa intendessero esprimere con il proprio codice)
- In sintesi:
 - il compilatore ci aiuta sugli aspetti statici (ad es. analizzando i tipi)
 - il compilatore non dice nulla di nuovo sugli aspetti dinamici (più di quanto non sia già implicato dagli aspetti statici)

I Bug

- I bug sono errori nell'evoluzione dinamica di un programma su cui il compilatore non ha potuto prevedere e dire nulla
- Il debugging è completamente a carico del programmatore
- Il costo di debugging è ritenuto di gran lunga la componente principale nel costo dei moderni progetti software

Ciclo di Debugging

- Come si effettua il debugging di un programma che compila? Con estenuanti cicli:

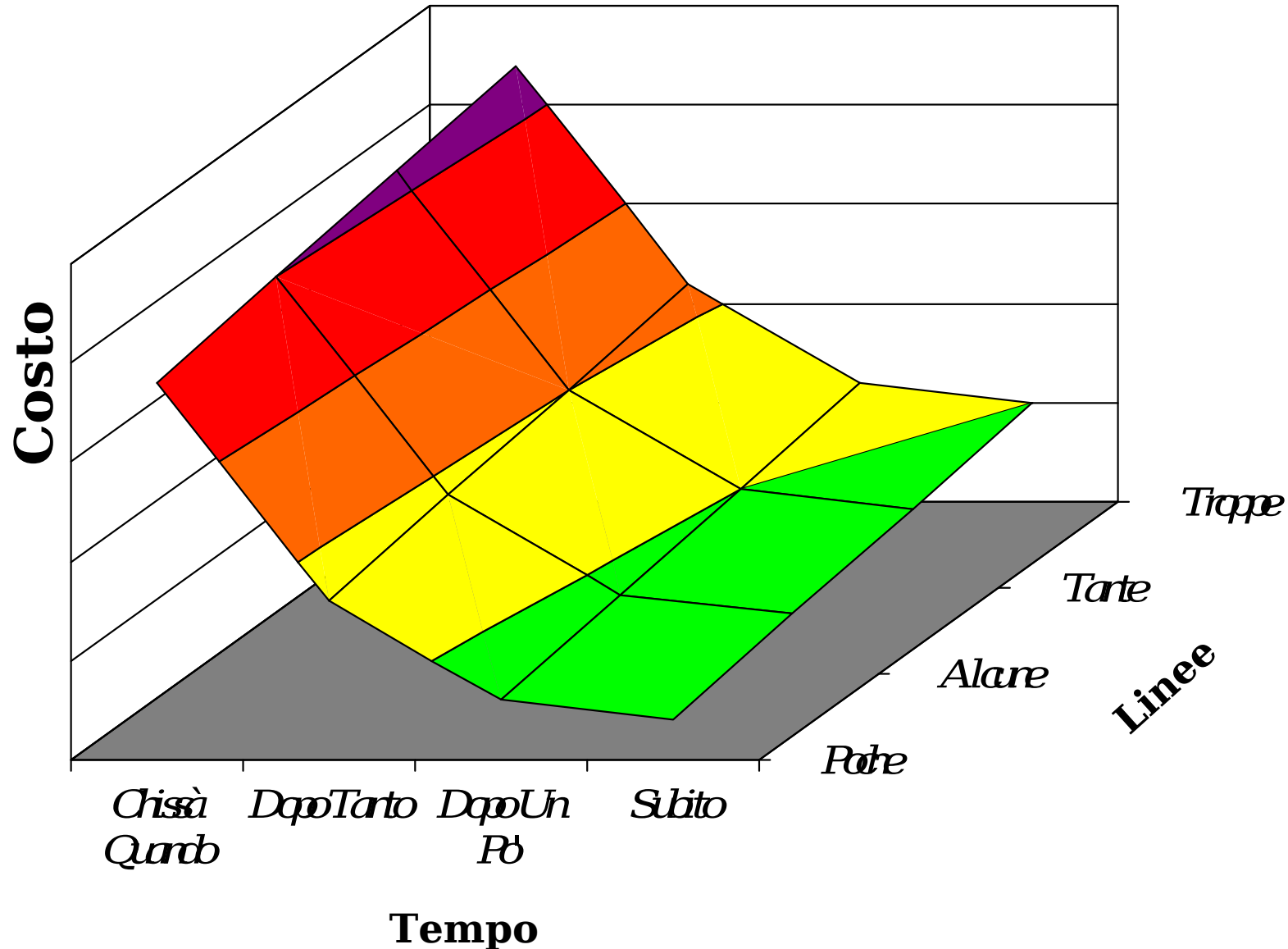


*A sua volta può richiedere:
sessioni di tracing/logging
sessioni con il debugger*

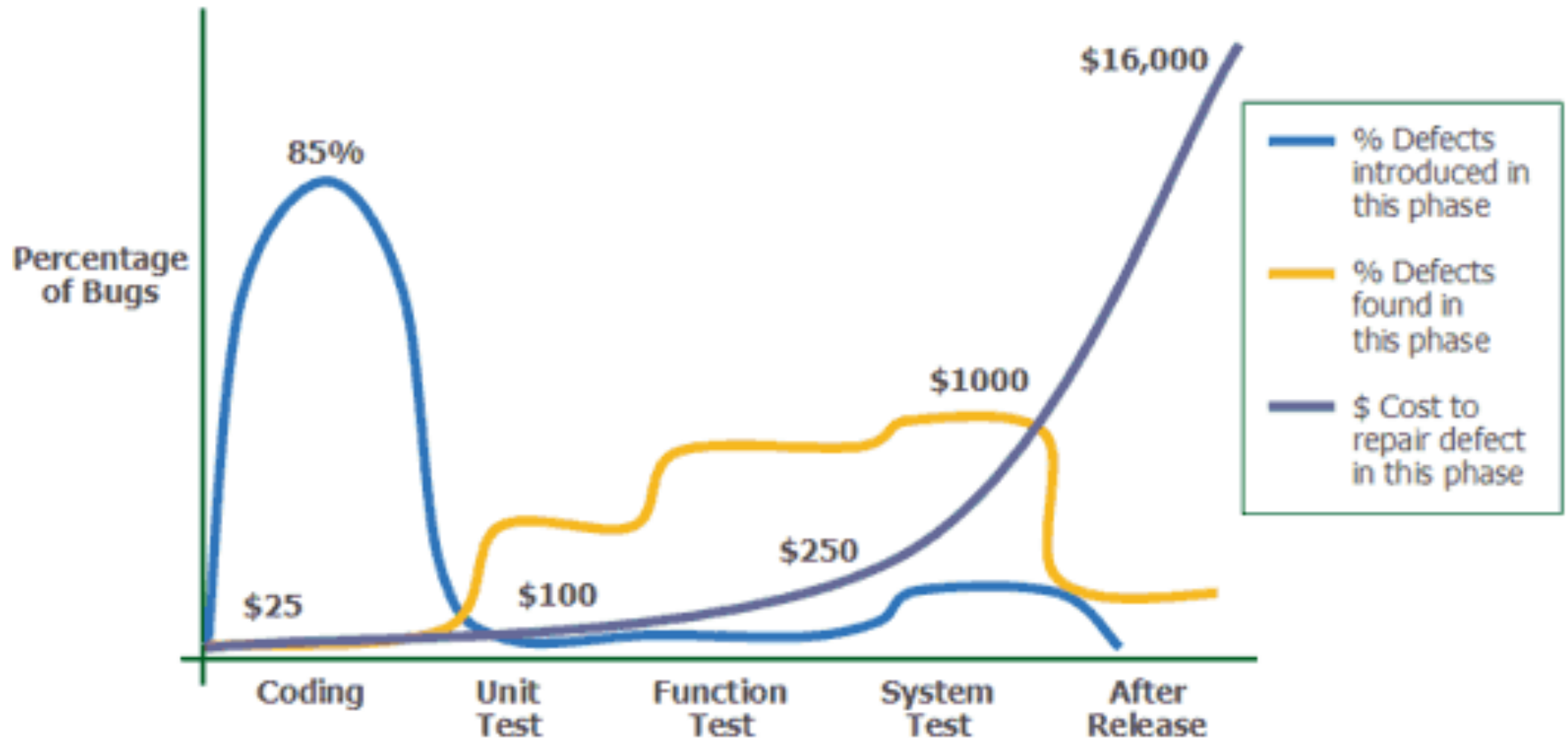
Costo del Debugging

- Debugging del codice: operazione molto costosa (nonostante gli ausili dell'IDE)
- E' noto che il costo della correzione di bug dipende da almeno due grandezze che ne determinano la *località* :
 - le “dimensioni” del contesto
 - ✓ numero di linee di codice in cui il bug può annidarsi
 - il “tempo” che il bug impiega per manifestarsi
 - ✓ misura temporale di quanto dista la causa del bug (durante un'esecuzione del codice) ed il rilevamento dei suoi effetti

Costo di un Bug e “Località”



Tipologie di Test



Le Tre Fasi di un Test

- Tutte le tipologie di test di un qualsiasi sistema prevedono la costruzione di uno *scenario di testing* che si articola sempre in tre fasi strettamente sequenziali
 1. mettere il sistema in un stato iniziale ben noto
 2. inviare una serie di sollecitazioni
 3. controllare che alla fine il sistema si trovi nello stato atteso
- I test possono fallire od avere successo

Obiettivi del Testing

- Se ben progettati e mantenuti, i test aiutano a confinare i bug nella “zona verde”, ovvero con spiccata località
 - i bug si manifestano immediatamente e palesemente
 - la ricerca del bug è confinabile in poche linee facilmente localizzabili
- Le esecuzioni che palesano un bug non sono mai troppo lunghe e complesse

Il Valore Aggiunto dal Testing

- Se il test ha successo:
 - si possiede una garanzia sul comportamento dinamico del codice (assenza di bug)
- Se il test non ha successo:
 - il bug dovrebbe risultare facilmente localizzabile nell'unità di codice sollecitata dal test
- In *entrambi* i casi c'è un significativo valore aggiunto

Testing vs Regression

- Se il test
 - funzionava subito prima di effettuare un modifica
 - ma smette di funzionare subito dopo aver effettuato la modifica
- E' altamente probabile che l'errore è stato appena introdotto con la modifica
 - Ricerca «locale» e quindi economica
- E' possibile prevenire la *regressione*

Esercizio

- Supponiamo di voler testare il metodo **max()** della classe **Sequenza** (quiz di benvenuto al corso)
 - Scriviamo in un documento di testo (.txt) diverse istanze dell'array di interi, per ogni sequenza scriviamo il massimo atteso
 - facciamo girare il programma su ciascuna sequenza e verifichiamo che il risultato sia quello atteso
- Osservazione:
 - possiamo scrivere i test senza preoccuparci dell'algoritmo per il calcolo del massimo (ovviamente è necessario scegliere con cura gli array di test)
- Conseguenza:
 - possiamo scrivere i test prima di scrivere il programma!

Testo QUIZ

- Scrivere il codice del metodo `massimo()` che deve restituire il più grande valore presente nella variabile di istanza `sequenza`, un array:

```
public class Sequenza {  
    private int[] sequenza;  
  
    public Sequenza(int n){  
        sequenza = new int[n];  
    }  
  
    public int massimo(){  
        // scrivere il codice di questo metodo:  
        // deve restituire il valore piu' grande  
        // presente nell'array sequenza  
    }  
  
    public void setElemento(int indice, int valore) {  
        sequenza[indice] = valore;  
    }  
}
```


Codice di Test

- Nella pratica, accanto al *codice di produzione* si sviluppa sempre del *codice di test*
- Unico motivo di esistere del codice di test è quello di verificare la correttezza a tempo di esecuzione del codice principale
- Il codice di test accompagna e supporta lo sviluppo del codice di produzione ma non fa parte del codice consegnato a fine progetto

Test Unitari Automatici

- Esistono diverse tipologie di test
- Nostra attenzione è limitata ad una in particolare: *unit-testing automatico*
 - test che si focalizzano su *frammenti (unità)* del sistema
 - senza alcun intervento umano (tranne la richiesta di esecuzione)
- Praticamente i test unitari si codificano nel medesimo linguaggio di programmazione utilizzato per lo sviluppo (Java)

Test Unitari - Unit Testing

- Test su *frammenti* di un sistema piuttosto che sull'intero sistema
- Concettualmente un test unitario si articola in questi passi
 - 1) mettere uno o più oggetti da testare in un stato iniziale ben noto
 - 2) invocare i metodi degli oggetti
 - 3) controllare che alla fine gli oggetti si trovino nello stato atteso

Automazione dei Test

- Abbiamo già eseguito test manuali (cfr. Esercizi precedenti)
 - basta riportare i valori di ingresso ed i valori di output attesi in un documento di testo e verificare che ogni esecuzione produca quanto atteso
- Chiaramente ogni esecuzione manuale richiede uno sforzo sia per inserire l'input che per ispezionare visivamente i risultati
 - ✓ Difficilmente si è disposti a ripetere l'operazione troppe volte
- L'automazione dei test è fondamentale
 - altrimenti viene meno la loro economicità

Automazione Artigianale

- Una possibile soluzione

- molto artigianale
- automatica

chiarisce il funzionamento dei test unitari

- Scriviamo un programma in cui
 - inizializziamo un certo numero di oggetti con sequenze di interi su cui basare il test (*fixture*)
 - invochiamo il metodo da testare e verifichiamo che il risultato restituito sia uguale a quello atteso
- Successivamente vedremo un framework (JUnit) che rende l'automazione ancora più spedita>>

Esempio Soluzione Artigianale (1)

```
public static void main(String[] args){
    Sequenza positivi;
    Sequenza negativi;
    Sequenza negEpos;
    Sequenza negEzero;
    Sequenza inPrimaPos;
    Sequenza inUltimaPos;

    positivi = new Sequenza(5);
    positivi.setElemento(0,1);
    positivi.setElemento(1,5);
    positivi.setElemento(2,8); // MAX!
    positivi.setElemento(3,3);
    positivi.setElemento(4,4);
    negativi = new Sequenza(5);
    negativi.setElemento(0,-6);
    negativi.setElemento(1,-1); // MAX!
    negativi.setElemento(2,-8);
    negativi.setElemento(3,-13);
    negativi.setElemento(4,-10);
```

Esempio Soluzione Artigianale (2)

```
negEpos = new Sequenza(5);  
negEpos.setElemento(0,100);  
negEpos.setElemento(1,-5);  
negEpos.setElemento(2,-80);  
negEpos.setElemento(3,1000); // MAX!  
negEpos.setElemento(4,10);
```

```
negEzero = new Sequenza(5);  
negEzero.setElemento(0,-1);  
negEzero.setElemento(1,0); // MAX!  
negEzero.setElemento(2,-80);  
negEzero.setElemento(3,-10);  
negEzero.setElemento(4,-10);
```

```
inPrimaPos = new Sequenza(5);  
inPrimaPos.setElemento(0, 1000); // MAX!  
inPrimaPos.setElemento(1, 0);  
inPrimaPos.setElemento(2, 80);  
inPrimaPos.setElemento(3,-10);  
inPrimaPos.setElemento(4,-10);
```

```
inUltimaPos = new Sequenza(5);  
inUltimaPos.setElemento(0, 1);  
inUltimaPos.setElemento(1, 0);  
inUltimaPos.setElemento(2, 80);  
inUltimaPos.setElemento(3,-10);  
inUltimaPos.setElemento(4, 1000); // MAX!
```

Esempio Soluzione Artigianale (3)

```
boolean esito = true;
esito &= (positivi.massimo() == 8);
System.out.println(positivi.massimo() == 8);
esito &= (negativi.massimo() == -1);
System.out.println(negativi.massimo() == -1);
esito &= (negEpos.massimo() == 1000);
System.out.println(negEpos.massimo() == 1000);
esito &= (negEzero.massimo() == 0);
System.out.println(negEzero.massimo() == 0);
esito &= (inPrimaPos.massimo() == 1000);
System.out.println(inPrimaPos.massimo() == 1000);
esito &= (inUltimaPos.massimo() == 1000);
System.out.println(inUltimaPos.massimo() == 1000);

System.out.println(esito);
}
```


Soluzione Artigianale

- La soluzione presentata, benché chiaramente *artigianale*, è completamente automatica
 - dopo ogni modifica al metodo sotto test possiamo velocemente far rigirare il programma di test e verificare se ci sono cambiamenti (regressioni) nei risultati
 - in presenza di fallimenti la ricerca degli errori risulta fortemente semplificata dalle informazioni desumibili già dal test fallito stesso

Automazione del Testing

- I test devono essere:
 - *automatici* (per mantenere rapido il ciclo di feedback)
 - devono essere eseguiti molte volte al giorno
 - *efficienti*
 - devono essere convenienti rispetto alle ispezioni manuali
 - *isolati e che garantiscano la località degli errori*
 - dal fallimento di un test alla rimozione del bug deve trascorrere poco tempo grazie alle caratteristiche di forte località del test per gli errori che rilevano
 - ed inoltre:
 - separati dal codice applicativo
 - eseguibili e verificabili separatamente

Automazione del Testing: JUnit

- Esistono vari strumenti per assistere il programmatore nel testing, ed in particolare nello unit-testing
- JUnit: <http://www.junit.org>
- Il più noto ed utilizzato framework (insieme di classi e convenzioni d'uso) per la scrittura di test unitari
- Fortemente integrato con gli ambienti di sviluppo più diffusi come Eclipse>>

JUnit: Test del Metodo massimo()

```
import static org.junit.jupiter.api.Assertions.*;  
import org.junit.jupiter.api.Test;  
public class SequenzaTest {
```

*import di classi ed
annotazioni Junit 5*

nome

@Test ← *Annotazione di metodo come test-case*

```
public void testMassimoPositivi() {  
    Sequenza seq = new Sequenza(5);  
    seq.setElemento(0,1);  
    seq.setElemento(1,5);  
    seq.setElemento(2,8);  
    seq.setElemento(3,3);  
    seq.setElemento(4,4);  
    assertEquals(8, seq.massimo());  
}
```

Assertione

test-case

```
@Test  
public void testMassimoNegativi() {  
    ...  
    ...  
}...}
```

test-case

JUnit: Struttura Classi di Test (1)

- Tutte le classi di test che scriveremo avranno questa struttura
- Ovviamente le classi di test vanno progettate sulla base delle peculiarità della classe testata
- Collochiamo la classe di test nello stesso package della classe che si sta testando
- Convenzione sui nomi basata sul suffisso:

Classe

Sequenza → SequenzaTest

Classe di Test

JUnit: Struttura Classi di Test (2)

- `import static org.junit.jupiter.api.Assertions.*;`
Serve per importare vari metodi statici del framework che permettono di fare *asserzioni*>>
- `import org.junit.jupiter.api.Test;`
Serve per importare l'*annotazione* del framework **@Test** utile a marcare i metodi i test-case
- Non è (più) necessario ma è (tuttora) buona norma usare '**test**' come prefisso del nome dei test-case

@Test

```
public void testCostruzioneComandiInvalidi() {  
    ...  
}
```

JUnit: Asserzioni

- Asserzione:
affermazione che può essere vera o falsa
- I risultati attesi sono documentati con delle *asserzioni* esplicite, non mediante stampe
 - richiederebbero dispendiose ispezioni visuali
- Se l'asserzione è
 - *vera* : il test ha avuto successo, è andato a buon fine
 - *falsa* : il test è fallito ed il codice testato non si comporta come atteso, quindi c'è un errore a tempo dinamico

JUnit: Metodi assertXYZ()

- Una asserzione non vera fa fallire il test-case
 - `assertEquals(Object expected, Object actual)`: afferma l'«uguaglianza» degli argomenti (>>)
 - `assertNull(Object object)`: afferma che il suo argomento è nullo (fallisce se non lo è)
 - molte altre varianti
 - `assertNotNull()`
 - `assertTrue()`
 - `assertFalse()`
 - `assertSame()...`

tutte sovraccariche ... e talvolta facilmente intercambiabili...
- Usare sempre la versione più pertinente!
 - meglio `assertFalse(b)` di `assertTrue(b==false)`,
 - meglio `assertNotNull(o)` di `assertTrue(o!=null)`

JUnit: assertEquals()

- `assertEquals(Object expected, Object actual)`
«expected» è il valore atteso, che ci si aspetta normalmente
«actual» è il valore effettivo, reale, quello ottenuto
 - afferma che il suo secondo argomento è uguale al primo argomento
 - va a buon fine se e solo se `expected.equals(actual)` restituisce `true`
- Una variante, spesso preferibile

`assertEquals(String message, Object expected, Object actual)`

- un messaggio diagnostico da stampare solo in caso di fallimento
- se ben ideato, dovrebbe permettere di comprendere il motivo del fallimento senza nemmeno aprire il debugger

Le “Tre Fasi” in Pratica

@Test

```
public void testMassimoPositivi() {  
    Sequenza seq = new Sequenza(5);  
    seq.setElemento(0,1);  
    seq.setElemento(1,5);  
    seq.setElemento(2,8);  
    seq.setElemento(3,3);  
    seq.setElemento(4,4);  
    assertEquals(8, seq.massimo());  
}
```

1) mettere un “frammento” del sistema in un stato *iniziale* noto

- il frammento comprende un *solo* oggetto **Sequenza** opportunamente popolato di valori (nell'es. si tratta di interi tutti positivi)

2) inviare una serie di sollecitazioni (`seq.massimo()`)

3) controllare tramite asserzioni che si raggiunga lo stato *finale* atteso (`assertEquals(...)`)

JUnit: Compilare i Test

Per compilare ed eseguire i test:

- Nel classpath devono essere presenti le librerie di JUnit
- ✓ Eclipse snellisce molti di questi passaggi sino a renderli quasi trasparenti
 - ed arriva a suggerire di aggiungere la libreria nel build path del vostro progetto non appena ne nota l'utilizzo

JUnit ed Eclipse

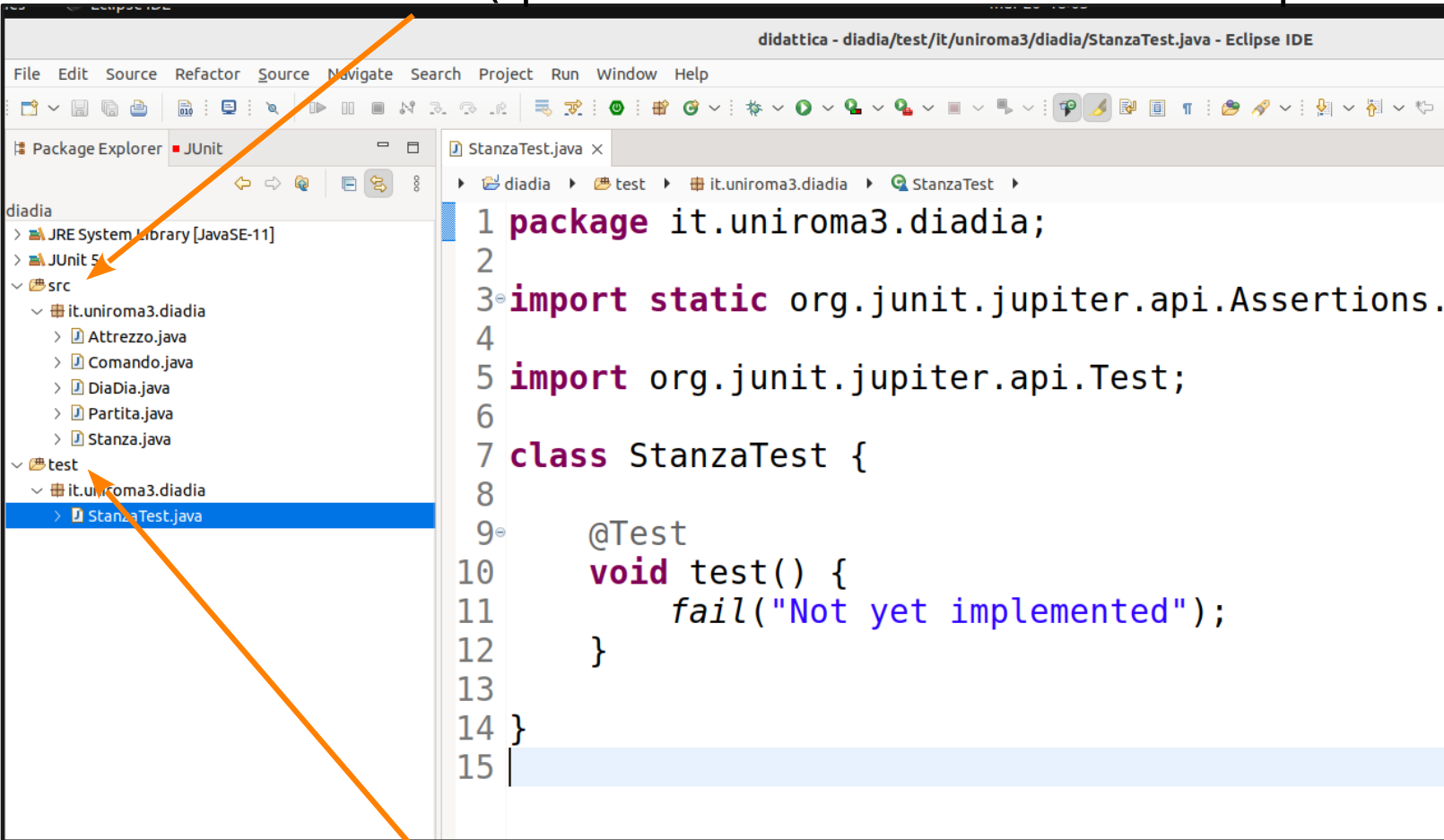
- JUnit è talmente popolare da venire fornito già integrato e fortemente supportato negli IDE
- Nel caso di Eclipse:
 - per creare una classe di test
click con tasto destro del mouse sulla classe
new-> JUnit Test Case (spuntare new JUnit 5)
 - per eseguire una classe di test
click con tasto destro del mouse sulla classe di test
run as -> JUnit test
 - barra verde: il test è andato a buon fine
 - barra blue: il test è fallito violando una asserzione
 - barra rossa: il test è fallito causando un errore

JUnit ed Eclipse

- Le classi di test devono essere nello stesso package delle classi da testare. Scomodo!
 - Si pensi ad una consegna del solo codice di produzione
- Ma in Eclipse possiamo collocare uno stesso package anche in directory (*source folder*) diverse
 - In ogni progetto,
 - nella source folder **src** mettiamo il codice di produzione
 - nella source folder **test** mettiamo le classi di test (organizzate con gli stessi package delle classi di produzione)
 - per creare una source folder: tasto destro del mouse sul progetto, quindi new->Source Folder
 - dentro la source folder **test** creiamo una copia “parallela” dei package del codice di produzione (new->package)

src vs test Cartella/Folder

Source folder **src** (qui vanno le classi del codice di produzione)



Source folder **test** (qui vanno le classi del codice di test)

JUnit: Fixture

- Per facilitare la scrittura dei test-case, è spesso comodo creare degli oggetti in uno stato iniziale noto e «pronto» per l'utilizzo da parte di tutti i test-case
- Può convenire fattorizzare il codice di creazione di questi oggetti. Ad es.
 - utilizzando metodi **setUp()**
 - mediante i cosiddetti *factory methods*
- Le *fixture* sono oggetti in uno stato iniziale noto ed ospitati in variabili d'istanza che le classi di test predispongono allo scopo

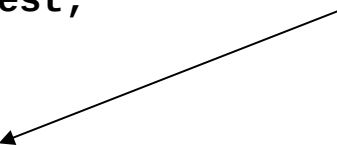
Fixture e JUnit

- Attraverso l'annotazione **@BeforeEach** è possibile indicare quali metodi eseguire *prima di ciascuna* invocazione di un test-case
- Tipicamente questi metodi inizializzano le fixture
- Spesso, ma non più obbligatoriamente, questi metodi vengono tuttora denominati **setUp()**
 - perché con le versioni di JUnit pre-annotazioni Java (JUnit 3.x) era un nome imposto per convenzione

Fixture e Metodo setUp() (1)

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
```

Fixture



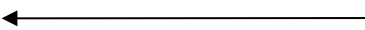
```
public class SequenzaTest {
    private Sequenza positivi;
    private Sequenza negativi;
```

@BeforeEach

```
public void setUp() {
    this.positivi = new Sequenza(5);
    this.positivi.setElemento(0,1);
    this.positivi.setElemento(1,5);
    this.positivi.setElemento(2,8);
    this.positivi.setElemento(3,3);
    this.positivi.setElemento(4,4);

    this.negativi = new Sequenza(5);
    this.negativi.setElemento(0, -6);
    ...
}
```

*Metodo eseguito prima di ogni
invocazione di test-case*



```
@Test
public void testMassimoPositivi() {...}
```

```
@Test
public void testMassimoNegativi() {...}
```

```
}
```

Fixture e Metodo setUp() (2)

```
...
public class SequenzaTest {
    ...
    @Test
    public void testMassimoPositivi() {
        assertEquals(8, this.positivi.massimo());
    }

    @Test
    public void testMassimoNegativi() {
        assertEquals(-1, this.negativi.massimo());
    }

    ...
}
```

setUp() e l'Importanza dei Nomi

- *Dopo diverso tempo dalla prima scrittura, a meno che il nome della fixture `this.positivi` sia perfettamente indicativo di “sequenza non vuota di interi tutti positivi” si finirà per dover leggere il corpo del metodo `setUp()` per poterne comprendere appieno il significato*
- Una situazione migliore:

```
...
public class SequenzaTest {
    ...
    @Test
    public void
    testMassimoDiSequenzaNonVuotaDiInteriTuttiPositivi() {
        assertEquals(8, this.positivi.massimo());
    }
    ...
}
```

setUp(): Controindicazioni

- Se nel **setUp()** si accumulano le fixture di diversi test-case (seq. positive, negative ecc. ecc. ...) si creano tanti oggetti che non hanno nulla a che vedere con il singolo test appena fallito, ad es. **testMassimoPositivi()**
- Si è costretti a leggere un lungo **setUp()** solo per comprendere un breve test-case
- Mettendo a fattor comune tutte le fixture utilizzate una sola volta si sta ledendo la leggibilità dei test-case rendendoli meno autocontenuti
- Tramite il **setUp()** si finisce per creare impliciti ma sottili accoppiamenti tra test-case

setUp(): Indicazioni

- In genere, nel `setUp()` ha senso fattorizzare solo la creazione di oggetti che vengano utilizzati da almeno **due** test-case distinti
- In tutti gli altri casi meglio non distribuire il codice di uno scenario di test in due distinti metodi `test()` + `setUp()`. Altrimenti:
 - Il test non è *isolato*: per comprenderne uno si finisce per dover capire (almeno una parte) di tutti
 - Il test non è *autocontenuto*: per ricostruire lo scenario di test bisogna cercare ben oltre il corpo del test-case stesso
- In definitiva, la *località* del test potrebbe risentirne

Qualità dei Test-Case (1)

- ATTENZIONE: la qualità di un test si avverte in particolare quando il test smette di funzionare e bisogna trovare l'errore all'origine del fallimento
 - *può capitare anche dopo molto tempo* dalla scrittura iniziale del codice
 - quando oramai lo stesso non risulta affatto “familiare”
 - magari subito dopo avere effettuato un refactoring...
- Lo sforzo necessario per rimuovere un errore appena introdotto è uno dei più importanti indicatori della qualità dei test che smettono di funzionare
- ✓ Questo sforzo, abbiamo già visto, dipende largamente dalla *località* di un test

Qualità dei Test-Case (2)

- Qual'è la lunghezza ottimale di un test-case?
 - 1 (dicesi *UNA*) – linea di codice totale!
- E' possibile perseguire questo obiettivo utilizzando alcuni accorgimenti
 - fixtures
 - factory methods
 - *minimalità*>>
- ✓ N.B. per la località dei test non è solo utile conseguire il risultato di aver test monolinea, ma è forse ancora più importante ricordarsi di perseguirlo
 - meno linee possibili per test-case
 - *molto* meglio 10 test-case con 1 asserzione ciascuno che 1 test-case con 10 asserzioni!

Factory Methods

- Per favorire la semplicità dei test si può pensare di fattorizzare il codice di creazione della fixture

```
public class SequenzaTest {
```

```
...  
    private Sequenza sequenza(int... array) {  
        Sequenza risultato = new Sequenza(array.length);  
        for(int i=0; i<array.length; i++) {  
            risultato.setElemento(i,array[i]);  
        }  
        return risultato;  
    }  
}
```

Equivale a
`private Sequenza sequenza(int[] array)`

```
@Test
```

```
public void testMassimoDiSequenzaNonVuotaDiInteriTuttiPositivi() {  
    assertEquals(8, sequenza(1,5,8,3,4).massimo());  
}  
...  
}
```


Testing - Punto di Vista di un Programmatore-Utilizzatore (1)

- Il factory method `sequenza()` ha reso evidente quanto sia “faticoso” creare una sequenza per gli utilizzatori della classe
 - la classe di test è solo una delle possibili classi *utilizzatrici/clienti*
- A ben vedere anche altri utilizzatori della classe possono condividere la stessa esigenza
- Se cambiamo `Sequenza` per facilitare il testing, rendiamo più semplice l’uso della classe da parte anche di tutti gli altri utilizzatori
 - Basta aggiungere il costruttore `Sequenza(int[] e)`?

Autocontenimento dei Test

```
...
public class SequenzaTest {
    ...
    @Test
    public void testMassimoDiSequenzaNonVuotaDiInteriTuttiPositivi() {
        assertEquals(8, new Sequenza(1,5,8,3,4).massimo());
    }
    ...
    @Test
    public void testMassimoDiSequenzaNonVuotaDiInteriTuttiNegativi() {
        assertEquals(-1, new Sequenza(-6,-1,-8,-13,-10).massimo());
    }
    ...
    @Test
    public void testMassimoInPrimaPosizione() {
        assertEquals(1000, new Sequenza(1000,0,80,-10,-10).massimo());
    }
    ...
}
```

Minimalità (1)

- I test visti sinora non sono *minimali*
- E' possibile esprimere lo stesso scenario di testing con test-case più brevi, e che fanno uso di oggetti di stato meno complesso
- Quale di questi test-case preferire? perché?

@Test

```
public void testMassimoDiSequenzaNonVuotaDiInteriTuttiPositivi() {  
    assertEquals(8, new Sequenza(1, 5, 8, 3, 4).massimo());  
}
```

...oppure...

@Test

```
public void testMassimoDiSequenzaNonVuotaDiInteriTuttiPositivi() {  
    assertEquals(2, new Sequenza(1, 2).massimo());  
}
```

???

Minimalità (2)

- L'uso di test minimali rende molto più semplice la ricerca degli errori
- Nulla è più minimale di una sequenza vuota!

```
@Test  
public void testMassimoDiSequenzaVuota() {  
    assertEquals(???, new Sequenza().massimo());  
}
```

- Qual'è il massimo di una sequenza vuota?

Testing - Punto di Vista di un Programmatore-Utilizzatore (2)

- Di nuovo il testing ha evidenziato un problema nel contratto di utilizzo che il metodo `massimo()` espone agli utilizzatori

JUnit 4

```
public class Sequenza {  
    ...  
    public int massimo() {  
        if (this.sequenza.length==0)  
            throw new java.util.NoSuchElementException();  
        // ...  
    }  
}
```

```
@Test(expected = java.util.NoSuchElementException.class)  
public void testMassimoDiSequenzaVuota() {  
    new Sequenza().massimo();  
}
```

“Si nasce minimali, non ci si diventa”

- Convienne sempre partire dai test più semplici, perché sono quelli che permetteranno di rimuovere la maggior parte dei bug il minor sforzo possibile
- E' poi naturale aumentare, via via, la complessità degli scenari di testing
 - ✓ per aumentare la propria confidenza sulla correttezza del proprio codice
- Ad esempio, se ipotizziamo di ordinare tutti i test-case secondo la complessità dello scenario di testing trattato...

Scenari di Testing di Complessità Crescente

```
...
public class SequenzaTest {
    @Test(expected = java.util.NoSuchElementException.class)
    public void testMassimoDiSequenzaVuota() {
        new Sequenza().massimo();
    }
    @Test
    public void testMassimoDiSequenzaSingleton() {
        assertEquals(1, new Sequenza(1).massimo());
    }
    @Test
    public void testMassimoInPrimaPosizione() {
        assertEquals(2, new Sequenza(2,1).massimo());
    }
    @Test
    public void testMassimoInSecondaPosizione() {
        assertEquals(2, new Sequenza(1,2).massimo());
    }
    @Test
    public void testMassimoDiSequenzaNonVuotaDiInteriTuttiNegativi() {
        assertEquals(-1, new Sequenza(-2,-1).massimo());
    }
    ...
}
```

*Fixture di complessità
crescente*



Quando Scrivere i Test?

- Uno dei più gravi e purtroppo frequenti errori di chi viene introdotto allo unit-testing è aspettare la fine della scrittura di tutto il codice principale per cominciare a scrivere il codice di test
- E' la scelta peggiore! si massimizzano i costi di scrittura dei test e si minimizzano i benefici
 - ✓ potranno esistere contemporaneamente molteplici errori, anche correlati, e se tanti test falliscono, non è più chiaro da dove cercarli... in due parole: *scarsa località*
- ✓ Risulta più conveniente scriverli continuativamente, *durante* o addirittura *prima* della scrittura del codice principale (>>)

Testing Continuo

- Il testing deve essere una attività associata ed affiancata all'ordinario sviluppo del codice principale: avviene progressivamente e continuativamente
- Principali motivazioni legate ai costi
 - la rimozione precoce degli errori riduce i costi di sviluppo
 - si costruisce contestualmente al codice principale un ambiente di test
 - si accumulano *batterie di test* molto utili per lo sviluppo e la manutenzione efficace del codice principale
 - i test possono essere riutilizzati durante la manutenzione del software ad esempio per evitare regressioni

Testing - Punto di Vista di un Programmatore-Utilizzatore (3)

- Chi scrive test si costringe nel ruolo del *Programmatore-Utilizzatore* e si focalizza sulla semplicità di utilizzo del proprio codice
- Per questo motivo il testing aiuta a cambiare la prospettiva di visione sul proprio codice, a concentrarsi sulle interfacce delle proprie classi e sulla distribuzione delle responsabilità
- Tipicamente il codice di qualità è più *testabile* e viceversa
- Esistono metodologie di sviluppo che portano all'estremo questa attitudine: *T.D.D.*

Test Driven Development

- Promuove l'uso dei test anche come strumento di progettazione
 - i test guidano lo sviluppo verso codice che sia semplice, facilmente testabile e di qualità
- Predica la scrittura dei test-case **prima** della scrittura del codice testato
 - anticipa nel tempo ed evidenzia il punto di vista del Programmatore-Utilizzatore
 - predilige micro-iterazioni
testing-coding-testing-coding...
che incentivano la minimalità dei test

Sviluppo Guidato dai Test

- Se scriviamo il codice di test prima del codice stesso siamo incentivati a:
 - precisare i metodi visibili all'esterno in quanto il codice di test è codice *cliente*
 - esterno alla classe alla stregua di tutte le altre classi clienti del codice testato
 - chiarire la semantica dei metodi
 - cercare di semplificare al massimo l'utilizzo del codice
 - Individuare i casi limite e chiarire la gestione delle situazioni anomale

Motivazioni del Testing: Conclusioni

- La rimozione precoce degli errori riduce i costi di sviluppo e migliora la qualità del codice
- I test inducono ad assumere anticipatamente il punto di vista del Programmatore-Utilizzatore e spingono gli sviluppatori verso soluzioni più semplici per gli utilizzatori
- Si documenta in maniera formale e precisa il funzionamento del codice

Jupiter - JUnit 5 vs JUnit 4

JUnit 4	JUnit 5
@BeforeClass, @AfterClass	@BeforeAll, @AfterAll
@Before, @After	@BeforeEach, @AfterEach
@Ignore	@Disabled
@Category	@Tag
Assert class.	Assertions class.
Optional assertion message is the first parameter.	Optional assertion message is the last parameter.
Assume class.	Assumptions class.
assumeNotNull and assumeNoException.	assumeNotNull and assumeNoException have been removed.

- Molti miglioramenti, nessuna rivoluzione, perché già JUnit 4 funzionava benissimo
- Alcune rendono il testing più facile in particolari scenari
 - test parametrici (>>)

JUnit 5 vs JUnit 4: In Pratica

- Nella pratica serve conoscere entrambe le versioni
 - “Greenfield” project: usare Jupiter – JUnit 5
 - “Brownfield” project: continuare ad usare JUnit 4
- Conviene cambiare?
 - Soprattutto per un uso basico, JUnit 4 già funzionava benissimo
 - Per utilizzi più avanzati, le soluzioni JUnit 4 risultano, in genere, sensibilmente più “macchinose” delle equivalenti in JUnit 5
- Un project “brownfield” su tutti: il SISTEMA QUIZ
 - Si basa su JUnit 4 e non vale la pena di aggiornarlo, al momento...
- Se si decide di aggiornare un progetto, come prima cosa ricordarsi di cambiare gli import...
 - Da JUnit 4:
 - `import static org.junit.Assert.*; // Asserzioni assertXYZ...`
 - `import org.junit.*; // Annotazioni @...`
 - A JUnit 5:
 - `import static org.junit.jupiter.api.Assertions.*; // assertXYZ...`
 - `import org.junit.jupiter.api.*; // Annotazioni @...`

Esercizi

- Scrivere (con Eclipse) una classe di test JUnit per la classe **Persone** (dal Quiz di preparazione alla prima verifica)
- In particolare testare il metodo `int conta0monimiDi(String nome)`
- Scrivere il codice del metodo `int conta0monimiDi(String nome)`
- Eseguire la classe di test JUnit (se il test fallisce, correggere il metodo sotto test e far girare nuovamente la classe di test)

```
public class Persone {  
    private String[] nomi;  
  
    public Persone(int n) {  
        this.nomi = new String[n];  
    }  
  
    public int conta0monimiDi(String nome) {  
        // metodo da scrivere  
    }  
  
    public void aggiungiNome(int indice, String nome){  
        this.nomi[indice] = nome;  
    }  
}
```


Programmazione Orientata agli Oggetti

Interfacce e Polimorfismo
Upcasting e Downcasting

Contenuti

- Riferimenti tipati
- Java Interface
- Principio di sostituzione
- Polimorfismo e late binding
- Tipo statico e tipo dinamico
- Interfacce come ruolo

Contenuti

- Riferimenti tipati
- Java Interface
- Principio di sostituzione
- Polimorfismo e late binding
- Tipo statico e tipo dinamico
- Interfacce come ruolo

Riferimenti Tipati (1)

- In Java i riferimenti sono tipati, ovvero specificano il tipo dell'oggetto referenziato
- La definizione:
Strumento s;
afferma che **s** è un riferimento ad un oggetto di tipo **Strumento**
- Questo significa che attraverso **s** possiamo invocare i servizi del tipo **Strumento**
 - ovvero che è possibile chiedere di eseguire i metodi offerti dal tipo **Strumento** all'oggetto referenziato da **s**

Riferimenti Tipati (2)

- Consideriamo la seguente classe **Musicista**

```
public class Musicista {  
    private String nome;  
  
    public Musicista(String nome) {  
        this.nome = nome;  
    }  
  
    public void suona(Strumento s) {  
        s.produciSuono();  
    }  
}
```

Riferimenti Tipati (3)

- Il metodo `suona(Strumento s)` prende come parametro un riferimento ad un oggetto il cui tipo è `Strumento`
- Nel corpo del metodo è possibile invocare su `s` tutti i metodi offerti dal tipo `Strumento`
 - intuiamo che `Strumento` offre il metodo `public void produciSuono()`

Costrutti Java per la Definizione di Nuovi Tipi

- Fino ad ora abbiamo visto un solo modo per definire nuovi tipi: la definizione di nuove classi (mediante il costrutto **class**)
- In Java (e in altri moderni linguaggi OO, come ad esempio C#, Scala) esistono molteplici costrutti per definire nuovi tipi
- È il costrutto **interface**

Contenuti

- Riferimenti tipati
- **Java Interface**
- Principio di sostituzione
- Polimorfismo e late binding
- Tipo statico e tipo dinamico
- Interfacce come ruolo

Java Interface (1)

- Possiamo dire che una **interface** specifica un tipo in termini dei servizi, ovvero dei metodi, che questi può offrire
- Una **interface** non specifica i dettagli implementativi dei vari servizi, specifica solamente in che modo i servizi possono essere invocati (nome, parametri, tipo restituito)
- In definitiva una **interface** consiste in una specifica delle signature (e dei tipi restituiti) dai metodi che il tipo può offrire

Java Interface (2)

- Esempio:

```
public interface Strumento {  
    public void produciSuono();  
}
```

- ✓ L'interface **Strumento** definisce il tipo di oggetti che possono offrire il metodo **produciSuono()**

Java Interface (3)

- Nelle interface specifichiamo solo le signature (e il tipo restituito) dei metodi che un tipo può offrire
- In una **interface** non c'è nessun dettaglio relativo alla implementazione
 - Niente variabili
 - Niente costruttori
 - Niente corpo dei metodi
- Le **interface** non si possono istanziare
- Ma una classe può implementare una (o più) interface
- Una classe che implementa una **interface** garantisce che le sue istanze rispettino il tipo specificato nella **interface**

Classi ed Interface (1)

```
public class Tamburo implements Strumento {  
    public void produciSuono() {  
        System.out.println("bum-bum-bum");  
    }  
}
```

- La parola chiave **implements** serve a specificare che la classe **Tamburo** implementa l'interfaccia **Strumento**
- Questo significa che gli oggetti **Tamburo** sono in grado di offrire i metodi del tipo **Strumento**

Classi ed Interface (2)

- Una classe che implementa una **interface** può avere altri metodi (oltre a quelli della **interface**) specifici della classe

```
public class Chitarra implements Strumento {  
    private int[] corde;  
  
    public Chitarra() {  
        corde = new int[6];  
    }  
  
    public void produciSuono() {  
        System.out.println("dlen-dlen-dlen");  
    }  
  
    public int accorda(int corda, int val) {  
        return corde[corda] += val;  
    }  
}
```

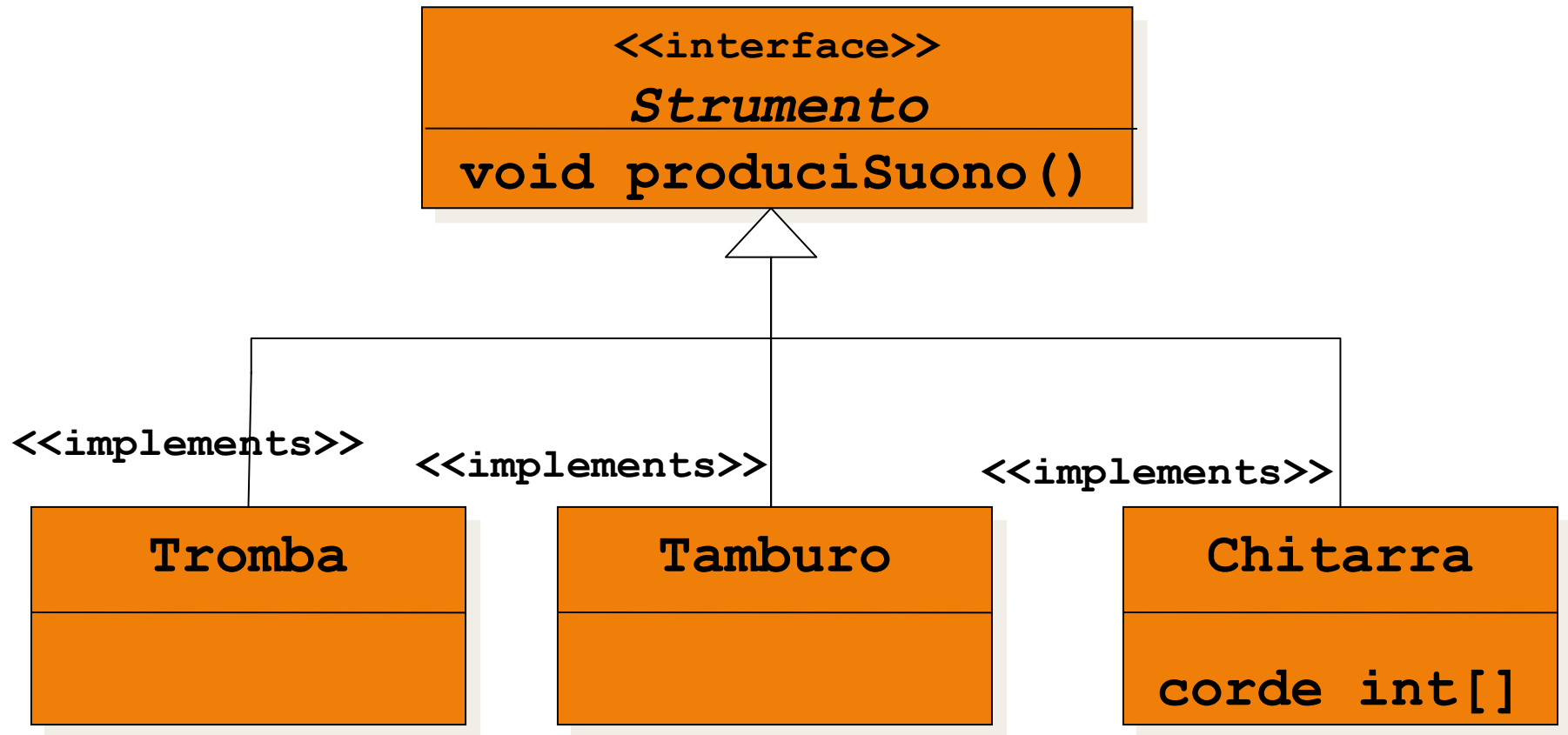
Esempio

```
public class Tamburo implements Strumento {  
    public void produciSuono() {  
        System.out.println("bum-bum-bum");  
    }  
}
```

```
public class Tromba implements Strumento {  
    public void produciSuono() {  
        System.out.println("pe-pe-re-pe-pe");  
    }  
}
```

```
public class Chitarra implements Strumento {  
    private int[] corde;  
    public Chitarra(){  
        corde = new int[6];  
    }  
    public void produciSuono() {  
        System.out.println("dlen-dlen-dlen");  
    }  
    public int accorda(int corda, int val) {  
        return corde[corda] += val;  
    }  
}
```

Diagramma delle Classi



Tipi, Sottotipi & Supertipi

- Abbiamo detto che una **interface** definisce un tipo
- Se la classe **C** implementa una **interface I** diciamo che:
 - **C** è un *sottotipo* di **I**
 - e che
 - **I** è un *supertipo* di **C**
- Ad esempio
 - **Tamburo** è un sottotipo di **Strumento**
 - **Strumento** è un supertipo di **Tamburo**

Contenuti

- Riferimenti tipati
- Java Interface
- **Principio di sostituzione**
- Polimorfismo e late binding
- Tipo statico e tipo dinamico
- Interfacce come ruolo

Principio di Sostituzione (1)

- In Java vale il *principio di sostituzione (di Liskov)*:

un sottotipo può essere usato al posto di un suo supertipo

- Rivediamo il metodo `suona` (`Strumento s`) della classe **Musicista**:

```
public void suona(Strumento s) {  
    s.produciSuono();  
}
```

- Se invochiamo il metodo `suona` (`Strumento s`), per il principio di sostituzione, possiamo passargli anche un riferimento ad un oggetto istanza di una qualunque classe che implementi l'interfaccia **Strumento**

Principio di Sostituzione (2)

- Esempio:

```
public static void main(String[] args){  
    Chitarra c = new Chitarra();  
    Strumento t = new Tamburo();  
    Musicista ludovico = new Musicista("Ludovico");  
    ludovico.suona(c);  
    ludovico.suona(t);  
}
```

- Nelle chiamate al metodo **suona(Strumento s)** abbiamo usato un riferimento ad un oggetto **Chitarra** (e poi un riferimento ad un oggetto **Tamburo**) al posto di un riferimento a **Strumento**

Principio di Sostituzione (3)

- Per il principio di sostituzione, un riferimento ad un sottotipo può essere assegnato ad un riferimento ad un suo supertipo
- Esempio:

```
Strumento s;
```

```
Chitarra c;
```

```
c = new Chitarra();
```

```
s = c;
```

Principio di Sostituzione (4)

- Commentiamo le precedenti istruzioni

Strumento s;

- Abbiamo definito una variabile **s**: contiene un riferimento ad un oggetto che rispetta il tipo **Strumento**

Chitarra c;

- Abbiamo definito una variabile **c**: contiene un riferimento ad un oggetto che rispetta il tipo **Chitarra**

c = new Chitarra();

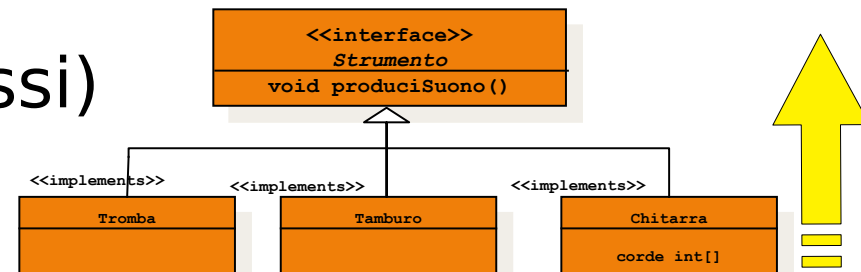
- Abbiamo creato un oggetto **Chitarra** e ne abbiamo assegnato il riferimento alla variabile **c**

s = c;

- Abbiamo assegnato il riferimento all'oggetto **c** alla variabile **s**
- È tutto lecito perché l'oggetto **c** è istanza della classe **Chitarra** che implementa il supertipo **Strumento**

Upcasting

- La promozione da un tipo ad un suo supertipo viene chiamata *upcasting*
- *upcasting* : un riferimento ad un oggetto è “promosso” in un riferimento ad un suo supertipo
- Il termine (“Up”=“verso l’alto”) è tradizionalmente legato al modo in cui vengono espresse graficamente le dipendenze supertipo/sottotipo (vedi diagramma delle classi)



Contenuti

- Riferimenti tipati
- Java Interface
- Principio di sostituzione
- Polimorfismo e late binding
- Tipo statico e tipo dinamico
- Interfacce come ruolo

Polimorfismo e Late Binding (1)

- Consideriamo la classe `Musicista`

```
public class Musicista {  
    private String nome;  
  
    public Musicista(String nome) {  
        this.nome = nome;  
    }  
  
    public void suona(Strumento s) {  
        s.produciSuono();  
    }  
}
```


Polimorfismo e Late Binding (2)

- Cosa succede a tempo di esecuzione, quando al parametro *s* è *legato* un oggetto?
- Sappiamo che il metodo `produciSuono()` viene invocato da un oggetto la cui classe implementa l'interfaccia **Strumento**
- Ma il codice da eseguire non è noto se non a tempo di esecuzione
- Il collegamento tra segnatura e corpo del codice da eseguire per `produciSuono()` viene stabilito solo a tempo di esecuzione (*late binding*)
- C'è un comportamento *polimorfo* del parametro formale **Strumento s**
 - può assumere forme/comportamenti diversi: tutti quelli dei suoi sottotipi

Contenuti

- Riferimenti tipati
- Java Interface
- Principio di sostituzione
- Polimorfismo e late binding
- Tipo statico e tipo dinamico
- Interfacce come ruolo

Tipo Statico e Tipo Dinamico

- Consideriamo la seguente istruzione:

```
Strumento s = new Chitarra();
```

- È lecita, per il principio di sostituzione
- Qual è il tipo della variabile `s`?
- Dobbiamo distinguere tra
 - Tipo statico
 - Tipo dinamico

Tipo Statico

- Il tipo statico è quello che viene usato nella dichiarazione della variabile
- Ad esempio, nella istruzione:

```
Strumento s = new Chitarra();
```

il tipo statico di **s** è **Strumento**

- Il tipo statico è determinato a tempo di compilazione
- Il compilatore permette di applicare i metodi del tipo statico (ovvero verifica che su una variabile siano invocati i metodi del suo tipo statico)
- Nel nostro esempio possiamo invocare su **s** solo i metodi di **Strumento**

```
Strumento s = new Chitarra();  
s.produciSuono(); // CORRETTO  
s.accorda(2,1);   // ERRATO: il tipo Strumento non  
                  // possiede il metodo accorda(int, int)
```

Tipo Dinamico

- Il tipo dinamico è quello dell'oggetto realmente istanziato e quindi referenziato in memoria
- Ad esempio, nella istruzione:
`Strumento s = new Chitarra();`
il tipo dinamico di `s` è `Chitarra`
- Il tipo dinamico stabilisce quale sarà l'implementazione usata
- Nel nostro esempio:
`Strumento s = new Chitarra();`
`s.produciSuono();`
- A tempo di esecuzione il codice del metodo `produciSuono()` che viene usato è quello definito nella classe `chitarra`

Tipo Statico e Tipo Dinamico

- Capire la differenza tra tipo statico e tipo dinamico è fondamentale
- Il tipo statico viene assegnato dal compilatore e determina l'insieme dei metodi che possono essere invocati
- Il tipo dinamico interviene a tempo di esecuzione e determina l'implementazione che viene eseguita

Tipo Statico vs Tipo Dinamico (1)

Qual'è il tipo di c?

```
Chitarra c = new Chitarra();
```

Tipo Statico

Tipo Dinamico

Qual'è il tipo di s?

```
Strumento s = new Chitarra();
```

Tipo Statico

Tipo Dinamico

Tipo Statico vs Tipo Dinamico (2)

- Il tipo dichiarato di una variabile è il suo *tipo statico*
- Il tipo dell'oggetto a cui una variabile si riferisce è il suo *tipo dinamico*
- Il compilatore si preoccupa di verificare violazioni del tipo statico

```
Strumento strumento = new Chitarra();  
strumento.accorda(2,1); // ERRORE a tempo di compilazione
```

- **accorda()** non è tra i metodi di **Strumento** (tipo statico della var. locale **strumento**)

Tipo Statico vs Tipo Dinamico (3)

- A tempo di esecuzione viene eseguito il metodo del tipo dinamico
 - ✓ d'altronde i metodi definiti nelle interfacce non possiedono implementazione se non quella delle classi che le implementano
- Nota che il compilatore non solo non *conosce*, ma neanche può *prevedere*, in generale, i tipi dinamici >>

Imprevedibilità dell'Esecuzione (1)

```
import java.util.Random;

public class OrchestraCausale {
    public static void main(String[] args){
        Strumento[] orchestra = new Strumento[10];
        Random r = new Random();

        for(int i=0; i<orchestra.length; i++) {
            int numeroAcaso = r.nextInt(3);
            if (numeroAcaso==0)
                orchestra[i] = new Chitarra();
            if (numeroAcaso==1)
                orchestra[i] = new Tamburo();
            if (numeroAcaso==2)
                orchestra[i] = new Tromba();
        }

        for(int i=0; i<orchestra.length; i++)
            orchestra[i].produciSuono();
    }
}
```

Imprevedibilità dell'Esecuzione (2)

- Nell'esempio precedente l'array è riempito casualmente a tempo di esecuzione: non sappiamo a priori quali strumenti vengono assegnati ai vari elementi dell'array
- A tempo di esecuzione, ogni elemento dell'array produce il suono corrispondente al tipo dinamico
- A tempo di compilazione, ogni elemento dell'array possiede tipo statico **Strumento**

Il compilatore non può prevedere il tipo dinamico degli oggetti effettivamente utilizzati a tempo di esecuzione

Tipo Statico e Tipo Dinamico: Overloading (1)

- L'overloading dei metodi viene risolto dal compilatore, quindi staticamente
- In particolare:
 - se abbiamo un metodo sovraccarico il compilatore guarda il tipo statico dei parametri per decidere qual'è il metodo da invocare
- Vedi esercizio seguente

Tipo Statico e Tipo Dinamico: Overloading (2)

```
interface Edificio {  
    public int altezza();  
}
```

```
public class Palazzo implements Edificio {  
    private int altezza;  
    public Palazzo(int altezza) {this.altezza = altezza;}  
    public int altezza() {return this.altezza;}  
}
```

```
public class Coloratore {
```

```
    public void colora(Edificio e) {  
        System.out.println("Colorato Edificio");  
    }
```

```
    public void colora(Palazzo p) {  
        System.out.println("Colorato Palazzo");  
    }
```

```
    public static void main(String args[]) {  
        Palazzo p = new Palazzo(4);  
        Edificio e = new Palazzo(3);  
        Coloratore c = new Coloratore();
```

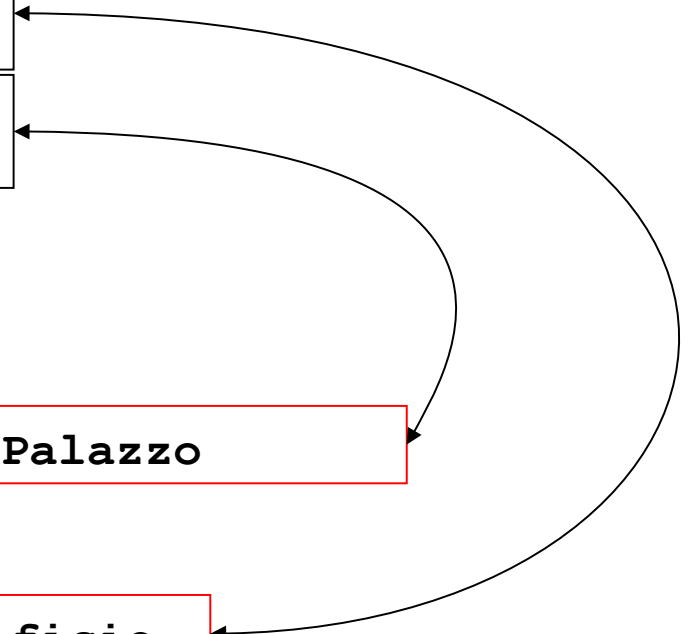
```
        c.colora(p);  
        c.colora(e);
```

```
    }
```

```
}
```

Tipo statico di p è Palazzo

Tipo statico di e è Edificio



Tipo Statico e Tipo Dinamico: Esempio

```
interface Veicolo {  
    public void func(Veicolo v);  
    public void func(Autotreno a);  
}  
  
public class Autotreno implements Veicolo {  
    public void func(Veicolo v) {  
        System.out.println("Autotreno.func(Veicolo) ");  
    }  
    public void func(Autotreno a) {  
        System.out.println("Autotreno.func(Autotreno) ");  
    }  
  
    public static void main(String args[]) {  
        Veicolo a = new Autotreno();  
        Autotreno b = new Autotreno();  
        a.func(b);  
  
        a.func(a);  
    }  
}
```

Tipo statico di b è
Autotreno

Tipo statico di a è Veicolo

Esercizi

- Fare le verifiche
 - L.java
 - Olimpiadi.java
 - Villa.java

Contenuti

- Riferimenti tipati
- Java Interface
- Principio di sostituzione
- Polimorfismo e late binding
- Tipo statico e tipo dinamico
- Interfacce come ruolo

Interface come *Ruolo* (1)

- Una classe può implementare più di una interface
- Potremmo dire che ciascuna interface implementata da una classe rappresenta uno specifico "ruolo" che la classe può assumere
- Ragionare sui ruoli (ed usare le potenzialità del polimorfismo) ci aiuta a produrre codice altamente riutilizzabile

Interface, Ruoli e Riuso (1)

- Consideriamo un problema noto che si presta naturalmente ad un comportamento polimorfo degli oggetti interessati: l'ordinamento
- Supponiamo di avere una classe che modella un "orario", espresso in ore e minuti
- Supponiamo di avere una collezione (per semplicità un array) di oggetti orario
- Supponiamo di voler ordinare questa collezione

Esempio: La Classe Orario

```
public class Orario {
    private int ore;
    private int minuti;

    public Orario(int ore, int minuti) {
        this.ore = ore;
        this.minuti = minuti;
    }

    public int getOre() {
        return this.ore;
    }

    public int getMinuti() {
        return this.minuti;
    }

    public boolean minoreDi(Orario o) {
        if (this.getOre() > o.getOre())
            return false;
        if (this.getOre() == o.getOre())
            return (this.getMinuti() < o.getMinuti());
        return true;
    }

    public String toString() {
        return this.getOre()+"-"+this.getMinuti();
    }
}
```

Interface, Ruoli e Riuso (2)

- Per ordinare la collezione creiamo una opportuna classe che offre questa funzionalità attraverso il metodo `ordina(Orario[])`
- Scriviamo il codice
(usiamo un qualsiasi algoritmo di ordinamento, cfr. corso Fondamenti, ad esempio il «selection sort»)
- vedi classe `OrdinatoreOrari`

La Classe OrdinatoriOrari

```
public class OrdinatoriOrari {  
  
    public static void ordina(Orario[] lista) {  
        int imin;  
        for (int ord=0; ord<lista.length-1; ord++) {  
            imin = ord;  
            for (int i=ord+1; i<lista.length; i++)  
                if (lista[i].minoreDi(lista[imin])) {  
                    Orario temp=lista[i];  
                    lista[i]=lista[imin];  
                    lista[imin]=temp;  
                }  
            }  
        }  
    }  
}
```

Interface, Ruoli e Riuso (3)

- Osserviamo bene il codice di **OrdinatoreOrari**
- Affinché gli oggetti dell'array possano essere ordinati, l'unica proprietà che questi oggetti devono avere è quella di possedere un metodo **minoreDi (Orario)**
- In altri termini l'ordinamento funziona su oggetti che sappiano interpretare il *ruolo* di «essere confrontati»
- Questo ruolo lo possiamo esplicitare in una opportuna interface

“Confrontabilità”, come Ruolo

- Creiamo l'interface **Comparable**: gli oggetti delle classi che la implementano sono in grado di essere confrontati tramite il metodo **minoreDi (Comparable)**

L'interface Comparabile

```
public interface Comparabile {  
    public boolean minoreDi (Comparabile c) ;  
}
```


Interface, Ruoli e Riuso (4)

- Possiamo ora generalizzare la nostra classe **Ordinatore** (e il relativo algoritmo di ordinamento) affinché funzioni su tutte le classi che sappiano interpretare il ruolo **Comparabile**

La Classe Ordinatori

```
public class Ordinatori {  
  
    public static void ordina(Comparabile[] lista) {  
        int imin;  
        for (int ord=0; ord<lista.length-1; ord++) {  
            imin = ord;  
            for (int i=ord+1; i<lista.length; i++)  
                if (lista[i].minoreDi(lista[imin])) {  
                    Comparabile temp=lista[i];  
                    lista[i]=lista[imin];  
                    lista[imin]=temp;  
                }  
            }  
        }  
    }  
}
```

La Classe Orario (rivisitata)

```
public class Orario implements Comparabile {  
    private int ore;  
    private int minuti;  
  
    public Orario(int ore, int minuti) {  
        this.ore = ore;  
        this.minuti = minuti;  
    }  
  
    public int getOre() {  
        return this.ore;  
    }  
  
    public int getMinuti() {  
        return this.minuti;  
    }  
}
```

```
public boolean minoreDi(Comparabile c) {  
    Orario o;  
    o = (Orario)c;  
    if (this.getOre() > o.getOre())  
        return false;  
    if (this.getOre() == o.getOre())  
        return (this.getMinuti() < o.getMinuti());  
    return true;  
}
```

Interface, Ruoli e Riutilizzo (5)

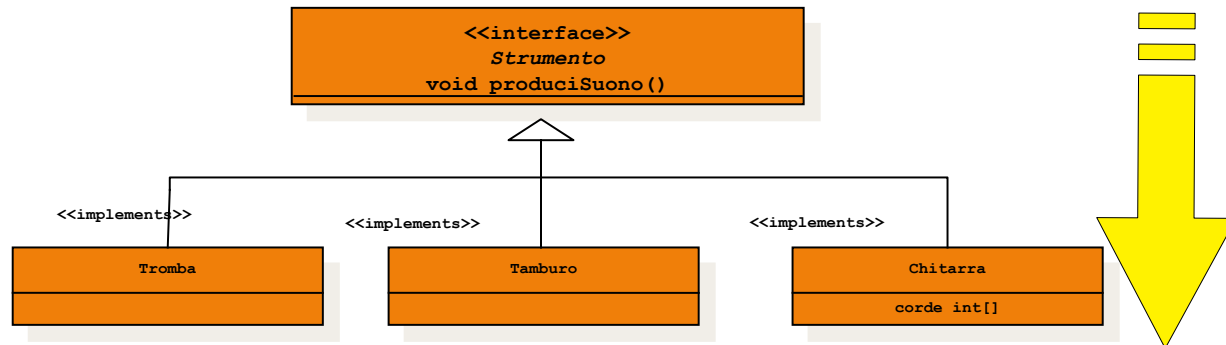
- Per rispettare l'interface **Comparabile** il metodo **minoreDi ()** deve prendere come parametro un oggetto **Comparabile**

```
public boolean minoreDi (Comparabile c)
```

- Quando però scriviamo il codice, dobbiamo poter usare i metodi specifici della classe **Orario** (altrimenti non potremmo implementare il metodo!)
- Il compilatore non ce lo permette: il tipo statico del parametro è **Comparabile**

Downcasting (1)

- È necessaria allora una “forzatura” sul tipo del parametro
- In particolare forziamo l'utilizzo (a tempo statico ed anche dinamico>>) del sottotipo
- Questa operazione viene chiamata *downcasting* (in opposizione all'*upcasting*)



Downcasting (2)

- Quando si opera il downcasting:
 - Informiamo il compilatore che vogliamo “forzarlo” ad usare un certo tipo statico
 - Lo stesso introduce (a tempo statico, durante la compilazione) nel codice oggetto un controllo da eseguirsi a tempo dinamico, durante l'esecuzione: la macchina virtuale verifica che l'operazione sia lecita e possibile
 - Ovvero verifica che l'oggetto sia di un tipo dinamico effettivamente sottotipo del tipo statico forzato
- In caso contrario il programma abortisce sollevando una eccezione di tipo `java.lang.ClassCastException`

Esercizio

- 1) Scrivere con JUnit test-case *minimali* per confermare il corretto funzionamento del metodo `minoreDi()` come implementato nella classe `Orario`
- 2) Scrivere con JUnit un test-case per:
 - definire e creare un array di 5 oggetti `Orario`
 - creare 5 oggetti orario, che rappresentino i seguenti orari: 12:30, 21:40, 9:20, 4:00, 1:35
 - mettere i riferimenti ai 5 oggetti creati negli elementi dell'array
 - Ordinare l'array
 - Verificare che sia correttamente ordinato

Esercizio

- Scrivere una classe **Studente**, che contenga i campi nome (una stringa), età (un intero), un costruttore con due parametri, e i metodi accessori
- La classe **Studente** deve implementare l'interfaccia **Comparabile**, descritta in precedenza (vedi codice di **Orario**)
- Scrivere un metodo che crei un array di oggetti **Studente** e lo ordini (per età) usando il metodo **Ordinatore.ordina()**
- Scrivere con JUnit una classe di test per verificare che l'array sia effettivamente ordinato, dopo l'invocazione del metodo **Ordinatore.ordina()**

Esercizio

- Introdurre nell'interfaccia **Comparabile** un nuovo metodo
`int compara(Comparabile c)`
che restituisce un valore negativo, pari a 0, positivo, se l'oggetto su cui è chiamato il metodo è rispettivamente minore, uguale, maggiore del valore del parametro
- Nella classe **Ordinatore**, scrivere il codice del metodo:
`public static int
 ricercaBinaria(Comparabile[] v, Comparabile cercato)`
che implementa l'algoritmo di ricerca binaria (cfr. corso di Fondamenti di Informatica); questo metodo restituisce un intero il cui valore corrisponde alla posizione dell'elemento **cercato** nell'array **v** oppure a -1 se tale elemento non è presente
- Scrivere, utilizzando JUnit, una classe di test per verificare il corretto funzionamento del metodo

Programmazione Orientata agli Oggetti

Polimorfismo:
Studio di caso

Riprendiamo lo Studio di Caso

- Nelle lezioni precedenti abbiamo già individuato (e rimosso) alcuni problemi nel codice dello studio di caso:
 - La responsabilità di gestire il labirinto deve essere assegnata ad una classe opportuna (la classe **Labirinto**)
 - Analogamente la responsabilità di gestire le informazioni relative al giocatore (borsa, CFU) devono essere assegnate ad una classe opportuna (la classe **Giocatore**): **da fare per esercizio!**
- La qualità del codice rimane bassa
 - La classe **DiaDia** implementa tutti(!) i possibili comandi del gioco
 - Analizziamo le conseguenze di quest'ultimo punto

Introdurre Nuovi Comandi

- Per introdurre un nuovo comando dobbiamo:
 - aggiungere un elemento nell'array **ElencoComandi**
 - aggiungere un metodo nella classe **DiaDia**
 - modificare il metodo **processaIstruzione (String)**
- Ma proviamo a ragionare in termini di responsabilità ed a "pensare in avanti"

Pensiamo in Avanti

- È ragionevole supporre che in futuro nel nostro gioco possano essere introdotti nuovi comandi
- Se per ogni comando introdotto dobbiamo modificare la classe **DiaDia** come abbiamo appena descritto, tale classe crescerà a dismisura

Una Soluzione (1)

- Il problema nasce dal fatto che la classe **DiaDia** conosce e realizza i dettagli di tutti i comandi
 - dovrebbe limitarsi a chiamare l'esecuzione di un comando, senza conoscerne i dettagli
- Le operazioni corrispondenti all'esecuzione di ogni comando dovrebbero essere codificate direttamente da un oggetto **Comando**
 - ma abbiamo tanti diversi comandi, ognuno con le sue peculiarità ...
- Per ovviare al problema un programmatore esperto ci suggerisce di sfruttare le potenzialità del polimorfismo, ristrutturando il codice come indicato di seguito

Una Soluzione (2)

- La classe **Comando** va trasformata in una interface, che rappresenti un generico comando
- L'interface **Comando** deve offrire il metodo
`public void esegui(Partita partita)`
- Tutti i comandi del gioco saranno realizzati da oggetti istanze di classi che implementano l'interface **Comando**:
 - l'implementazione del metodo `esegui(Partita partita)` codifica la semantica del comando specifico
- (Per ora) la classe **DiaDia** sulla base delle istruzioni lette da tastiera istanzia l'implementazione opportuna del comando
- Al comando istanziato chiederà quindi di eseguire il metodo `esegui(Partita partita)`, senza preoccuparsi di come avverrà l'esecuzione

La Classe DiaDia

```
private boolean processaIstruzione(String istruzione) {
    Comando comandoDaEseguire;
    FabbricaDiComandi factory = new FabbricaDiComandi()

    comandoDaEseguire = factory.costruisciComando(istruzione);
    comandoDaEseguire.esegui(this.partita);
    if (this.partita.vinta())
        System.out.println("Hai vinto!");
    if (!this.partita.giocatoreIsVivo())
        System.out.println("Hai esaurito i CFU...");
    return this.partita.isFinita();
}
```

- L'oggetto **factory** (istanza di **FabbricaDiComandi**) ha la responsabilità di creare un oggetto **comando**. Non ci interessano le specificità di ogni singolo comando disponibile
 - vedremo in seguito (>>) come è fatta la classe **FabbricaDiComandi**
- Invochiamo semplicemente il metodo **esegui()** (che è polimorfo): in sostanza lasciamo al comando la responsabilità di eseguire il comando
- Spariscono dalla classe tutti i metodi che implementano i comandi.
 - ✓ Per disporre di un nuovo comando basta introdurre la sua classe, senza dover modificare la classe **DiaDia**

L'Interface *Comando*

```
public interface Comando {  
  
    /**  
     * esecuzione del comando  
     */  
    public void esegui(Partita partita) ;  
  
}
```

La Classe ComandoVai

- Proviamo a creare una implementazione (la classe `ComandoVai`)
- La classe `ComandoVai` implementa il comando che permette di cambiare stanza
- Scriviamone il codice

Implementazione di *Comando*

```
public class ComandoVai implements Comando {  
    private String direzione;  
  
    public ComandoVai(String direzione) {  
        this.direzione = direzione;  
    }  
  
    /**  
     * esecuzione del comando  
     */  
    @Override    
    public void esegui(Partita partita) {  
        // qui il codice per cambiare stanza ...  
    }  
}
```

La classe ComandoVai (2)

```
@Override
```

```
public void esegui(Partita partita) {  
    Stanza stanzaCorrente = partita.getStanzaCorrente();  
    Stanza prossimaStanza = null;  
    if (direzione==null) {  
        System.out.println("Dove vuoi andare?  
                             Devi specificare una direzione");  
        return;  
    }  
    prossimaStanza = stanzaCorrente.getStanzaAdiacente(this.direzione);  
    if (prossimaStanza==null) {  
        System.out.println("Direzione inesistente");  
        return;  
    }  
    partita.setStanzaCorrente(prossimaStanza);  
    System.out.println(partita.getStanzaCorrente().getNome());  
    partita.getGiocatore().setCfu(partita.getGiocatore().getCfu()-1);  
}
```

Osservazioni (1)

- Chi ha la responsabilità di creare gli oggetti **Comando**?
- Questa responsabilità è ragionevole sia affidata non ad un metodo della classe **DiaDia**, ma ad una classe dedicata
- Una classe che *fabbrica* comandi
 - **FabbricaDiComandi**
 - fabbrica un oggetto **Comando** a partire dall'istruzione digitata
- Rimangono alcuni problemi (per ora ci accontentiamo...)
 - In particolare:
 - accoppiamento forte con la gestione dell'I/O
 - il codice a fisarmonica non viene definitivamente eliminato, è stato solamente confinato dentro **FabbricaDiComandi**

Osservazioni (2)

- I comandi possono avere un parametro
- Come facciamo ad impostarne il valore?
 - attraverso il costruttore
Es. `ComandoVai(String direzione)`
 - oppure, introducendo (nella interface) un metodo setter
`setParametro(String parametro)`
- La seconda soluzione impone che *tutte* le classi che implementano **Comando** abbiano questo metodo
 - anche quelle che rappresentano comandi senza parametri (come «aiuto» o «fine»)
 - ✓ Non necessariamente un problema (il corpo del metodo sarà vuoto); sicuramente poco *elegante*
- Le due soluzioni sono equivalenti. Preferiamo comunque la seconda
 - (per motivi evidenti solo in seguito >>)

L'Interface *Comando* (rivista)

```
public interface Comando {  
  
    /**  
     * esecuzione del comando  
     */  
    public void esegui(Partita partita);  
  
    /**  
     * set parametro del comando  
     */  
    public void setParametro(String parametro);  
  
}
```

La Classe ComandoVai (rivista)

```
public class ComandoVai implements Comando {
    private String direzione;
    @Override
    public void esegui(Partita partita) {
        Stanza stanzaCorrente = partita.getStanzaCorrente();
        Stanza prossimaStanza = null;
        if (this.direzione==null) {
            System.out.println("Dove vuoi andare?
                                Devi specificare una direzione");

            return;
        }
        prossimaStanza = stanzaCorrente.getStanzaAdiacente(this.direzione);
        if (prossimaStanza==null) {
            System.out.println("Direzione inesistente");
            return;
        }
        partita.setStanzaCorrente(prossimaStanza);
        System.out.println(partita.getStanzaCorrente().getNome());
        partita.getGiocatore().setCfu(partita.getGiocatore().getCfu()-1);
    }
    @Override
    public void setParametro(String parametro) {
        this.direzione = parametro;
    }
}
```


Creazione di Oggetti Comando

```
public class FabbricaDiComandi {  
  
    public Comando costruisciComando(String istruzione) {  
        Scanner scannerDiParole = new Scanner(istruzione);  
        String nomeComando = null;  
        String parametro = null;  
        Comando comando = null;  
  
        if (scannerDiParole.hasNext())  
            nomeComando = scannerDiParole.next(); // prima parola: nome del comando  
        if (scannerDiParole.hasNext())  
            parametro = scannerDiParole.next(); // seconda parola: eventuale parametro  
  
        if (nomeComando == null)  
            comando = new ComandoNonValido();  
        else if (nomeComando.equals("vai"))  
            comando = new ComandoVai();  
        else if (nomeComando.equals("prendi"))  
            comando = new ComandoPrendi();  
        else if (nomeComando.equals("posa"))  
            comando = new ComandoPosa();  
        else if (nomeComando.equals("aiuto"))  
            comando = new ComandoAiuto();  
        else if (nomeComando.equals("fine"))  
            comando = new ComandoFine();  
        else if (nomeComando.equals("guarda"))  
            comando = new ComandoGuarda();  
        else comando = new ComandoNonValido();  
        comando.setParametro(parametro);  
        return comando;  
    }  
}
```

Osservazioni

- Abbiamo migliorato significativamente la qualità del codice della classe **DiaDia**
 - risulta ora più coesa (con quali responsabilità?)
 - non è più accoppiata ai *dettagli* dei singoli comandi
 - abbiamo rimosso il codice a *fisarmonica*
 - ora confinato nella classe **FabbricaDiComandi** (anche se in una forma molto più semplice e pulita): non deteriora la qualità del codice di **DiaDia**
 - questa anomalia sarà completamente risolta in seguito (>>)
 - Predisponiamo il codice a questa prevedibile evoluzione:
 - creiamo una interface **FabbricaDiComandi**,
 - ridenominiamo la classe attuale (che implementa tale interface) **FabbricaDiComandiFisarmonica**
 - In pratica astraiamo dai dettagli implementativi della fabbrica, in attesa di una implementazione alternativa (>>)

Creazione di Oggetti Comando

```
public interface FabbricaDiComandi {
    public Comando costruisciComando(String istruzione);
}

public class FabbricaDiComandiFisarmonica implements FabbricaDiComandi {
    @Override
    public Comando costruisciComando(String istruzione) {
        Scanner scannerDiParole = new Scanner(istruzione);
        String nomeComando = null;
        String parametro = null;
        Comando comando = null;

        if (scannerDiParole.hasNext())
            nomeComando = scannerDiParole.next(); // prima parola: nome del comando
        if (scannerDiParole.hasNext())
            parametro = scannerDiParole.next(); // seconda parola: eventuale param.

        if (nomeComando == null)
            ...
        return comando;
    }
}
```

Programmazione Orientata agli Oggetti

Esercitazione:
Interfacce, Polimorfismo

Esercitazione **FormeGeometriche**

(TRATTO DALL'ESAME DEL GIUGNO 2003)

- Una software house sta sviluppando una libreria per la gestione di forme geometriche. Allo stato attuale nella libreria ci sono le classi **Punto**, **Cerchio** e **Rettangolo** (vedi codice)
- Sono già date le seguenti classi:
 - **Punto**
 - **Cerchio**
 - **Rettangolo**

La Classe Punto

```
public class Punto {  
    private int x,y;  
  
    public Punto (int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
  
    public void setX(int x){  
        this.x = x;  
    }  
  
    public void setY(int y){  
        this.y = y;  
    }  
  
    public int getX(){  
        return this.x;  
    }  
  
    public int getY(){  
        return this.y;  
    }  
}
```

La Classe Cerchio

```
public class Cerchio {  
    private int raggio;  
    private Punto centro;  
  
    public Cerchio(Punto centro, int raggio) {  
        this.raggio = raggio;  
        this.centro = new Punto(centro.getX(), centro.getY());  
    }  
  
    public void trasla(int deltaX, int deltaY) {  
        this.centro.setX(this.centro.getX() + deltaX);  
        this.centro.setY(this.centro.getY() + deltaY);  
    }  
  
    public Punto getCentro() { return this.centro; }  
  
    public int getRaggio() { return this.raggio; }  
  
}
```

La Classe Rettangolo

```
public class Rettangolo {  
  
    private int altezza, base;  
    private Punto vertice;  
  
    public Rettangolo(Punto vertice, int altezza, int base) {  
        this.altezza = altezza;  
        this.base = base;  
        this.vertice = new Punto(vertice.getX(), vertice.getY());  
    }  
  
    public void trasla(int deltaX, int deltaY) {  
        this.vertice.setX(this.vertice.getX() + deltaX);  
        this.vertice.setY(this.vertice.getY() + deltaY);  
    }  
  
    public Punto getVertice() { return this.vertice; }  
  
    public int getBase()      { return this.base;    }  
  
    public int getAltezza()   { return this.altezza; }  
  
}
```

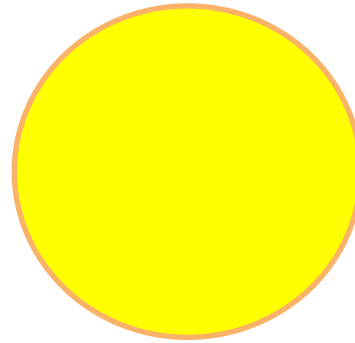

Esercizio 1: Polimorfismo

- Si vuole introdurre una classe **GruppoDiForme** che rappresenta un raggruppamento di forme. In particolare, le forme di un raggruppamento possono essere rettangoli, cerchi e ***altri raggruppamenti***. Un gruppo di forme può essere traslato (vengono traslate tutte le forme che lo compongono)
- La classe **GruppoDiForme** deve offrire i metodi:
 - **`void trasla(int deltaX, int deltaY)`**
trasla tutto il raggruppamento (cioè tutti gli oggetti che compongono il raggruppamento)
 - **`void aggiungiForma(Forma forma)`**
aggiunge una forma al raggruppamento

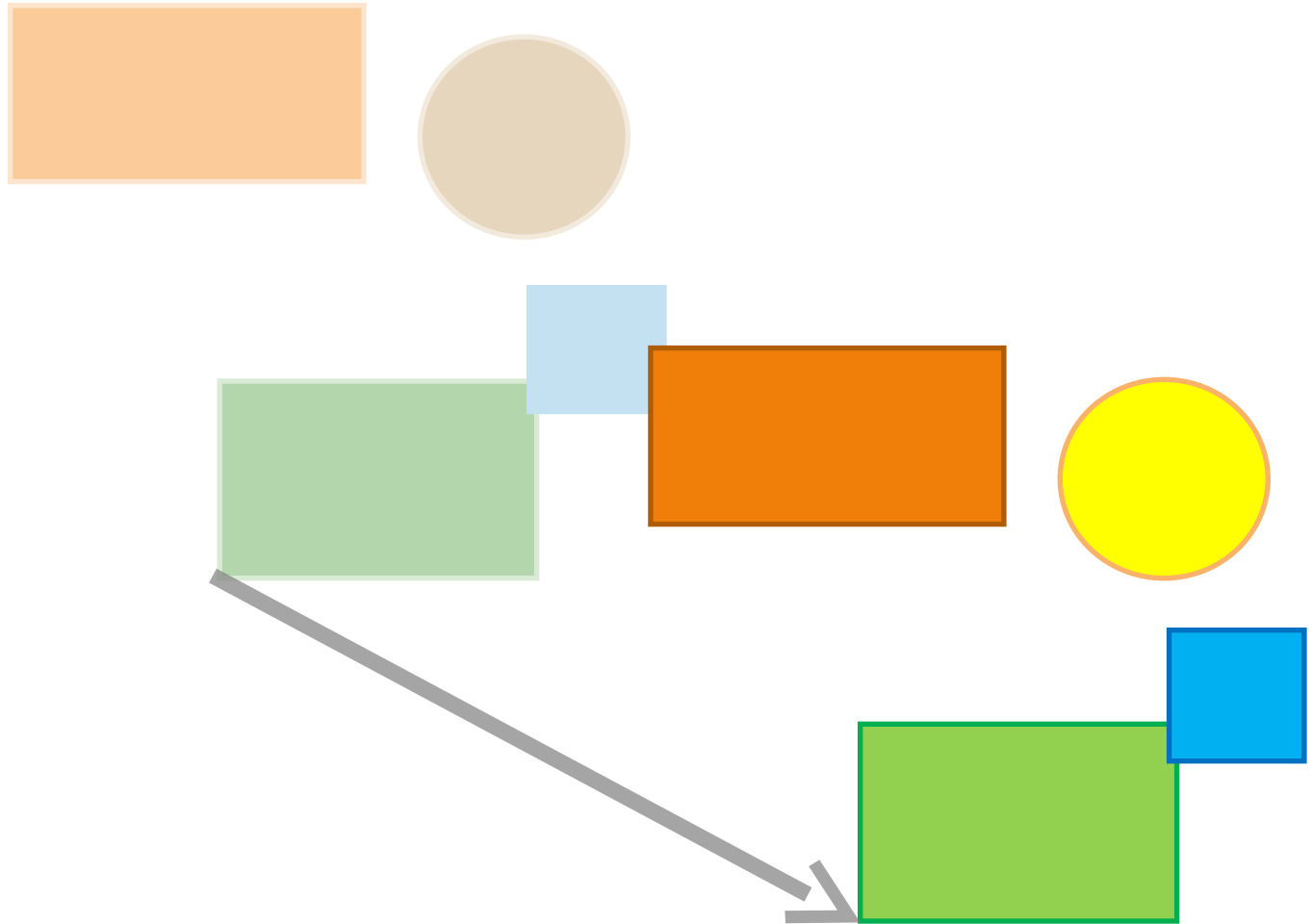
Esercizio 1: Polimorfismo (Continua)

- **Suggerimento:** astrarre i concetti di forma geometrica (rettangolo, cerchio, gruppo) in una interfaccia **Forma**. Quindi, nell'ordine:
 1. Scrivere l'interfaccia **Forma**
 2. Rendere **Cerchio** e **Rettangolo** specializzazioni di **Forma**
 3. Scrivere le classi di test **CerchioTest** e **RettangoloTest**
 4. Scrivere la classe **GruppoDiForme**: Un gruppo di forme è composto da un array di riferimenti a oggetti che implementano **Forma**. Per semplicità si supponga che un gruppo di forme possa essere composto al massimo da 10 forme.
 5. Scrivere la classe **GruppoDiFormeTest**

Un GruppoDiForme



Un GruppoDiForme *traslato()*



Esercizio 2: Testing sulle Forme Semplici

- Utilizzando JUnit creare classi di test per **Cerchio** e **Rettangolo**.
- Aggiungere una serie di test-case *minimali* relativi al metodo **trasla()**
- Ad esempio, un primo test-case **testTrasla()** di **Cerchio**:
 - istanzia un cerchio unitario ($r=1$) di centro sull'origine (0,0)
 - trasla di (+0, +0)
 - asserisce che dopo la traslazione il cerchio non si è spostato
- Un secondo test-case potrebbe traslare di (+1,+0)
- Un terzo test-case potrebbe ...

Esercizio 3: Testing su GruppoDiForme

- Creare una classe di test per **GruppoDiForme** e scrivere test-case *minimali* del metodo `trasla()` che eseguano le seguenti istruzioni
- Distinguere almeno questi scenari di testing a complessità crescente, nell'ordine:
 - un gruppo *vuoto*
 - un gruppo *semplice*, con una sola forma non ulteriormente decomponibile
 - un gruppo *composito*, ovvero di un gruppo contenente un gruppo semplice
 - un gruppo *complesso*, ovvero di un gruppo contenente un gruppo composito
- Scrivere diversi test-case *minimali* per ciascun scenario

Esercizio 4: Downcasting per `assertEquals()` nel Metodo `equals()`

- Il metodo (di JUnit) `assertEquals()` si basa sull'esecuzione del metodo `equals()` sugli oggetti passati come argomento
- La segnatura esatta del metodo `equals` DEVE essere:
@Override

```
public boolean equals(Object o)
```
- Per accorciare i test-case già prodotti:
 - munire la classe `Punto` di un metodo `equals()`
 - Oggetti istanza della classe `Punto` distinti ma di pari coordinate uguali sono considerati equivalenti
 - assicurarsi che i test usino `assertEquals()` passandogli oggetti istanza di `Punto`
 - controllare che risulti effettivamente invocato il metodo `Punto.equals(Object o)`

Programmazione Orientata agli Oggetti

Approfondimenti Interface
Estensione (prima parte)
La classe `Object`

Sommario

- Estensione di Interfacce
- Estensione di Classi (prima parte)
- La classe `Object`

Sommario

- Estensione di Interfacce
- Estensione di Classi (prima parte)
- La classe `Object`

Estensione di Interface (1)

- Talvolta può essere utile definire una nuova interface a partire da una interface esistente
- Questo significa definire una nuova interface che offre qualche servizio (metodo) **aggiuntivo** rispetto ad una interface nota

Estensione di Interface (2)

- Esempio: data l'interface **A**

```
public interface A {  
    public void a1(int i);  
    public String a2();  
}
```

- Supponiamo di dover definire l'interface **B**, che debba offrire gli stessi metodi di **A**, ed in più il metodo **b1()**

Estensione di Interface (3)

- Potremmo definire la nuova interface in questo modo

```
public interface B {  
    public void a1(int i);  
    public String a2();  
    public int b1();  
}
```

- Ma le due interface non avrebbero alcuna relazione esplicita (soprattutto, i *tipi* che definiscono non la possiedono affatto)
- Di conseguenza non potremmo referenziare con un oggetto **B** un riferimento ad **A**, anche se concettualmente sembrerebbe sensato

```
A a = new ClasseCheImplementa_A();  
B b = new ClasseCheImplementa_B();  
a=b; // ERRORE
```

Estensione di Interface (4)

- Sarebbe utile e sensato riuscire a specificare che **B** è un sottotipo di **A**
- In Java (e analogamente in altri linguaggi) questo è possibile definendo una interface come una *estensione* di un'altra interface
- Relativamente al nostro esempio possiamo scrivere

```
public interface B extends A {  
    public int b1();  
}
```

Estensione di Interface (5)

- In questo modo stiamo definendo una nuova interface (**B**) a partire da una già esistente (**A**)
- In particolare stiamo dicendo che:
 - **B è un sottotipo di A**
 - **B** offre tutti i metodi di **A** più il metodo **b1 ()**
- Quindi *vale* il principio di sostituzione
- In questo caso le istruzioni:

```
A a = new ClasseCheImplementa_A();
```

```
B b = new ClasseCheImplementa_B();
```

```
a = b; // OK B è un sottotipo di A
```

sono corrette

Estensione di Interface: Corrette Motivazioni

- L'estensione di interfacce *non* è una scorciatoia per non ripetere la scrittura di metodi
- E' invece un sofisticato meccanismo per definire sottotipi con effetti importanti sulla modellazione del dominio (vedi corso APS >>)
- Usare correttamente l'estensione delle interface richiede esperienza
 - Il legame tra le due interface (tipo/sottotipo) è forte e deve essere ben giustificato dal dominio

Utilizzo dell'Estensione di Interface

- In questo corso non arriveremo praticamente mai a far utilizzo dell'estensione delle interface già in fase di progettazione
- Però dobbiamo essere in grado di capirne la semantica, perché *useremo* spesso e volentieri molte interface definite per estensione di altre
 - Sono infatti frequentemente utilizzate in alcune delle API di Java approfondite in seguito, come ad es. nelle collezioni>>

Meccanismi per la Creazione di Tipi

- Attraverso l'estensione delle interfacce è possibile definire nuovi tipi a partire da tipi già esistenti
- Riassumiamo tutti i meccanismi visti sinora per introdurre nuovi tipi in Java annunciando le linee guida per il loro utilizzo
 - Esistono altri meccanismi (classi astratte, tipi enumerativi, classi nidificate) che per ragioni di natura prettamente didattica conviene rimandare
- ✓ Sfruttiamo invece l'occasione per introdurre la classe `Object`, che al contrario conviene comprendere il prima possibile

Creazione di Tipi Ex-Novo

- Le *Interfacce*
 - permettono di definire nuovi tipi senza definire l'implementazione dei metodi che formano la specifica di tipo
- Le *Classi*
 - permettono di definire nuovi tipi ma richiedono l'implementazione di tutti i metodi che formano la specifica di tipo
- Le *Classi Astratte* (>>)
 - strumento “intermedio”: permette di lasciare qualche metodo astratto, ovvero senza implementazione, pur consentendone la definizione

Definizione di Nuovi Tipi per Estensione

- Estensione di interfacce
 - nuove interfacce definite come estensione di altre
 - nessun metodo nelle due interfacce possiede implementazione
- Estensione di classi
 - nuove classi definite come estensione di altre già esistenti
 - i metodi ereditano anche l'implementazione, che se necessario può essere sovrascritta (*override*)
- Per entrambe:
 - l'insieme dei metodi del tipo esteso comprende quelli pubblici del tipo base, più altri di nuova definizione

Sommario

- Estensione di Interfacce
- Estensione di Classi (prima parte)
- La classe Object

Estensione di Classi: Introduzione

- Uno dei meccanismi più caratteristici (e più difficili da usare *correttamente*) dei linguaggi OO è l'estensione (o ereditarietà)
- Con l'estensione possiamo definire una nuova classe a partire da una classe esistente
 - **aggiungendo** campi (variabili e/o metodi) a quelli della classe originale
 - **sovrascrivendo** metodi della classe originale

Estensione di Classi: Terminologia

- La classe di partenza viene chiamata *superclasse*, o *classe base*, o *classe genitore*
- La classe definita per estensione a partire da una classe base viene chiamata *classe estesa*, o *classe derivata*, o *sottoclasse*, o *classe figlia*
 - ✓ nell'ambito del corso preferiamo usare i termini *classe base* e *classe estesa/derivata*
- Siccome la classe derivata può a sua volta essere utilizzata come classe base di una nuova classe, si dice anche che le classi sono organizzate in una *gerarchia*

Estensione di Classi: Caratteristiche Generali

- La classe estesa conserva ("eredita") tutti i campi della classe base
- Rispetto alla classe base, la classe estesa di solito:
 - può avere qualche membro (campo e/o metodo) in aggiunta
 - può ridefinire il comportamento di qualche metodo
- La classe base viene considerata un supertipo della classe estesa
 - Quindi vale il *principio di sostituzione*: un'istanza della classe estesa può essere considerata anche come un'istanza della superclasse

Estensione di Classi: Esempio (1)

```
public class Persona {  
    private String nome;  
  
    public Persona(String nome) {  
        this.nome = nome;  
    }  
  
    public void setNome(String nome) {  
        this.nome = nome;  
    }  
  
    public String getNome() {  
        return this.nome;  
    }  
  
    public String toString() {  
        return this.getNome();  
    }  
}
```

Definizione di una Classe estesa

- In Java per indicare la definizione di una nuova classe per estensione di una già esistente si usa la parola chiave **extends**

```
class Studente extends Persona {  
  
    // metodi e campi  
  
}
```

Ereditarietà: Caratteristiche Generali (1)

- Tutte le variabili e tutte le operazioni definite nella classe base sono «ereditate» nella classe estesa
- Rispetto alla classe base, la classe estesa
 - può avere qualche membro (campo e/o metodo) *in più*
 - può sovrascrivere il comportamento di qualche metodo
- La classe base viene considerata un supertipo della classe estesa
 - vale il principio di sostituzione: un'istanza della classe estesa può essere considerata anche come un'istanza della superclasse

Estensione di Classi: Esempio (2)

```
public class Studente extends Persona {  
    private String matricola;  
  
    public Studente(String nome, String matricola) {  
        // vediamo dopo  
    }  
  
    public void setMatricola (String matricola) {  
        this.matricola = matricola;  
    }  
  
    public String getMatricola() {  
        return this.matricola;  
    }  
    @Override  
    public String toString() {  
        // vediamo dopo  
    }  
}
```

Definizione di una Classe Estesa (1)

- La classe `Studente` rispetto alla classe `Persona` possiede una nuova variabile di istanza:
 - `matricola`... e i corrispondenti due nuovi metodi accessori
 - `void setMatricola(String)`
 - `String getMatricola()`
- Le variabili di istanza e i metodi vengono *ereditati*:
 - le istanze della classe estesa hanno le stesse variabili della classe base *più* quelle eventualmente aggiunte
 - tutti i metodi pubblici della classe base sono disponibili nella nuova classe, senza necessità di ridefinirli
 - ✓ N.B.: esattamente come per tutte le altre classi *esterne* alla classe base, anche la classe estesa può accedere ai membri (variabili di istanza o metodi) pubblici della classe base ma *non* a quelli privati

Definizione di una Classe Estesa (2)

- I nuovi membri della classe estesa non hanno nulla di particolare
- Se si dispone di un oggetto **Studente** è possibile invocare i metodi della classe base **Persona**
 - esempio: se si dispone di un riferimento ad un oggetto **Studente** è possibile invocare i metodi della classe base

```
Studente anonimo = new Studente("", "");  
anonimo.setNome("Paolo");
```

- In generale, possiamo usare tutti i metodi pubblici della classe base (ereditati) oltre quelli della classe estesa

Estensione e Polimorfismo (1)

- Una classe estesa è un sottotipo della classe base (la classe base è un supertipo della classe estesa)
- Infatti la classe estesa offre l'interfaccia (e l'implementazione di alcuni) dei metodi della classe base
- Quindi, in base al principio di sostituzione, la classe estesa può essere usata sempre laddove è richiesto un oggetto della classe base
- ✓ N.B.: esattamente come nel caso di una classe che implementa un'interfaccia fornendone un sottotipo *concreto*

Estensione e Polimorfismo (2)

- **Studente** automaticamente possiede tutti i metodi di **Persona**, senza bisogno di definirli
- La classe estesa ha quindi l'interfaccia e *l'implementazione* dei metodi della classe base
- **Studente** è un sottotipo di **Persona**: può essere usata al posto di **Persona**

Estensione di Classi (1)

- Vale il principio di sostituzione: il sottotipo può certamente essere usato al posto di un supertipo

```
public class ProvaPersona {  
    public static void main(String[] args) {  
        Persona p = new Studente("Paolo", "123456");  
        p.setNome("Anna");  
        System.out.println(p.getNome());  
        Studente s = new Studente("Luigi", "654321");  
        s.setNome("Antonio");  
    }  
}
```

Estensione di Classi (2)

- Attenzione: possono essere invocati solo i metodi (pubblici) del tipo statico

```
public class RiProvaPersona {  
    public static void main(String[] args) {  
        Persona p = new Studente("Paolo", "123456");  
        p.setNome("Anna");  
        p.setMatricola("33333"); // NON COMPILA!  
    }  
}
```

Ereditarietà: Caratteristiche Generali (2)

- Tutte le variabili e tutte le operazioni definite nella classe base sono "*ereditate*" nella classe estesa
- Rispetto alla classe base, la classe estesa
 - ha qualche membro (campi e/o metodi) *in più*
 - può sovrascrivere il comportamento di qualche metodo
- La classe base viene considerata un supertipo della classe estesa
 - vale il principio di sostituzione: un'istanza della classe estesa può essere considerata anche come un'istanza della superclasse ed usata al suo posto

Overriding (“Riscrittura”)

- Alcune implementazioni dei metodi offerti nella classe base possono essere *non adatte* alla classe estesa
- Tipico esempio il metodo **String toString()**
la stampa dovrebbe permettere di distinguere le istanze della classe base da quelle della classe estesa
 - ✓ Ad es. nella stringa restituita per gli studenti vogliamo che compaia anche la matricola (che non ha senso per tutte le persone)
- Questo comportamento si ottiene facendo l'*overriding* (*sovrascrittura*) del metodo
 - ✓ Attenzione: non confondere *overriding* ed *overloading*

Estensione di Classi: Esempio (3)

```
public class Studente extends Persona {  
    private String matricola;  
  
    public Studente(String nome, String matricola) {  
        // vediamo dopo  
    }  
  
    public void setName(String nome) {  
        this.nome = nome;  
    }  
  
    public String getMatricola() {  
        return this.matricola;  
    }  
    @Override  
    public String toString() {  
        return this.nome + " " + this.matricola;  
    }  
}
```

NON COMPILA: si prova ad accedere
a campi privati (la variabile di istanza `nome`)

Estensione: Overriding

- La soluzione precedente non è praticabile perché i metodi della classe estesa (**Studente**) non possono accedere ai campi privati della classe base (**Persona**)
 - anche se ogni oggetto **Studente** ha ereditato una variabile di istanza in cui viene memorizzata la stringa che rappresenta il nome, non è accessibile!
- Se i metodi della classe estesa vogliono accedere ai campi della classe base *devono* usare l'interfaccia pubblica della classe base come tutte le altre classe *esterne* alla stessa

Estensione di Classi: Esempio (4)

```
public class Studente extends Persona {  
    private String matricola;  
  
    public Studente(String nome, String matricola) {  
        // vediamo dopo  
    }  
  
    public void setMatricola (String matricola) {  
        this.matricola = matricola;  
    }  
  
    public String getMatricola() {  
        return this.matricola;  
    }  
    @Override  
    public String toString() {  
        return this.getNome() + " " + this.getMatricola();  
    }  
}
```

Preferire sempre e comunque l'utilizzo
dei metodi accessori rispetto all'uso
diretto delle variabili di istanza

Accesso al metodo pubblico

Overriding e Polimorfismo (1)

- Un'istanza della classe estesa può essere usata al posto di una istanza della classe base
- Anche qui, si manifesta il polimorfismo (già visto per le interfacce) e il legame al codice avviene a tempo di esecuzione (late binding). Rispetto alle interfacce:
 - se il metodo non è ridefinito, si utilizza (si eredita) l'implementazione della superclasse
 - se il metodo è ridefinito (overriding) si utilizza l'implementazione della classe estesa
- In definitiva si sceglie *sempre* l'implementazione del tipo dinamico

Overriding e Polimorfismo (2)

Il tipo *dinamico* della variabile locale **studente** risulta essere **Studente**, all'invocazione di `toString()` viene eseguito il codice del corpo del metodo presente nella classe **Studente**

```
public class AltraProvaStudente{  
    public static void main(String[] args) {  
  
        Studente studente = new Studente("Paolo", "123456");  
        Persona persona = new Studente("Anna", "654321");  
  
        System.out.println(studente.toString());  
        System.out.println(persona.toString());  
    }  
}
```

Il tipo *dinamico* della variabile locale **persona** è **Studente**, all'invocazione di `toString()` viene eseguito il codice del corpo del metodo presente nella classe **Studente**

Estensione: Creazione di Istanze

- Nella creazione di oggetti istanze di una classe estesa bisogna tener presente la relazione esistente con le istanze della classe padre
 - ✓ ogni istanza della classe estesa è anche una istanza della superclasse
- Alcuni meccanismi offerti dal linguaggio Java nella gestione dei costruttori per classi estese si comprendono meglio tenendo a mente che
 - ✓ ciascuna classe deve essere l'unica responsabile dell'inizializzazione delle proprie istanze
 - ✓ per creare una istanza di una classe estesa bisogna *prima* creare l'istanza della classe base «che è in lei»
 - ✓ bisogna concludere la creazione e l'inizializzazione di una istanza prima di fare qualsiasi altra cosa con la stessa
- ✓ A ben vedere, il servizio di creazione di `Object` (>> e derivati) può essere offerto *solo* dalla JVM

Estensione: Costruttori (1)

- Il costruttore di una classe estesa deve inizializzare:
 - *direttamente* le proprie variabili di istanza
 - *indirettamente* quelle ereditate dalla classe base
- Per il principio dell'information hiding, la classe estesa non può avere la responsabilità di inizializzare direttamente le variabili di istanza della classe base
- Per non violarlo, il costruttore della classe estesa deve poter delegare l'inizializzazione delle variabili di istanza della classe base ad un costruttore della stessa
 - questa operazione in Java si effettua chiamando dal corpo del costruttore della classe estesa il costruttore della classe base mediante la parola chiave **super()** e specificando i parametri attuali del costruttore della classe base tipicamente sulla base dei parametri formali ricevuti (>>)

Estensione: Costruttori (2)

```
public class Studente extends Persona {  
    private String matricola;  
  
    public Studente(String nome, String matricola) {  
        super(nome);  
        this.matricola = matricola;  
    }  
  
    public void setMatricola (String matricola) {  
        this.matricola = matricola;  
    }  
  
    public String getMatricola() {  
        return this.matricola;  
    }  
  
    @Override  
    public String toString() {  
        return this.getNome() + " " + this.getMatricola();  
    }  
}
```

Estensione: Costruttori (3)

- Il corpo dei costruttori di una classe estesa *deve sempre* avere una chiamata al costruttore della classe base mediante l'uso della parola chiave **super** come *prima istruzione*
- In assenza di una chiamata esplicita, il compilatore ne inserisce automaticamente una al costruttore *no-arg* della superclasse
 - Attenzione: *solo* in assenza di tale costruttore nella superclasse si verifica un errore a tempo di compilazione
- **La chiamata al costruttore della classe base deve essere la prima istruzione nel corpo del costruttore della classe estesa**
- ✓ Come già per i metodi, ricordiamo che anche per i costruttori è possibile definire diverse versioni sovraccariche; valgono le regole già viste per l'overloading di metodi

Estensione (Prima Parte): Ricapitoliamo

- La classe estesa:
 - ha tutte le proprietà della classe base
 - è in grado di eseguire tutti i metodi (pubblici) della classe base
 - non ha accesso ai membri privati della classe base (nessuna eccezione al principio dell'information hiding)
- Inoltre:
 - può possedere variabili di istanza proprie, oltre a quelle ereditate dalla classe base
 - può avere metodi propri
 - può specializzare il comportamento di alcuni metodi della classe base
 - può avere versioni sovraccariche del costruttore

Sommario

- Estensione di Interfacce
- Estensione di Classi (prima parte)
- **La classe Object**

La Classe Object

- Tutte le classi estendono (direttamente od indirettamente) la classe `Object`
 - si usa dire che `Object` è la radice della gerarchia dei tipi Java
- `Object` è una classe predefinita, che viene automaticamente estesa da ogni nuova classe (direttamente o indirettamente)
- La classe `Object` ha un insieme di metodi molto generici, ereditati e (volendo sovrascritti) da ogni nuova classe
 - tra questi metodi ce ne sono alcuni già noti (ed altri che lo saranno presto)

... extends Object

- Tutte le classi estendono automaticamente la classe **Object**
 - E' una classe predefinita, che viene automaticamente ed implicitamente estesa da ogni nuova classe (direttamente o indirettamente)
- ✓ Scrivere:

```
public class MiaClasse {  
}
```

è del tutto equivalente a scrivere:

```
public class MiaClasse extends Object {  
}
```

Classe Object: Alcuni Metodi

- Vedere documentazione javadoc
 - `String toString()`
 - `boolean equals(Object o)`
 - ...
- Di tutti questi metodi le nostre classi ereditano l'implementazione, oltre che la segnatura
- Le nostre classi possono ridefinirne l'implementazione (ma a tal fine *devono* rispettarne la segnatura)

Metodo `String toString()` (1)

- Questo è il motivo per il quale non è strettamente necessario definire il metodo `toString()` dentro le classi di nostra definizione per poterlo usare
- Se non lo definiamo, verrà comunque ereditata la definizione del metodo `toString()` propria della classe `java.lang.Object`
- Questa si preoccupa di stampare un messaggio testuale che dipende dall'indirizzo in memoria dell'oggetto sul quale viene invocato
- Di solito risulta poco esplicativo ed utile, e perciò conviene quasi sempre ridefinirlo sulla base delle specificità della classe definita

Metodo `String toString()` (2)

```
public class Persona {  
    private String nome;  
    ...  
    public void setNome(String nome) {  
        this.nome = nome;  
    }  
  
    public static void main(String[] args) {  
        Persona p = new Persona("Paolo");  
        System.out.println(p.toString());  
    }  
}
```

Stampa (qualcosa simile a): `Persona@10b62c9`

Il metodo `toString()` viene ereditato da `Object`:
vale l'implementazione di `Object`

Metodo String toString() (3)

```
public class Persona {  
    private String nome;  
  
    ...  
    public void setNome(String nome) {  
        this.nome = nome;  
    }  
    @Override  
    public String toString() {  
        return this.nome;  
    }  
  
    public static void main(String[] args) {  
        Persona p = new Persona();  
        p.setNome("Antonio");  
        System.out.println(p.toString());  
    }  
}
```

Stampa: **Antonio**

Il metodo `toString()` è stato sovrascritto:
vale l'implementazione riscritta

Metodo boolean `equals()` (Object o)

- Analogamente non è strettamente necessario definire il metodo `equals()` dentro le classi di nostra definizione per poterlo usare
- Anche in questo caso, se non lo definiamo, verrà comunque ereditata la definizione del metodo `equals()` propria della classe `java.lang.Object`
 - questa confronta l'indirizzo in memoria dell'oggetto sul quale viene invocato con il riferimento passato come parametro (analogamente all'operatore `==`)
 - di solito non è questa la semantica desiderata
 - definisce il criterio di equivalenza di oggetti distinti ma istanza della stessa classe (specie se le istanze saranno utilizzate dentro *collezioni*>>)

Metodo equals () & Testing

```
public class TestPersona {
```

```
    @Test
```

```
    public void testEquals() {
```

```
        Persona p1 = new Persona("Paolo");
```

```
        Persona p2 = new Persona("Paolo");
```

```
        assertEquals(p1, p2);
```

```
    }
```

```
}
```

FALLISCE

Il metodo `equals()` viene ereditato da `Object`:

✓ vale l'implementazione del metodo nella classe `Object`

Metodo equals() : Esempio

```
public class Persona {  
    private String nome;  
    ...  
    public void setNome(String nome) {  
        this.nome = nome;  
    }  
  
    @Override // overrides toString() di java.lang.Object  
    public String toString() {  
        return this.nome;  
    }  
  
    @Override // overrides equals(Object o) di java.lang.Object  
    public boolean equals(Object o) {  
        Persona that = (Persona)o; // ← (downcast)  
        return this.getNome().equals(that.getNome());  
    }  
}
```

Il metodo equals(Object o) è stato ridefinito:

✓ il precedente test ora va a buon fine!

Metodo equals() : Downcast Necessario

- Attenzione alla segnatura

```
boolean equals(Object o)
```

- L'argomento è di tipo **Object**!
- Per questo abbiamo dovuto fare un cast
(tecnicamente un *downcast*: abbiamo forzato il
tipo statico al sottotipo atteso a tempo dinamico)

```
Persona that = (Persona)o;
```

- Senza questo downcast non potremmo invocare i
metodi propri della classe (**Persona** nell'esempio) su
cui si basa il confronto (l'uguaglianza degli oggetti
String ottenute invocando **getNome()** nell'es.)

✓ N.B. A sua volta **String** ridefinisce **equals()** ...

Metodo equals() : Un Errore Troppo Frequente (1)

- Un errore è quello di usare come parametro formale un riferimento ad un oggetto della classe sulla quale si sta ridefinendo il metodo

```
boolean equals(Persona persona) // ERRORE
```

- Questa *non* è una *sovrascrittura* del metodo

```
boolean equals(Object persona)
```

- Utilizzare sempre l'annotazione `@Override` per ribadire al compilatore che si sta eseguendo una sovrascrittura

- ✓ segnalerà questo tipo di problemi già in fase di compilazione ed eviterà che si debba ricercarli sulla base degli effetti (non sempre evidenti) in fase di esecuzione

Metodo equals() : Un Errore Troppo Frequente (2)

```
public class Persona {
    private String nome;

    ...
    public void setName(String nome) {
        this.nome = nome;
    }

    public String toString() {
        return this.nome;
    }

    public boolean equals(Persona that) {
        return this.getNome().equals(that.getNome());
    }
}

public class TestPersona {

    @Test
    public void testEquals() {
        Persona p1 = new Persona("Paolo");
        Persona p2 = new Persona("Paolo");
        assertEquals(p1, p2);
    }
}
```

FALLISCE

perché viene invocato il metodo `equals(Object o)`: non essendo stato sovrascritto per l'errata segnatura, viene invece usato quello ereditato da `Object` che **confronta gli indirizzi in memoria degli oggetti**

Conclusioni

- Molti altri dettagli nell'estensione di classi meritano nuovamente uno spazio dedicato più avanti nel corso
- E' tuttavia utile capire già ora come tutte le nostre classi estendono e sono sottotipi di `java.lang.Object`, la radice della gerarchia dei tipi in Java, dal quale ereditano alcuni metodi di pubblica e generale utilità come
 - `String toString()`
 - `boolean equals(Object o)`
- ✓ Questi metodi vengono frequentemente sovrascritti nelle nostre classi per adattarli alle loro peculiarità

Programmazione Orientata agli Oggetti

Estensione (Seconda Parte)

Esercizio

```
public class C {  
    public void dim(C c) {  
        System.out.println("C.dim(C) ");  
    }  
  
    public void dim(L l) {  
        System.out.println("C.dim(L) ");  
    }  
  
    public void dim(K k) {  
        System.out.println("C.dim(K) ");  
    }  
}
```

```
public class K extends C {  
    public void dim(C c) {  
        System.out.println("K.dim(C) ");  
    }  
  
    public void dim(L l) {  
        System.out.println("K.dim(L) ");  
    }  
  
    public void dim(K k) {  
        System.out.println("K.dim(K) ");  
    }  
}
```

```
public class L extends C {  
    public void dim(C c) {  
        System.out.println("L.dim(C) ");  
    }  
  
    public void dim(L l) {  
        System.out.println("L.dim(L) ");  
    }  
  
    public void dim(K k) {  
        System.out.println("L.dim(K) ");  
    }  
  
    public static void main(String args[]) {  
        C a = new K();  
        C b = new L();  
        a.dim(b);  
        L a1 = new L();  
        a1.dim(a);  
    }  
}
```

Sommario

- Estensione nel caso di studio
- Chiamate a metodi della superclasse
- Accesso protetto ai membri
- Overriding di metodi
- Ancora sull'ereditarietà multipla
- La gerarchia dei tipi
- Considerazioni finali sui tipi
- Esercizio: *Overriding for Overloading*

Introduzione

- Riprendiamo lo studio di caso **diadia**
 - supponiamo di voler inserire nel labirinto delle stanze particolari, stanze "magiche", il cui comportamento differisce da quello usuale
- Una stanza magica, esattamente come la stanza ordinaria, ha una descrizione, possiede una collezione di uscite, e può ospitare una collezione di attrezzi

La Stanza Magica (1)

- Una stanza magica ha delle particolarità, che la rendono diversa dalla stanza ordinaria:
 - dopo N volte che in tale stanza viene posato (aggiunto) un qualsiasi attrezzo da parte del giocatore, la stanza inizierà a comportarsi «magicamente»
 - quando la stanza si comporta magicamente, ogni volta che posiamo un attrezzo, la stanza "inverte" il nome dell'attrezzo e ne raddoppia il peso. Ad esempio: se posiamo (togliamo dalla borsa e aggiungiamo alla stanza) l'attrezzo con nome '**chiave**' e peso 2, la stanza memorizza un attrezzo con nome '**evaihc**' e peso 4
 - quando la stanza non si comporta magicamente, il comportamento rimane quello usuale

La Stanza Magica (2)

- Vogliamo introdurre questa *variante* nel nostro gioco
- Una stanza magica deve poter essere usata in qualsiasi punto in cui usiamo oggetti **Stanza**
- La classe della stanza magica pur essendo molto simile a quella della stanza ordinaria, differisce per:
 - alcuni dati in più da gestire
 - alcuni metodi in più che può offrire
 - il comportamento di un metodo
- ✓ Possiamo usare l'estensione e sfruttare il polimorfismo per introdurre questa caratteristica nel gioco

Definizione di Classe Estesa (1)

- La classe **StanzaMagica** rispetto alla classe **Stanza**
 - possiede nuove variabili:
 - **contatoreAttrezziPosati**: memorizza il numero di attrezzi posati (aggiunti)
 - **sogliaMagica**: memorizza il numero di attrezzi da posare prima che si attivi il comportamento «magico» della stanza
 - **SOGLIA_MAGICA_DEFAULT**: valore di default per la soglia
 - possiede un nuovo metodo privato
private Attrezzo modificaAttrezzo(Attrezzo attrezzo) che restituisce un attrezzo a partire dall'attrezzo passato come parametro
 - ridefinisce il metodo **addAttrezzo(Attrezzo attrezzo)** per implementare l'effetto magico
 - ha due costruttori: uno prende nome e soglia, l'altro solo il nome (e imposta la soglia al valore di default)

Esempio: StanzaMagica

```
class StanzaMagica extends Stanza {  
    final static private int SOGLIA_MAGICA_DEFAULT = 3;  
    private int contatoreAttrezziPosati;  
    private int sogliaMagica;  
  
    public StanzaMagica(String nome) {  
        this(nome, SOGLIA_MAGICA_DEFAULT);  
    }  
  
    public StanzaMagica(String nome, int soglia) {  
        super(nome);  
        this.contatoreAttrezziPosati = 0;  
        this.sogliaMagica = soglia;  
    }  
  
    @Override  
    public boolean addAttrezzo(Attrezzo attrezzo) {  
        ...  
    }  
  
    private Attrezzo modificaAttrezzo(Attrezzo attrezzo) {  
        ...// (>>)  
    }  
}
```

Esempio: StanzaMagica

- Un nuovo metodo privato `modificaAttrezzo()` :

```
class StanzaMagica extends Stanza {  
  
    ... ..  
  
    private Attrezzo modificaAttrezzo(Attrezzo attrezzo) {  
        StringBuilder nomeInvertito;  
        int pesoX2 = attrezzo.getPeso() * 2;  
        nomeInvertito = new StringBuilder(attrezzo.getNome());  
        nomeInvertito = nomeInvertito.reverse();  
        attrezzo = new Attrezzo(nomeInvertito.toString(),  
                                pesoX2);  
        return attrezzo;  
    }  
}
```

Definizione di Classe Estesa (2)

- Con oggetti **StanzaMagica** è comunque possibile usare i metodi pubblici definiti nella superclasse quali **getDescrizione()**, **getStanzeAdiacenti()**

```
StanzaMagica labIA = ...
```

```
String s = labIA.getDescrizione();
```

- Questi metodi non sono definiti esplicitamente in **StanzaMagica**, ma vengono *ereditati*
- Allo stesso modo vengono *ereditate* le variabili di istanza
 - (ma solo quelle non private solo visibili nella sottoclasse)

Overriding

- Alcuni metodi della classe base possono essere *ridefiniti* (sovrascritti) nella classe estesa
- Per modificare il comportamento (l'implementazione) di un metodo si effettua un *overriding* (*sovrascrittura*) del metodo
- Nel nostro esempio:

@Override

```
public boolean addAttrezzo(Attrezzo attrezzo)
```

Overriding (Tentativo 1)

```
class StanzaMagica extends Stanza {  
    private int contatoreAttrezziPosati;  
    private int sogliaMagica;  
    ...  
    @Override  
    public boolean addAttrezzo(Attrezzo attrezzo) {  
        this.contatoreAttrezziPosati++;  
        if (this.contatoreAttrezziPosati > this.sogliaMagica)  
            attrezzo = this.modificaAttrezzo(attrezzo);  
        if (this.numeroAttrezzi < this.attrezzi.length) {  
            this.attrezzi[this.numeroAttrezzi] = attrezzo;  
            this.numeroAttrezzi++;  
            return true;  
        }  
        else return false;  
    }  
  
    private Attrezzo modificaAttrezzo(Attrezzo attrezzo) {  
        ...  
    }  
}
```

le variabili `attrezzi` e `numeroAttrezzi` sono ereditate dalla classe base ma non sono accessibili (in quanto private)

Estensione: Overriding

- La soluzione precedente *non* compila: i metodi della classe estesa **StanzaMagica** non possono accedere ai campi *privati* della classe base **Stanza**
 - N.B. anche se ogni oggetto **StanzaMagica** implicitamente possiede le variabili di istanza per rappresentare gli attrezzi contenuti nella stanza
- La classe estesa può accedere solo ai membri pubblici della classe base come tutte le altre classi *esterne*
 - ✓ Coerentemente con il principio dell'*information hiding*
- Nel nostro caso possiamo però pensare di limitarci a riutilizzare il metodo pubblico reso disponibile dalla superclasse **Stanza**:
boolean addAttrezzo(Attrezzo attrezzo)

Overriding (Tentativo 2)

```
class StanzaMagica extends Stanza {  
    private int contatoreAttrezziPosati;  
    private int sogliaMagica;  
    ...  
  
    @Override  
    public boolean addAttrezzo(Attrezzo attrezzo) {  
        this.contatoreAttrezziPosati++;  
        if (this.contatoreAttrezziPosati > sogliaMagica)  
            return addAttrezzo(Attrezzo attrezzo);  
        return true;  
    }  
  
    private Attrezzo modificaAttrezzo(Attrezzo attrezzo) {  
        ...  
    }  
}
```

NON FUNZIONA: viene chiamato
ricorsivamente (all'infinito) il metodo
addAttrezzo(Attrezzo attrezzo) !

Overriding (Tentativo 2)

- La soluzione precedente non funziona perché il metodo `addAttrezzo(Attrezzo attrezzo)` chiama se stesso!
 - `java.lang.StackOverflowError`
 - ✓ come implicato dal late-binding: per l'esecuzione si usa l'implementazione del tipo dinamico, ovvero il corpo dello stesso metodo che si sta definendo
- E' necessario indicare che vogliamo usare il metodo della superclasse
- Questo è reso possibile usando nuovamente la parola chiave **super**
 - ✓ ma con un nuovo significato

Estensione: Overriding

```
class StanzaMagica extends Stanza {  
    private int contatoreAttrezziPosati;  
    private int sogliaMagica;  
    ...  
  
    @Override  
    public boolean addAttrezzo(Attrezzo attrezzo) {  
        this.contatoreAttrezziPosati++;  
        if (this.contatoreAttrezziPosati > this.sogliaMagica)  
            attrezzo = this.modificaAttrezzo(attrezzo);  
        return super.addAttrezzo(attrezzo);  
    }  
  
    private Attrezzo modificaAttrezzo(Attrezzo attrezzo) {  
        ...  
    }  
}
```

Esercizio

- Scrivere una classe di test **StanzaMagicaTest** per testare la classe **StanzaMagica** che preveda anche test-case per verificarne il comportamento “magico”
- Implementare **StanzaMagica** usando l'ereditarietà secondo le linee guida discusse in queste dispense

Visibilità protected (1)

- I membri **private** della classe base non sono accessibili dall'esterno nemmeno da una sottoclasse
- In Java esiste un altro modificatore di accesso, che consente di definire campi a cui sia consentito l'accesso da sottoclassi
 - È il modificatore di accesso **protected**
- Un membro di una superclasse con accesso protetto è visibile a tutte le sottoclassi
 - ✓ Indipendentemente dal package di appartenenza

Visibilità protected (2)

```
class Stanza {  
    private String descrizione;  
    protected Attrezzo[] attrezzi;  
    protected numeroAttrezzi;  
    ...  
}
```

- In questo modo, qualsiasi classe che estenda `Stanza` può accedere alle sue variabili di istanza `attrezzi` e `numeroAttrezzi`
- Si rende possibile una definizione alternativa del metodo `addAttrezzo(Attrezzo attrezzo)` di `StanzaMagica`, con accesso diretto ai campi ereditati da `Stanza`

Accesso ai Membri dalla Classe Estesa

```
class StanzaMagica extends Stanza {  
    private int contatoreAttrezziPosati;  
    private int numeroPassaggi;  
    ...  
    @Override  
    public boolean addAttrezzo(Attrezzo attrezzo) {  
        this.contatoreAttrezziPosati++;  
        if (this.contatoreAttrezziPosati > this.sogliaMagica)  
            attrezzo = this.modificaAttrezzo(attrezzo);  
        if (this.numeroAttrezzi < this.attrezzi.length) {  
            this.attrezzi[this.numeroAttrezzi] = attrezzo;  
            this.numeroAttrezzi++;  
            return true;  
        }  
        else return false;  
    }  
    ...  
}
```

La classe estesa `StanzaMagica` ha la possibilità di accedere ai membri *protetti* (`attrezzi` e `numeroAttrezzi`) della classe base `Stanza`

Visibilità protected (3)

- Più precisamente, è possibile accedere ad un membro **protected**:
 - da tutte le classi estese
 - da tutte le classi dello stesso package!
- I membri protetti sono una violazione (seppur controllata e voluta) dell'information hiding
 - vanno pertanto usati con molta accortezza
 - se possibile, meglio evitare il loro utilizzo
- L'utilizzo più appropriato è nella progettazione di framework, ovvero librerie che consentano agevolmente l'estensione da parte di sviluppatori esperti
 - ✓ ... ben oltre i nostri obiettivi formativi

Livelli di visibilità in Java

Access Levels

Modifier	Class	Package	Subclass	World
<code>public</code>	Y	Y	Y	Y
<code>protected</code>	Y	Y	Y	N
<i>no modifier</i>	Y	Y	N	N
<code>private</code>	Y	N	N	N

scontato.

comprensibile...

da ricordare!

- Il livello di visibilità ottenuto senza modificatore viene anche denominato «package-private»
- Il livello di visibilità «protected» è più permissivo del livello «package-private» ed è il livello più permissivo in assoluto dopo «public»
- Il livello di visibilità «package-private» è il meno permissivo in assoluto dopo «private»

Esercizio

- Reimplementare **StanzaMagica** anche utilizzando l'estensione tra classi, impostando il modificatore di accesso delle variabili **attrezzi** e **numeroAttrezzi** della classe base **Stanza** a **protected** (anziché **private**)
- Le successive evoluzioni del nostro studio di caso saranno un'occasione per vedere quali problemi la violazione dell'information hiding può comportare durante la manutenzione del codice...

Overriding ed Information Hiding

- Solo i metodi pubblici e protetti della classe base possono essere sovrascritti
- Se proviamo a ridefinire un metodo **private** della classe base quello che otteniamo è un nuovo metodo
 - N.B. il nuovo metodo «nasconde» l'omonimo metodo offerto nella superclasse ma *non* lo sovrascrive affatto
- In generale è possibile sovrascrivere un metodo e cambiare il modificatore di accesso, ma solo mantenendo od *ampliandone* la visibilità
 - Perché? Sugg.: pensare al principio di sostituzione
- Quindi per sovrascrivere un metodo, i livelli di visibilità permessi sono:
 - *sovrascritto* → *sovrascrivente*
 - `protected` → `protected`
 - `protected` → `public`
 - `public` → `public`

Visibilità dei Metodi Sovrascritti

- Pertanto il seguente codice è corretto:

```
public class Superclasse {  
    protected void metodo() {}  
}  
  
public class Sottoclasse extends Superclasse {  
    @Override  
    public void metodo() {} // da protected a public OK  
}
```

- mentre invece il seguente non compila:

```
public class Superclasse {  
    public void metodo() {}  
}  
  
public class Sottoclasse extends Superclasse {  
    @Override  
    protected void metodo() {} // da public a protected ERRORE
```

Cambiare Segnatura in Override

- Permettere di sovrascrivere un metodo solo mantenendone od *ampliandone* la visibilità, è la scelta più sensata?
 - ✓ Rispondere applicando il p.d.s.:
 - un metodo che sovrascrive e rimpiazza un altro metodo *deve* certamente permettere tutti gli utilizzi originali
 - Al più, può consentirne *anche* di nuovi
 - ✓ Ampliare la visibilità è coerente con il p.d.s.!
- Finora abbiamo solo visto casi di metodi sovrascritti con ampliamento della visibilità, ma della stessa identica segnatura. Ad es. in `Stanza` e `StanzaMagica`:

```
public boolean void addAttrezzo(Attrezzo a);
```
- Appliciamo lo stesso ragionamento per capire se ha senso cambiare il tipo restituito e/o quello dei parametri dei metodi sovrascritti

Overriding: Metodi di Risultato Polimorfo (1)

- Consideriamo questi due metodi

- `public Studente immatricola()`
 - `public Persona immatricola()`

con **Persona** supertipo di **Studente**

- Quale di questi due metodi in una classe estesa può sovrascrivere l'altro collocato nella classe base? Perché?

- Sugg.: applicare il principio di sostituzione che *deve* restare valido

- ✓ Scriviamo due classi **Organizzazione** ed **Universita**, con la classe **Universita** che estende la classe **Organizzazione**...

Overriding: Metodi di Risultato Polimorfo (2)

```
public class Persona {  
}  
  
public class Studente extends Persona {  
    public String getMatricola() {...}  
}  
  
public class Organizzazione {  
  
    public Studente immatricola() {  
        return new Studente();  
    }  
}  
  
public class Università extends Organizzazione {  
  
    @Override  
    public Persona immatricola() {  
        return new Persona();  
    }  
}
```

NON COMPILA:

```
$ javac *.java
```

```
Università.java:6: error:  
immatricola() in Università cannot  
override immatricola() in  
Organizzazione
```

```
    public Persona immatricola() {  
        ^
```

return type Persona is
not compatible with
Studente

```
Università.java:5: error: method  
does not override or implement a  
method from a supertype
```

```
    @Override  
    ^
```

2 errors

Overriding: Metodi di Risultato Polimorfo (3)

```
public class Persona {  
}  
  
public class Studente extends Persona {  
    public String getMatricola() {...}  
}  
  
public class Organizzazione {  
    public Persona immatricola() {  
        return new Persona();  
    }  
}  
  
public class Universita extends Organizzazione {  
    @Override  
    public Studente immatricola() {  
        return new Studente();  
    }  
}
```

COMPILA!

Un metodo può
sovrascrivere un metodo di
identica segnatura ma tipo
restituito più generale

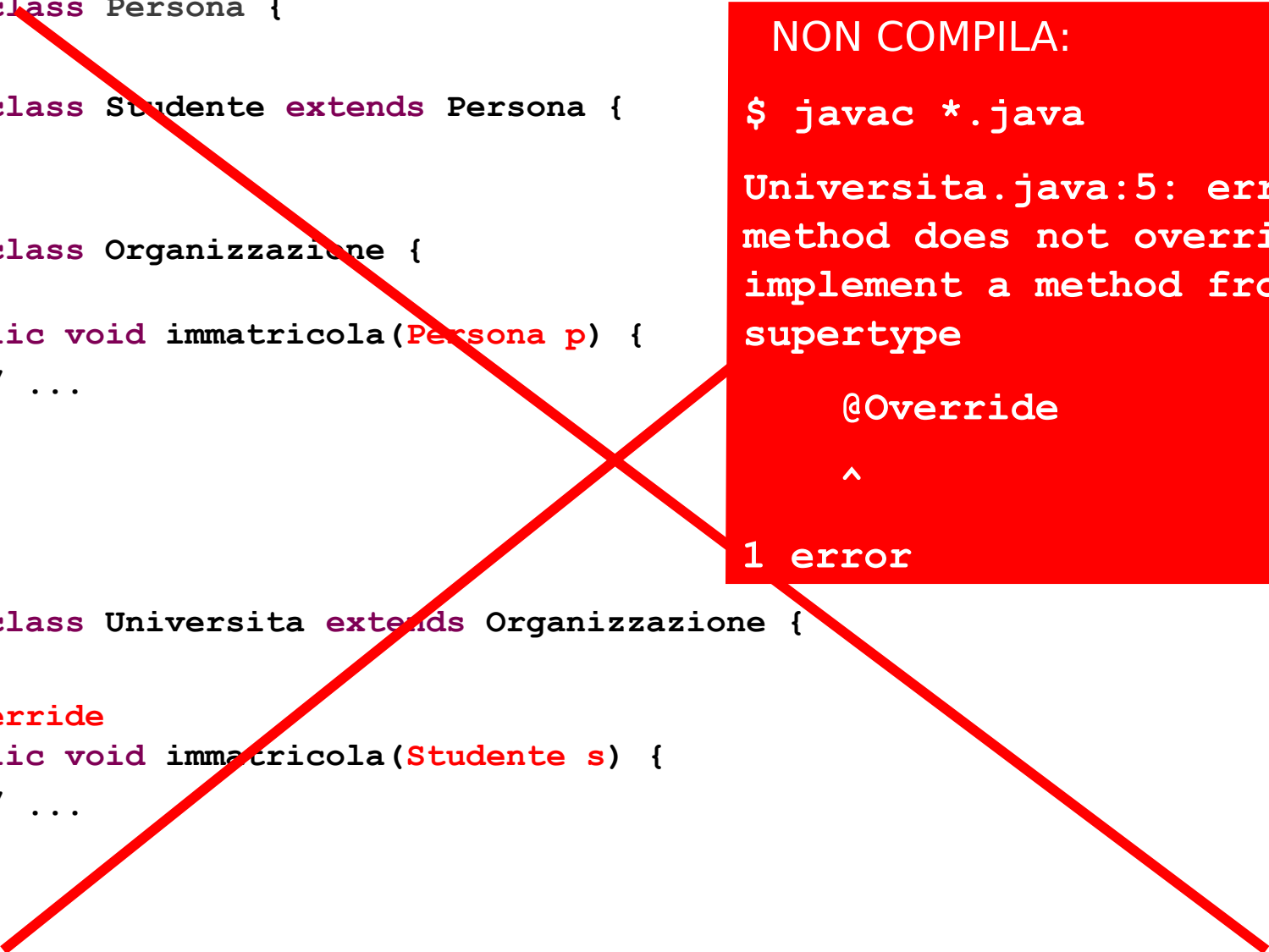
Si usa dire che il tipo
restituito dai metodi
sovrascritti è *covariante*
con la derivazione tra
classi: entrambi tendono a
generalizzarsi od a
specializzarsi muovendosi
in su od in giù lungo la
gerarchia dei tipi

Overriding: Metodi con Parametri Polimorfi (1)

- Consideriamo questi due metodi
 - `public void immatricola(Studiante s)`
 - `public void immatricola(Persona p)`con `Persona` supertipo di `Studiante`
- Quale di questi due metodi in una classe derivata può sovrascrivere l'altro collocato nella classe base e... perché?
- ✓ Scriviamo due classi `Organizzazione` ed `Universita`, con la classe `Universita` che estende la classe `Organizzazione`...

Overriding: Metodi con Parametri Polimorfi (2)

```
public class Persona {  
}  
public class Studente extends Persona {  
}  
  
public class Organizzazione {  
    public void immatricola(Persona p) {  
        // ...  
    }  
}  
  
public class Università extends Organizzazione {  
  
    @Override  
    public void immatricola(Studente s) {  
        // ...  
    }  
}
```



NON COMPILA:

```
$ javac *.java
```

Università.java:5: error:
method does not override or
implement a method from a
supertype

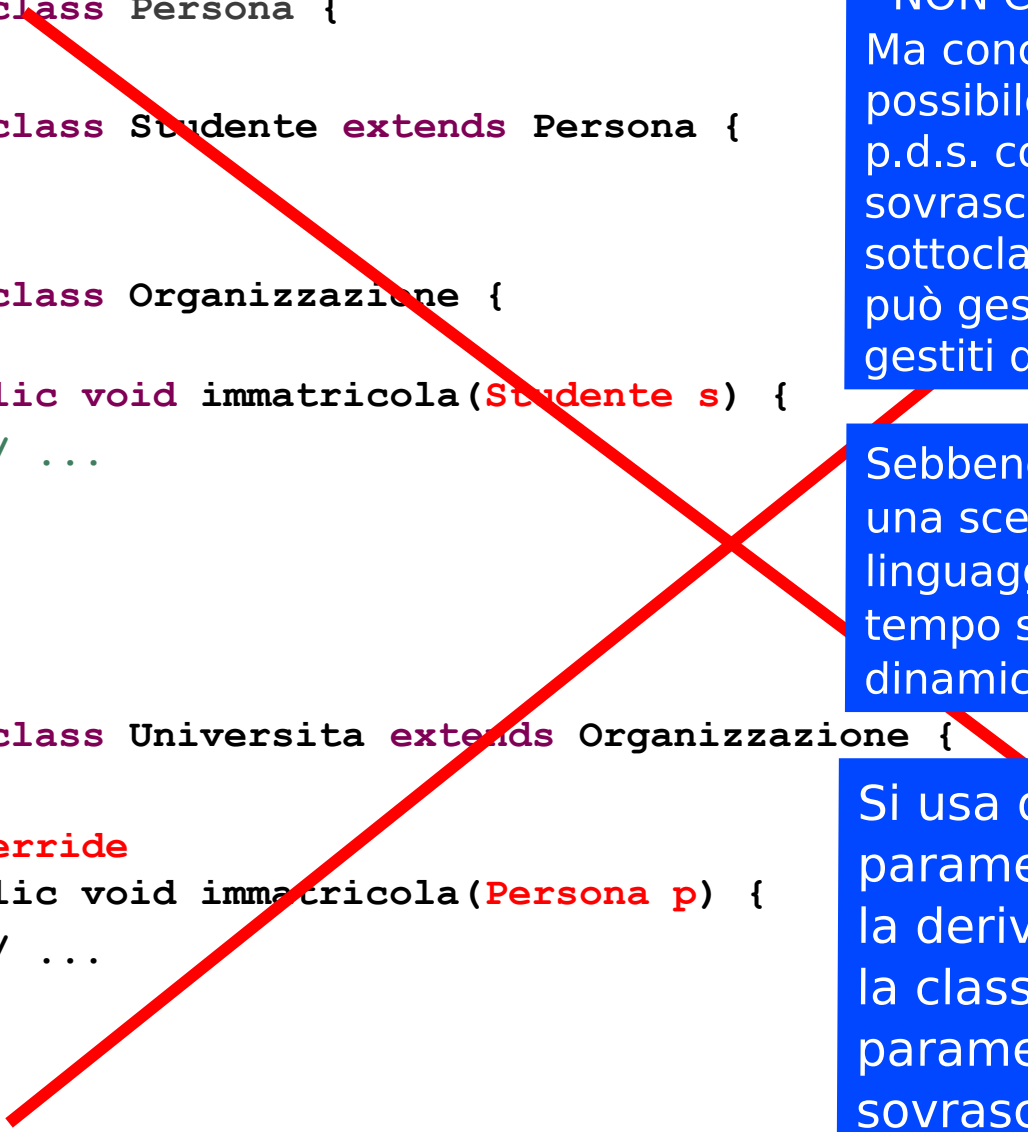
@Override

^

1 error

Overriding: Metodi con Parametri Polimorfi (3)

```
public class Persona {  
}  
  
public class Studente extends Persona {  
}  
  
public class Organizzazione {  
  
    public void immatricola(Studente s) {  
        // ...  
    }  
}  
  
public class Universita extends Organizzazione {  
  
    @Override  
    public void immatricola(Persona p) {  
        // ...  
    }  
}
```



NON COMPILA lo stesso!
Ma concettualmente dovrebbe essere possibile: ovvero si rispetterebbe il p.d.s. considerandoli uno la versione sovrascritta dell'altro perché la sottoclasse ospita un metodo che può gestire un sovrainsieme dei casi gestiti dal metodo sovrascritto.

Sebbene sorprendente, è comunque una scelta coerente con il resto del linguaggio che risolve l'*overloading* a tempo statico e non a tempo dinamico. >>

Si usa dire che il tipo dei parametri è *controvariante* con la derivazione tra classi: quando la classe viene specializzata, i parametri (dei metodi sovrascritti) tendono a generalizzarsi e viceversa

Overriding: Metodi con Parametri Polimorfi (4)

```
public class Persona {  
}  
  
public class Studente extends Persona {  
}  
  
public class Organizzazione {  
  
    public void immatricola(Studente s) {  
        System.out.println("Organizzazione.immatricola(Studente)");  
    }  
  
}  
  
public class Università extends Organizzazione {  
  
    public void immatricola(Persona p) {  
        System.out.println("Università.immatricola(Persona)");  
    }  
  
}
```

COMPILA ma non è un overriding!

Togliendo l'annotazione `@Override`, compila e la sottoclasse `Università` finisce per disporre di due metodi (di cui uno ereditato) sovraccarichi (overloading) e distinti, ma NON sovrascritti (overriding)

Un Overriding che Diventa Overloading

```
static public void main(String args[]) {  
    Persona p = new Persona();  
    Persona ps = new Studente();  
    Studente s = new Studente();  
    Organizzazione org = new Organizzazione();  
    Organizzazione studi = new Università();  
    Università rm3 = new Università();  
//org.immatricola(p); // ERRORE: NON COMPILA  
//org.immatricola(ps); // ERRORE: NON COMPILA  
    org.immatricola(s);  
//studi.immatricola(p); // ERRORE: NON COMPILA  
//studi.immatricola(ps); // ERRORE: NON COMPILA  
    studi.immatricola(s);  
    rm3.immatricola(p);  
    rm3.immatricola(ps);  
    rm3.immatricola(s);  
}
```

Togliendo l'annotazione `@Override`
COMPILA ma non è un overriding!

Università dispone di due metodi
sovraccarichi (overloading): quale
invocare viene deciso direttamente
dal compilatore solo sulla base del
tipo statico

L'esecuzione stampa:

```
Organizzazione.immatricola(Studente)  
Organizzazione.immatricola(Studente)  
Università.immatricola(Persona)  
Università.immatricola(Persona)  
Organizzazione.immatricola(Studente)
```

Riassunto: Covarianza & Controvarianza

Organizzazione

`protected Persona immatricula(Studente)`

tipo + generale

Studente

Persona

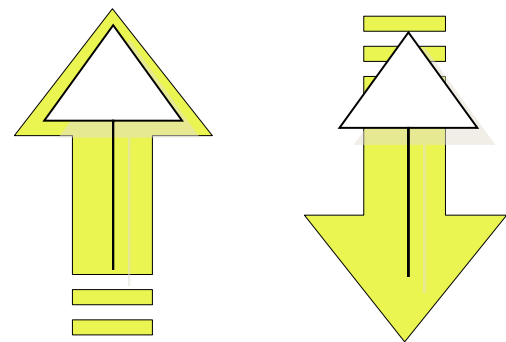
Università

`@Override`

`public Studente immatricula(Persona)`

tipo + speciale

Specializzando $\uparrow$ il tipo che ospita un metodo,
affinché si possa sovrascrivere:
visibilità $\downarrow$ – tipo restituito $\uparrow$ – tipo parametro $\downarrow$



covarianza controvarianza

La Gerarchia delle Classi Java: La Classe `Object`

- Abbiamo già visto che in Java tutte le classi estendono automaticamente la classe `Object`
- E' una classe predefinita, che viene automaticamente estesa da ogni nuova classe (direttamente o indirettamente)
- N.B. scrivere:

```
public class MiaClasse {  
}
```

è del tutto equivalente a scrivere:

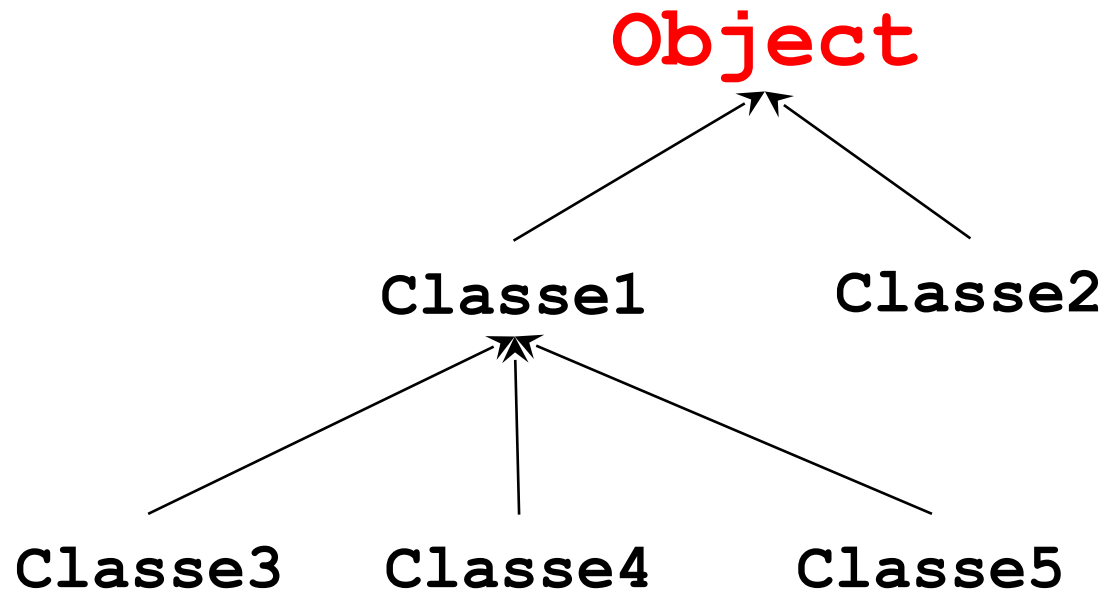
```
public class MiaClasse extends Object {  
}
```


Gerarchie di Classi

- In Java una classe può essere estesa da molte classi, ma ogni classe estende sempre *una ed una sola* classe
 - tranne `Object`, che è la radice predefinita della gerarchia di classi
- Non ci può essere «*ereditarietà multipla*» delle implementazioni

La Gerarchia delle Classi Java

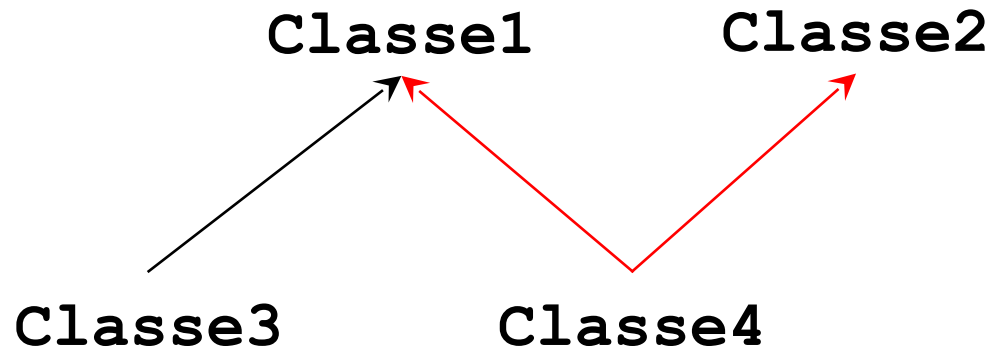
- Un'unica radice: `Object`
- Ogni classe* ha una e una sola superclasse
- Ogni classe può avere zero o più sottoclassi



* con l'eccezione di `Object`

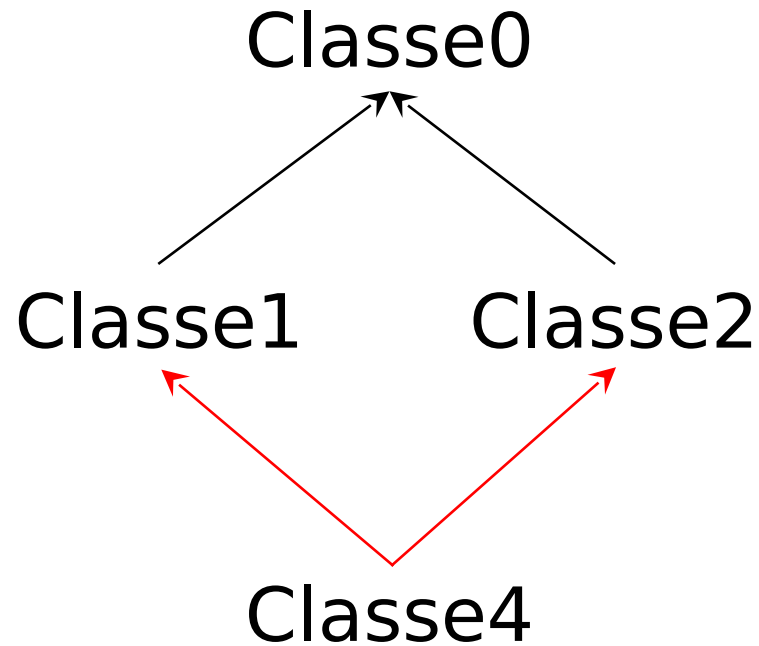
Ereditarietà Multipla non Ammessa

- Se **Classe4** ereditasse sia da **Classe1** che da **Classe2**
- ✓ N.B. In Java **non** è possibile!



- Quali problemi?

Problemi con l'Ereditarietà Multipla: *l'Ereditarietà a Diamante*

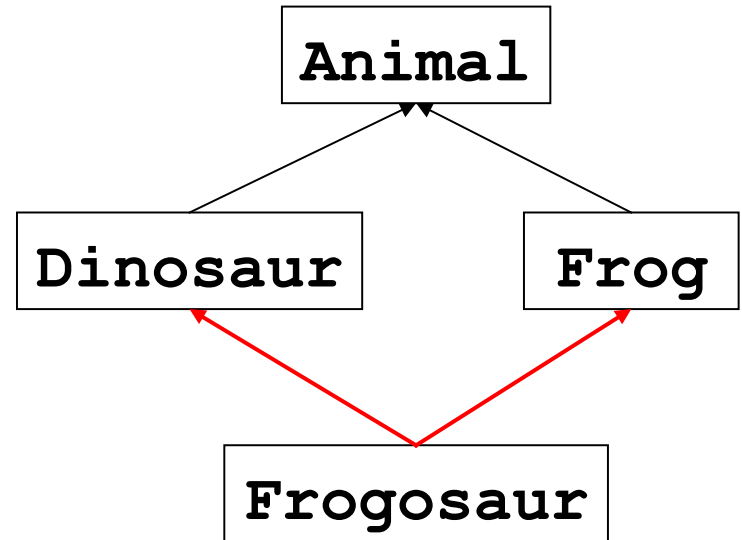


La gerarchia del *Frogosauro*

```
class Animal {  
    void talk() {  
        System.out.println("...");  
    }  
}
```

```
class Frog extends Animal {  
    void talk() {  
        System.out.println("Ribit, ribit.");  
    }  
}
```

```
class Dinosaur extends Animal {  
    void talk() {  
        System.out.println("I'm a dinosaur: I'm cool! ");  
    }  
}
```



II *Frogosau*so

// NON COMPILA

```
class Frogosaur extends Frog, Dinosaur {  
}
```

✓ Cosa dovrebbe fare la seguente chiamata a `talk()`?

```
Animal animal = new Frogosaur();  
animal.talk();
```

Problemi di Ereditarietà Multipla (1)

- Se un membro (metodo o campo) è definito in entrambe le classi base, da quale delle due la classe estesa «eredita» l'implementazione?
 - E se le due classi base a loro volta estendono una superclasse comune ?
 - Il problema è legato alle implementazioni, che vengono ereditate e potenzialmente «confuse»
- ✓ Per questo motivo, in Java si adotta una scelta molto conservativa:
- una classe può implementare *tante* interfacce
 - ma può estendere sempre e solo una *unica* superclasse

Problemi di Ereditarietà Multipla (2)

- In realtà il problema di quale implementazione scegliere, anche in questi casi, è perfettamente risolvibile e risolto in alcuni linguaggi di programmazione
 - (C++, Scala, in una forma molto limitata anche Java 8+)
- Il *Problema del Frogosauro* fa quasi parte del folklore oramai, quasi impossibile non parlarne!
- ✓ Ma solo lascamente chiarisce il vero problema:
 - “mischiare” due implementazioni diverse è estremamente complicato e quasi sempre non necessario
 - comodo in pochi e particolari casi (implementazioni che sono sostanzialmente ortogonali: per i più interessati vedere *mixin*) ben oltre gli obiettivi formativi di questo corso

Implementazione di più Interfacce

- Al contrario in Java è possibile che una classe implementi molteplici interface, una politica molto permissiva:

```
public interface Persona {  
    ...  
}  
  
public class Dirigente implements Impiegato, Persona {  
    ...  
}
```

- ✓ Una classe già ampiamente utilizzata implementa tre interfacce: `java.lang.String`

```
public final class String implements  
    Serializable, Comparable<String>, CharSequence
```

Java Interface ed Ereditarietà Multipla (1)

- Il problema dell'ereditarietà multipla (delle implementazioni) **non sussiste affatto con le interface**
- Consideriamo infatti una classe che implementa più di una interface
- Per essere concreta e risultare istanziabile:
 - è costretta a fornire direttamente tutte le implementazioni di tutti metodi implementati
 - ✓ si evitano così alla radice tutti i problemi legati all'eredità multipla delle implementazioni

Java Interface ed Ereditarietà Multipla (2)

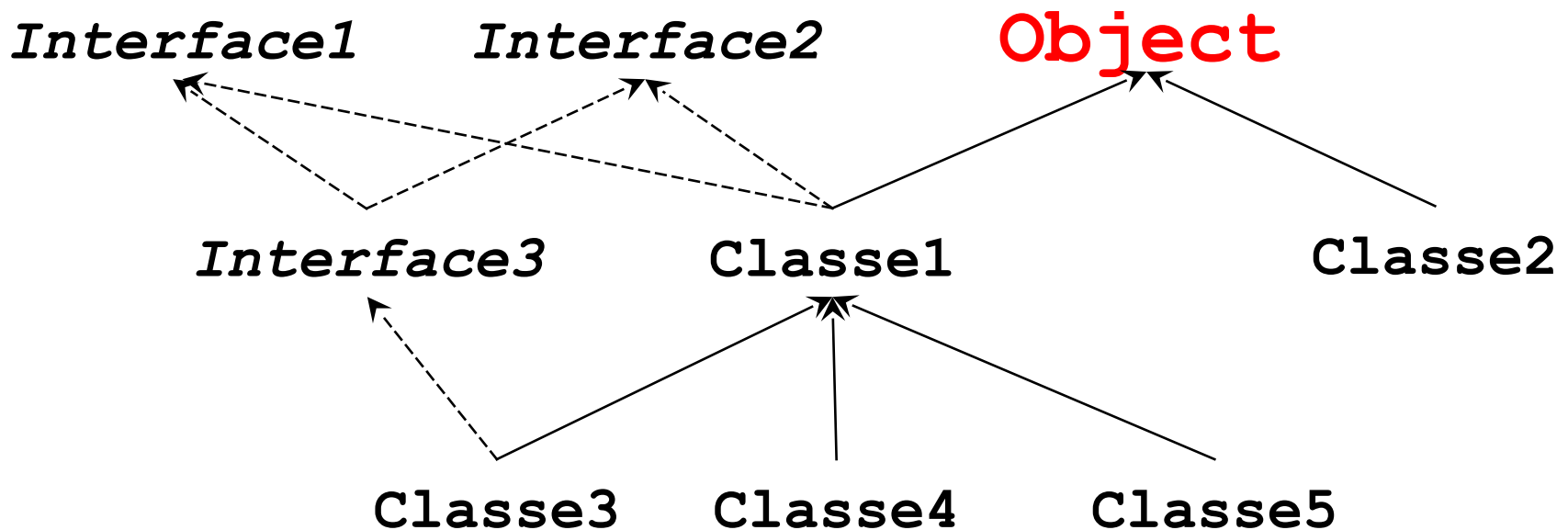
- Abbiamo anche visto che una interface può essere definita per estensione da un'altra interface
- In generale una interface può estendere anche più di una interface scrivendo direttamente:

```
public interface A {}  
public interface B {}  
public interface C {}  
  
public interface I extends A,B,C {}
```

- Pertanto la gerarchia dei *tipi* è più articolata di quella delle classi in quanto contempla anche i tipi definiti tramite le interface...

La Gerarchia dei Tipi Java

- ✓ Un'unica radice: **Object**
- Ogni classe* ha una ed una sola superclasse
- Ogni classe può avere zero o più sottoclassi
- Ogni classe può implementare zero o più interface
- Ogni interface può estendere zero o più interface



* con l'eccezione di **Object**

Gerarchia delle Classi Lineare

- Ogni classe estende sempre una ed una sola classe
- Tranne `Object`, che è la radice predefinita della gerarchia di classi (e tipi)
- Ma una classe può essere estesa da molte classi
- Non ci può essere ereditarietà multipla delle *implementazioni*
- Si usa dire che la gerarchia delle classi è *lineare*
 - ✓ le implementazioni di tutti i metodi di una classe si trovano in una sequenza lineare di classi e superclassi che conducono sino alla radice `Object`
 - ✓ Attenzione: non più perfettamente vero in Java 8+
 - metodi di interface con implementazioni di *default*

L'importanza dei Tipi (1)

- La scelta dei corretti tipi da definire in un programma è un delicato esercizio di modellazione
- Si può affrontare a ragion veduta solo dopo studio, pratica ed interi corsi specificatamente dedicati alla *Analisi & Progettazione* (come APS)
- E' opportuno cercare *subito* di prevenire alcuni degli errori più ricorrenti nei primi utilizzi dei meccanismi di definizione dei tipi
 - Non considerare gli aspetti *dinamici*
(il comportamento degli oggetti)
 - Ovvero considerare solo gli aspetti *statici*
(dati che modellano)

L'importanza dei Tipi (2)

- Su altri errori tipici dovuti alla mancanza di pratica meglio tornarci in seguito (>>)
- Tra i più rilevanti la *tipizzazione anemica*
 - possono scherzosamente chiamarsi sulla base del nome di un tipo di cui si ha un'irrefrenabile tendenza ad abusare, quasi fosse una «malattia»
 - La «*stringhite*» (>>)
 - La «*mappite*» (>>)
- Sono «sintomi» dell'incapacità di scegliere correttamente i tipi da definire

Considerazioni Finali: Tipi e Sottotipi

- Un frequente errore è sicuramente legato alla tendenza a fissare le relazioni tra tipi concentrandosi *solo* sugli aspetti *statici* delle classi e trascurando invece quelli *dinamici*
- Domanda: Ma... *Quadrato* è sottotipo di *Rettangolo*?
Farsi sempre guidare dal *principio di sostituzione*: ogni qualvolta mi aspetto un *Rettangolo* posso utilizzare al suo posto un *Quadrato*?
- Risposta: *dipende!* Se devo calcolarne il perimetro sì, ma se devo raddoppiarne l'altezza senza cambiarne la base sicuramente no!
 - ✓ Il p.d.s. deve valere anche a tempo dinamico, anche dopo i cambiamenti di stato degli oggetti coinvolti

Tipi e Sottotipi:

String VS StringBuilder (1)

- Altro esempio; cfr. **String VS StringBuilder**:
- **String** si usa per oggetti *immutabili*:
 - Una volta creato un oggetto `java.lang.String`, non è poi più possibile cambiarne lo stato
 - ✓ (E' tuttavia possibile creare *nuovi* oggetti stringhe sulla base del primo)
- Per costruire progressivamente stringhe particolarmente lunghe, è spesso preferibile utilizzare la versione modificabile delle stringhe: classe `java.lang.StringBuilder`
- **StringBuilder** si usa per oggetti *mutabili*:

```
public final class StringBuilder ...???... {  
    StringBuilder append(String str);  
  
    ...  
    String toString();  
}
```

- **StringBuilder** deve essere sottotipo di **String**???

Tipi e Sottotipi:

String VS StringBuilder (2)

✓ Cfr. String/StringBuilder/CharSequence:

```
public final class String extends Object
    implements
        Serializable, Comparable<String>, CharSequence {...}
```

```
public final class StringBuilder extends Object
    implements Serializable, CharSequence {...}
```

```
public interface CharSequence {
    public char charAt(int index);
    public int length();
    public CharSequence subSequence(int start, int end);
    public String toString();
}
```

✓ **StringBuilder NON è un sottotipo (né potrebbe esserlo) di String ma di un loro supertipo comune CharSequence;** si prevede esplicitamente la convertibilità degli oggetti **StringBuilder** a **String** tramite **toString()**

Esercizio:

Overriding for Overloading (1)

- Rafforziamo la comprensione del legame tra l'overloading e l'overriding in Java svolgendo un particolare esercizio
- Vogliamo invocare un metodo sovraccarico proprio sulla base del tipo dinamico del suo parametro e *NON* sulla base del suo tipo statico
- N.B. *NON* è possibile farlo con i meccanismi offerti direttamente dal linguaggio Java
 - In Java i metodi sovraccarichi sono risolti sulla base del tipo **STATICO** e mai sulla base del tipo **DINAMICO**!
- Mostriamo come mediante l'overriding di un metodo polimorfo ed il late-binding sia possibile «simulare» la risoluzione di un metodo sovraccarico sulla base del tipo dinamico di un parametro

Esercizio:

Overriding for Overloading (2)

- Consideriamo un semplice esempio: è data una gerarchia di tipi avente come radice l'interface ***Forma*** e sottotipi concreti ***Quadrato***, ***Cerchio***,... ma non è possibile cambiarne il codice
- ✓ Per quanto l'esempio sia volutamente stilizzato, nella pratica questa situazione si presenta non di rado
 - ad es. durante l'uso di gerarchie di tipi definite da librerie esterne il cui codice risulta imm modificabile
 - ad es. quelle che gestiscono composizioni di oggetti tutti di uno stesso supertipo comune ma di diversi sottotipi concreti

Esempio: Una Gerarchia di Tipi (Sorgente Non Modificabile)

```
public interface Forma {  
    }  
public class Cerchio implements Forma {  
    private int raggio;  
    public Cerchio(int r) { this.raggio = r; }  
    public int getRaggio() { return this.raggio; }  
}  
  
public class Quadrato implements Forma {  
    private int lato;  
    public Quadrato(int l) { this.lato = l; }  
    public int getLato() { return this.lato; }  
}
```

- Si vuole calcolare l'area delle forme, che dipende dal tipo, ma NON è possibile modificare l'interface *Forma* per aggiungere il metodo

```
public float getArea();
```

e quindi non è possibile implementarlo in *Quadrato*, *Cerchio*, ...

Risoluzione Tramite Overriding (1)

```
public class CalcolatoreDiArea {  
    public float areaDi(Cerchio c) {  
        int r = c.getRaggio();  
        return 3.14f * r * r;  
    }  
  
    public float areaDi(Quadrato q) {  
        int l = q.getLato();  
        return l * l;  
    }  
  
    public static void main(String args) {  
        CalcolatoreDiArea calcolatore = new CalcolatoreDiArea();  
        Cerchio cerchio = new Cerchio(1);  
        Quadrato quadrato = new Quadrato(2);  
        System.out.println(calcolatore.areaDi(cerchio));  
        System.out.println(calcolatore.areaDi(quadrato));  
    }  
}
```

COMPILA

Ma solo perché si è avuto cura di fare coincidere tipo statico e tipo dinamico delle variabili locali **cerchio** e **quadrato**.

Non è sempre possibile...

Risoluzione Tramite Overriding (2)

```
public class CalcolatoreDiArea {  
    public float areaDi(Cerchio c) {  
        int r = c.getRaggio();  
        return 3.14f * r * r;  
    }  
  
    public float areaDi(Quadrato q) {  
        int l = q.getLato();  
        return l * l;  
    }  
  
    public static void main(String args) {  
        CalcolatoreDiArea calcolatore = new CalcolatoreDiArea();  
        Forma cerchio = new Cerchio(1);  
        Forma quadrato = new Quadrato(2);  
        System.out.println(calcolatore.areaDi(cerchio)); // ERRORE: NON COMPILA  
        System.out.println(calcolatore.areaDi(quadrato)); // ERRORE: NON COMPILA  
    }  
}
```

NON COMPILA:

The method `areaDi(Cerchio)` in the type `CalcolatoreDiArea` is not applicable for the arguments `(Forma)`

- Si supponga invece di voler calcolare l'area totale di una collezione di forme di cui non sia possibile prevedere il tipo a tempo dinamico...

Risoluzione Tramite Overriding (3)

```
public class CalcolatoreDiArea {  
    public float areaDi(Cerchio c) {  
        int r = c.getRaggio();  
        return 3.14f * r * r;  
    }  
    public float areaDi(Quadrato q) {  
        int l = q.getLato();  
        return l * l;  
    }  
    static public float sommaAll(CalcolatoreDiArea calcolatore, Forma[] forme) {  
        float acc = 0;  
        for(Forma forma : forme) {  
            acc += calcolatore.areaDi(forma); // ERRORE: NON COMPILA  
        }  
        return acc;  
    }  
    public static void main(String args) {  
        CalcolatoreDiArea calcolatore = new CalcolatoreDiArea();  
        Forma[] forme = { new Cerchio(1), new Quadrato(2) } ;  
        System.out.println(sommaAll(calcolatore, forme));  
    }  
}
```

NON COMPILA:

The method `areaDi(Cerchio)`
in the type `CalcolatoreDiArea` is
not applicable for the arguments
(`Forma`)

Overriding for Overloading (1)

- Una libreria che pubblica una gerarchia di tipi (con sorgente imm modificabile) può essere predisposta per la definizione di codice *cliente* che sappia specializzare i propri comportamenti sulla base del tipo *dinamico*
- L'interface **Forma** viene cambiata per ospitare un metodo che permetta a tutti i tipi di forme di «accettare» un generico **Calcolatore**:

```
public interface Forma {  
    public float accetta(Calcolatore c);  
}
```

- Questo deve saper distinguere tutti i tipi di forme della gerarchia

```
public interface Calcolatore {  
    public float calcola(Cerchio c);  
    public float calcola(Quadrato q);  
}
```

- ✓ Parliamo di «generici» calcoli; il calcolo dell'area è solo un esempio di uno dei loro possibili «istanziamenti»...

Overriding for Overloading (2)

- Il metodo `calcola()` trova questa implementazione nei sottotipi concreti `Cerchio` e `Quadrato`:

```
public class Cerchio    // lo stesso dicasi per Quadrato
    implements Forma {
    @Override
    public float accetta(Calcolatore c) { // N.B. il tipo statico
        // di this è quello della classe corrente, qui Cerchio
        return c.calcola(this); // tipo statico Cerchio
    }
}
```

- ✓ La chiamata al metodo polimorfo `float accetta(Calcolatore)` di `Forma` come sovrascritto (overload) in `Cerchio` finisce per servire a fissare il tipo statico dell'argomento alla chiamata al metodo sovraccarico (override) `Calcolatore.calcola(Cerchio)`!
- ✓ In `Quadrato` servirà per chiamare l'altro metodo sovraccarico `Calcolatore.calcola(Quadrato)`

Overriding for Overloading (3)

```
public interface Forma {  
    public float accetta(Calcolatore c);  
}  
  
public class Cerchio implements Forma {...  
    @Override  
    public float accetta(Calcolatore c) {  
        return c.calcola(this);  
    }  
}  
  
public class Quadrato implements Forma {...  
    @Override  
    public float accetta(Calcolatore c) {  
        return c.calcola(this);  
    }  
}  
  
public interface Calcolatore {  
    public float calcola(Cerchio c);  
    public float calcola(Quadrato q);  
}
```

E' possibile mettere a fattor comune
in una superclasse tutti i metodi
accetta(Calcolatore)???

Tipo statico Cerchio

Tipo statico Quadrato

Overriding for Overloading (4)

```
public class CalcolatoreDiArea implements Calcolatore {  
    @Override  
    public float calcola(Cerchio c) {  
        return 3.14f * c.getRaggio() * c.getRaggio();  
    }  
    @Override  
    public float calcola(Quadrato q) {  
        return q.getLato() * q.getLato();  
    }  
  
    static public float sommaAll(Calcolatore calc, Forma[] forme) {  
        float acc = 0;  
        for(Forma forma : forme) {  
            acc += forma accetta(calc); // COMPILA!  
        }  
        return acc;  
    }  
  
    public static void main(String args) {  
        Calcolatore calcAree = new CalcolatoreDiArea();  
        Forma[] forme = { new Cerchio(1), new Quadrato(2) } ;  
        System.out.println(sommaAll(calcAree, forme));  
    }  
}
```

Ulteriori Calcolatori (Dipendenti dal Tipo Dinamico)

- E' banalmente possibile implementare altre tipologie di Calcolatore. Ad es.:

```
public class CalcolatoreDiPerimetro implements Calcolatore {  
    @Override  
    public float calcola(Cerchio c) {  
        Return 2 * 3.14f * c.getRaggio();  
    }  
    @Override  
    public float calcola(Quadrato q) {  
        return 4 * q.getLato();  
    }  
}
```

- Il metodo statico `sommaAll()` continua a funzionare:

```
public static void main(String args) {  
    Calcolatore calcPerim = new CalcolatoreDiPerimetro();  
    Forma[] forme = { new Cerchio(1), new Quadrato(2) } ;  
    System.out.println(sommaAll(calcPerim, forme));  
}
```

Conclusioni:

Overriding for Overloading (1)

- Riassumendo: si è aggiunto un livello di indirizzione affinché il corpo del metodo polimorfo accetta (Calcolatore) di Forma fissi il tipo statico dell'argomento della chiamata al metodo sovraccarico calcola(...) di Calcolatore
- Per ottenere una chiamata sovraccarica risolta (a tempo di esec.) sulla base del tipo *dinamico* del suo unico parametro vengono fatte:
 - ✓ prima, una chiamata *polimorfa* risolta *dinamicamente*
 - ✓ poi, una chiamata *sovraccarica* risolta *staticamente*

Conclusioni:

Overriding for Overloading (2)

- Lo scopo di questo esercizio era chiarire i legami (ma anche ribadire le significative differenze) esistenti tra due meccanismi tipici della programmazione orientata agli oggetti
 - ✓ *Overriding vs Overloading*
- Si potrebbe ripetere lo stesso esercizio anche per metodi che ricevono DUE o più parametri polimorfi (ad es. 2+ forme)
 - ✓ Il numero di varianti di metodo in overload si moltiplica ad ogni parametro aggiuntivo per ogni possibile tipo contemplato nella gerarchia
 - ✓ Sino a risolverli tutti tramite una sequenza di chiamate polimorfe, una per ciascun parametro
- Il risultato sarebbe significativamente più lungo e meno leggibile, ma non per questo concettualmente più complicato
- Nella pratica, anche grazie a questi meccanismi alternativi, la mancanza in Java della risoluzione dei metodi sovraccarichi sulla base del tipo dinamico di uno o più parametri non viene percepita come una significativa carenza

Programmazione Orientata agli Oggetti

Generics: Concetti Base

Obiettivi della Lezione

- I generics sono uno strumento per scrivere classi (e metodi) ***parametriche rispetto ad un tipo***
- Ci concentriamo soprattutto su come usare classi generiche
 - Al termine del corso lo studente dovrà essere in grado di usare classi generiche (in particolare quelle del package `java.util` relative alla gestione di collezioni di oggetti)
- La progettazione di classi generiche va oltre gli obiettivi del corso
 - Anche se, da un punto di vista puramente didattico, risulta invece utile introdurre i *Generics* progettando una semplice classe contenitrice di oggetti: **Coppia**

Introduzione

- Notare che le coppie sono una forma molto rudimentale di *collezione*
- In effetti i *Generics* furono introdotti in Java 5 proprio per migliorare il JCF, la libreria dedicata alle collezioni già presente sin da Java 2 (ovvero 1.2)
- Obiettivo: una classe generica rispetto al tipo dei due oggetti ospitati, purché sia lo stesso per entrambi. Ad esempio la classe dovrà gestire:
 - Coppie di stringhe (istanze della classe **String**)
 - Coppie di attrezzi (istanze della classe **Attrezzo**)
 - Coppie di URL (istanze della classe **URL**)
 - ...

La Classe Generica Coppia

- La classe **Coppia** deve offrire dei servizi per gestire una coppia di oggetti del **medesimo** tipo:
 - Metodi per ottenere/cambiare
 - il primo elemento della coppia
 - il secondo elemento della coppia
 - Un costruttore che riceve come parametri due riferimenti ad oggetti del medesimo tipo

Una Possibile Soluzione Basata sul Polimorfismo

- Una possibile soluzione (l'unica possibile prima dell'introduzione dei *Generics* in Java 5) consiste nello sfruttare il polimorfismo, ed in particolare il principio di sostituzione
- Definiamo una classe che gestisce una coppia di oggetti istanza di `Object`
 - per il principio di sostituzione (e per la gerarchia dei tipi Java a singola radice in `Object`) la nostra classe può gestire coppie di oggetti istanza di qualsiasi classe (in quanto sottotipo di `Object`)

La Coppia di Object

```
public class Coppia {  
    private Object primo;  
    private Object secondo;  
  
    public Coppia() {}  
  
    public Coppia(Object primo, Object secondo) {  
        this.primo = primo;  
        this.secondo = secondo;  
    }  
  
    public Object getPrimo() {  
        return this.primo;  
    }  
  
    public Object getSecondo() {  
        return this.secondo;  
    }  
  
    public void setPrimo(Object primo) {  
        this.primo = primo;  
    }  
  
    public void setSecondo(Object secondo) {  
        this.secondo = secondo;  
    }  
}
```

Tipo degli Elementi Ospitati

- Considereremo di seguito del codice che fa riferimento alla semplice classe **Persona**

```
class Persona {  
    private String nome;  
  
    public Persona(String nome) {  
        this.nome = nome;  
    }  
  
    public String getNome() {  
        return this.nome;  
    }  
}
```

Controllo sui Tipi Senza Generics: Scomodi (ed Inutili?) Downcast

- Il seguente codice compila, e funziona correttamente:

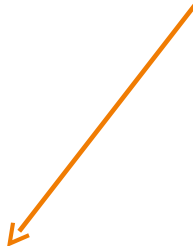
```
public class CoppiaSenzaGenericsTest {  
    @Test  
    public void testCheCompilaEdEsegue() {  
        Coppia coppia = new Coppia();  
        String pippo = new String("Pippo");  
        String pluto = new String("Pluto");  
        Persona p1 = new Persona(pippo);  
        coppia.setPrimo(p1);  
        Persona p2 = new Persona (pluto);  
        coppia.setSecondo(p2);  
        Persona persona = (Persona)coppia.getPrimo();  
        assertSame(pippo, persona.getNome());  
    }  
}
```

Controllo sui Tipi Senza Generics: non Compila Quando Vorremmo...

- Per un utilizzo più semplice delle coppie, vorremmo poter scrivere il seguente codice:

```
public class CoppiaSenzaGenericsTest {  
    @Test  
    public void testCheNonCompila() {  
        Coppia coppia = new Coppia();  
        String pippo = new String("Pippo");  
        String pluto = new String("Pluto");  
        Persona p1 = new Persona(pippo);  
        coppia.setPrimo(p1);  
        Persona p2 = new Persona(pluto);  
        coppia.setSecondo(p2);  
        assertSame(pippo, coppia.getPrimo().getNome());  
    }  
}
```

NON COMPILA!
Il tipo statico
`Object`
non possiede il
metodo `getNome()`



Controllo sui Tipi Senza Generics: non Vorremmo che Compilasse!

- Al contrario, il seguente codice compila correttamente ma l'esecuzione fallisce:

```
public class CoppiaSenzaGenericsTest {  
    @Test  
    public void testCheCompilaMaNonEsegue() {  
        Coppia coppia = new Coppia();  
        String pippo = new String("Pippo");  
        String pluto = new String("Pluto");  
        Persona p1 = new Persona(pippo);  
        Persona p2 = new Persona(pluto);  
        coppia.setPrimo(pippo);  
        coppia.setSecondo(pluto);  
        assertSame(pippo, ( (Persona) coppia.getPrimo() ).getNome() );  
    }  
}
```

ClassCastException a tempo di esecuzione!

Controllo sui Tipi Statici vs Dinamici

- Il problema nasce dal controllo a tempo dinamico operato dal downcast esplicito:
 - nell'oggetto `coppia` è atteso come primo elemento un oggetto di tipo dinamico `Persona`
- Invece vi si trova un riferimento ad un oggetto di tipo dinamico `String`
- Tutto perfettamente lecito per il compilatore che effettua verifiche solo sul tipo statico:
 - l'istruzione `coppia.setPrimo(pippo)` riceve come parametro attuale la variabile locale `pippo`, di tipo statico `String`, sottotipo del tipo `Object` atteso

Tipizzazione Lasca

- A ben vedere sono tutte conseguenze di una tipizzazione *lasca*
 - ✓ **obiettivo:** *coppie di oggetti dello stesso tipo*
- A differenza di come originariamente desiderato, per ovviare alla scarsa espressività del sistema dei tipi Java, siamo finiti per progettare una classe che può ospitare un coppia di oggetti qualsiasi
 - ✓ **risultato:** *coppie di oggetti non necessariamente dello stesso tipo!*
- **Solo un'approssimazione del tipo desiderato**
 - pratica frequentemente utilizzata precedentemente all'introduzione dei *Java Generics* nella piattaforma Java

Conseguenze della Tipizzazione Lasca

- Un controllo lasco dei tipi a tempo di compilazione comporta almeno le seguenti conseguenze:
 - Costringe ad inserire downcast ogni volta che accediamo ad un elemento della coppia
 - ✓ Prima di Java 5 molti sviluppatori consideravano i downcasting quantomeno *ineleganti*, ma pochi lo consideravano un sostanziale limite, nella pratica, del linguaggio
 - Rimanda a tempo di esecuzione alcuni errori che risulterebbero facilmente rilevabili già a tempo di compilazione con una migliore analisi dei tipi statici
 - ✓ Questo è il vero problema! riconsiderare il costo dei bug rispetto al costo degli errori di compilazione>>
- In fase di definizione della coppia, si vorrebbe poter specificare un unico tipo per *entrambi* i riferimenti ad oggetti che la coppia è destinata a memorizzare

Tipi e Metodi Generici

- I generics sono uno strumento per scrivere classi, ed interfacce, il cui tipo diventa *parametrico rispetto ad uno o più tipi*
- Si applica anche ai metodi per renderne parametrica la *segnatura*
- Nella definizione di un tipo generico, il codice viene scritto in maniera parametrica rispetto ad un *tipo formale*
- Nell'uso di un tipo generico, il tipo *formale* deve essere istanziato con un tipo *attuale* per renderlo effettivamente utilizzabile e completamente definito

La Classe Generica Coppia<T> (1)

- La definizione di una classe generica prevede la dichiarazione del parametro formale di tipo racchiuso tra parentesi acute

```
public class Coppia<T> {  
  
    ...  
  
}
```

- In questo modo si specifica che all'interno della definizione della classe `Coppia`, il simbolo `T` indica il tipo sulla base del quale la definizione di classe è parametrica
- Convenzione sul nome del parametro formale di tipo
 - Singola lettera maiuscola: `T`, `E`, `V` ...

La Classe Generica Coppia<T> (2)

- Nella definizione di campi e metodi all'interno della classe **T** viene usato come una dichiarazione di tipo:

```
public class Coppia<T> {  
    private T primo;  
    private T secondo;  
  
    public Coppia(T primo, T secondo) {  
        this.primo = primo;  
        this.secondo = secondo;  
    }  
  
    public T getPrimo() {  
        return this.primo;  
    }  
  
    public T getSecondo() {  
        return this.secondo;  
    }  
    ...  
}
```

Oggetti Coppia<T> *Mutabili*

```
public class Coppia<T> {  
    private T primo;  
    private T secondo;  
  
    public Coppia(T primo, T secondo) {  
        this.primo = primo;  
        this.secondo = secondo;  
    }  
  
    public T getPrimo() {  
        return this.primo;  
    }  
  
    public T getSecondo() {  
        return this.secondo;  
    }  
  
    public void setPrimo(T primo) {  
        this.primo = primo;  
    }  
  
    public void setSecondo(T secondo) {  
        this.secondo = secondo;  
    }  
}
```


Usare una Classe Generica (T=Persona)

- Quando usiamo una classe generica, dobbiamo *istanziarne completamente il tipo* fornendo il tipo *attuale* di tutti i tipi *formali* di cui fa uso
- Ad esempio, usiamo la nostra classe generica **Coppia<T>**, per gestire coppie di oggetti **Persona**

```
public class CoppiaTest {  
    @Test  
    public void testCoppiaDiPersone() {  
        Coppia<Persona> coppia;  
        Persona p1 = new Persona("Stanlio");  
        Persona p2 = new Persona("Olio");  
        coppia = new Coppia<Persona>(p1, p2);  
        assertSame(p1, coppia.getPrimo());  
        assertSame(p2, coppia.getSecondo());  
    }  
}
```

Usare una Classe Generica (T=Color)

- Vediamo la classe parametrica `Coppia<T>` istanziata su un altro tipo qualsiasi (ad es. `java.awt.Color`)

```
import java.awt.Color;
```

```
...
```

```
public class CoppiaTest {
```

```
    @Test
```

```
    public void testCoppiaDiColori() {
```

```
        Coppia<Color> colori;
```

```
        Color rosso = new Color(255,0,0);
```

```
        Color blue = new Color(0,0,255);
```

```
        colori = new Coppia<Color>(rosso, blue);
```

```
        assertEquals(rosso, colori.getPrimo());
```

```
        assertEquals(blue, colori.getSecondo());
```

```
    }
```

```
}
```

Controllo sui Tipi, con Generics

- Riconsideriamo il codice di prima, quello che avremmo voluto *non* compilasse affatto:

```
public class CoppiaTest {  
    @Test  
    public void testCheSmetteDiCompilare() {  
        Coppia<Persona> coppia = new Coppia<Persona>();  
        String pippo = new String("Pippo");  
        String pluto = new String("Pluto");  
        Persona p1 = new Persona(pippo);  
        Persona p2 = new Persona(pluto);  
        coppia.setPrimo(pippo);  
        coppia.setSecondo(pluto);  
        assertSame(pippo,  
                    ((Persona) coppia.getPrimo()).getNome());  
    }  
}
```

NON COMPILA!

Tipo Formale - Tipo Attuale

- Non è difficile trovare una similitudine tra
 - il concetto di parametro formale/attuale inerente l'invocazione dei metodi
 - il concetto di tipo formale/attuale inerente la tipizzazione di classi generiche
- Solo superficialmente sono concetti simili: tra le tante differenze, non dimenticare mai la prima e più importante:
 - il legame tra parametri formali/attuali è operato dalla JVM a tempo di esecuzione
 - il legame tra tipi formali/attuali è operato dal compilatore a tempo di compilazione *solo* sulla base dei tipi *statici*

Generics a più Parametri

- È possibile definire classi, interfacce e metodi generici con più parametri di tipo
 - Sintatticamente, si separano i vari parametri con una virgola

```
public class Esempio<T, S> {...}
```

Generics e Tipi Primitivi

- Java è un linguaggio ibrido ci sono informazioni che si rappresentano senza utilizzare oggetti, ad es. `int`
- Non è possibile istanziare i tipi di una classe, di una interfaccia o di un metodo generico con tipi primitivi
- Per ovviare al problema è possibile rappresentare i tipo primitivi mediante le cosiddette *classi wrapper*

```
public class TestCoppia {  
    @Test  
    public void testCheNonCompila() {  
        Coppia<int> coppia;           // ERRORE: NON COMPILA!  
        int i1 = 100;  
        int i2 = 200;  
        coppia = new Coppia<int>(i1, i2); // ERRORE  
    }  
}
```

Classi *Wrapper* (1)

- Per ogni tipo primitivo esiste una corrispondente classe *wrapper* che consente di «oggettificare» il dato primitivo, costruendoci un oggetto «attorno», per ospitarlo:

- `int` → `Integer`
- `double` → `Double`
- `float` → `Float`
- `char` → `Character`
- `boolean` → `Boolean`

Classi *Wrapper* (2)

```
int i;  
i = 18;  
Integer iwrap = new Integer(i);
```

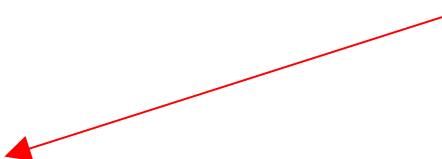
...

```
int value = iwrap.intValue();
```

...

"scarto" il
valore

il valore della variabile
`int` è "avvolto" in un
oggetto `Integer`



Classi *Wrapper* (3)

- Le classi wrapper sono definite nel package `java.lang`
 - quindi non è necessario importarle esplicitamente
 - per approfondimenti sui dettagli dei loro metodi è sufficiente vedere la documentazione
- Metodi più frequentemente usati:
 - metodi `xxxValue()`
 - metodi `valueOf()` e `parseXxx()`
 - metodo `equals()`

Generics e Tipi Primitivi

- Esempio:

```
public class TestCoppia {  
    @Test  
    public void testCheCompila() {  
        Coppia<Integer> coppia;           // OK  
        Integer i1 = new Integer(100);  
        Integer i2 = new Integer(200);  
        coppia = new Coppia<Integer>(i1, i2); // OK  
    }  
}
```

✓ Decisamente prolisso

Boxing/Unboxing

- Per ovviare alla eccessiva verbosità, dalla versione 5 di Java (non a caso la stessa dell'introduzione dei *generics*), la gestione di oggetti wrapper è semplificata dalle funzionalità di *boxing* e *unboxing*
- In sostanza una tecnica di conversione automatica di tipi primitivi nei corrispondenti oggetti di un tipo *wrapper* e viceversa
 - per certi aspetti simile alla promozione di tipi che già si opera ad es. da `int` a `float`
 - per altri ancora, differente: sono coinvolti sia tipi che non sono oggetti, sia tipi che invece lo sono

Boxing

- *Boxing*: è possibile assegnare direttamente tipi primitivi a oggetti wrapper
- Le seguenti istruzioni sono equivalenti:

```
int i = 0;  
Integer iWrap;  
iWrap = i;  
iWrap = 5;
```

```
int i = 0;  
Integer iWrap;  
iWrap = new Integer(i);  
iWrap = new Integer(5);
```

- È il compilatore che inserisce automaticamente le istruzioni per gestire boxing/unboxing!

Unboxing

- *Unboxing*: è possibile assegnare direttamente oggetti wrapper a tipi primitivi
- Le seguenti istruzioni sono equivalenti:

```
int i = 0;  
Integer iWrap;  
iWrap = 5;  
i = iWrap;
```

```
int i = 0;  
Integer iWrap;  
iWrap = new Integer(5);  
i = iWrap.intValue();
```

- È il compilatore che inserisce automaticamente le istruzioni per gestire boxing/unboxing!

Boxing, Unboxing e Collezioni

- Grazie a boxing e unboxing, anche la gestione di collezioni che memorizzano informazioni riconducibili a tipi primitivi risulta semplificata
- Le seguenti operazioni sono lecite (sempre grazie a boxing e unboxing):

```
Coppia<Integer> c;  
c = new Coppia<Integer>();  
int i = 4;
```

```
c.setPrimo(i);  
c.setSecondo(5);
```

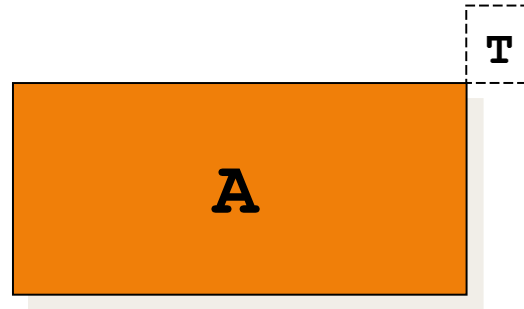
```
c.setPrimo(new Integer(i));  
c.setSecondo(new Integer(5));
```

Attenzione allo Zucchero (Sintattico)

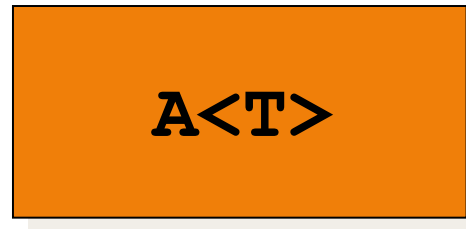
- Le versioni più recenti del linguaggio hanno semplificato la gestione dei tipi primitivi nascondendo alcune conversioni di tipo
- Tuttavia, è necessario comprendere a fondo
 - la differenza tra il concetto di tipo primitivo e la loro controparte ad oggetti, i wrapper
 - quali operazioni non sono necessarie solo grazie ai servizi offerti dalle ultime versioni del compilatore Java (sarebbero necessarie con versioni precedenti)
 - quali operazioni il compilatore inserisce per conto nostro
- Perché conviene avere queste competenze?
 - per stimare meglio il numero di oggetti creati dalle nostre applicazioni
 - per migliorare la nostra capacità di ricerca delle origine degli errori sia a tempo di compilazione che di esecuzione
 - per riuscire ad usare versioni precedenti del compilatore

Generics: Rappresentazione Diagrammatica

- Rappresentazione diagrammatica di una classe generica



- oppure



Generics: Wildcard (1)

- A seguire approfondiamo alcuni concetti più avanzati relativi ai generics per mezzo di esempi
- N.B. In questo corso ci concentriamo sull'uso di classi generiche e non sulla loro progettazione
 - Ma l'utilizzo di alcune librerie pressuppone la capacità di comprendere alcun tipi e signature di metodi generici
- ✓ sviluppare la propria capacità di astrazione è comunque sempre importante nella programmazione

Generics: Wildcard (2)

- Aggiungiamo alla classe `Coppia<T>` il metodo `copyAll()`
 - copia nella coppia corrente gli elementi di un'altra coppia che viene passata come parametro
- La coppia che passiamo come parametro per «fornire» gli elementi da copiare deve essere istanziata su un qualunque sottotipo degli oggetti della coppia corrente che finirà per ospitarli
- Questa particolarità si esprime con il carattere jolly ? e (di nuovo) con la parola chiave `extends`

Generics: Upper-Bounded Wildcard (1)

- Vediamone la segnatura del metodo di istanza contenuto della classe generica `Coppia<T>`:

```
public class Coppia<T> {
```

```
...
```

```
    public void copyAll(Coppia<? extends T> c)
```

```
...
```

```
}
```

Coppia T ^{copia} $\rightarrow$ *sotto T*

$\hookrightarrow$ T o sottotipo di T



- ✓ Cosa significa questa segnatura?
- ✓ Di che tipo deve essere il parametro?

Generics: Upper-Bounded Wildcard (2)

Coppia<? extends T> c

- Significa: un riferimento ad un oggetto (qui usato come parametro di un metodo) del tipo generico **Coppia** istanziato sullo stesso tipo **T**, o su un suo sottotipo, su cui è istanziata la coppia/oggetto corrente **Coppia<T>**
- Esempio di utilizzo:

```
Coppia<Strumento> strumenti;  
Coppia<Chitarra> chitarre;
```

↓ SOTTOTIPO

TIPO ATTUALE

UNA COPPIA DI STRUMENTI NON È
SUPERTIPO DI UNA COPPIA DI CHITARRA

...
↓ QUA CI COS'EREBBE PERFORAZZA STRUMENTI

```
strumenti.copyAll(chitarre); // OK! T = ???
```

- Definizione del metodo **copyAll()** (nella classe **Coppia<T>**):

```
public void copyAll(Coppia<? extends T> coppia) {  
    this.setPrimo(coppia.getPrimo());  
    this.setSecondo(coppia.getSecondo());  
}
```

↑ O UN TIPO O UN SUO SOTTOTIPO

Metodi Statici Generici

- È possibile definire anche metodi statici generici (cioè parametrici rispetto ad un tipo formale)
- Un metodo generico definisce i tipi formali nella segnatura del metodo, subito prima del tipo restituito. Ad es.:

```
public static <T> int metodoGenerico(  
                                Coppia<T> c,  
                                T e  
                                )
```

- ✓ Anche in questo caso, come già per le classi generiche, è possibile invocare il metodo solo dopo aver fornito (a tempo statico, in fase di compilazione) tutti i tipi attuali necessari a completare definitivamente la segnatura

Metodi Generici: Wildcard

- Definiamo ora la classe **Coppia**^s, classe contenitrice per ospitare metodi generici e di utilità generale per manipolare oggetti di tipo **Coppia**
- In particolare la classe **Coppia** offre alcuni metodi (statici), di cui stiamo per definire le signature, perseguendo la loro generalità rispetto al tipo degli oggetti ospitati nelle coppie:
 - ^{Coppia<T>} `reverse(Coppia coppia)`
prende come parametro una coppia e ne inverte gli elementi (il primo elemento diventa il secondo e viceversa)
 - ^{Void} `fill(Coppia coppia, T e)`
prende due parametri: una coppia ed un elemento; riempie entrambi gli elementi della coppia con l'elemento ricevuto

Metodi Generici: Esempio

```
public class Coppias {  
    public static <T> void reverse(Coppia<T> c) {  
        T tmp;  
  
        tmp = c.getPrimo();  
        c.setPrimo(c.getSecondo());  
        c.setSecondo(tmp);  
    }  
    ...  
}
```

Generics: Lower-Bounded Wildcard (1)

- Il metodo `fill(??? coppia , ??? e)`
 - imposta entrambi gli elementi della coppia che viene passata come primo parametro, con lo stesso riferimento ad oggetto nel secondo parametro
- E' un metodo parametrico: il tipo del secondo parametro deve essere un qualunque sottotipo del tipo istanziato dalla coppia
 - ✓ ovvero: il tipo su cui la coppia è istanziata deve essere un supertipo del tipo del parametro
- Si esprime così:

COPPIA È DI TIPO
T O SUPERTIPO DI T

```
static <T> void fill(Coppia<? super T> coppia,  
                    T elemento)
```


Generics: Lower-Bounded Wildcard (2)

`Coppia<? super T>`

- ✓ Significa: un riferimento ad un oggetto `Coppia` istanziato su `T` o su un qualsiasi supertipo di `T`

- Esempio di utilizzo:

```
Coppia<Strumento> strumenti;
```

Generics: Bounded Wildcard

```
static <T> void copy(Coppia<? super T> dest,  
                    Coppia<? extends T> src)
```

✓ Attenzione a non confonderlo con `copyAll()`

• Esempio di utilizzo:

```
Coppia<Strumento> strumenti = new Coppia<Strumento>();
```

```
Coppia<Chitarra> chitarre = new Coppia<Chitarra>();
```

...

```
Coppias.copy(strumenti, chitarre); // OK T = ???
```

• Definizione:

```
static <T> void copy(Coppia<? super T> dest,  
                    Coppia<? extends T> src) {  
    dest.setPrimo(src.getPrimo());  
    dest.setSecondo(src.getSecondo());  
}
```

Annotations:
- **CONSUMA** (arrow pointing to `dest`)
- **FORNISCE** (arrow pointing to `src`)
- **↳ SORGENE** (arrow pointing to `src`)

La Regola Mnemonica *PECS*

- Semplice regola per ricordare il tipo dei parametri formali nelle signature di collezioni

Producer Extends Consumer Super

- Utilizzare `<? extends T>` per i parametri di collezioni che “producono” elementi
 - ad es. il parametro del metodo `copyAll()`
- Utilizzare `<? super T>` per i parametri di collezioni che “consumano” elementi
 - ad es. il parametro del metodo `fill()`
- Esempio di utilizzo contestuali di entrambi: i due parametri del metodo statico `copy()`

Personaggi ed Interpreti

- Ogni riferimento ad API esistenti o a metodi realmente esistenti *NON* è affatto puramente casuale
 - Coppia nel ruolo di Collection, List...
 - Coppias nel ruolo di Collections (>>)
- Dentro java.util.Collections si trovano:

```
static <T> void fill(List<? super T> list, T obj)
static <T> boolean addAll(Collection<? super T> c,
                        T... elements)
```

```
static <T> void copy(List<? super T> dest,
                    List<? extends T> src)
```

```
static void reverse(List<?> list)
```



Preferibile quando non è
strettamente necessario dare
un nome al tipo formale

Esercizi

- Scrivere il codice della classe generica **Coppia<T>**
- Scrivere il codice della classe **Coppias**
- Scrivere, utilizzando JUnit, classi di test per le classi **Coppia<T>** e **Coppias**
- Cercare di capire sino al dettaglio quante più possibili signature generiche dei metodi statici fornite nella classe **java.util.Collections**

Esercizi

- Supponendo che **Studiante** sia sottotipo di **Persona**, è vero che **Coppia<Studiante>** risulta essere sottotipo di **Coppia<Persona>**?
- Ripetere l'esercizio di sopra, nel caso di coppie *immutabili*, ovvero prive dei metodi **setXXX()**
- Il tipo di **Coppia<T>** è *covariante* o *controvariante* rispetto al tipo **T**? *È COVARIANTE SOLTANTO SE È IMMUTABILE*
 - Nel caso di coppie *mutabili* ?
 - Nel caso di coppie *immutabili* ?

Riferimenti ed Approfondimenti

- Alcuni articoli spiegano *molti* altri dettagli:

<http://www.oracle.com/technetwork/java/javase/generics-tutorial-159168.pdf>

- Per sapere (quasi) tutto sui java generics:

<http://www.angelikalanger.com/GenericsFAQ/JavaGenericsFAQ.html>

Programmazione Orientata agli Oggetti

Collezioni
Liste

Sommario

- Introduzione alle Collezioni
 - Interface **Collection<E>**
 - Iterare una collezione: **Iterator<E>**
 - Rimuovere elementi da una collezione
- Liste
 - aggiungere elementi
 - iterare sugli elementi della lista
- Ordinamento di liste
 - **Comparable, Comparator**

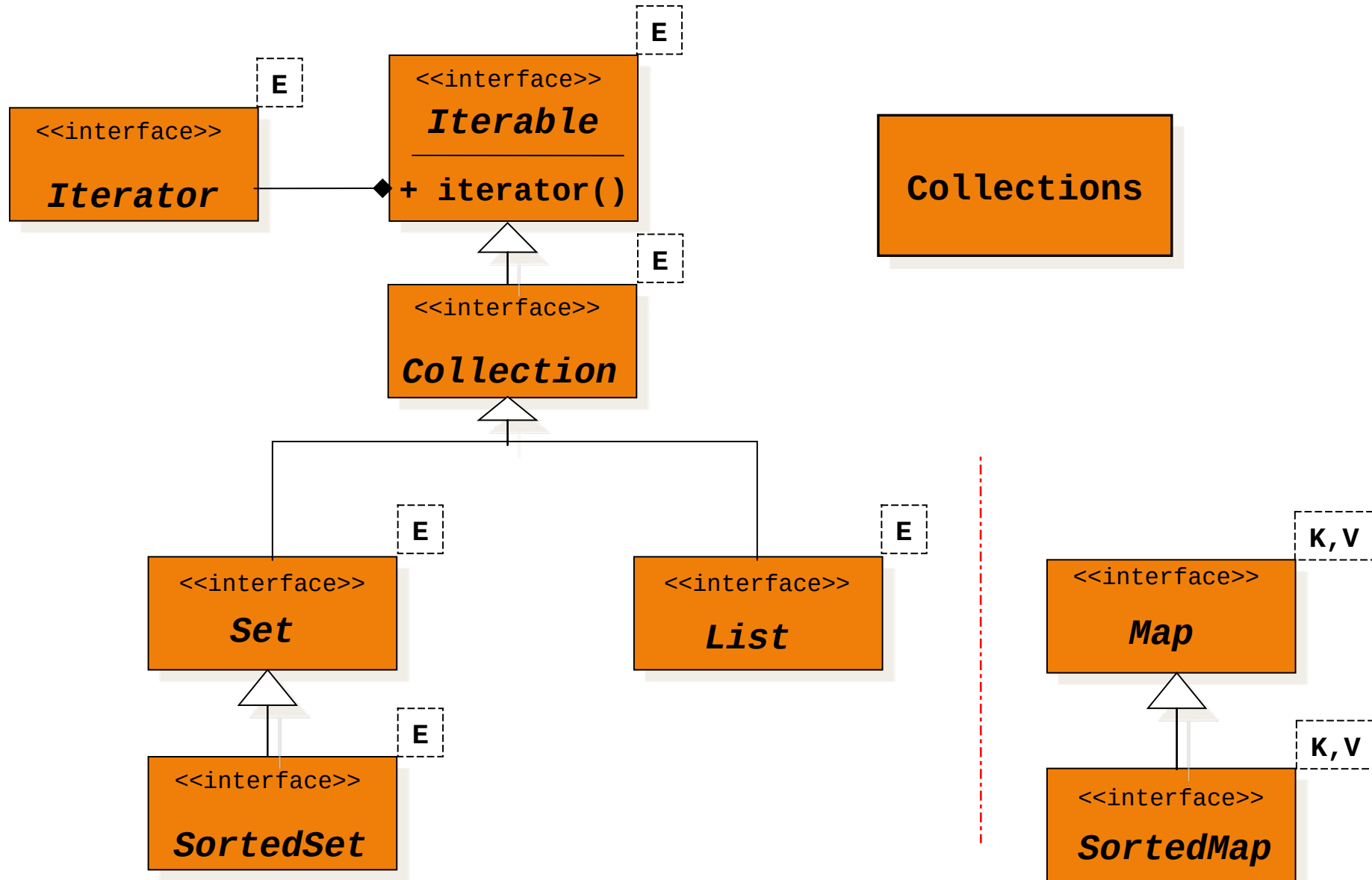
Introduzione

- Molte applicazioni richiedono di gestire collezioni di oggetti
- **Gli array sono uno strumento di basso livello**
 - La dimensione di una collezione in genere non è nota a priori e può variare notevolmente
 - Negli esercizi fatti finora abbiamo ipotizzato un numero massimo di elementi proprio per facilitarne l'utilizzo
 - un indicatore di riempimento serve a ricordarsi il numero di elementi già memorizzati nell'array
 - Possiamo avere bisogno di molte modalità di accesso
 - (non solo indicizzato; ad es. LIFO, FIFO, ecc. ecc.)
 - Ci può essere la necessità di mantenere gli elementi ordinati

Le Collezioni del Package `java.util`

- Nella libreria di base di Java abbiamo un package che ci offre un vasto insieme di interfacce e di classi per la gestione di collezioni di oggetti
- Introdotte già in Java 2:
 - ma sostanzialmente rivisitate in seguito alla introduzione dei Generics in Java 5
 - ✓ N.B. I Generics di Java 5 sono stati introdotti principalmente per il loro utilizzo nelle collezioni preesistenti
 - significativamente estese nelle versioni successive
 - ✓ per coprire scenari di utilizzo via via conclamatesi come importanti
 - ad es. `java.util.concurrent`: collezioni concorrenti
- ✓ Certamente tra le librerie più utilizzate in assoluto

Il Java Collection Framework (JCF)



Un Primo Sguardo: Collection<E> (1)

- L'interface **Collection<E>** dichiara i metodi di una generica collezione:
- Generalizza sia **List<E>** sia **Set<E>**

List<E>:

- Collezioni sequenziali i cui elementi possiedono una posizione
- Senza gestione dei duplicati

Set<E>:

- Collezioni che non ammettono duplicati
- Gli elementi non possiedono posizione

Un Primo Sguardo: Collection<E> (2)

- L'interface **Collection<E>** dichiara i metodi di una generica collezione
- Questi metodi permettono di svolgere operazioni quali:
 - aggiungere un elemento alla collezione
 - verificare la dimensione della collezione
 - verificare se la collezione è vuota
 - aggiungere tutti gli elementi di un'altra collezione
 - ottenere un *iteratore* con cui scandire la collezione

Un Primo Sguardo: Set<E>

- L'interface **Set<E>** estende **Collection<E>**: è una collezione che non può contenere *duplicati*
- Offre tutti e soli i metodi della interface **Collection**, con la restrizione che le classi che la implementano si impegnano a non ammettere la presenza di elementi *duplicati*
 - sarà necessario utilizzare un meccanismo di modellazione del criterio di equivalenza tra elementi dell'insieme
 - bisognerà utilizzarlo per definire un criterio di equivalenza tra gli elementi ospitati nell'insieme

Un Primo Sguardo: List<E>

- L'interface **List<E>** estende **Collection<E>** e corrisponde ad una sequenza, ovvero una collezione ordinata di elementi
- Le liste, rispetto agli insiemi, possono contenere elementi duplicati
- Oltre alle operazioni offerte dal supertipo **Collection<E>**, l'interface **List<E>** include altre operazioni specifiche, quali:
 - *Accesso posizionale*: permette di accedere agli elementi di tipo **E** in base alla loro posizione nella lista (in maniera simile a quanto avviene per gli array)
 - *Ricerca*: permette di ricercare la posizione di un elemento nella lista

Un Primo Sguardo: Map<K, V>

- L'interface **Map<K, V>** offre le operazioni di una mappa, o dizionario: una mappa è una collezione di coppie **chiave-valore**
- L'interface **Map<K, V>** dichiara i metodi per operazioni quali:
 - *Accesso per chiave*: ottenere il valore associato ad una chiave
 - Cancellare una coppia in cui compare una chiave
 - Inserire una nuova coppia nella mappa
 - Ottenere una collezione contenente tutte le chiavi o tutti i valori

Un Primo Sguardo: Implementazioni

- **List<E>**
 - **ArrayList<E>**
 - **LinkedList<E>**
- **Set<E>**
 - **HashSet<E>**
 - **TreeSet<E>**
- **Map<K, V>**
 - **HashMap<K, V>**
 - **TreeMap<K, V>**

... le più diffusamente utilizzate, ma ne esistono molte altre di uso più specifico

Un Primo Sguardo: Collections

- La classe `java.util.Collections` (al plurale: attenzione alla 's' finale!)
 - offre un vasto insieme di metodi (statici) generici che implementano utili e diffusi algoritmi per la manipolazione di liste quali:
 - ordinamento
 - ricerca max e min
 - shuffle (mescolamento casuale)
 - reverse
 - fill...
- ✓ A meno di forti (anzi fortissime) motivazioni in senso contrario, implementare funzionalità equivalenti a questi (o ad uno degli altri) metodi statici offerti da `Collections` è solo una perdita di tempo

Sommario

- Introduzione alle Collezioni
 - **Interface Collection<E>**
 - Iterare una collezione: **Iterator<E>**
 - Rimuovere elementi da una collezione
- Liste
 - aggiungere elementi
 - iterare sugli elementi della lista
- Ordinamento di liste
 - **Comparable, Comparator**

Interface `Collection<E>`

- L'interface `Collection<E>` dichiara i metodi di una collezione generica
- Questi metodi permettono di svolgere operazioni di tre categorie:
 - Manipolazione di base
 - Bulk
 - Conversione da e verso array

Interface Collection<E>

```
public interface Collection<E> extends Iterable<E> {  
    //Basic operations  
    int size();  
    boolean isEmpty();  
    boolean contains(Object element);  
    boolean add(E element); //optional  
    boolean remove(Object element); //optional  
    Iterator<E> iterator();  
    //Bulk operations  
    boolean containsAll(Collection<?> c);  
    boolean addAll(Collection<? extends E> c); //optional  
    boolean removeAll(Collection<?> c); //optional  
    boolean retainAll(Collection<?> c); //optional  
    void clear(); //optional  
    //Array operations  
    Object[] toArray();  
    <T> T[] toArray(T[] a);  
}
```



A dashed box containing a red question mark is positioned to the right of the `contains` method signature. Dotted lines with arrows point from this box to the `contains` method signature and the `containsAll` method signature.

Collection<E>: Metodi Base

Consultare i Javadoc! In sintesi:

- **boolean isEmpty();**
ritorna **true** se la collezione è vuota
- **int size();**
ritorna il numero di elementi presenti nella collezione
- **boolean contains(Object element);**
ritorna **true** se la collezione contiene un elemento uguale a quello passato come parametro (l'uguaglianza è verificata dal metodo **equals()**)
- **boolean add(E element);**
aggiunge alla collezione l'elemento passato; ritorna **true** se la collezione è cambiata dopo la chiamata a questo metodo
- **boolean remove(Object element);**
rimuove dalla collezione gli elementi uguali all'oggetto passato come parametro (l'uguaglianza è verificata dal metodo **equals()**). Ritorna **true** se la collezione è cambiata dopo l'invocazione del metodo
- **Iterator<E> iterator();**
restituisce un oggetto **Iterator**, per iterare sugli elementi della collezione

Collection<E>: Metodi Bulk

Consultare i Javadoc! In sintesi:

- **boolean containsAll(Collection<?> c);**
ritorna **true** se la collezione contiene tutti gli elementi della collezione passata come parametro
- **boolean addAll(Collection<? extends E> c);**
aggiunge alla collezione tutti gli elementi della collezione passata come parametro; ritorna **true** se la collezione è cambiata dopo l'invocazione di questo metodo
- **boolean removeAll(Collection<?> c);**
rimuove dalla collezione tutti gli elementi uguali (l'uguaglianza è verificata dal metodo **equals()**) che sono contenuti nella collezione passata come parametro; ritorna **true** se la collezione è cambiata dopo l'invocazione di questo metodo
- **boolean retainAll(Collection<?> c);**
rimuove dalla collezione tutti gli elementi che non sono presenti nella collezione passata come parametro; ritorna **true** se la collezione è cambiata dopo l'invocazione di questo metodo
- **void clear();**
rimuove tutti gli elementi dalla collezione

Sommario

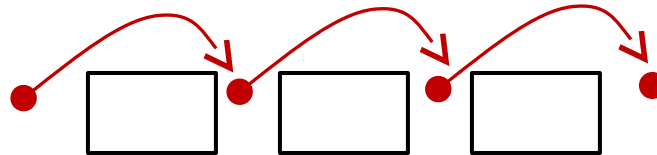
- Introduzione alle Collezioni
 - Interface **Collection<E>**
 - Iterare una collezione: **Iterator<E>**
 - Rimuovere elementi da una collezione
- Liste
 - aggiungere elementi
 - iterare sugli elementi della lista
- Ordinamento di liste
 - **Comparable, Comparator**

Iterazione: Interface `Iterator<E>`

- L'iterazione di una collezione avviene attraverso un oggetto *iteratore* dedicato allo scopo
- Gli iteratori sono creati invocando il factory method **`iterator()`** sulla collezione che si vuole scandire
- L'oggetto ottenuto implementa l'interface **`Iterator<E>`**, munita dei metodi
 - **`boolean hasNext()`**
 - **`E next()`**
 - **`void remove()`**
- ✓ Solo i primi due metodi sono considerati strettamente caratterizzanti gli iteratori

Semantica degli Iteratori

- Nella sostanza, un *cursore* che scandisce la collezione sottostante ricordando la sua posizione nella scansione
- Posizioni lecite: subito *prima* o subito *dopo* un elemento della collezione che si sta iterando
 - ✓ N.B. mai “sopra” un elemento



Collezioni di Tante Tipologie, un Unico Modo per Enumerarle

- ✓ L'iterazione si effettua sempre nello stesso identico modo indipendentemente dal tipo di collezione sottostante che lo ha *generato*
- Con notevole “economia di pensiero”
- Per questo motivo è possibile discuterli ancora prima delle implementazioni che sanno generarli, con riferimento all'interface **Collection<E>**

Iterator<E>: Metodi

- **boolean hasNext();**
ritorna **true** se e solo se esiste un altro elemento da scandire
- **E next();**
restituisce il prossimo elemento della collezione nella scansione corrente ed avanza

Iterator<E>: Iterazione

- La chiamata ripetuta di **next()** permette di scorrere gli elementi della collezione **uno alla volta**
- Se si raggiunge la fine della collezione viene sollevata una eccezione (che interrompe il programma)
java.util.NoSuchElementException
- Per evitare questa situazione, prima di chiamare **next()** si usa il metodo **hasNext()**, che ritorna **true** se e solo se esiste un altro elemento su cui iterare

Esercizio: la Semantica di Iterator

- Per comprendere la semantica dei metodi di una classe non esiste metodo migliore di una batteria di test-case che la documenti precisamente
- Per **Iterator<E>** scriviamo test per i due metodi principali dell'interfaccia **Iterator<E>**
- Utilizziamo come implementazione concreta della collezione **ArrayList<E>** (**>>**)

Unit-Testing per Documentare la Semantica di Iterator<E>

```
import ...  
public class IteratorTest {  
  
    private List<String> vuota;  
    private List<String> singoletto;  
    private String solitario;  
  
    @Before  
    public void setUp() {  
        this.vuota = new ArrayList<>();  
        this.singoletto = new ArrayList<String>();  
        this.solitario = new String("solitario");  
        this.singoletto.add(this.solitario);  
    }  
  
    @Test ... ..  
}
```

// da Java 7
ArrayList<>();

Diamond Operator

Test di `Iterator.hasNext()`

`@Test`

```
public void testHasNext_noListaVuota() {  
    Iterator<String> it = this.vuota.iterator();  
    assertNotNull(it);  
    assertFalse(it.hasNext());  
}
```

`@Test`

```
public void testHasNext_primaSiPoiNoSuSingoletto() {  
    Iterator<String> it = this.singoletto.iterator();  
    assertNotNull(it);  
    assertTrue(it.hasNext());  
    it.next();  
    assertFalse(it.hasNext());  
}
```

Test di `Iterator.next()`

`@Test`

```
public void testNext_singoletto() {  
    Iterator<String> it =  
        this.singoletto.iterator();  
    assertNotNull(it);  
    assertTrue(it.hasNext());  
    assertSame(this.solitario, it.next());  
}
```

Test di Iterator.next()

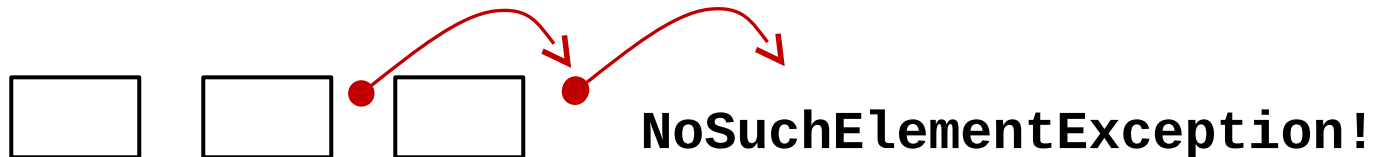
@Test

```
public void testNext_suListaDiDueElementi() {  
    List<String> doppietta = new ArrayList<>();  
  
    doppietta.add(new String("primo"));  
    doppietta.add(new String("secondo"));  
    Iterator<String> it = doppietta.iterator();  
    assertNotNull(it);  
    assertTrue(it.hasNext());  
    assertEquals("primo", it.next());  
    assertTrue(it.hasNext());  
    assertEquals("secondo", it.next());  
    assertFalse(it.hasNext());  
}
```

Test di `Iterator.next()`

```
@Test(expected = NoSuchElementException.class)
public void testNext_oltreLaFineSollevaEccezione() {
    Iterator<String> it = this.vuota.iterator();
    it.next();
}
```

- Posizioni lecite: subito *prima* o subito *dopo* un elemento della collezione che si sta iterando
 - ✓ raggiunto l'ultimo elemento non si può andare oltre: il metodo **`Iterator.next()`** solleva una eccezione (>>)



Iterazione di Array - Ciclo *for*

```
public class Borsa {  
    private Attrezzo[] attrezzi;  
    private int numeroAttrezzi;  
    ...  
    public int getPeso(){  
        int pesoTotale = 0;  
        for(int i=0; i<this.numeroAttrezzi; i++) {  
            Attrezzo a = this.attrezzi[i];  
            pesoTotale += a.getPeso();  
        }  
        return pesoTotale;  
    }  
    ...  
}
```

Iterazione Mediante Iteratori

```
import java.util.List;
import java.util.ArrayList;

public class Borsa {
    private List<Attrezzo> attrezzi;
    ...
    public int getPeso(){
        int pesoTotale = 0;
        Iterator<Attrezzo> iteratore =
            this.attrezzi.iterator();
        while (iteratore.hasNext()) {
            Attrezzo a = iteratore.next();
            pesoTotale += a.getPeso();
        }
        return pesoTotale;
    }
    ...
}
```

Esercizio List<E>: Iterazione

- L'utilizzo degli iteratori è decisamente ripetitivo
- Sempre le stesse identiche operazioni:
 - **Iterator<Attrezzo> iteratore =**
this.attrezzi.iterator();
Abbiamo chiesto alla lista di creare un oggetto per gestire l'iterazione su una collezione di oggetti **Attrezzo...**
 - **while (iteratore.hasNext())**
...fintanto che ci sono ancora elementi da scandire...
 - **Attrezzo a = iteratore.next();**
...attraverso il metodo **next()** otteniamo dall'iteratore il prossimo elemento della scansione
- E' ben motivata una forma sintattica abbreviata...

Iterazione: *for-each* (1)

- Per iterare su **tutti** gli elementi di una collezione è possibile usare la forma "for-each" dell'istruzione **for**

```
for( Tipo elemento : iterable )  
    <<blocco_di_operazioni_su_elemento>>
```
- Dove *iterable* è un qualsiasi sottotipo di **java.lang.Iterable<E>**, una interface che offre il factory method:
 - **Iterator<E> iterator()**
- ✓ La sintassi for-each è conveniente *solo se non* è necessario accedere all'indice di iterazione

Iterazione: *for-each* (2)

- ✓ Si tratta solo di “zucchero sintattico”: il compilatore traduce

```
for( E elemento : iterable )  
    <<blocco_di_codice_su elemento>>  
... in ...  
    for (Iterator<E> iter = iterable.iterator();  
        iter.hasNext(); ) {  
        E elemento = iter.next();  
        <<blocco_di_codice_su elemento>>  
    }
```
- Tutte le collezioni del JCF sono già *Iterable*
 - Si può usare il *for-each* su qualunque classe, anche di nuova definizione, purché implementi ***Iterable<E>***
 - Ed anche sugli array che pure non lo sono affatto (sottotipi di ***Iterable<E>***)
 - ✓ grazie ad una gestione peculiare e specifica da parte del compilatore

Esercizio:

Scansione di Liste con *for-each*

```
import java.util.List;
import java.util.ArrayList;

public class Borsa {
    private List<Attrezzo> attrezzi;
    ...
    public int getPeso(){
        int pesoTotale = 0;
        for(Attrezzo a : this.attrezzi)
            pesoTotale += a.getPeso();
        return pesoTotale;
    }
    ...
}
```

Sommario

- Introduzione alle Collezioni
 - Interface **Collection<E>**
 - Iterare una collezione: **Iterator<E>**
 - **Rimuovere elementi da una collezione**
- Liste
 - aggiungere elementi
 - iterare sugli elementi della lista
- Ordinamento di liste
 - **Comparable, Comparator**

Legame Iterator / Collezione

- Tra un iteratore e la collezione che lo ha creato permane un legame anche successivamente alla sua creazione
 - Interessante anche per i meccanismi di costruzione dei tipi utilizzati nell'occasione (>>)
- L'esistenza di un terzo metodo nell'interface **Iterator<E>** per la rimozione di elementi rende più evidente la natura di questo legame:

void remove();

- rimuove dalla collezione l'ultimo elemento che è stato restituito da una precedente chiamata di **next()**
- ✓ attraverso l'iteratore vengono operate modifiche sulla collezione sottostante

Test di `Iterator.remove()`

@Test

```
public void testRemove() {  
    Iterator<String> it =  
        this.singoletto.iterator();  
    String solitario = it.next();  
    assertFalse(this.singoletto.isEmpty());  
    it.remove();  
    assertTrue(this.singoletto.isEmpty());  
}
```

Il Metodo `remove()` di Iterator

- Il metodo `remove()` rimuove l'elemento restituito dall'ultima chiamata di `next()`
- Non è ammesso chiamare `remove()` a meno che prima non si sia provveduto ad invocare `next()`
- Ad es. per eliminare due elementi consecutivi:

```
it.next();  
it.remove();  
it.remove();    // ERRORE
```

prima bisogna richiamare `next()`:

```
it.next();  
it.remove();  
it.next();  
it.remove();    // OK
```

Rimuovere Elementi da una Collezione

- Pertanto esistono due diversi modi per rimuovere un elemento da una collezione, ciascuno dettato da specifiche esigenze:
 - per la rimozione di un elemento uguale (secondo quanto stabilito dal metodo `boolean equals(Object o)`) ad un elemento dato (passato come parametro) si usa il metodo `remove(Object o)` di `Collection`
 - per la rimozione di un elemento durante la scansione si usa il metodo `remove()` di `Iterator`

Il Metodo `remove(Object o)` di `Collection<E>`

`boolean remove(Object o)`

Removes a single instance of the specified element from this collection, if it is present (optional operation). More formally, removes an element *e* such that $(o == null ? e == null : o.equals(e))$, if this collection contains one or more such elements. Returns true if this collection contained the specified element (or equivalently, if this collection changed as a result of the call).

- **Parameters:**

- *o* - element to be removed from this collection, if present

- **Returns:**

- true if an element was removed as a result of this call

(dalla documentazione)

Il Metodo `remove()` di `Iterator<E>`

- **`void remove()`**
 - Removes from the underlying collection the last element returned by the iterator (optional operation). This method can be called only once per call to `next`. The behavior of an iterator is unspecified if the underlying collection is modified while the iteration is in progress in any way other than by calling this method.

(dalla documentazione)

Come Rimuovere Elementi?

- Attenzione: è un errore cercare di rimuovere elementi da una collezione con il metodo
 - `boolean remove(Object o)`di **Collection** proprio mentre si sta ancora visitando la stessa collezione con un iteratore
 - la collezione verrebbe modificata "sotto i piedi" dell'iteratore che la sta ancora scandendo
- Se si stanno cercando elementi da rimuovere attraverso un iteratore, *deve* poi essere usato il metodo **remove()** dell'iteratore stesso per renderlo «consapevole» dei cambiamenti alla «sua» collezione

java.util.ConcurrentModificationException

```
import java.util.*;

public class ConcurrentModificationMain {
    public static void main(String args[]) {
        List<Object> list = new ArrayList<>();
        Iterator it = list.iterator();
        list.add(new Object());
        it.next(); // Qui solleva ConcurrentModificationException
    }
}
```

- ✓ L'eccezione è sollevata solo alla prima occasione utile (semantica *fail-fast*, vedere Javadoc)

Exception in thread "main"

```
java.util.ConcurrentModificationException
    at java.util.AbstractList$Itr.checkForComodification(AbstractList.java:448)
    at java.util.AbstractList$Itr.next(AbstractList.java:419)
    at ConcurrentModificationMain.main(ConcurrentModificationMain.java:9)
```

Sommario

- Introduzione alle Collezioni
 - Interface **Collection<E>**
 - Iterare una collezione: **Iterator<E>**
 - Rimuovere elementi da una collezione
- **Liste**
 - aggiungere elementi
 - iterare sugli elementi della lista
- Ordinamento di liste
 - **Comparable, Comparator**

Liste: Interface List<E> (1)

- Una lista è una collezione che mantiene gli elementi ordinati secondo l'ordine di inserimento (il primo elemento aggiunto alla lista, è in prima posizione, il secondo in seconda posizione, ..., l'ultimo elemento aggiunto è in ultima posizione)
- Cfr. ASD (<<)

Liste: Interface List<E> (2)

- L'interface **List<E>** estende l'interface **Collection<E>**
- Oltre ai metodi della interface **Collection<E>**, **List<E>** offre specifici metodi che consentono accesso e inserimento indicizzati degli elementi. Ad esempio:
 - **E get(int index):** *Returns the element at the specified position in this list*
 - **int indexOf(Object o):** *Returns the index of the first occurrence of the specified element in this list, or -1 if this list does not contain the element*

Esercizio (per Casa)

- Analizzare, compilare ed eseguire la classe di test **ListTest** riportata subito di seguito.
 - Aggiungere opportuni metodi di test per verificare e comprendere la semantica dei metodi:
 - **indexOf(Object o);** in particolare cerca la stessa istanza in memoria od un oggetto **equals()** ???
 - **contains(Object o);** idem...
 - **retainAll(Collection<?> c)**
 - **containsAll(Collection<?> c)**

Esercizio (per Casa, *continua*)

```
public class ListTest {  
    private Collection<Integer> c;  
    private Collection<Integer> t;  
  
    @Before  
    public void setUp () {  
        c = new LinkedList<Integer>();  
        t = new ArrayList<Integer>();  
        c.add(1);  
        c.add(2);  
        c.add(3);  
        t.add(1);  
        t.add(2);  
    }  
  
    @Test  
    public void testRemoveAll() {  
        assertTrue(c.removeAll(t));  
        Iterator<Integer> it = c.iterator();  
        assertTrue(it.hasNext());  
        assertEquals(3, it.next().intValue());  
        assertFalse(it.hasNext());  
    }  
}
```


Implementazioni di List<E>

- Il package `java.util` offre due diverse implementazioni di `List<E>`
 - `ArrayList<E>`
 - `LinkedList<E>`
- Forniamo qualche (grossolana) indicazione su come scegliere l'implementazione più opportuna
- ✓ **NOTA:** Questi aspetti sono stati già approfonditi nel corso "*Algoritmi e Strutture Dati*"

ArrayList<E>: Implementazione

- Gli elementi sono memorizzati in un contenitore rappresentato con array ed indicatore di riempimento
 - Al momento della creazione, la dimensione dell'array (ovvero la capacità della collezione) è inizializzata ad un valore prestabilito
 - Quando il numero di elementi è prossimo alla capacità dell'array, viene istanziato un nuovo array di dimensione maggiore (ad esempio doppia) nel quale vengono travasati tutti gli elementi dell'array originario
- ✓ NOTA: Questi aspetti sono stati approfonditi nel corso "*Algoritmi e Strutture Dati*"

LinkedList<E>: Implementazione

- Gli elementi sono memorizzati in una lista concatenata
 - Ogni elemento della lista contiene
 - un riferimento all'elemento successivo
 - un riferimento all'oggetto memorizzato
 - Non è necessario stabilire una capacità iniziale
- ✓ NOTA: Questi aspetti sono stati approfonditi nel corso "*Algoritmi e Strutture Dati*"

LinkedList<E> o ArrayList<E>?

- Molto schematicamente:
 - **ArrayList<E>** conviene se:
 - la dimensione è abbastanza stabile
 - è necessario un accesso indicizzato (la classe **ArrayList** offre un metodo opportuno)
 - **LinkedList<E>** conviene se:
 - la dimensione può variare anche significativamente
 - gli accessi sono perlopiù sequenziali
- Nella pratica la complessità delle JVM moderne rende le differenze spesso impercettibili e/o comunque molto difficilmente prevedibili
 - ✓ Basare le ottimizzazioni *sempre* su misurazioni sperimentali che ne palesino ed accertino la reale necessità

Implementazioni di List<E>: Costruttori

- I costruttori sono sovraccarichi. In particolare facciamo osservare che esiste un costruttore che permette la creazione di una lista a partire da una collezione
- Costruttori di **ArrayList<E>**
 - **ArrayList<E>()** *Constructs an empty list with an initial capacity of ten*
 - **ArrayList(**Collection<? extends E>** c)** *Constructs a list containing the elements of the specified collection, in the order they are returned by the collection's iterator*
 - **ArrayList<E>(int initialCapacity)** *Constructs an empty list with the specified initial capacity*
- Costruttori di **LinkedList<E>**
 - **LinkedList<E>** *Constructs an empty list*
 - **LinkedList<E>(**Collection<? extends E>** c)** *Constructs a list containing the elements of the specified collection, in the order they are returned by the collection's iterator*

Esercizio: List<E>

- La classe **ArrayList<E>** implementa l'interfaccia **List<E>** (e quindi è sottotipo anche di **Collection<E>**)
- Proviamo a rivedere il codice della classe **Borsa** nello studio di caso:
 - anziché usare un array per memorizzare l'insieme di attrezzi, usiamo un **ArrayList<E>**
- Vediamo come gestiamo
 - aggiunta di un elemento
 - scansione della lista

Esercizio List<E>: Aggiunta di Elementi

```
public class Borsa {  
    private Attrezzo[] attrezzi;  
    private int numeroAttrezzi;  
    public Borsa() {  
        this.numeroAttrezzi = 0;  
        this.attrezzi = new Attrezzo[10];  
    }  
    public boolean addAttrezzo(Attrezzo attrezzo){  
        if (numeroAttrezzi == 10)  
            return false;  
        this.attrezzi[this.numeroAttrezzi] = attrezzo;  
        this.numeroAttrezzi++;  
        return true;  
    } ...  
}
```

array

```
import java.util.List;  
import java.util.ArrayList;  
public class Borsa {
```

```
    private List<Attrezzo> attrezzi;  
    public Borsa() {  
        this.attrezzi = new ArrayList<Attrezzo>();
```

ArrayList

```
    }  
    public boolean addAttrezzo(Attrezzo attrezzo) {  
        return this.attrezzi.add(attrezzo);  
    }...
```

```
}
```

Esercizio List<E>: Osservazioni

- Dobbiamo importare:

`java.util.List`

`java.util.ArrayList`

- Principali benefici rispetto all'uso degli array
 - non ci dobbiamo preoccupare di stabilire a priori le dimensioni massime della collezione
 - non ci dobbiamo preoccupare di gestire l'indicatore di riempimento, che memorizza il numero di elementi effettivamente memorizzati nell'array

Esercizio List<E>: Aggiunta di Elementi

- L'aggiunta di elementi in un **ArrayList<E>** viene realizzata tramite il metodo **add(E e1)**
- Questo metodo aggiunge un riferimento ad oggetto (istanza di tipo **E**) nell'ultima posizione della collezione
- Gli elementi della lista rimangono ordinati secondo l'ordine di inserimento
 - l'oggetto inserito per primo è nella prima posizione
 - l'oggetto inserito per secondo è nella seconda posizione
 - ...
 - l'oggetto inserito per ultimo è in ultima posizione

Esercizio List<E>: Osservazioni

- La lista aumenta la sua capacità se necessario
 - Mantiene un conteggio del numero di elementi
 - Il metodo **int size()** restituisce questo valore
- ✓ I dettagli di come tutto ciò viene realizzato ci viene nascosto
- È importante?
 - Non conoscere questi dettagli ci impedisce di usare la collezione?
- ✓ Quanto risulta difficile cambiare la scelta dell'implementazione?
- Ad es. passare da **ArrayList** a **LinkedList**...

Metodi Specifici di `LinkedList<E>`

- `LinkedList<E>` offre anche alcuni metodi “fuori” dalla interface `List<E>` e quindi non offerti anche da `ArrayList<E>`
 - Similarmente `ArrayList<E>` offre costruttori (basati sulla capacità) che `LinkedList<E>` non offre
- Sono metodi tesi ad evidenziare l'accesso efficiente da parte delle `LinkedList<E>` in testa ed in coda
- Ad es.
 - `addFirst()/Last()`
 - `getFirst()/Last()`
 - `removeFirst()/Last()`
 - ...
- ✓ Attenzione: utilizzarli rende meno intercambiabili le due implementazioni
- Per questo si consiglia di dichiarare i tipi, a meno di *forti* motivazioni in senso contrario, utilizzando *sempre* le interface

Esercizio List<E>: Rimozione di un Elemento

```
import java.util.List;
import java.util.ArrayList;

public class Borsa {
    private List<Attrezzi> attrezzi;
    ...
    public Attrezzo removeAttrezzo(String nomeAttrezzo) {
        Attrezzo a = null;
        Iterator<Attrezzo> iteratore =
            this.attrezzi.iterator();
        while (iteratore.hasNext()) {
            a = iteratore.next();
            if (a.getNome().equals(nomeAttrezzo)) {
                iteratore.remove();
                return a;
            }
        }
        return null;
    }
    ...
}
```

Con ArrayList

Esercizio: `ListIterator<E>`

- In realtà l'interface ***List*** affianca al factory method ***iterator()*** di ***Collection*** un metodo specifico per le liste (e non esistente per tutte le generiche collezioni)
- Il metodo **`List<E>.listIterator()`** restituisce un ***ListIterator<E>***
 - ✓ ***ListIterator*** estende ***Iterator***
- Studiare (consultando i javadoc) le differenze tra le due interface ***Iterator*** e ***ListIterator***
- Scrivere dei test di unità sulla semantica dei metodi di ***ListIterator*** verificando che continuino ad aver successo anche i test-case già scritti per il suo supertipo ***Iterator***

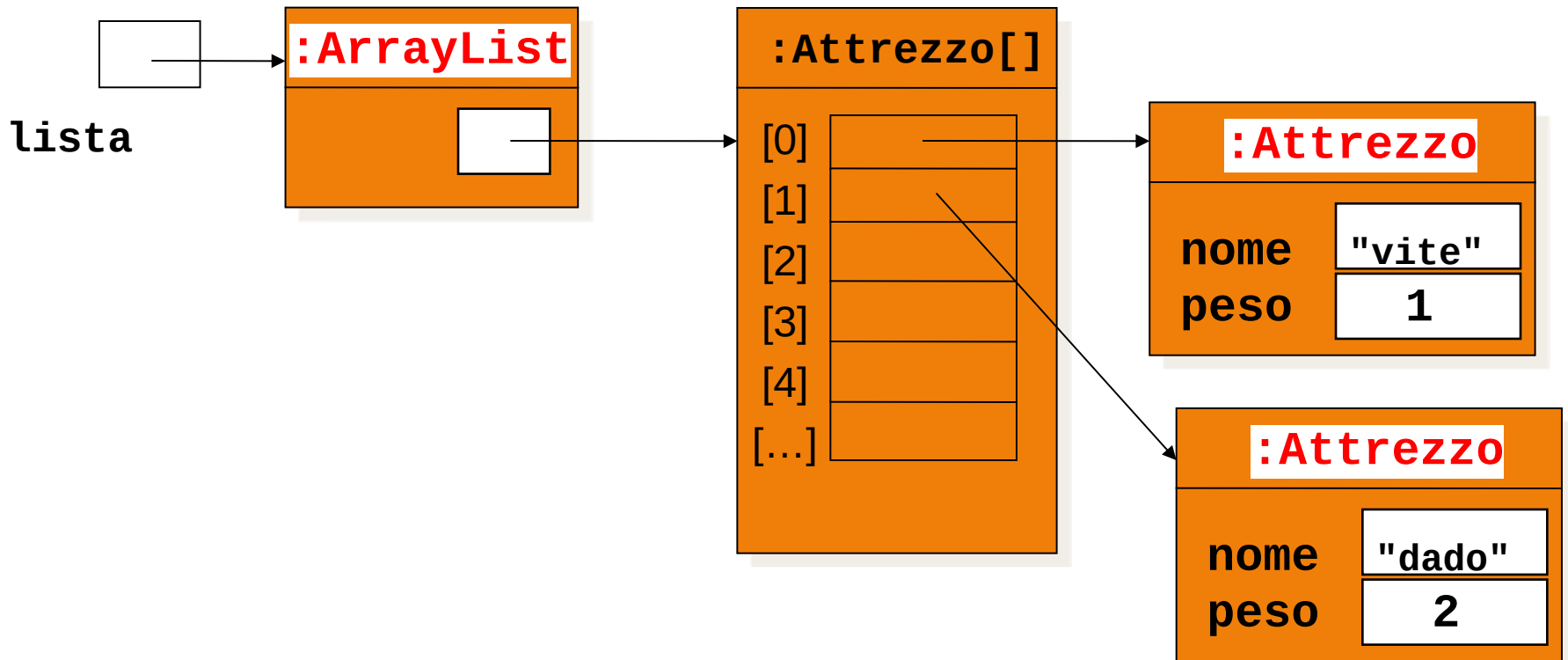
Liste: Diagramma degli Oggetti

- Nel seguito introduciamo una notazione grafica per la rappresentazione di oggetti **ArrayList** e **LinkedList**
 - la rappresentazione proposta è una astrazione (molto semplificata, ma utile a fini didattici) della rappresentazione interna delle due implementazioni

ArrayList: Diagramma degli Oggetti

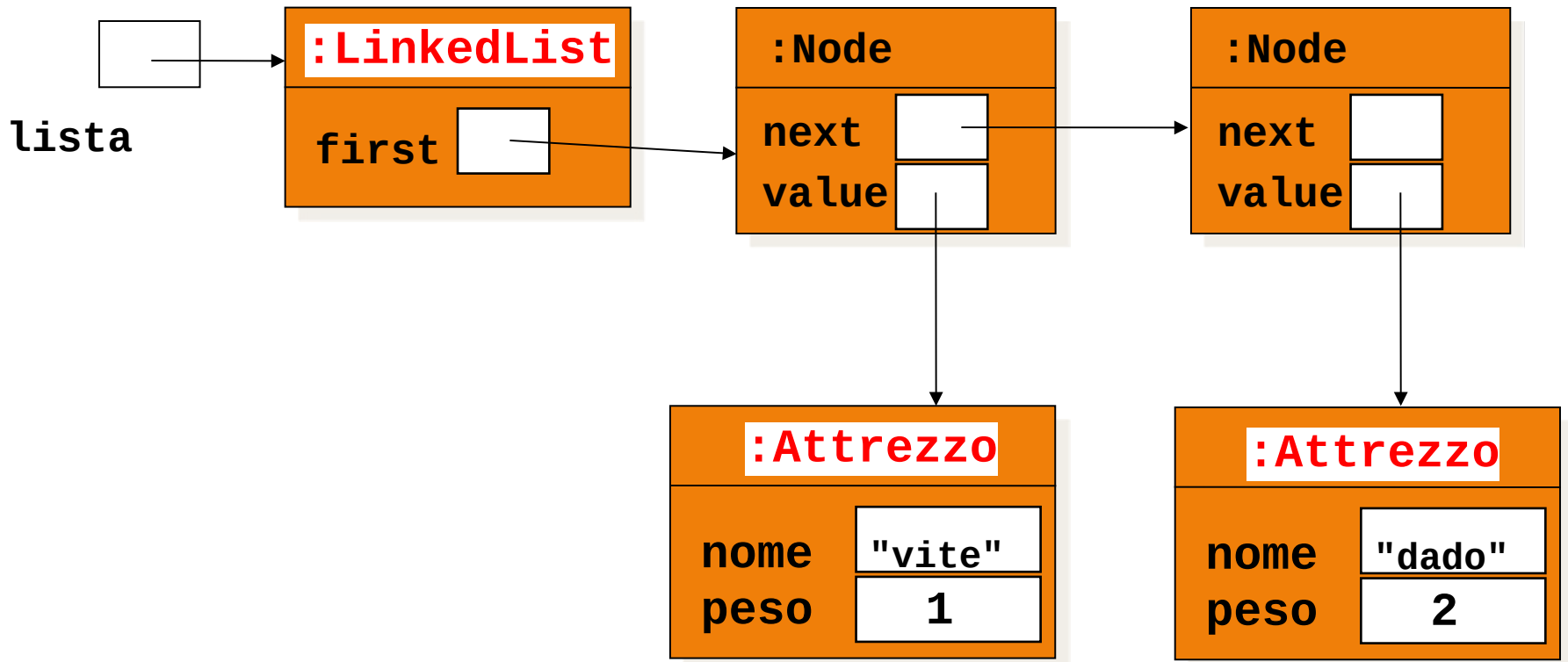
```
List<Attrezzo> lista;  
lista = new ArrayList<Attrezzo>();  
lista.add(new Attrezzo("vite",1);  
lista.add(new Attrezzo("dado",2);
```

// DIAGRAMMA DA DISEGNARE QUANDO L'ESECUZIONE RAGGIUNGE QUESTO PUNTO



LinkedList: Diagramma degli Oggetti

```
List<Attrezzo> lista;  
lista = new LinkedList<Attrezzo>();  
lista.add(new Attrezzo("vite",1);  
lista.add(new Attrezzo("dado",2);  
// DIAGRAMMA IN QUESTO PUNTO
```



Sommario

- Introduzione alle Collezioni
 - Interface Collection<E>
 - Iterare una collezione: Iterator<E>
 - Rimuovere elementi da una collezione
- Liste Generiche
 - aggiungere elementi
 - iterare sugli elementi della lista
- **Ordinamento di liste**
 - **Comparable, Comparator**

Ordinamenti e Ricerche

- Il JCF (in particolare la classe **Collections**) include metodi che implementano algoritmi efficienti per
 - ordinare una lista
 - ricercare la posizione di un elemento in una lista ordinata
 - ricercare il min/max in una lista

Relazione d'Ordine

- Queste operazioni sono possibili solo se esiste una relazione d'ordine per il tipo degli elementi ospitati nella collezione
 - in altri termini, gli elementi della lista devono sapersi confrontare
 - oppure ci deve essere un oggetto esterno che sa come confrontare due oggetti della lista
- ✓ Anche una relazione d'ordine su un supertipo degli elementi ospitati va bene

Definizione di un Criterio di Ordinamento

- La responsabilità di modellare il criterio di ordinamento può essere affidata, in alternativa:
 - alla stessa classe degli oggetti contenuti, che deve implementare una apposita interfaccia `java.lang.Comparable<T>`
(il cosiddetto ordinamento «*naturale*» o «*interno*»)
 - ad una classe esterna alla classe degli oggetti contenuti; tale classe esiste solo con l'obiettivo di confrontarli, si chiama *comparatore* e rispetta l'interfaccia `java.util.Comparator<T>`
(il cosiddetto ordinamento «*esterno*»)

L'Interface `java.lang.Comparable<T>`

- L'interface `java.lang.Comparable<T>` consiste di un solo metodo:

`public int compareTo(T that)`

- Restituisce un valore che è:
 - minore, uguale, maggiore di zero a seconda che l'oggetto corrente `this` sia rispettivamente:
 - minore, uguale, maggiore dell'oggetto il cui riferimento è ricevuto come parametro formale `that`

Utilizzi di `java.lang.Comparable<T>`

- Molte importanti classi della libreria standard già implementano l'interfaccia **`java.lang.Comparable<T>`**, adottando una semantica più o meno scontata.
- Ad es.
 - **`java.lang.String`**
 - **`java.util.Calendar`**
 - **`java.util.Date`**
 - **`java.io.File`**
 - **`java.net.URI`**
 - tutte le classi wrapper
 - ... e molte altre ancora

L'interface Comparable<T>: Esempio

```
public class Persona implements Comparable<Persona> {  
    private String nome;  
    private int eta;  
    public Persona(String nome, int eta) {  
        this.nome = nome;  
        this.eta = eta;  
    }  
  
    public String getNome() {  
        return this.nome;  
    }  
  
    public int getEta() {  
        return this.eta;  
    }  
  
    @Override  
    public int compareTo(Persona that) {  
        return this.nome.compareTo(that.getNome());  
    }  
}
```

Test di Persona.compareTo()

```
public class PersonaTest {  
  
    @Test  
    public void testCompareTo() {  
        Persona p1 = new Persona("Paolo", 10);  
        Persona p2 = new Persona("Valter", 5);  
  
        assertTrue(p1.compareTo(p2) < 0);          // <0  
  
        Persona p3 = new Persona("Paolo", 10);  
        assertEquals(0, p1.compareTo(p3)); // 0  
  
        Persona p4 = new Persona("Anna", 8);  
        assertTrue(p1.compareTo(p4) > 0);          // >0  
    }  
  
}
```


Ordinamento «Naturale»

- Un ordinamento naturale di oggetti è definito dalla relazione d'ordine implementata dal metodo **compareTo()** nell'interface **Comparable<T>**
- E' possibile quindi operare su una **List<T>** contenente oggetti che implementano **Comparable<T>** con metodi che utilizzino tale criterio di ordinamento:
 - si possono effettuare ricerche efficienti
 - si può calcolare il massimo e il minimo
 - si può effettuare l'ordinamentomediante i metodi offerti da **Collections**

Ordinamento di Liste: `Collections.sort()`

- Metodo statico **`Collections.sort()`**, in due versioni sovraccariche corrispondenti ai due diversi modi («naturale» vs «esterno») di fornire un criterio di ordinamento
- Segnatura per l'ordinamento naturale:

```
public static <T extends Comparable<? super T>>  
    void sort(List<T> list)
```

Test dell'Ordinamento «Naturale»

```
public class OrdinamentoNaturaleTest {  
    @Test  
    public void testSort() {  
        List<Persona> l = new LinkedList<>();  
        l.add(new Persona("Valter", 5));  
        l.add(new Persona("Paolo", 10));  
        l.add(new Persona("Giacomo", 7));  
        l.add(new Persona("Alessandro", 8));  
        Collections.sort(l);  
        assertEquals("Alessandro", l.get(0).getNome());  
        assertEquals("Giacomo", l.get(1).getNome());  
        assertEquals("Paolo", l.get(2).getNome());  
        assertEquals("Valter", l.get(3).getNome());  
    }  
}
```

Se **Persona** non implementasse **Comparable<Persona>**,
si solleverebbe un errore già a tempo di compilazione

Ottenere *Min* & *Max* di una Lista

- Data una lista **List<T>** i cui elementi implementino l'interface **Comparable<T>**, può essere calcolato l'elemento massimo/minimo (rispetto all'ordinamento naturale) mediante i metodi statici di **Collections**:
 - **Collections.min()**
 - **Collections.max()**

```
public static <T extends Object & Comparable<? super T>>  
    T min/max(Collection<? extends T> coll)
```

...che semplifichiamo in:

```
public static <T extends Comparable<? super T>>  
    T min/max(Collection<? extends T> coll)
```

Test del Calcolo Min & Max di una Lista

```
public class MinMaxTest {  
    @Test  
    public void testMinMax() {  
        List<Persona> l = new LinkedList<>();  
  
        l.add(new Persona("Valter"), 5);  
        l.add(new Persona("Paolo"), 10);  
        l.add(new Persona("Giacomo"), 7);  
        l.add(new Persona("Alessandro"), 8);  
  
        assertEquals("Alessandro", Collections.min(l).getNome());  
        assertEquals("Valter", Collections.max(l).getNome());  
    }...  
}
```

✓ Se **Persona** non implementasse **Comparable<Persona>**,
si solleverebbe un errore a tempo di compilazione

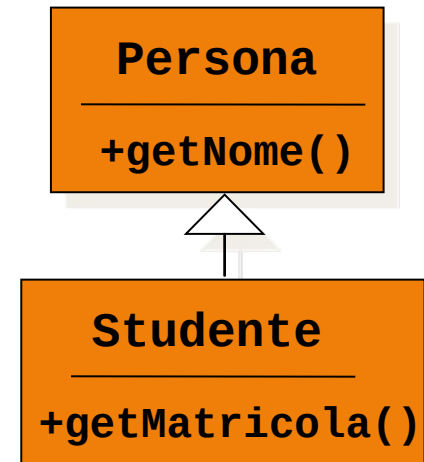
Ordinamento «Naturale»: Limiti (1)

- L'ordinamento naturale può essere definito al più una sola volta in ogni classe
- In effetti può essere utilizzata una sola volta per ogni intera *gerarchia di tipi* !
- Per capire perché, consideriamo due classi, **Persona** e **Studiante**, ove **Studiante** estende **Persona**, e prevediamo per entrambe un ordinamento «naturale»
 - le persone sono ordinate per nome
 - gli studenti sono ordinati per matricola

Ordinamento «Naturale»: Limiti (2)

```
public class Persona implements Comparable<Persona> {  
    private String nome;  
    public Persona(String nome) { this.nome = nome; }  
    public String getName() { return this.nome; }  
    @Override  
    public int compareTo(Persona that) {  
        return this.getName().compareTo(that.getName());  
    }  
}
```

```
public class Studente extends Persona implements Comparable<Studente> {  
    private String matricola;  
    public Studente(String nome, String matricola) {  
        super(nome);  
        this.matricola = matricola;  
    }  
    public String getMatricola() { return this.matricola;  
    @Override  
    public int compareTo(Studente that) {  
        return this.getMatricola().compareTo(that.getMatricola());  
    }  
}
```



NON COMPILA!

The interface **Comparable** cannot be implemented more than once with different arguments: **Comparable<Persona>** and **Comparable<Studente>**

Limitazioni dei Java Generics (1)

- Alla stessa classe/interface (in generale tipo) non è permesso implementare la stessa interface generica più di una volta con tipi attuali distinti!
- Chiaramente controintuitivo; contraddice almeno questi aspetti che lasciavano sperare altrimenti:
 - una classe Java può implementare diverse interface
 - si possono definire metodi sovraccarichi di stesso nome ma diversa segnatura per l'uso di tipi polimorfi (anche se pescati dalla medesima gerarchia)
- In effetti è come se la piattaforma si rifiutasse di considerare diversi due tipi generici validi se differiscono *solo* per uno dei tipi attuali utilizzati
- ✓ Ed è esattamente così...

Limitazioni dei Java Generics (2)

- *java.lang.Comparable* presente sin da Java 1.0
 - ✓ ovviamente non utilizzava i generics: introdotti solo in Java 5
 - ✓ Si basava su **Object**: ***Comparable*.boolean compareTo(Object o)**
- L'irrinunciabile retrocompatibilità ha imposto forti assunzioni sull'uso dei generics: in particolare NON risultò possibile cambiare le informazioni sui tipi (dinamici) disponibili a tempo di esecuzione rispetto alle versioni pre-generics
 - Detto diversamente, ciascun tipo deve essere *sempre* distinguibile a tempo *dinamico* anche se cancelliamo i tipi attuali utilizzati nella definizione dei tipi generici e finalizzati a tempo *statico*
 - A tempo dinamico non rimane traccia dei generics, che sono sempre risolti in un tipo non generico (chiamato «erasure») già a tempo di compilazione
- ✓ Per motivazioni similari, non è possibile creare array generics
 - Per es. **T[] ag = new T[10]; // T tipo NON COMPILA**

Limitazioni dell'Ordinamento «Naturale» o «Interno»

- A causa di questa sfortunata interazione con le limitazioni dei generics, l'ordinamento naturale può essere definito in uno solo dei tipi di una gerarchia
 - nell'es. di prima, dentro **Studente** oppure dentro **Persona**, ma non in entrambe
- Pertanto, considerando:
 - la necessità di definire molteplici criteri di ordinamento su oggetti dello stesso tipo (o della stessa gerarchia di tipi)
 - nonché la necessità di disaccoppiare la definizione del criterio di ordinamento da quella della classe le cui istanze risulteranno ordinate
- ✓ risulta ben motivato l'utilizzo di soluzioni (per la definizione di un criterio di ordinamento) «esterne» alla classe

Ordinamento «Esterno»

- Se vogliamo ordinare una lista secondo un criterio diverso dall'ordinamento naturale?
- Segnatura per l'ordinamento esterno:

```
public static <T> void sort(  
    List<T> listaDaOrdinare,  
    Comparator<? super T> comparatore  
)
```
- Si affida ad un *oggetto esterno*, passato come parametro, per effettuare i confronti necessari all'ordinamento
 - ✓ unica soluzione possibile quando serve più di un criterio di ordinamento

L'Interface

`java.util.Comparator<T>`

- L'interfaccia `java.util.Comparator<T>` consiste di un solo metodo:

```
public int compare(T o1, T o2)
```

che deve restituire un valore che è

- minore, uguale, maggiore di zero a seconda che l'oggetto riferito da **o1** sia...
- minore, uguale, maggiore dell'oggetto riferito dal parametro **o2**
- ✓ (per ricordarselo: “come fosse **o1 - o2** per un ord. crescente”)
- ✓ N.B. la segnatura è simile ma *non* identica a quella del metodo **compareTo()** di **Comparable<T>**

Esercizio (cont.)

- Supponiamo di voler ordinare una lista di oggetti **Persona** per età
- Introduciamo un opportuno comparatore esterno:

```
import java.util.Comparator;

public class ComparatorePerEta
    implements Comparator<Persona> {

    @Override
    public int compare(Persona p1, Persona p2) {
        return p1.getEta() - p2.getEta();
    }

}
```

Test di Comparator<Persona>.compare()

```
// import omessi
```

```
public class ComparatorePerEtaTest {
```

```
    @Test
```

```
    public void testCompare() {
```

```
        Persona paolo = new Persona("Paolo", 61);
```

```
        Persona anna = new Persona("Anna", 55);
```

```
        ComparatorePerEta comparator =  
            new ComparatorePerEta();
```

```
        assertTrue(comparator.compare(paolo, anna) > 0);
```

```
        assertTrue(comparator.compare(anna, paolo) < 0);
```

```
        assertEquals(0, comparator.compare(paolo, paolo));
```

```
        assertEquals(0, comparator.compare(anna, anna));
```

```
    }
```

```
}
```

Test dell'Ordinamento «Esterno»

```
public class OrdinamentoEsternoTest {
    @Test
    public void testSort() {
        List<Persona> l = new LinkedList<>();

        l.add(new Persona("Valter", 5));
        l.add(new Persona("Paolo", 10));
        l.add(new Persona("Giacomo", 7));
        l.add(new Persona("Alessandro", 8));
        ComparatorePerEta comparatore =
            new ComparatorePerEta();
        Collections.sort(l, comparatore);
        assertEquals("Valter",      l.get(0).getNome());
        assertEquals("Giacomo",     l.get(1).getNome());
        assertEquals("Alessandro",  l.get(2).getNome());
        assertEquals("Paolo",       l.get(3).getNome());
    }
}
```

Note Finali

- L'interface **Comparable<T>** è nel package **java.lang**, quindi non è necessario importarla
- L'interface **java.util.Comparator<T>** è nel package **java.util**, quindi va **importata esplicitamente**
- In **Collections** esistono anche i metodi per il calcolo del min/max secondo l'ordinamento esterno.

Ad es.:

```
static <T> T min(Collection<? extends T> coll,  
                Comparator<? super T> comp)
```


Esercizio

- Senza cambiare la classe **Libro** (riportata di seguito), scrivere il metodo **elencoOrdinatoPerPagine()** della classe **Biblioteca** affinché restituisca l'elenco dei libri ordinato per numero di pagine

Esercizio (cont.)

```
public class Libro implements Comparable<Libro> {  
  
    private String titolo;  
    private int pagine;  
  
    public Libro(String titolo, int pagine) {  
        this.titolo = titolo;  
        this.pagine = pagine;  
    }  
  
    public String getTitolo() {  
        return this.titolo;  
    }  
  
    public int getPagine() {  
        return this.pagine;  
    }  
    @Override  
    public int compareTo(Libro libro) {  
        return this.getTitolo().compareTo(libro.getTitolo());  
    }  
}
```

Esercizio (cont.)

```
import java.util.List;
import java.util.ArrayList;

public class Biblioteca {

    private List<Libro> elenco;

    public Biblioteca() {
        this.elenco = new ArrayList<>();
    }

    public void aggiungiLibro(Libro libro) {
        this.elenco.add(libro);
    }

    public List<Libro> elencoOrdinatoPerPagina() {
        ComparatorePerPagina comp = new ComparatorePerPagina();
        Collections.sort(this.elenco, comp);
        return this.elenco;
    }

}
```

Esercizio (cont.)

```
public class ComparatorePerPagine
    implements Comparator<Libro> {

    @Override
    public int compare(Libro l1, Libro l2) {
        return l1.getPagine() - l2.getPagine();
    }

}
```

Conclusioni

- Introduzione alle Collezioni
 - Interacce principali
 - Principali implementazioni
 - Iteratori, e **for-each**
- Java Collection Framework
- Liste
- Ordinamento di liste
 - Ordinamento *Naturale*
 - Ordinamento *Esterno*
 - Generics nelle gerarchie di tipi

Programmazione Orientata agli Oggetti

Collezioni:
Insiemi Generici

Sommario

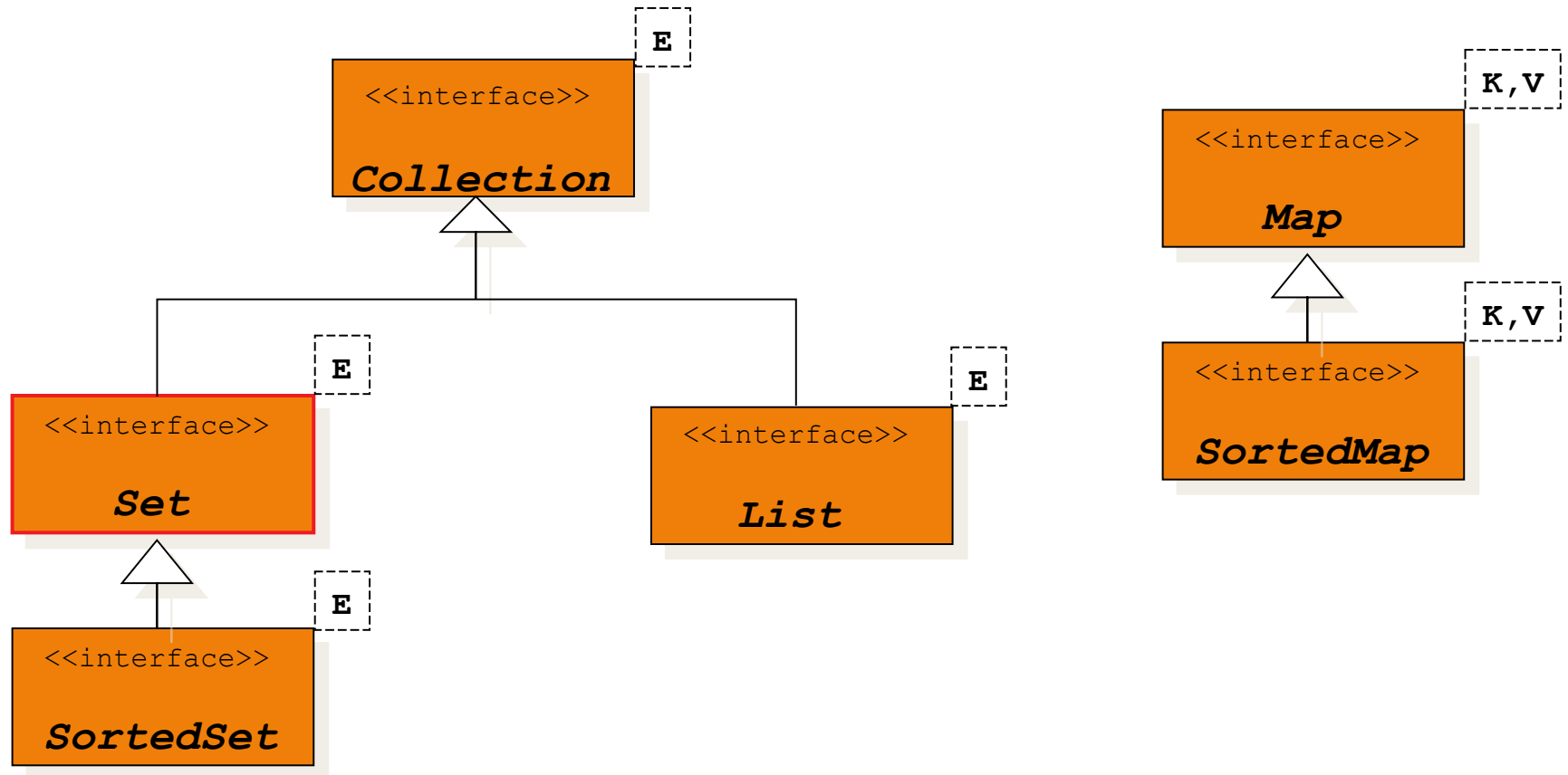
- L'interface **Set<E>**
- Criteri equivalenza tra elementi
- **HashSet<E>**
 - Tavole Hash (intuizione)
 - **hashCode()**, **equals()**
- **TreeSet<E>**
 - **compare()**, **compareTo()**

Sommario

- **L'interface `Set<E>`**
- Criteri equivalenza tra elementi
- **`HashSet<E>`**
 - Tavole Hash (intuizione)
 - `hashCode()`, `equals()`
- **`TreeSet<E>`**
 - `compare()`, `compareTo()`

Collezioni: Principali Interface

- Le principali *interface* del package **java.util**



- Per ognuna di queste interface il package offre diverse implementazioni

Insiemi: Interface *Set*<E>

- Un insieme (set) è una collezione *che non contiene duplicati*
- L'interface **Set**<E>, che estende l'interface **Collection**<E>, offre tutti e soli i metodi della interface **Collection**<E>, con la restrizione (riportata nel contratto) che le classi che la implementano si impegnano a non ammettere la presenza di elementi *duplicati*
- E' necessario stabilire un criterio di equivalenza tra gli elementi dell'insieme al fine di rilevare (e gestire) l'inserimento di duplicati

Sommario

- L'interface `Set<E>`
- Criteri equivalenza tra elementi
- Tavole Hash (intuizione)
- `HashSet<E>`
 - `hashCode()`, `equals()`
- `TreeSet<E>`
 - `compare()`, `compareTo()`

Criterio di Equivalenza tra Elementi di un Insieme (1)

- Nel Java Collection Framework abbiamo due implementazioni di **Set<E>**, rispettivamente **SortedSet<E>**:
 - **HashSet<E>**
 - **TreeSet<E>**
- Attenzione: nelle due classi concrete il criterio di equivalenza tra elementi non è definito nella stessa maniera:
 - **HashSet<E>**
si basa sui metodi **equals()** e **hashCode()**
 - **TreeSet<E>**
viene «indotto» dal criterio di ordinamento (**>**)

Criterio di Equivalenza tra Elementi di un Insieme (2)

- Per usare correttamente le implementazioni di **Set<E>** è necessario che le classi degli oggetti destinati a svolgere il ruolo di elementi dell'insieme, definiscano il criterio di equivalenza oggetti/elementi desiderato
- **HashSet<E>** richiede che tali classi ridefiniscano opportunamente due metodi (ereditati da **Object**) che servono a verificare ed evitare la presenza di duplicati
 - il metodo *equals()*: `public boolean equals(Object o)`
 - **ed anche** il metodo *hashCode()*: `public int hashCode()`
- **TreeSet<E>** richiede
 - che la classe degli elementi definisca un ordinamento naturale (quindi che implementi **Comparable<E>**)
 - oppure, che al momento della costruzione dell'insieme venga passato un comparatore (**Comparator<E>**) che definisca una relazione d'ordine per gli oggetti di tipo **E**

Insiemi: Criteri di Equivalenza

- Esempio: se sono interessato ad usare un insieme di oggetti **Persona** (**Set<Persona>**):
 - Se decidiamo di usare l'implementazione **HashSet**, la classe **Persona** deve:
 - ridefinire il metodo *equals()*
`public boolean equals(Object that)`
 - ed anche ridefinire il metodo *hashCode()*
`public int hashCode()`
 - Se decidiamo di usare l'implementazione **TreeSet**:
 - la classe **Persona** deve implementare l'interface **Comparable<Persona>**
 - oppure, l'oggetto **TreeSet** deve essere costruito passando un oggetto **Comparator<Persona>**

Sommario

- L'interface `Set<E>`
- Criteri equivalenza tra elementi
- **`HashSet<E>`**
 - **Tavole Hash (intuizione)**
 - `hashCode()`, `equals()`
- **`TreeSet<E>`**
 - `compare()`, `compareTo()`

Insiemi: Implementazione `HashSet<E>`

- Per comprendere appieno le motivazioni alla base di questa scelta progettuale è necessario discutere alcuni dettagli relativi all'implementazione
- L'implementazione di `HashSet<E>` si basa su una struttura dati fondamentale: *Tavola Hash*
- Ricapitoliamo velocemente alcuni aspetti (per una trattazione più approfondita, vedi il corso di *Algoritmi e strutture dati*)

Insiemi: Implementazione con Tavole Hash

- Una *funzione di hash* è una funzione che calcola un numero intero, detto *codice hash*, a partire dai dati di un oggetto, di modo che molto probabilmente (anche se non certamente) *oggetti non equivalenti* abbiano codici diversi
- Se invece due oggetti equivalenti finiscono per avere lo stesso codice di hash, si crea una situazione indesiderata
 - tale situazione è denominata *collisione*
- ✓ Una buona funzione di hash deve infatti minimizzare le collisioni per migliorare le prestazioni complessive

Codici *Hash*

- Esempi di codici hash per una stringa
 - Il numero di caratteri che compongono la stringa
 - "Pippo": 5
 - "Pluto": 5
 - "Paperino": 8
 - Non è un buon codice hash: troppe collisioni
 - La somma dei codici ASCII dei caratteri che compongono la stringa
 - "Pippo": $80+73+80+80+79=392$
 - "Pluto": $80+76+85+84+79=400$
 - "Paperino": $80+65+80+69+82+73+78+79=626$
 - ... meglio ...
- Osservazione fondamentale: se i codici hash sono diversi allora le stringhe non sono eguali; il contrario non è necessariamente vero

Tavole Hash (1)

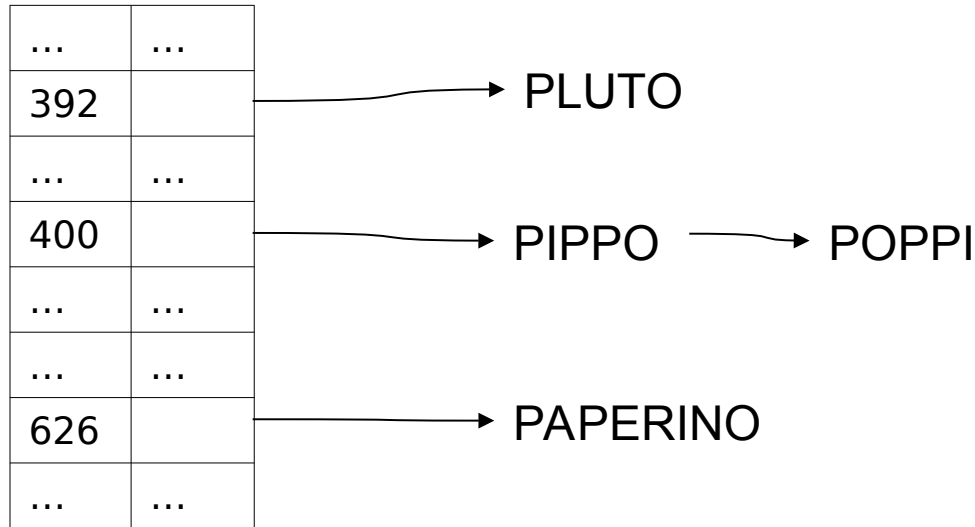
- Le funzioni di hash vengono usate per creare collezioni (mappe e insiemi) molto efficienti
- ✓ L'idea è quella di usare rappresentazioni basate su array da indicizzare mediante i codici hash degli elementi

...	...
392	PLUTO
...	...
400	PIPPO
...	...
...	...
626	PAPERINO
...	...

Tavole Hash (2)

- Questa semplice idea però ha due problemi:
 - La dimensione dell'array
 - ✓ Se non basta a contenere tutti i possibili codici hash?
 - Le collisioni
 - ✓ Se trovo il posto già occupato da un altro elemento?
- Per ovviare al primo problema si deve scegliere una funzione di hash che generi codici all'interno di una gamma di valori ragionevolmente limitata
- Per ovviare al secondo problema si usano liste concatenate (vedi corso ASD)

Tavole Hash (3)



- Se la funzione di hash è ben definita (range di valori contenuto, poche collisioni) le operazioni di verifica di appartenenza, rimozione e inserimento in una collezione posso essere realizzate a costo costante

Tavole Hash (4)

- Le implementazioni degli insiemi (e delle mappe, vedremo>>) di basano sulle tavole hash
- In Java, il codice hash di un oggetto deve essere restituito dal metodo `public int hashCode()`
- Se vogliamo usare l'implementazione `HashSet<E>` di `Set<E>`, gli oggetti della collezione devono avere **sia** il metodo `hashCode()`, **sia** il metodo `equals()` opportunamente e coerentemente definiti
 - ✓ se non li definiamo, questi metodi vengono ereditati da `java.lang.Object` secondo una semantica che, di solito, *non* è quella desiderata: `hashCode()` restituisce [un numero che dipende dal]l'indirizzo in memoria dell'oggetto, `equals()` realizza una semantica equivalente a quella dell'operatore `==` per il confronto di riferimenti, e *non* dei contenuti degli oggetti referenziati

Sommario

- L'interface **Set<E>**
- Criteri equivalenza tra elementi
- **HashSet<E>**
 - Tavole Hash (intuizione)
 - **hashCode()**, **equals()**
- **TreeSet<E>**
 - **compare()**, **compareTo()**

Eguaglianza: il Metodo `equals()`

- Per ogni classe va definito il metodo `boolean equals(Object o)`
- Deve soddisfare le seguenti proprietà per qualsiasi riferimento non nullo `x`, `y`, `z`:
 - *Riflessività*: `x.equals(x)` deve restituire `true`
 - *Simmetria*: `x.equals(y)` deve restituire `true` se e solo se `y.equals(x)` restituisce `true`
 - *Transitività*: se `x.equals(y)` restituisce `true` e `y.equals(z)` restituisce `true`, allora `x.equals(z)` deve restituire `true`
 - `x.equals(null)` deve restituire `false`
 - Se due riferimenti `x` e `y` sono identici (`x==y`), allora `x.equals(y)` deve restituire `true`

Eguaglianza: il Metodo `hashCode()`

- Ogni volta che in una classe definiamo il metodo `boolean equals(Object o)`, **dobbiamo** definire anche il metodo `int hashCode()`
- Il metodo `int hashCode()` può essere definito come il calcolo di una semplice combinazione lineare (ad es. somma) dei codici hash delle variabili usate per verificare l'uguaglianza
 - tutte le classi della libreria standard hanno una implementazione (“naturalmente” definita) del metodo `hashCode()`
 - N.B.: se le variabili sono tipi primitivi numerici possiamo usare direttamente il loro valore (al limite convertendolo prima in un `int`)

Esempio

```
public class Calciatore {
    private String nome;
    private String cognome;
    private int numeroMaglia;
    ... // metodi getter omessi
    @Override
    public boolean equals(Object o) {
        Calciatore that = (Calciatore)o;
        return    this.getNome().equals(that.getNome()) &&
                  this.getCognome().equals(that.getCognome()) &&
                  this.getNumeroMaglia() == that.getNumeroMaglia();
    }
    @Override
    public int hashCode() {
        return    this.getNome().hashCode()
                  + this.getCognome().hashCode()
                  + this.getNumeroMaglia();
    }
}
```

Gestione Duplicati in HashSet<E>

- In sintesi, per verificare la presenza di duplicati, **HashSet<E>** usa **equals(Object o)** in combinazione con il valore restituito dal metodo **hashCode()**
- In particolare l'implementazione di **HashSet**, per verificare se un elemento - *e* - è già presente nell'insieme:
 - prima, effettua una verifica molto *efficiente*: nell'insieme esistono già elementi con lo stesso codice hash di *e* ???
 - quindi, ma solo in caso affermativo (*collisione*) si verifica l'effettiva uguaglianza degli oggetti invocando il metodo **equals(Object o)**
 - ✓ per una valutazione «completa» e risolutiva dell'equivalenza, sebbene meno efficiente rispetto al metodo **hashCode()**

Due Metodi al Posto di Uno (1)

- Quindi i due metodi `equals()` ed `hashCode()` sono assolutamente *dipendenti* l'uno dall'altro e devono essere definiti in maniera coerente
- Perché costringere i programmatori utilizzatori a dover conoscere questi dettagli, non bastava il metodo `equals()`?
- La risposta è che il metodo `hashCode()` è l'unica strada conosciuta per ottenere facilmente implementazioni efficienti. E' una scelta chiaramente:
 - a favore del programmatore-*realizzatore*
 - ✓ e quindi dell'efficienza
 - a scapito del programmatore-*utilizzatore*
 - ✓ e quindi della semplicità di utilizzo

Due Metodi al Posto di Uno (2)

- Per ottenere implementazioni efficienti i calcoli effettuati dal metodo `hashCode()` dovrebbero essere meno onerosi rispetto a quelli effettuati da `equals()`

- `int hashCode()` viene invocato sempre
 - `boolean equals(Object o)` solo in caso di collisioni

- Attenzione: un metodo `hashCode()` così definito:

`@Override`

```
public int hashCode() { return 0; }
```

- sarebbe sempre corretto e per nulla oneroso
 - ✓ produrrebbe sempre collisioni! La tavola hash degenera in una lista concatenata di oggetti in perenne conflitto tra loro



Due Metodi al Posto di Uno (3)

- E se **per errore** non (ri)definissimo il metodo **hashCode()** pur avendo ridefinito il metodo **equals()**?
- Si eredita quello definito nella radice della gerarchia di classi Java, ovvero dalla classe **java.lang.Object**: finiamo per usare come codice hash l'indirizzo di memoria dell'oggetto (ogni oggetto possiede un codice diverso)
 - Quindi oggetti distinti che un'applicazione reputa equivalenti, siccome possiedono codici hash diversi, finirebbero per essere considerati distinti anche se il metodo **equals()** afferma invece il contrario
 - Un semplice test aiuta ad evidenziare la pericolosità di questo tipo di errori difficilmente rilevabili...

Gestione Duplicati in HashSet<E>: Esempio

- Consideriamo la seguente classe
N.B.: manca il metodo hashCode()

```
public class Persona {  
  
    private String nome;  
  
    Persona(String nome)      { this.nome = nome; }  
    public String toString() { return this.nome; }  
    public String getNome()  { return this.nome; }  
  
    @Override  
    public boolean equals(Object o) {  
        Persona that = (Persona)o;  
        return this.getNome().equals(that.getNome());  
    }  
}
```

Test Gestione Duplicati in HashSet<E>

```
public class HashSetTest {  
    @Test public void testAddDuplicatiEnon() {  
        Set<Persona> s = new HashSet<>();  
        assertEquals(0, s.size());  
  
        Persona paolo = new Persona("Paolo");  
        Persona valter = new Persona("Valter");  
  
        assertTrue(s.add(paolo));  
        assertEquals(1, s.size());  
  
        assertTrue(s.add(valter));  
        assertEquals(2, s.size());  
  
        assertFalse(s.add(paolo));  
        assertEquals(2, s.size());  
  
        Persona paolo2 = new Persona("Paolo");  
        assertFalse(s.add(paolo2));  
        assertEquals(2, s.size());  
    }  
}
```


`equals()` Orfano di `hashCode()`

- Se eseguiamo il test "sorprendentemente" *fallisce*, perché alcuni duplicati non vengono rilevati anche se il metodo `equals()` stabilisce correttamente il criterio di equivalenza desiderato
- Il problema è dovuto al fatto che non abbiamo ridefinito coerentemente il metodo `hashCode()`
- proviamo a stampare i codici hash, inserendo nel codice:

```
Persona p1 = new Persona("Paolo");  
Persona p2 = new Persona("Paolo");  
System.out.println("hash code p1: " + p1.hashCode());  
System.out.println("hash code p2: " + p2.hashCode());
```

Coerenza di `hashCode()` ed `equals()` (1)

- L'implementazione usata dalla macchina evidentemente assegna due codici hash diversi ai due oggetti **Persona** uguali (secondo la nostra definizione di `equals()`)
- Per ovviare al problema dobbiamo ridefinire nella classe **Persona** il metodo `hashCode()` coerente con il criterio di equivalenza stabilito dal metodo `equals()`
- Ogni qualvolta si scrive un metodo `equals()` è necessario (OBBLIGATORIO) scrivere anche il corrispondente metodo `hashCode()`

Coerenza di `hashCode()` ed `equals()` (2)

- In sintesi, per implementare efficacemente il metodo `hashCode()` nella produzione del codice hash conviene usare le stesse informazioni usate dal metodo `equals()`
- Esempio: se `equals()` si basa su *nome* e *cognome*, è bene che anche `hashCode()` utilizzi le stesse informazioni
 - ritornando ad esempio alla classe **Persona**, si usa `hashCode()` del *nome*
 - avessimo `equals()` che “insiste” su *nome* e *cognome*, un semplice metodo `hashCode()` potrebbe essere la somma degli `hashCode()` delle due stringhe

Gestione Duplicati in HashSet<E>: Esempio

- Aggiungiamo un opportuno metodo `hashCode()`

```
public class Persona {
    private String nome;

    public Persona(String nome) { this.nome = nome; }
    public String toString() { return this.nome; }
    public String getNome() { return this.nome; }

    @Override
    public int hashCode() {
        return this.getNome().hashCode();
    }

    @Override
    public boolean equals(Object o) {
        Persona that = (Persona)o;
        return this.getNome().equals(that.getNome());
    }
}
```

Test Gestione Duplicati in HashSet<E>

- **HashSetTest**, con la nuova definizione della classe **Persona**, ora mostra come (correttamente) non vengano inseriti duplicati
- A confermare la natura del problema visto in precedenza, ristampiamo anche i codici hash così come forniti dal nuovo metodo **hashCode()**
 - ✓ ora oggetti distinti ma che consideriamo equivalenti possiedono lo stesso codice hash

hashCode ()

- Molte classi della libreria standard già forniscono una implementazione del metodo `hashCode ()`
- Per scrivere il metodo `hashCode ()` delle nostre classi conviene usare una combinazione (ad esempio la somma) dei codici hash restituiti dai metodi `hashCode ()` delle variabili di istanza
 - nell'esempio precedente abbiamo usato il valore restituito dal metodo `hashCode ()` di `this.nome`, che è di tipo `String`



Insieme: Implementazione `HashSet<E>`

- Ricapitoliamo:
 - Se usiamo una collezione `HashSet<E>`:
 - gli elementi della collezione devono avere i metodi `hashCode()` e `equals()` appropriatamente definiti e in maniera mutuamente compatibile affinché `hashCode()` realizzi una funzione di hash rispetto al criterio di equivalenza stabilito da `equals()`
 - Indipendentemente dal fatto che gli oggetti della classe saranno elementi di un `HashSet`:
 - ogni qualvolta si scrive un metodo `equals()` è necessario (leggi OBBLIGATORIO) scrivere anche il corrispondente metodo `hashCode()` e viceversa

Sommario

- L'interface `Set<E>`
- Criteri equivalenza tra elementi
- **`HashSet<E>`**
 - Tavole Hash (intuizione)
 - `hashCode()`, `equals()`
- **`TreeSet<E>`**
 - Alberi binari di ricerca
 - `compare()`, `compareTo()`

Insiemi: Criterio di Equivalenza

- Due implementazioni di `Set<E>` e di `SortedSet<E>`, rispettivamente, nel JCF:
 - `HashSet<E>`
 - `TreeSet<E>`
- Nelle due classi concrete il criterio di equivalenza tra elementi non è definito nella stessa maniera:
 - `HashSet<E>`
si basa sui metodi `equals()` e `hashCode()`
 - `TreeSet<E>`
si basa sul metodo `Comparable.compareTo()` o `Comparator.compare()`

Set<E>: Implementazione TreeSet<E> (1)

- L'implementazione **TreeSet<E>** garantisce (oltre all'assenza di duplicati) che gli elementi **siano ordinati in accordo ad un criterio di ordinamento**
 - l'ordinamento naturale degli elementi, *oppure*
 - un ordinamento stabilito da un comparatore esterno, ma noto all'insieme stesso
 - ✓ ovvero ottenuto al momento della creazione dell'insieme attraverso uno dei costruttori

Set<E>: Implementazione TreeSet<E> (2)

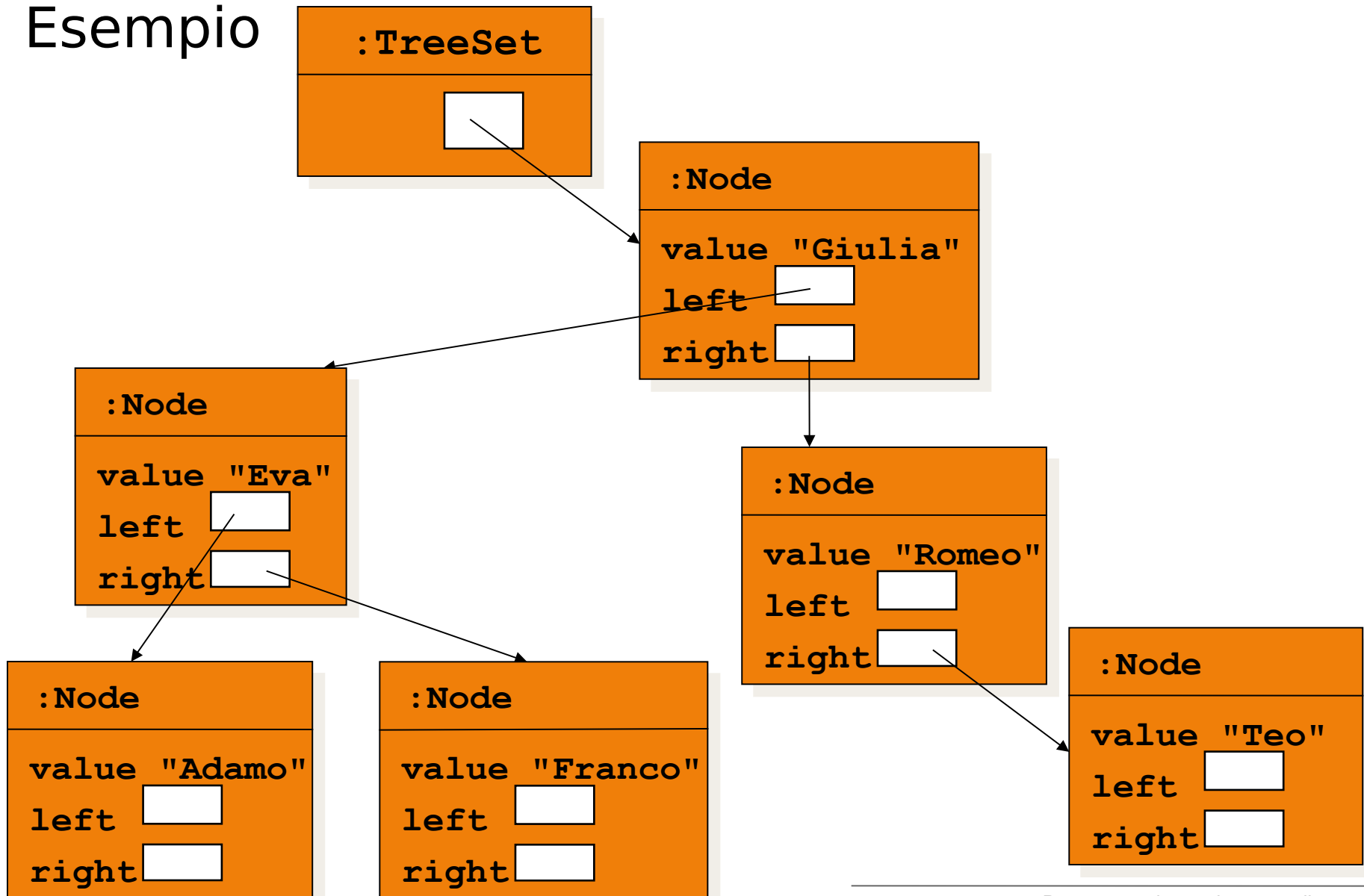
- Costruttori:
 - **TreeSet()**: Constructs a new, empty tree set, sorted according to the natural ordering of its elements.
 - **TreeSet(Collection<? extends E> c)**: Constructs a new tree set containing the elements in the specified collection, sorted according to the *natural ordering* of its elements.
 - **TreeSet(Comparator<? super E> comparator)**: Constructs a new, empty tree set, sorted according to the specified comparator.
- ✓ Attenzione: se gli elementi non implementano **Comparable<E>** (o se non viene usato un comparatore **Comparator<E>** in fase di costruzione dell'oggetto **TreeSet<E>**) **si può sollevare un errore a tempo di esecuzione anche se il codice compila!**

TreeSet<E>: Alberi di Ricerca Binari (1)

- L'implementazione **TreeSet** è basata su alberi di ricerca binari
 - gli elementi dell'albero sono nodi che possiedono riferimenti ad ai nodi *figli*
 - in un albero binario ogni nodo ha due figli, *nodo destro* e *nodo sinistro*, che rispettano la seguente proprietà:
 - i valori di tutti i discendenti alla sinistra di qualunque nodo sono inferiori al valore del dato memorizzato in quel nodo, mentre tutti i discendenti alla destra contengono valori maggiori

TreeSet<E>: Alberi di Ricerca Binari (2)

- Esempio



TreeSet<E>:

Alberi di Ricerca Binari (3)

- Per utilizzare una simile struttura dati è necessario che gli elementi si possano confrontare ed ordinare
 - per stabilire se un nodo deve andare a destra o a sinistra dobbiamo sapere se i suoi dati sono minori o maggiori dei dati del nodo genitore
- A tal fine, gli elementi devono implementare l'interface **Comparable<E>**
 - in alternativa è possibile che **TreeSet<E>** affidi la gestione di questo aspetto ad un comparatore (implementazione di **Comparator<E>**) esterno

Gestione Duplicati in TreeSet<E>: Esempio (1)

- Consideriamo nuovamente la classe **Persona**:

```
public class Persona {  
    private String nome;  
  
    Persona(String nome)        { this.nome = nome; }  
    public String toString()    { return this.nome; }  
    public String getNome()     { return this.nome; }  
  
    @Override  
    public int hashCode() {  
        return this.getNome().hashCode();  
    }  
  
    @Override  
    public boolean equals(Object p) {  
        return this.getNome().equals(((Persona)p).getNome());  
    }  
}
```


Test Gestione Duplicati in TreeSet<E>

```
public class TreeSetTest {  
    @Test public void testAddDuplicatiEnonConClassCastExc() {  
        Set<Persona> s = new TreeSet<Persona>();  
  
        Persona paolo = new Persona("Paolo");  
        Persona valter = new Persona("Valter");  
        assertTrue(s.add(paolo));  
        assertTrue(s.add(valter));  
        assertFalse(s.add(valter));  
        assertEquals(2, s.size());  
        Iterator<Persona> it = s.iterator();  
        assertSame(paolo, it.next());  
        assertSame(valter, it.next());  
    }  
}
```

ClassCastException
Non funziona!

TreeSet<E>: Criterio di Equivalenza non Definito

- Se compiliamo e facciamo girare il codice precedente, si verifica un errore a tempo di esecuzione
- L'errore per un programmatore inesperto può essere di non facile interpretazione: è una **ClassCastException**
- Il problema nasce dal fatto che un criterio di ordinamento NON è stato specificato:
 - Né **Persona** implementa **Comparable**
 - Né il **TreeSet** riceve un comparatore usando un costruttore che permetta di specificarlo al momento della sua creazione

Gestione Duplicati in TreeSet<E>: Esempio (2)

- La classe ora viene modificata per implementare **Comparable<Persona>**

```
public class Persona implements Comparable<Persona> {  
    private String nome;  
  
    Persona(String nome)      { this.nome = nome; }  
  
    public String toString() { return this.nome; }  
    public String getNome()  { return this.nome; }  
  
    @Override  
    public int hashCode() {  
        return this.getNome().hashCode();  
    }  
  
    @Override  
    public int compareTo(Persona that) {  
        return this.getNome().compareTo(that.getNome());  
    }  
  
    @Override  
    public boolean equals(Object o) {  
        return this.getNome().equals(((Persona)o).getNome());  
    }  
}
```

Gestione Duplicati in TreeSet<E>: Esempio (3)

- In alternativa: la classe **Persona** non viene fatta implementare **Comparable<Persona>**, ma specifichiamo il criterio di ordinamento passando un oggetto **Comparator<Persona>** al costruttore dell'insieme

```
public class Persona { // N.B. non implementa Comparable<Persona>
    private String nome;

    Persona(String nome)      { this.nome = nome; }
    public String toString()  { return this.nome; }
    public String getNome()   { return this.nome; }
}
```

```
public class ComparatorePersone implements Comparator<Persona> {

    @Override
    public int compare(Persona p1, Persona p2) {
        return p1.getNome().compareTo(p2.getNome());
    }
}
```

Gestione Duplicati in TreeSet<E>: Esempio (4)

```
public class TreeSetTest {  
    @Test public void testAddDuplicatiEnonConClassCastExc() {  
        ComparatorePersone cmp = new ComparatorePersone();  
        Set<Persona> s = new TreeSet<Persona>(cmp);  
  
        Persona paolo = new Persona("Paolo");  
        Persona valter = new Persona("Valter");  
        assertTrue(s.add(paolo));  
        assertTrue(s.add(valter));  
        assertFalse(s.add(valter));  
        assertEquals(2, s.size());  
        Iterator<Persona> it = s.iterator();  
        assertSame(paolo, it.next());  
        assertSame(valter, it.next());  
    }  
}
```

Gestione Duplicati in TreeSet<E>: Esempio (5)

- ✓ per implementare `ComparatorePersone` ci siamo basati sul fatto che `java.lang.String` a sua volta implementa `Comparable<String>`
- A questo punto se facciamo girare nuovamente il codice di `TreeSetTest`, il comportamento è quello atteso:
 - viene fatto un solo inserimento
 - non si verificano errori a tempo di esecuzione,
 - gli elementi sono ordinati secondo l'ordinamento naturale (quello lessicografico delle stringhe)

TreeSet<E>: Gestione Duplicati

- Riassumendo, per verificare la presenza di duplicati (e per mantenere ordinata la collezione)
 - **TreeSet<E>** usa il metodo **compareTo()** quindi gli oggetti che appartengono alla collezione devono implementare **Comparable<E>**
 - Oppure l'oggetto **TreeSet<E>** deve essere creato passando al costruttore un oggetto **Comparator<E>**



Eguaglianza e Set<E>

- Ovviamente le implementazioni di `compareTo()`, `hashCode()` e `equals()` devono avere una semantica coerente
 - Ogni volta che definiamo il metodo `equals()` dobbiamo definire anche il metodo `hashCode()`
 - Se definiamo il metodo `compareTo()` (cioè se la classe implementa `Comparable<E>`), questo è bene che sia coerente con `equals()`
- Dati due riferimenti ad oggetti **x** e **y**, allora
 - x.compareTo(y)** restituisce 0
 - se e solo se
 - x.equals(y)** restituisce **true**

Implementazioni di Set<E>: Verifica Duplicati

- Riassumendo:
 - **TreeSet<E>** verifica l'esistenza di duplicati
 - tramite il metodo `compareTo()`, ed il tipo **E** deve implementare l'interface `Comparable<E>`;
 - *oppure*, tramite il metodo `compare()` dell'interfaccia `Comparator` una cui istanza gli è stata fornita all'atto dell'istanziamiento (vedi i costruttori di **TreeSet<E>** nella documentazione)
 - **HashSet<E>** verifica l'esistenza di duplicati tramite i metodi `hashCode()` e `equals()` del tipo **E**

Set<E>: Quale Implementazione?

- **TreeSet<E>** mantiene ordinata la collezione, ma sia gli aggiornamenti che gli accessi casuali risultano meno efficienti della controparte non ordinata
 - ✓ perché bisogna mantenere l'ordinamento della rappresentazione sottostante
 - va utilizzata solo se esiste l'esigenza vera di mantenere ordinata la collezione
- Altrimenti preferire la più efficiente **HashSet<E>**
- Questi aspetti sono stati affrontati approfonditamente nel corso di *Algoritmi e Strutture Dati*

Esercizio

- Date le classi **Studente** e **Aula**

```
public class Studente {  
    private String nome;  
  
    public Studente(String nome) { this.nome = nome; }  
    public String getNome() { return this.nome; }  
}
```

```
public class Aula {  
    private Set<Studente> studenti;  
  
    public Aula() { /* scrivere il codice */ }  
    public boolean addStudente(Studente studente) {  
        // scrivere il codice  
    }  
}
```

Esercizio (cont.)

- Scrivere il codice del costruttore di **Aula** e del metodo **addStudente (Studente s)**
- Domanda 1:
 - produrre una soluzione considerando che è possibile modificare la classe **Studente**
- Domanda 2:
 - produrre una soluzione considerando che NON è possibile modificare la classe **Studente**, ma è possibile definire altre classi
- Domanda 3:
 - come la domanda 1, ma l'insieme degli studenti deve essere ordinato e non è possibile introdurre altre classi

Una Soluzione alla Domanda 1

```
public class Studente {
    private String nome;

    public Studente(String nome) { this.nome = nome; }
    public String getNome() {return this.nome; }

    @Override
    public int hashCode() { return this.nome.hashCode(); }

    @Override
    public boolean equals(Object o) {
        Studente that = (Studente)o;
        return this.getNome().equals(that.getNome());
    }
}

public class Aula {
    private Set<Studente> studenti;

    public Aula(){ this.studenti = new HashSet<Studente>(); }
    public boolean addStudente(Studente studente) {
        return this.studenti.add(studente);
    }
}
```

Una Soluzione alla Domanda 2

```
public class Studente {
    private String nome;

    public Studente(String nome) {this.nome = nome;}
    public String getNome() {return this.nome;}
}

public class Aula {
    private Set<Studente> studenti;

    public Aula() {
        this.studenti = new TreeSet<>(new ComparatoreStudenti());
    }

    public boolean addStudente(Studente studente) {
        return this.studenti.add(studente);
    }
}
```

Una Soluzione alla Domanda 2 (cont.)

```
import java.util.*;

public class ComparatoreStudenti
    implements Comparator<Studente> {

    @Override
    public int compare(Studente s1, Studente s2) {
        return s1.getNome().compareTo(s2.getNome());
    }
}
```


Una Soluzione alla Domanda 3

```
public class Studente implements Comparable<Studente> {
    private String nome;

    public Studente(String nome) { this.nome = nome; }
    public String getNome()      { return this.nome; }

    @Override
    public int hashCode() { return this.getNome().hashCode(); }

    @Override
    public boolean equals(Object o) {
        Studente that = (Studente)o;
        return this.getNome().equals(that.getNome());
    }

    @Override
    public int compareTo(Studente that) {
        return this.getNome().compareTo(that.getNome());
    }
}

public class Aula {
    private Set<Studente> studenti;

    public Aula() { this.studenti = new TreeSet<Studente>(); }
    public boolean addStudente(Studente studente) {
        return this.studenti.add(studente);
    }
}
```

Suggerimento

- Eclipse genera automaticamente il codice dei metodi `equals()` e `hashCode()`
 - ✓ si osservi che vengono generati entrambi contestualmente
 - ✓ è sufficiente indicare su quali variabili di istanza deve basarsi il criterio di equivalenza
- Durante l'apprendimento NON UTILIZZARLO!
 - ✓ Meglio mantenere la piena consapevolezza del codice utilizzato e, *soprattutto*, di come scriverlo

NavigableSet<E>: da Java 6

- In Java 6 le funzionalità offerte da `SortedSet<E>` sono state riconsiderate ed estese
- Allo scopo si aggiunge una specializzazione di `SortedSet<E>` denominata `NavigableSet<E>`
 - ✓ implementata anche da `TreeSet<E>`
- Qualche semplice esempio di metodi di cui si avvertiva la necessità (consultare i javadoc):

`E pollFirst() / E pollLast()`

Retrieves and removes the first (lowest) [last (highest)] element, or returns null if this set is empty

`NavigableSet<E> descendingSet()`

Returns a reverse order view (>>) of the elements contained in this set

NavigableSet<E>: Viste Attive

Un esempio più articolato:

```
NavigableSet<E> subSet(E fromElement, boolean fromInclusive,  
                        E toElement, boolean toInclusive)
```

Returns a view of the portion of this set whose elements range from fromElement to toElement.

If fromElement and toElement are equal, the returned set is empty unless fromInclusive and toInclusive are both true

- Questo metodo, ed altri strettamente correlati (cfr. **headSet()**, **tailSet()**), producono come risultato delle «viste attive» sull'insieme ordinato di partenza
 - *The returned set is backed by this set, so changes in the returned set are reflected in this set, and vice-versa.*
- ✓ Cambiamenti della vista si riflettono sull'insieme originale
Se non è il comportamento desiderato, basta «materializzare» la vista usandola come argomento del costruttore di una nuova collezione

NavigableSet<E> e Retrocompatibilità

- ✓ Alcune scelte sembrano discutibili, ma vanno certamente valutate nel contesto in cui sono nate, ovvero con l'esigenza di mantenere la retro-compatibilità con le implementazioni e le interfacce già esistenti e diffuse

@Override

SortedSet<E> `headSet(E toElement)`

- *Returns a view of the portion of this set whose elements are strictly less than `toElement`.*

NavigableSet<E>

`headSet(E toElement, boolean inclusive)`

- *Methods `subSet(E, E)`, `headSet(E)`, and `tailSet(E)` are specified to return `SortedSet` to allow existing implementations of `SortedSet` to be compatibly retrofitted to implement `NavigableSet`, but extensions and implementations of this interface are encouraged to override these methods to return `NavigableSet`*

Programmazione Orientata agli Oggetti

Collezioni:
Mappe Generiche

Sommario

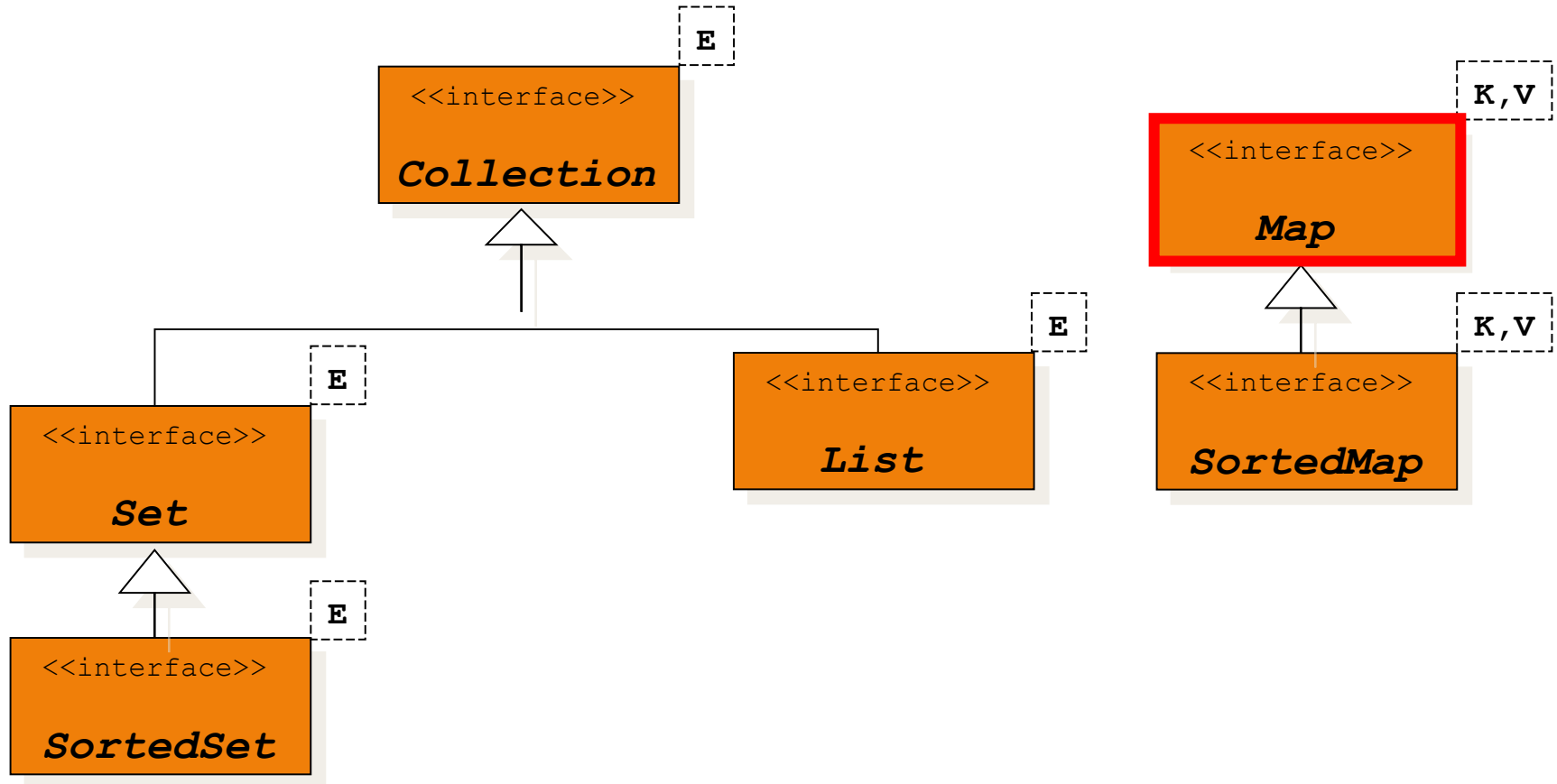
- Interface **Map<K, V>**
 - Operazioni tipiche sulle mappe
- Implementazioni di **Map<K, V>**
 - unicità delle chiavi

Sommario

- **Interface Map<K, V>**
 - Operazioni tipiche sulle mappe
- Implementazioni di **Map<K, V>**
 - unicità delle chiavi

Collezioni: Interface

- Le principali interface del package `java.util`



- Per ognuna di queste interface il package offre diverse implementazioni

Mappe

- Una mappa (o dizionario, o array associativo) è una collezione di coppie chiave-valore (entry)
- Le chiavi, in quanto tali, sono uniche
 - In una mappa non possono esistere due chiavi eguali
- Ad ogni chiave può essere associato un solo valore
 - ✓ ma il valore può essere una collezione
- Esempi:
 - rubrica telefonica: a ciascuna persona (chiave) è associato un numero di telefono (il numero è il valore)
 - indice analitico di un libro: ad ogni voce (chiave) è associato un insieme di numeri di pagine (l'insieme è il valore)

Mappe: Operazioni Principali

- Operazioni tipiche su una mappa sono:
 - Inserimento di una coppia chiave-valore
 - Inserisci nella rubrica: "Paolo Rossi" -> 0288821212
 - Richiesta del valore associato ad una chiave
 - Dammi il numero di telefono di "Elena Bruni"
 - Aggiornamento di un valore
 - Cambia il numero di telefono di "Paolo Rossi" in 063232122
 - Rimozione di una coppia chiave-valore
 - Rimuovi "Mario Verdi" dalla mia rubrica
 - Verifica di esistenza di una chiave
 - C'è "Chiara Scuri" nella mia rubrica?
 - Richiesta dell'insieme delle chiavi
 - Dammi i nomi di tutte le persone della mia rubrica

I Metodi Base di Map<K, V>

- **V put(K chiave, V valore) ;**
inserisce la coppia chiave-valore nella mappa;
 - se la chiave esiste già, allora il valore viene aggiornato e il metodo ritorna il valore vecchio
 - se la chiave non esiste viene inserita una nuova coppia, e il metodo ritorna `null`
- **V get(Object chiave) ;**
restituisce il valore associato alla chiave, `null` se la chiave non esiste nella mappa
- **V remove(Object chiave) ;**
rimuove la coppia associata alla chiave e lo ritorna il valore
- **boolean containsKey(Object chiave) ;**
verifica se la chiave è presente nella mappa

Metodi Bulk e Viste di Map<K, V>

- **void putAll (Map<K, V> mappa)**
inserisce nella mappa tutte le coppie della mappa
passata come parametro
- **void clear()**
elimina tutte le coppie dalla mappa
- **Set<K> keySet()**
restituisce in un insieme tutte le chiavi
- **Collection<V> values()**
restituisce in una collezione tutti i valori

Sommario

- Interface **Map<K, V>**
 - Operazioni principali
- Implementazioni di **Map<K, V>**
 - unicità delle chiavi

Mappe ed Insiemi

- Nel Java Collection Framework **Map<K,V>** e **Set<K>** si basano su concetti molti simili
 - ✓ In realtà l'implementazione di **HashSet<K>** si basa su una istanza di **HashMap<K, ?>**
- Abbiamo diverse implementazioni di **Map<K,V>**, e **SortedMap<K,V>**, rispettivamente:
 - **HashMap<K,V>**
 - **TreeMap<K,V>**
- I meccanismi per la definizione dei criteri di equivalenza sulle chiavi sono gli stessi già visti per **Set<K>**, **SortedSet<K>**

Mappe: Implementazioni

- Per definire i criteri di equivalenza esistono due meccanismi:
 - l'unicità delle chiavi in **HashMap<K, V>** è definita attraverso i metodi **hashCode()** e **equals()** della classe delle chiavi
 - l'unicità delle chiavi in **TreeMap<K, V>** è «indotta» dal criterio di ordinamento delle chiavi stesse:
 - Ordinamento «Naturale»: attraverso il metodo **compareTo()** delle chiavi (e le chiavi devono quindi implementare **Comparable<K>**), oppure
 - *Ordinamento* «Esterno»: tramite il metodo **compare()** di un comparatore esterno **Comparator<K>** (passato al costruttore della mappa)

Esercizio

- Scrivere una batteria di metodi di test per comprendere a fondo la semantica di tutti i metodi citati nelle slide precedenti con riferimento alle due implementazioni di

Map<K, V>

- **HashMap<K, V>**
- **TreeMap<K, V>**

Esempio: Rubrica

```
import java.util.*;
public class Rubrica {
    private Map<String,Integer> rubrica;

    public Rubrica() {
        this.rubrica = new HashMap<String,Integer>();
        // this.rubrica = new HashMap<>(); // solo da Java7...
    }

    public void inserisci(String nome, Integer numero) {
        this.rubrica.put(nome, numero);
    }

    public void rimuovi(String nome) {
        this.rubrica.remove(nome);
    }

    public Set<String> nomiInRubrica() {
        return this.rubrica.keySet();
    }

    public Integer dammiIlNumeroDi(String nome) {
        return this.rubrica.get(nome);
    }
}
```

...

Esempio: Rubrica (cont.)

```
...
public Integer dammiIlNumeroDi(String nome) {
    return this.rubrica.get(nome);
}

public Integer aggiornaNumero(String nome, Integer numero) {
    return this.rubrica.put(nome, numero);
}

@Override
public String toString(){
    StringBuilder str = new StringBuilder();
    str.append("-----\n");
    str.append("Rubrica\n");
    Set<String> nomi = this.rubrica.nomiInRubrica();
    for(String s : nomi) {
        str.append(s);
        str.append(": ");
        str.append(this.rubrica.dammiIlNumeroDi(s));
        str.append("\n");
    }
    str.append("-----\n");
    return str.toString();
}
```

Esempio: Rubrica (cont.)

```
... public static void main(String[] args) {
    Rubrica r = new Rubrica();
    String s1 = new String("Paolo"), s2 = new String("Fabio");
    String s3 = new String("Anna"), s4 = new String("Carla");

    r.inserisci(s1, 390112461); // inserisco Paolo->390112461 in rubrica
    r.inserisci(s2, 390108361); // inserisco Fabio->390108361 in rubrica
    r.inserisci(s3, 39062888); // inserisco Anna->39062888 in rubrica
    System.out.println(r.toString()); // stampo la rubrica
    // verifico se ho il numero di Carla
    Integer numeroCercato = r.dammiIlNumeroDi(s4);
    if (numeroCercato == null)
        System.out.println("Il numero di "+s4+" non esiste");
    else
        System.out.println("Il numero di "+s4+" è "+numeroCercato);

    r.rimuovi(s2); // tolgo i dati relativi a Fabio dalla rubrica

    // cambio il numero di Paolo (e mi faccio stampare il numero vecchio)
    Integer nuovoNumero = 39066777;
    Integer vecchioNumero = r.aggiornaNumero(s1, nuovoNumero);
    System.out.println("Aggiornato il numero di "+s1+
        " da "+vecchioNumero+" a "+nuovoNumero);

    System.out.println(r.toString()); // stampo la rubrica
}
}
```

Esercitazione:

Caso di Studio diadiala

- Possiamo usare le mappe per semplificare il codice del nostro studio di caso
- Possiamo gestire in maniera molto più semplice l'implementazione della classe **Stanza**, in particolare per ciò che riguarda l'associazione **direzione → stanzaAdiacente**
 - attualmente è implementata tramite due array

Esercitazione: Testing & Refactoring

1. Rieseguire i test già accumulati prima di ogni refactoring
2. Cambiare la rappresentazione interna di **Stanza**
3. Rieseguire gli stessi test e verificare che tutto continui a funzionare nonostante le modifiche (senza regressione)
 - In presenza di fallimenti, utilizzare i test per localizzare e rimuovere velocemente, ed a basso costo, i bug accidentalmente *appena* introdotti

Riscriviamo il Codice di Stanza

```
public class Stanza {
    private String descrizione;
    private List<Attrezzo> attrezzi;
    private Map<String, Stanza> stanzeAdiacenti;

    public Stanza(String descrizione) {
        this.uscite = new HashMap<>();
        this.attrezzi = new ArrayList<>();
        this.descrizione = descrizione;
    }

    public Stanza getStanzaAdiacente(String direzione) {
        return this.stanzeAdiacenti.get(direzione);
    }

    void impostaStanzaAdiacente(String direzione,
                                Stanza stanzaAdiacente) {
        this.stanzeAdiacenti.put(direzione, stanzaAdiacente);
    }
    ...
}
```

Esercizio Mappe

- Date le classi **Studente** e **Aula**

```
public class Studente {  
    private String matricola;  
    private String meseDiNascita;  
  
    public Studente(String matricola, String meseDiNascita) {  
        this.matricola = matricola;  
        this.meseDiNascita = meseDiNascita;  
    }  
  
    public String getMatricola()    { return this.matricola;          }  
  
    public String getMeseDiNascita() { return this.meseDiNascita;      }  
  
    @Override  
    public int hashCode()           { return this.matricola.hashCode(); }  
  
    @Override  
    public boolean equals(Object o) {  
        Studente that = (Studente)o;  
        return this.getMatricola().equals(that.getMatricola());  
    }  
}
```


Esercizio Mappe (cont.)

```
public class Aula {  
    private Set<Studente> studenti;  
  
    public Aula() {  
        this.studenti = new HashSet<Studente>();  
    }  
  
    public boolean addStudente(Studente studente) {  
        return this.studenti.add(studente);  
    }  
  
    public Map<String, List<Studente>> meseDiNascita2studenti() {  
        // scrivere il codice di questo metodo  
    }  
}
```

- Scrivere il codice del metodo **meseDiNascita2studenti()** :
 - ritorna una mappa che associa a ciascun mese una lista con tutti e soli gli studenti nati in quel mese

Esercizio Mappe (cont.)

- Esempio:

- Usiamo `<"00","gen">` per denotare un oggetto **Studente** di matricola "00" e mese di nascita gennaio
- se un'aula contiene l'insieme dei seguenti studenti:
`{<"00","gen">, <"11","gen">, <"54","ott">, <"24","mar">, <"32","mar">}`

allora il metodo `meseDiNascita2studenti()` deve restituire una mappa con le seguenti coppie chiave → valore :

`"gen" → {<"00","gen">, <"11","gen">}`

`"ott" → {<"54","ott">}`

`"mar" → {<"32","mar">, <"24","mar">}`

Una Soluzione

```
public Map<String, List<Studente>> meseDiNascita2studenti() {  
    List<Studente> tmp;  
    Map<String, List<Studente>> mappa;  
    mappa = new HashMap<String, List<Studente>>();  
  
    for(Studente studente : this.studenti){  
        tmp = mappa.get(studente.getMeseDiNascita());  
        if (tmp==null)  
            tmp = new ArrayList<Studente>();  
        tmp.add(studente);  
        mappa.put(studente.getMeseDiNascita(), tmp);  
    }  
    return mappa;  
}
```

Un'Altra Soluzione

```
public Map<String, List<Studente>> meseDiNascita2studenti() {  
    List<Studente> tmp;  
    Map<String, List<Studente>> mappa;  
    mappa = new HashMap<String, List<Studente>>();  
    for(Studente studente : this.studenti) {  
        tmp = mappa.get(studente.getMeseDiNascita());  
        if (tmp==null) {  
            tmp = new ArrayList<Studente>();  
            mappa.put(studente.getMeseDiNascita(), tmp);  
        }  
        tmp.add(studente);  
    }  
    return mappa;  
}
```

Un'Altra Soluzione Ancora!

```
public Map<String, List<Studente>> meseDiNascita2studenti() {  
    List<Studente> tmp;  
    Map<String, List<Studente>> mappa;  
    mappa = new HashMap<String, List<Studente>>();  
    for(Studente studente : this.studenti){  
        if (mappa.containsKey(studente.getMeseDiNascita())) {  
            tmp = mappa.get(studente.getMeseDiNascita());  
            tmp.add(studente);  
        }  
        else {  
            tmp = new ArrayList<Studente>();  
            tmp.add(studente);  
            mappa.put(studente.getMeseDiNascita(), tmp);  
        }  
    }  
    return mappa;  
}
```

Mappe, Liste, Insiemi

- Le mappe risultano spesso più comode di liste ed insiemi
 - ✓ N.B. non si può accedere all'elemento di un insieme se non disponendo di un suo duplicato!
- Riconsideriamo la classe **Borsa**: e la rappresentazione della collezione degli attrezzi nella borsa
- Confrontiamo le due alternative rappresentazioni:
 - Lista: `List<Attrezzo>`, VS
 - Mappa: `Map<String, Attrezzo>`,
`nomeAttrezzo → Attrezzo`
- N.B.: non è possibile memorizzare oggetti **Attrezzo** distinti ma con lo stesso nome
- L'accesso per nome risulta agevole

Esercizio: Mappe & Boxing/Unboxing (1)

```
public Map<String, Integer> countWordOccurrences(String text) {  
    Map<String, Integer> word2count = new HashMap<>();  
    Scanner scanner = new Scanner(text);  
  
    while (scanner.hasNext()) {  
        String word = scanner.next();  
        int count = word2count.get(word);  
        if (count==0) {  
            count = 1;  
        }  
        else {  
            count = count + 1;  
        }  
        word2count.put(word, count);  
    }  
    return word2count;  
}
```

✓ Dov'è l'errore ?

Esercizio: Mappe & Boxing/Unboxing (2)

- ✓ Suggerimento: il codice precedente, una volta compilato, è equivalente al seguente

```
public Map<String, Integer> countWordOccurrences(String text) {  
    Map<String, Integer> word2count = new HashMap<>();  
    Scanner scanner = new Scanner(text);  
  
    while (scanner.hasNext()) {  
        String word = scanner.next();  
        int count = word2count.get(word).intValue();  
        if (count==0) {  
            count = 1;  
        }  
        else {  
            count = count + 1;  
        }  
        word2count.put(word, new Integer(count));  
    }  
    return word2count;  
}
```


NavigableMap<K, V>: da Java 6

- In Java 6 le funzionalità offerte da **SortedMap<K, V>** sono state riconsiderate ed estese
- E' stata introdotta una specializzazione di **SortedMap<K, V>** denominata **NavigableMap<K, V>**
 - ✓ Implementata anche da **TreeMap<K, V>**
- Alcuni nuovi metodi risultano molto utili, ad es.:

```
NavigableMap<K, V> subMap(K fromKey,  
                           boolean fromInclusive,  
                           K toKey,  
                           boolean toInclusive)
```

- Valgono tutte le stesse considerazioni già fatte per **NavigableSet<E>** (>>)

NavigableMap<K, V> e Retrocompatibilità

- ✓ Alcune scelte sembrano discutibili. Sono state dettate dall'esigenza di mantenere la retro-compatibilità con le implementazioni e le interfacce già esistenti e diffuse

@Override

SortedMap<K> headMap (K toKey)

- Returns a view of the portion of this map whose keys are strictly less than *toKey*.

**NavigableMap<K> headMap (K toKey,
boolean inclusive)**

- Methods *subMap (K, K)*, *headMap (K)*, and *tailMap (K)* are specified to return *SortedMap* to allow existing implementations of *SortedMap* to be compatibly retrofitted to implement *NavigableMap*, but extensions and implementations of this interface are encouraged to override these methods to return *NavigableMap*. Similarly, *SortedMap.keySet ()* can be overridden to return *NavigableSet*.

Sovrautilizzo del JCF

- Un ultimo invito alla cautela nell'uso immotivamente complicato delle collezioni nel JCF
- Ad es., se vi ritrovate ad utilizzare mappe eccessivamente nidificate, come la seguente...

```
Map<String, Map<String, Set<Studente>>>
```

...avete di fronte un segnale forte, anzi fortissimo(!), che la distribuzione delle responsabilità tra le classi del vostro progetto necessita di essere riconsiderata

- ✓ State “evitando” la modellazione del dominio di interesse con *nuovi* tipi dedicati e vi state “rifugiando” nell'utilizzo di *vecchi* tipi già forniti (ad es. dal JCF)

Collezioni vs Distribuzione delle Responsabilità

- Redistribuendo le responsabilità del progetto (tipicamente aggiungendo anche nuove classi di dominio come quelle utili a modellare esplicitamente le associazioni tra oggetti, cfr. **Rubrica**)
 - i livelli di nidificazione delle **Map** si «abbattono»
 - alcuni **Set/List/Map** diventano variabili di istanza di classi di dominio e non tipi attuali di **Map** generiche eccessivamente nidificate e difficili da «visualizzare»
 - il testing diventa molto(!) più agevole

Ridistribuzione delle Responsabilità vs Collezioni Nidificate

- ✓ Esistono pertanto sostanziali motivi per preferire ad una struttura dati del tipo...

```
Map<String, Map<String, Set<Studente>>>;
```

...una serie di classi di dominio come ad es.:

- Classe **Ateneo** [con variabile di istanza]:

- `Map<String, Dipartimento> nome2dipartimento;`

- Classe **Dipartimento**

- `Map<String, CorsoDiStudio> nome2cds;`

- Classe **CorsoDiStudio**

- `Set<Studente> studentiIscritti;`

Modello di Dominio “Anemico”

- Spesso il «sintomo» precedente si affianca all'uso eccessivo di `java.lang.String` come strumento di modellazione universale

<http://c2.com/cgi/wiki/StringlyTyped>

- Sebbene ogni concetto sia rappresentabile con collezioni di `String` opportune, questo non deve distrarci dai vantaggi che scaturiscono dalla scelta dei giusti tipi per la rappresentazione dei concetti di dominio (vedi APS >>)
 - ✓ rimandare eccessivamente un'adeguata modellazione di dominio fa solo “lievitare” il conto che prima o poi si sarà costretti a pagare
 - ad es. il progetto passa ad altri che non riescono ad interpretare il codice: diventa più conveniente riscriverlo che comprenderlo

Programmazione Orientata agli Oggetti

Riflessione (Introspezione)

Sommario

- Introduzione
- **java.lang.Class<T>**
- Utilizzo della riflessione
- Unchecked Cast Warning
- Riflessione nello studio di caso
 - **FabbricaDiComandiRiflessiva**
- Equivalenza tra oggetti di una gerarchia
 - Esercizio

Introduzione

- La riflessione (o *introspezione*) è una interessante caratteristica del linguaggio Java
- Permette di scrivere codice che introspettivamente analizza (e modifica) lo codice compilato (nello stesso linguaggio Java)
- Del resto i tool che utilizziamo sono, in definitiva, programmi che manipolano altro codice Java
 - Spesso e volentieri anch'essi scritti in Java
 - ✓ Due esempi su tutti: Eclipse, JUnit

La Classe `java.lang.Class<T>` (1)

- Per ogni tipo `T` definito da un programma Java esiste un oggetto che descrive il tipo stesso
 - E che, in un certo senso, descrive anche il contenuto del corrispondente file `.class` contenente il codice oggetto
- Istanza della classe generica `java.lang.Class<T>`
 - Offre metodi che consentono di analizzare la definizione della classe
 - ✓ ad esempio è possibile ottenere l'elenco dei metodi, l'elenco dei supertipi, ecc.
- ✓ Consultare la documentazione javadoc di `Class<T>`

La Classe `java.lang.Class<T>` (2)

- La JVM fornisce forti garanzie su questi oggetti:
 - per ogni tipo `T` esiste un corrispondente ed unico esemplare istanza della classe `java.lang.Class<T>`
 - la creazione di tale oggetto avviene al momento del caricamento del file `.class`, ovvero quando «serve» per l'esecuzione
 - tipicamente perché ad es. viene creata una istanza del tipo oppure viene acceduto un suo campo statico oppure viene caricata un'altra classe che ne fa uso
 - in base alle specifiche della piattaforma, non tutti gli oggetti `Class<T>` necessari durante un'esecuzione devono essere creati sin dal suo inizio
 - ✓ anzi... alcune ottimizzazioni tendono ad evitarlo

Riferimenti ad Oggetti Class<T>

- Tutti i tipi (comunque definiti: classi, interface, ...) possiedono una variabile statica pubblica chiamata **class** che memorizza un riferimento al corrispondente oggetto di tipo **java.lang.Class**
- Esempio:

```
Class<it.uniroma3.diadia.comandi.ComandoVai> classeDiVai =  
    it.uniroma3.diadia.comandi.ComandoVai.class;
```

- Tutte gli oggetti ereditano un metodo di istanza (polimorfo) di **java.lang.Object** che restituisce un riferimento all'oggetto associato alla classe del suo *tipo dinamico*
- Ad esempio:

```
Class<ComandoVai> classeDiVai = new ComandoVai().getClass();
```

Interrogare Oggetti Class<T>

- Possiamo interrogare l'oggetto di tipo **Class<T>** per ottenere le proprietà del tipo **T**

- Es.: stampiamo l'elenco di tutti i metodi di un tipo

```
import java.lang.reflect.Method;
import java.util.iterator;
public class EsempioRiflessione {
    public static void main(String[] args) {
        Class<Iterator> classeDiIterator = Iterator.class;
        for (Method m : classeDiIterator.getMethods())
            System.out.println(m);
    }
}
```

- Possibile esecuzione (con Java 7):

```
$ java EsempioRiflessione
public abstract boolean java.util.Iterator.hasNext()
public abstract java.lang.Object java.util.Iterator.next()
```

Uso della Riflessione

- La riflessione viene applicata nello sviluppo di programmi con funzionalità complesse
- ✓ Ad es. in alcuni applicativi/framework di quelli “scritti in Java per gestire codice Java”
 - per lo sviluppo di IDE
 - in alcuni framework, come JUnit
 - il suo *runner* riesce a trovare i metodi annotati con **@Test** utilizzando proprio l’introspezione
 - per la persistenza dei dati (ORM >>>)
- Tuttavia, la riflessione risulta particolarmente utile anche in contesti meno sofisticati:
 - offre la possibilità di creare oggetti a partire dal nome della classe memorizzata in un oggetto **String**
 - ✓ quindi in effetti decidendo il tipo dell’oggetto creato a tempo dinamico e *non* già a tempo statico!

Creazione Introspettiva di Oggetti

- Esistono poche modalità per la creazione di oggetti e tutte comportano l'invocazione esplicita od implicita (cfr. letterali **String**, *Boxing*) di un costruttore con l'operatore **new**
- Ad esempio:

```
import it.uniroma3.diadia.comandi.*;
```

```
...
```

```
ComandoVai vai = new ComandoVai();
```

- Con la riflessione è possibile invece anche creare oggetti di tipo **T** invocando un metodo **newInstance()** offerto da un oggetto di tipo **Class<T>**, con questa segnatura:

```
T newInstance()
```

- Ad esempio:

```
Class<ComandoVai> classeDiVai = ComandoVai.class;
```

```
Comando vai = classeDiVai.newInstance();
```

Caricamento a Tempo di Esecuzione di Oggetti Class

- Un'altra interessante possibilità è offerta dalla classe **Class<T>** tramite il metodo:

```
public static Class<?> forName(String className)
```

permette di caricare una classe fornendo il suo nome completamente qualificato (come **String**) e restituisce il corrispondente oggetto **Class<?>**

- Esempio:

```
warning: [unchecked] unchecked cast
Class<ComandoVai> classeDiVai = (Class<ComandoVai>)
Class.forName("it.uniroma3.diadia.comandi.ComandoVai");
```

```
Class<ComandoVai> classeDiVai = (Class<ComandoVai>)
Class.forName("it.uniroma3.diadia.comandi.ComandoVai");
```

- ✓ Il caricamento presuppone un accesso al contenuto di **ComandoVai.class**
 - possono sollevarsi **ClassNotFoundException**

Creazione di Oggetti di Tipo Scelto a Tempo di Esecuzione

- Usando questi metodi possiamo creare oggetti a partire dal nome di una classe (deciso a tempo dinamico) contenuto in un oggetto **String**

- Esempio:

warning: unchecked cast

```
Class<ComandoVai> classeDiVai = (Class<ComandoVai>)  
    Class.forName("it.uniroma3.diadia.comandi.ComandoVai");  
Comando vai = (Comando)classeDiVai.newInstance();
```

- Il warning sul downcast serve ad avvertirci che il compilatore non è in grado di quanto non faccia con il seguente codice dove chiaramente solleva una **ClassCastException** durante l'esecuzione. Dove?...

```
Class<ComandoVai> classeDiVai = (Class<ComandoVai>)  
    Class.forName("it.uniroma3.diadia.Gioco");  
Comando vai = (Comando)classeDiVai.newInstance();
```

Unchecked Cast Warning

- Quando si lavora con l'introspezione, la tipizzazione diventa più lasca, e l'uso dei Java Generics più spesso mostra le limitazioni della loro tardiva (ma retrocompatibile) introduzione in Java 5+
- Il compilatore emettere degli avvertimenti legati ad un uso che riconosce troppo «permissivo» e privo di garanzie dei tipi
 - ✓ Quasi una «resa» consapevole da parte del compilatore
- La sua analisi statica dei tipi non riesce a fornire le garanzie tipiche dei linguaggi fortemente e staticamente tipati (ovvero l'assenza di **ClassCastException** durante l'esecuzione)
 - ✓ «avvertimento» non «errore»: la compilazione rimane possibile
- Un *Unchecked Cast Warning* viene emesso dal compilatore per informare che l'uso dei tipi non è abbastanza stringente da permettergli di offrire garanzie a tempo statico sull'assenza di errori di tipo a tempo dinamico

Unchecked Cast Warning (cont.)

- Il problema è esacerbato dalla decisione di lasciar convivere le versioni pre-generics delle classi con quelle propriamente generiche per la retrocompatibilità

- ✓ **java.lang.Class** non generica esiste da sempre

- L'implementazione dei Java Generics, in definitiva, si basa sull'ampliamento dei controlli sui tipi a tempo statico ed un meccanismo di trasformazione dei tipi generici verso quelli non generics, i soli disponibili a tempo dinamico (vedi *erasure <<*)

```
List lPreG = new ArrayList();  
lPreG.add(new Integer(3));
```

Tempo statico (compilazione):
warning: unchecked cast

```
ArrayList<String> listG = (ArrayList<String>) lPreG;  
String s = listG.get(0);
```

Tempo dinamico: ClassCastException
java.lang.ClassCastException:
Integer cannot be cast to String

- Se il contesto non permette una tipizzazione forte (come purtroppo spesso accade con quest API) le garanzie offerte vengono rilassate (agli stessi livelli considerati “normali” nell’era pre-generics)

Riflessione - Studio di Caso (1)

- Possiamo scrivere codice in cui vengono creati dinamicamente oggetti a partire dal nome della loro classe
- Un esempio nel nostro studio di caso: possiamo creare gli oggetti **Comando** derivando il nome della classe del sottotipo concreto da utilizzare a partire dall'istruzione digitata dall'utente

“vai nord” → `it.uniroma3.diadia.ambienti.ComandoVai`

- Scriviamo una classe che implementi **FabbricaDiComandi** sfruttando la riflessione
 - ✓ Finalmente si elimina uno dei più “resistenti” esemplari di «codice-fisarmonica» presenti sin dalla versione base

Riflessione - Studio di Caso (2)

- L'idea è quella di costruire il nome della classe che processa un comando a partire dal nome del comando contenuto nell'istruzione digitata. Ad esempio:
"vai" → "it.uniroma3.diadia.comandi.ComandoVai"
- Quindi, si prevedono i seguenti passi:
 - (1) Lettura istruzione
"vai sud"
 - (2) Costruzione nome della classe
"it.uniroma3.diadia.comandi.ComandoVai"
 - (3) Caricamento dell'oggetto **Class** con **Class.forName()**
 - (4) Istanziamo un oggetto di tale classe con **newInstance()**
- Un piccolo refactoring preparatorio...

FabbricaDiComandi

- Ridefiniamo l'interfaccia **FabbricaDiComandi** come segue (aggiungendo la clausola **throws** all'unico metodo):

```
public interface FabbricaDiComandi {  
    public Comando costruisciComando(String istruzione)  
        throws Exception; // (>> eccezioni)  
}
```
- Così non siamo costretti a gestire subito le diverse tipologie di eccezioni che possono venire sollevate
- ✓ *Dopo la lezione sulle eccezioni* riconsidereremo questa scelta (>>)
 - ✓ al momento meglio non distrarci dall'introspezione
 - ✓ trascuriamo anche la gestione dei comandi non validi

FabbricaDiComandiRiflessiva

```
public class FabbricaDiComandiRiflessiva implements FabbricaDiComandi {  
    public Comando costruisciComando(String istruzione) throws Exception {  
        Scanner scannerDiParole = new Scanner(istruzione); // es. 'vai sud'  
        String nomeComando = null; // es. 'vai'  
        String parametro = null; // es. 'sud'  
        Comando comando = null;  
  
        if (scannerDiParole.hasNext())  
            nomeComando = scannerDiParole.next(); // prima parola: nome del comando  
        if (scannerDiParole.hasNext())  
            parametro = scannerDiParole.next(); // seconda parola: eventuale parametro  
  
        StringBuilder nomeClasse  
            = new StringBuilder("it.uniroma3.diadia.comandi.Comando");  
        nomeClasse.append( Character.toUpperCase(nomeComando.charAt(0)) );  
        // es. nomeClasse: 'it.uniroma3.diadia.comandi.ComandoV'  
        nomeClasse.append( nomeComando.substring(1) );  
        // es. nomeClasse: 'it.uniroma3.diadia.comandi.ComandoVai'  
        comando = (Comando)Class.forName(nomeClasse.toString()).newInstance();  
        comando.setParametro(parametro);  
  
        return comando;  
    }  
}
```

Osservazioni

- Il metodo `newInstance()` invoca il costruttore *no-arg*, che quindi *deve* essere visibile
 - (Finalmente!) il vero motivo per
 - il metodo `setParametro(String)` nella interface **Comando**
 - le classi dei comandi munite di un costruttore *no-args*
 - Al contrario, prevedendo costruttori diversi di comando in comando ci saremmo privati della possibilità di crearli tutti allo stesso modo mediante l'uso del semplice metodo `newInstance()`
 - ✓ questo presuppone l'assenza di argomenti
- N.B. esistono (da Java 5) anche meccanismi per invocare costruttori con parametri
 - ✓ rispetto ai nostri obiettivi formativi, tali tecnicismi non aggiungono molto (vedere i javadoc...)

Sommario

- Introduzione
- **java.lang.Class<T>**
- Utilizzo della riflessione
- Unchecked Cast Warning
- Riflessione nello studio di caso
 - **FabbricaDiComandiRiflessiva**
- **Equivalenza tra oggetti di una gerarchia**
 - Esercizio

Riflessione, Criteri di Equivalenza & Polimorfismo

- La scelta di un opportuno criterio d'equivalenza tra due oggetti risulta significativamente più complicato quando questi fanno parte di una gerarchia di tipi
- E' un problema frequente!
 - ✓ succede ogni qualvolta si crea un **Set<E>** con **E** polimorfo
 - ✓ ed ogni qualvolta si crea una **SetMap<k, V>** con **K** polimorfo
- La riflessione permette però di codificare con relativa facilità alcuni controlli sul tipo dinamico di un oggetto
- Il metodo polimorfo **this.getClass()** restituisce un riferimento al singleton associato con il tipo dinamico dell'oggetto «corrente»
 - Permette il controllo sul tipo dinamico di un oggetto

Equivalenza e Polimorfismo

- Supponiamo ad esempio di considerare una popolazione di due sottoinsiemi disgiunti di persone: studenti e docenti
 - ad es. in un dipartimento universitario
- Modelliamo questa situazione introducendo due classi minimali: **Persona** e **Studiante**:

```
public class Persona {
    private String nome;
    public Persona(String nome) { this.nome = nome; }
    public String getNome()      { return this.nome; }
}

public class Studiante extends Persona {
    private String matricola;
    public Studiante(String nome, String matricola) {
        super(nome);
        this.matricola = matricola;
    }
    public String getMatricola() { return this.matricola; }
}
```

Un Criterio di Equivalenza

- I docenti sono ben pochi ed il loro ricambio è molto lento
 - *Assunzione di Dominio*: il nominativo sarà un identificatore per tutta la vita dell'applicativo
- Gli studenti sono molti di più ed il loro ricambio è più veloce
 - *Assunzione di Dominio*: serve una matricola per identificarli perché altrimenti presto vi saranno casi di omonimie

```
public class Persona { ...
    @Override public boolean equals(Object o) {
        if (o==null) return false;
        return this.getNome().equals(((Persona)o).getNome());
    }
    @Override public int hashCode() {
        return this.getNome().hashCode();
    }
}

public class Studente extends Persona { ...
    @Override public boolean equals(Object o) {
        if (o==null) return false;
        return this.getMatricola().equals(((Studente)o).getMatricola());
    }
    @Override public int hashCode() {
        return this.getMatricola().hashCode();
    }
}}
```

Set di Oggetti Polimorfi

- L'attuale definizione dei due metodi, sembra essere molto naturale, e si presta alla scrittura di alcuni metodi (ad es. nella classe **Dipartimento**)
- Ad esempio:
 - **Set<Persona> getAllDocentiAfferenti()**od anche
 - **Set<Studente> getAllStudentiIscrittiAdUnCDS()**
- ✓ Questi metodi non mescolano i due tipi di oggetti e risultano di facile scrittura
- E se invece scrivessimo il metodo
 - **Set<Persona> getAllPersoneDelDipartimento()**che restituisce l'insieme di tutte le persone, sia docenti (di tipo dinamico **Persona**) che studenti (di tipo dinamico **Studente**)???

Problemi della Definizione «Naturale»

- L'attuale definizione dei due metodi `equals()` ed `hashCode()`, sebbene sembri la più naturale, risulta invece problematica non appena si creano collezioni che mischino oggetti di entrambi i tipi
 - ✓ Perché a ben vedere, assume confronti solo tra oggetti del medesimo tipo dinamico (o sottotipi per il p.d.s.)
- ✓ Non è una vera relazione di equivalenza in quanto asimmetrica:
 - `p.equals(s) → true`
 - `s.equals(p) → ClassCastException!`
- Come cambiarle per far funzionare il seguente codice?

```
Persona p = new Persona("Antonio");
Studiante s = new Studiante("Antonio", "54321");
Set<Persona> persone = new HashSet<>();
persone.add(p);
persone.add(s);
assertEquals(???, persone.size());
```

Controlli sul Tipo Dinamico

- La riflessione permette di codificare con relativa facilità alcuni controlli sul tipo dinamico di un oggetto
- Basta aggiungere il tipo dinamico all'elenco delle proprietà confrontate
 - ✓ meglio premettere i controlli ai downcast, come segue:

```
@Override
```

```
public boolean equals(Object o) {
```

```
    if (o==null || o.getClass()!=this.getClass()) return false;
```

```
    .../* resto del metodo per confrontare le altre proprietà
```

```
        di due oggetti aventi lo stesso tipo dinamico... */
```

```
}
```

```
@Override
```

```
public int hashCode() {
```

```
    return this.getClass().hashCode()+ ... /* altri hash da sommare... */;
```

```
}
```

Il Tipo Dinamico nei Criteri di Equivalenza (1)

- Aggiungiamo il controllo sul tipo dinamico: oggetti di due tipi diversi non sono mai equivalenti

```
public class Persona { ...
    @Override public boolean equals(Object o) {
        if (o==null || o.getClass()!=this.getClass()) return false;
        Persona that = (Persona)o;
        return this.getNome().equals(that.getNome());
    }
    @Override public int hashCode() {
        return this.getClass().hashCode()+this.getNome().hashCode();
    }
}

public class Studente extends Persona { ...
    @Override public boolean equals(Object o) {
        if (o==null || o.getClass()!=this.getClass()) return false;
        Studente that = (Studente)o;
        return this.getMatricola().equals(that.getMatricola());
    }
    @Override public int hashCode() {
        return this.getClass().hashCode()+this.getMatricola().hashCode();
    }
}
```


Il Tipo Dinamico nei Criteri di Equivalenza (2)

- Asimmetria risolta. Dati:

```
Persona p = new Persona("Antonio");
```

```
Studiante s = new Studiante("Antonio", "54321");
```

ora risulta:

- `p.equals(s)` → `false`
- `s.equals(p)` → `false`!

- ✓ Una soluzione percorribile *solo* nelle seguenti ipotesi
 - i docenti (**Persona**) sono separati dagli studenti (**Studiante**)
 - non interessa creare oggetti **Persona** per modellare studenti (per i quali usiamo *sempre* e *solo* oggetti della classe **Studiante**)
- Se ora però, allarghiamo la popolazione, sino a coprire l'intero **Ateneo**, risulta presto necessario risolvere le omonimie tra docenti, ad es. mediante il C.F. (necessario comunque per gli aspetti fiscali)
- Per non costringere la segreteria a raccogliere anche i C.F. degli studenti (assumiamo non strettamente necessario) preferiamo creare una nuova sottoclasse di **Persona**, chiamata **Docente**, piuttosto che aggiungere il campo direttamente nella superclasse

Gerarchia Totale e Disgiuntiva

```
public class Docente extends Persona {
    private String cf;
    public Docente(String nome, String cf) {
        super(nome);
        this.cf = cf;
    }
    public String getCF() { return this.cf; }
    @Override
    public boolean equals(Object o) {
        if (o==null || o.getClass()!=this.getClass()) return false;
        Docente that = (Docente)o;
        return this.getCF().equals(that.getCF());
    }
    @Override
    public int hashCode() {
        return this.getClass().hashCode()+this.getCF().hashCode();
    }
}
```

- Gerarchia *totale*: in questo modo docenti e studenti avranno un tipo esplicitamente dedicato e non vengono mai istanziati oggetti che non siano un sottotipo di **Persona**
 - vedremo che si potrebbe marcare la classe **Persona** come **abstract** solo per esserne sicuri già a tempo di compilazione (>>)
- Una parte del codice può agevolmente essere messa a fattor comune

Refactoring di equals() & hashCode()

```
public abstract class Persona { ...
    @Override public boolean equals(Object o) {
        if (o==null || o.getClass()!=this.getClass()) return false;
        Persona that = (Persona)o;
        return this.getNome().equals(that.getNome());
    }
    @Override public int hashCode() {
        return this.getClass().hashCode() + this.getNome().hashCode();
    }
}

public class Docente extends Persona {...
    @Override
    public boolean equals(Object o) {
        return super.equals(o) && this.getCF().equals(((Docente)o).getCF());
    }
    @Override
    public int hashCode() {
        return super.hashCode() + this.getCF().hashCode();
    }
}

public class Studente extends Persona {...
    @Override
    public boolean equals(Object o) {
        return super.equals(o) && this.getMatricola().equals(((Studente)o).getMatricola());
    }
    @Override
    public int hashCode() {
        return super.hashCode() + this.getMatricola().hashCode();
    }
}
```

Gerarchia Parziale

- E se invece avessimo perseguito la strada senza la classe **Docente**, ovvero aggiungendo il campo per il C.F. a **Persona**, lasciandolo comunque nullo per tutti gli studenti?
- Si tratta di una gerarchia *parziale*: esistono oggetti che sono istanza della superclasse ma **NON** della sottoclasse

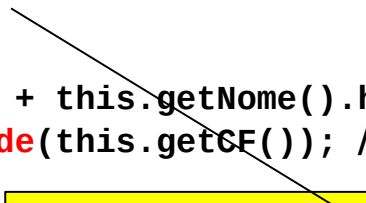
```
public class Persona {  
    private String nome;  
    private String cf;  
    public Persona(String nome, String cf) {  
        this.nome = nome  
        this.cf = cf;  
    }  
    public String getNome() { return this.nome; }  
    public String getCF() { return this.cf; }  
}  
  
public class Studente extends Persona {  
    private matricola;  
    public Studente(String nome, String matricola) {  
        super(nome, null); // Studenti senza C.F.  
        this.matricola = matricola;  
    }  
    public String getMatricola() { return this.matricola; }  
}
```

Gerarchia Parziale con Campi Null

- Bisogna cambiare anche i metodi `equals()` e `hashCode()` di **Persona**, mostrando il sostanziale forte accoppiamento tra tutte le implementazioni di questi metodi nella gerarchia...

```
public class Persona {...
    @Override public boolean equals(Object o) {
        if (o==null || o.getClass()!=this.getClass()) return false;
        Persona that = (Persona)o;
        return    this.getNome().equals(that.getNome())
                && java.util.Objects.equals(this.getCF(), that.getCF()); // N.B. gestione null
    }
    @Override public int hashCode() {
        return this.getClass().hashCode() + this.getNome().hashCode()
                + java.util.Objects.hashCode(this.getCF()); // N.B. gestione null
    }
}

public class Studente extends Persona {...
    @Override
    public boolean equals(Object o) {
        return super.equals(o) && this.getMatricola().equals(((Studente)o).getMatricola());
    }
    @Override
    public int hashCode() {
        return super.hashCode() + this.getMatricola().hashCode();
    }
}
```



Vedi javadoc! Consultare anche:
java.util.Objects.hash(Object... values)

I Problemi Non Finiscono Mai...

- Sinora tutte le varianti ipotizzavano che la diversità del tipo dinamico fosse sempre un fattore decisivo per concludere la NON equivalenza di due oggetti
- ✓ In alcune situazioni potrebbe non essere desiderabile
- Alcuni docenti sono ex studenti dello stesso ateneo...
 - ✓ Come rilevare l'equivalenza dei corrispondenti oggetti di tipo distinto? Ad es. per contare il numero di persone mai transitate, a qualsiasi titolo, in un ateneo?
 - al momento non è affatto possibile perché il nome non è un identificatore e gli studenti non possiedono C.F.
- Ma se invece raccogliessimo il C.F. anche degli studenti...
 - ✓ Come converrebbe cambiare la gerarchia delle persone?

I Problemi ... (cont.)

- Come converrebbe cambiare la gerarchia delle persone per poter contare il numero esatto di persone mai transitate in ateneo?
 - ✓ rimuoviamo tutti metodi `equals()` ed `hashCode()` nella gerarchia tranne che nella superclasse **Persona**, ove insiste SOLO sul C.F.!
- Ma se poi volessimo scrivere un metodo
`static public Set<Studente> iscrittiAppello()`
senza tediare gli studenti con un modulo di iscrizione che raccoglie il C.F. ma solamente la matricola?
- Se per questi oggetti **Studente** volessimo sovrascrivere il criterio di equivalenza basato sul C.F. di **Persona**
- In sostanza non saremmo più liberi di farlo senza contraddire l'ipotesi che tutte le persone abbiano un C.F.
 - ipotesi oramai *cablata* nella definizione del criterio di equivalenza in **Persona**
 - e nemmeno sovrascrivibile dalle altre classi della gerarchia (!)

Conclusioni: Equivalenza & Polimorfismo (1)

- La definizione di un criterio di equivalenza deve tenere conto di tutta la gerarchia
 - ✓ rappresenta un *intrinseco* e *forte* accoppiamento tra porzioni correlate ma distinte di codice
- Non esiste un automatismo per definire criteri di equivalenza per tipi polimorfi: rimane una scelta di modellazione del dominio
- *Preferire soluzioni semplici*: la necessità di soluzioni complicate e/o la definizione di molteplici criteri di equivalenza sulle stesse classi nasconde sottostanti problemi di modellazione e di scelta dei tipi
- Ad esempio:
 - ✓ Introdurre una nuova e distinta classe **Prenotazione** che ospiti sia riferimenti agli oggetti **Studente** che un criterio di equivalenza (definito localmente) basato sulla matricola
 - Si risolverebbero facilmente i problemi appena discussi
 - ✓ In un certo senso, spostiamo il criterio di equivalenza desiderato più «vicino» al suo immediato utilizzatore e «fuori» dalla gerarchia di tipi

Conclusioni: Equivalenza & Polimorfismo (2)

- Il meccanismo di controllo del tipo dinamico va utilizzato o meno a seconda che sia richiesto dal criterio ritenuto più adatto nel dominio
- In funzione del dominio, possono esistere situazioni in cui conviene
 - confrontare il tipo dinamico
 - *oppure*, non controllare il tipo dinamico
 - *oppure*, controllarlo o meno in funzione di alcune informazioni sugli oggetti coinvolti (>>)!
- La definizione del più opportuno criterio di equivalenza tra oggetti è un importante problema di modellazione
- E' pertanto impossibile definire dei criteri che risultino generali, ancor meno in presenza di gerarchie di tipi

Conclusioni: Equivalenza & Polimorfismo (3)

- Indicazioni per evitare i più comuni errori:
 - laddove si utilizza **Set<E>**, **Map<K, V>** chiedersi *sempre* se per tutti i tipi *attuali* di **E** e **K** sia *necessario* definire un opportuno criterio di equivalenza
 - Direttamente mediante la coppia di metodi **equals()**/**hashCode()**
 - Indirettamente definendo un criterio di ordinamento totale (naturale e/o esterno, metodi: **compareTo()** e/o **compare()**) che induce anche un criterio di equivalenza
 - Se alla luce di questo ragionamento, lo stesso tipo dovrebbe supportare diversi criteri di equivalenza incompatibili, rivedere la modellazione
 - Valutare se non sia il caso di introdurre altri tipi per modellare alcuni concetti di dominio al momento «sfuggiti»

Esercizio

- Si riconsideri l'esercitazione
P00-polimorfismo-esercitazione (<<)
- L'interface **Forma** è successivamente implementata da **Quadrato**, **Rettangolo**, **GruppoDiForme**
- Esercizio: munire tutte le forme geometriche di un opportuno criterio di equivalenza che tenga conto della posizione, del colore, oltre che della tipologia di ciascuna forma

Esercizio (Suggerimento)

- Alcuni quesiti da porsi immediatamente:
 - i. Quando due oggetti **Rettangolo** sono equivalenti?
 - ii. Quando due oggetti **Cerchio** sono equivalenti?
 - iii. Ha senso confrontare un **Rettangolo** vs un **Cerchio**? **Rettangolo** vs **GruppoDiForme**? **Cerchio** vs **GruppoDiForme**?
 - iv. Quando due oggetti **GruppoDiForme** sono equivalenti?

Esercizio (cont.)

1. Scrivere una batteria di test di unità per ciascuna classe concreta (**Cerchio/Rettangolo/GruppoDiForme**) per codificare quali sono i comportamenti attesi dal criterio di equivalenza tra forme omogenee
2. L'interfaccia **Forma** va modificata?
3. I metodi **equals()**/**hashCode()** in quali classi vanno introdotti?
4. Controllare che i test abbiano successo
5. Aggiungere altri test di unità coinvolgendo come fixture insiemi di una sola tipologia di forma:
 - **Set<Cerchio>**
 - **Set<Rettangolo>**
 - **Set<GruppoDiForme>**

Esercizio (cont.)

6. Aggiungere dei test per verificare il corretto funzionamento dei metodi **`equals()`**/**`hashCode()`** quando sollecitati su forme di tipo dinamico diverso
7. Modificare il codice per soddisfare i test di prima
8. Aggiungere i test per verificare il comportamento di una collezione di diversi tipi di forme utilizzando come fixture un insieme
 - **`Set<Forma>`**
9. Come modificare il codice per soddisfare i nuovi test?

Esercizio (cont.)

10. (Continuare solo dopo la lezione sulle classi astratte)

Introdurre una nuova classe astratta **AbstractForma**

11. Rifattorizzare quanto più codice possibile facendo uso della nuova classe

12. Confermare che i test di unità già sviluppati continuino ad aver successo dopo il refactoring

13. Cambiare i requisiti e modificare di conseguenza il codice secondo un nuovo criterio di equivalenza che includa anche la classe **Punto** come caso particolare di **Forma**

- Si considerino tutte le forme, indipendentemente dal tipo, come ad es. i cerchi (e i rettangoli, rispett.), equivalenti ai punti nel casi degeneri (rispett. raggio e base/altezza pari a zero)

Approfondimenti

- Per evitare gli errori più comuni nella definizione di un criterio di equivalenza, e per un approfondimento sul legame tra il principio di sostituzione ed i criteri di equivalenza tra tipi polimorfi:

How to Write an Equality Method in Java

<http://www.artima.com/lejava/articles/equality.html>

by Martin Odersky, Lex Spoon, and Bill Venners

June 1, 2009

Programmazione Orientata agli Oggetti

Gestione delle Eccezioni

Sommario

- Introduzione alle eccezioni
- Gestione eccezioni in Java
 - Lato Client
 - Lato Server
- La gerarchia delle eccezioni Java
 - *Checked vs Unchecked* Exception
- Testing ed eccezioni
- Conclusioni

Alcune Frequenti Cause di «Anomalie»

- Talvolta le esecuzioni possono risultare «anomale». Alcune delle cause più frequenti:
 - ✓ Implementazione non corretta
 - l'applicazione non è conforme alle specifiche
 - Cfr. *Analisi e Progettazione del Software*
 - ✓ Errori logici
 - Cfr. *Fondamenti di Informatica*
- A causa di queste anomalie un oggetto può trovarsi in uno stato *inconsistente*, ovvero non più rappresentativo delle istanze nel dominio che intende modellare

Non Sempre Errori del Programmatore

- Alcune situazioni anomale possono essere causate dall'ambiente «esterno» al programma:
 - URL o nome file errato
 - Hard Disk pieno
 - Interruzione di rete
 - Mancanza di permessi appropriati per una risorsa esterna (ad es. file, connessioni verso DBMS)
 - ...
- Situazioni «eccezionali» che è però possibile gestire per contenerne gli effetti negativi, e migliorare le possibilità di recupero
- ✓ N.B. eccezionali non significa *inverosimili*!

Eccezioni

- Uno strumento supportato da alcuni linguaggi specificatamente per gestire situazioni anomale
 - Consentono l'implementazione del codice di gestione di situazioni anomale e/o «eccezionali»
 - difficile ottenere soluzioni eleganti per gestire situazioni *eccezionali*
 - per definizione, sono tutte diverse una dall'altra!
-
- La gestione del caso anomalo, va inquadrata allo stesso modo del caso ordinario?
 - Come evitare di offuscare il codice per la gestione del caso *ordinario* con il codice per la gestione dei casi *eccezionali* ?

Oggetti «Client» e «Server» (1)

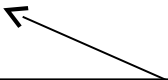
- Programma OO: descrivono l'interazione tra oggetti «client» e oggetti «server»
 - Gli oggetti *server* offrono servizi (metodi)
 - Vengono erogati (invocati) su richiesta degli oggetti *client*
- E' lecito porsi almeno queste domande:
 - ✓ se un oggetto server «fallisce», come comunica l'anomalia all'oggetto client?
 - ✓ a fronte di un fallimento del server:
 - come se ne accorge il client?
 - cosa può fare e come recupera la situazione?

Oggetti «Client» e «Server» (2)

```
class ClasseClient {  
    private ClasseServer server;  
    public void metodoClient() {  
        int i;  
        i = server.metodoServer(0);  
    }  
}
```



```
class ClasseServer {  
    public int metodoServer(int arg) {  
        ...  
    }  
}
```



Metodo in cui si possono
verificare situazioni *anomale*

Eccezioni in Java

- La gestione delle eccezioni nella piattaforma Java prevede dei meccanismi attraverso i quali è possibile:
 - «sollevare» eccezioni:
 - per segnalare ai client la situazione anomala interrompendo il normale flusso di esecuzione
 - «catturare» eccezioni:
 - per implementare eventuali azioni di gestione dell'anomalia
 - «imporre» la gestione:
 - il client è forzato a prendere in carico esplicitamente la gestione dell'eccezione sollevate dal server

Eccezioni vs Oggetti

- Idea generale: le eccezioni sono oggetti
 - estendono `java.lang.Throwable`
 - ✓ vedi gerarchia dei tipi delle eccezioni Java (>>)
- Possono essere «sollevati» / «lanciati» esplicitamente utilizzando l'istruzione **throw**:
`throw new IllegalArgumentException()`
- è possibile (consultare i javadoc):
 - allegare messaggi diagnostici all'eccezione
 - incapsulare eventuali *altre* eccezioni all'origine dell'eccezione sollevata >>
 - conoscere lo *stack-trace* dell'esecuzione che ha sollevato l'eccezione

java.lang.Throwable

Costruttori:

Throwable() *Constructs a new throwable with null as its detail message.*

Throwable(String message)

Constructs a new throwable with the specified detail message.

Throwable(String message, Throwable cause)

Constructs a new throwable with the specified detail message and cause.

Alcuni dei metodi più significativi:

Throwable getCause()

Returns the cause of this throwable or null if the cause is nonexistent...

String getMessage()

Returns the detail message string of this throwable.

StackTraceElement[] getStackTrace()

*...programmatic access to the stack trace ... by **printStackTrace()***

void printStackTrace()

Prints this throwable and its backtrace to the standard error stream.

Es.: IllegalArgumentException

- Utilizzata ad indicare argomenti che violano il *contratto di un metodo*
- Ad es. usata da: `Collections.nCopies()`:
`public static <T> List<T> nCopies(int n, T o)`
*returns: an immutable list consisting of n copies of the specified object. **throws**: IllegalArgumentException - if n < 0*
- I costruttori ricalcano quelli di `java.lang.Throwable`:

`IllegalArgumentException(String s)`

Constructs an `IllegalArgumentException` with the specified detail message.

`IllegalArgumentException(String message, Throwable cause)`

Constructs a new exception with the specified detail message and cause.

`IllegalArgumentException(Throwable cause)`

Constructs a new exception with the specified cause ...

Lato Server:

Lanciare un'Eccezione con throw

- Per indicare che si sono rilevate anomalie, si lancia un'eccezione; per lanciare un'eccezione:
 - Prima viene costruito l'oggetto `Throwable`:
`e = new IllegalArgumentException("eta>0");`
 - Poi l'oggetto viene “lanciato” con l'istruzione:
`throw e;`
 - Spesso, più direttamente:
`throw new IllegalArgumentException("eta>0");`
- I tipi di eccezioni che un metodo lancia possono divenire parte integrante della sua stessa segnatura (>>) mediante la clausola **throws**
 - ✓ nei javadoc basta scrivere: `@throws ExType <descrizione>`

Istruzione throw & Clausola throws

```
class ClasseClient {  
    private ClasseServer server;  
    public void metodoClient() {  
        int i;  
        i = server.metodoServer(0);  
    }  
}  
/**  
 * @throws IllegalArgumentException  
 */  
class ClasseServer {  
    public int metodoServer(int arg)  
        throws IllegalArgumentException {  
        if (arg<=0) {  
            throw new IllegalArgumentException("arg>0");  
        }  
        ...}  
}
```

Equivale ad affermare che
questo metodo può lanciare
una IllegalArgumentException



Effetti di una Eccezione

- Il metodo che lancia una eccezione finisce prematuramente
 - ✓ senza eseguire l'istruzione **return**
- Non viene restituito nessun valore
- Il controllo (lato client) NON ritorna al punto di chiamata (da parte del client) del metodo del server
- ✓ A tutti gli effetti, viene interrotta ed abbandonata la *normale* sequenza di esecuzione a favore di una sequenza di esecuzione *eccezionale*

Eccezioni che «Bucano» gli Stack

- Le eccezioni «bucano» lo stack, ovvero causano:
 - la terminazione del metodo che le lanciano (e quindi la rimozione dallo stack del relativo *r.d.a.*, cfr. FdI)
 - la riattivazione del metodo (client) invocante, al quale viene in effetti restituito il controllo, che può esercitare in vari modi (>>)
- Le eccezioni, se non gestite, arrivano sino al metodo da cui l'intera esecuzione è cominciata
 - ovvero, per programmi eseguiti da riga di comando, sino al metodo `main()`
 - se non viene gestita nemmeno al livello iniziale, l'esecuzione del programma da parte della JVM abortisce
 - la JVM termina: stampa *stack-trace* e *messaggio di errore allegato all'eccezione*

Lato Client: Gestione dell'Eccezione

- Se un metodo client chiama un metodo server che lancia una eccezione, allora può/deve «gestirla»
- Esistono due alternative per la gestione dal lato metodo client di una eccezione sollevata dal lato metodo server:
 - I. *Presa in carico diretta*: cattura e gestione dell'eccezione
 - II. *Rinuncia alla gestione diretta e propagazione verso il metodo chiamante*: si rimanda la gestione al livello immediatamente precedente (nelle nidificazioni delle chiamate di metodo)

Cattura e Gestione Diretta (1)

- Per catturare un'eccezione
 - le chiamate ad un metodo che lancia una eccezione che si vuole catturare devono essere effettuate all'interno di un blocco `try` {...}
 - l'eventuale eccezione viene catturata e gestita in un blocco `catch`(...) {...}

```
try {  
    <blocco codice che può sollevare un'eccezione>  
} catch (ExceptionType e) {  
    <blocco codice di gestione eccezione>  
}
```

Cattura e Gestione Diretta (2)

```
class ClasseClient {  
    ClasseServer server;  
    public void metodoClient() {  
        int i;  
        try {  
            i = server.metodoServer(0);  
        }  
        catch (IllegalArgumentException e) {  
            ... «codice gestione eccezione e»  
        }  
        ...  
    }  
}
```

Chiamata ad un metodo che
dichiara di lanciare
IllegalArgumentException

Codice di gestione di una eccezione
IllegalArgumentException
«catturata» nella variabile locale **e**

```
class ClasseServer {  
    public int metodoServer(int arg)  
        throws IllegalArgumentException {  
        if (arg<=0)  
            throw new IllegalArgumentException("arg>0");  
        ...  
    }  
}
```

Pericoloso Errore Metodologico

- Eliminare le eccezioni ignorandole!
 - Siccome sulla riga `o.metodo()` si osserva una *N.P.E.*, allora si è tentati di risolvere il problema con

```
try {  
    o.metodo();  
} catch (NullPointerException npe) {  
    /* ignora e vai dritto */  
}
```
 - Classica “toppa peggio del buco”:
 - non solo non si è affatto risolto il problema: lo si sta anche rendendo ancora più difficile da individuare!
 - ✓ sarà più arduo capirne la vera origine dell’errore con un’eccezione in più a “nasconderla”!
-

Cattura e Gestione Diretta: FabbricaDiComandi

- In precedenza per lo studio di caso: nella gerarchia con radice in **FabbricaDiComandi** abbiamo cambiato la segnatura del metodo **costruisciComando()**

- ✓ (<<) per non distogliere l'attenzione dall'*introspezione*

- Ora il metodo non dichiara più di lanciare eccezioni

```
public interface FabbricaDiComandi {  
    public Comando costruisciComando(String istruzione)  
        throws Exception;  
}
```

- Gestiamo *direttamente* le eventuali eccezioni sollevate
 - nel corpo del metodo di **FabbricaDiComandiRiflessiva**
 - ✓ risulta migliorata la distribuzione delle responsabilità?
-

FabbricaDiComandiRiflessiva

```
public class FabbricaDiComandiRiflessiva implements FabbricaDiComandi {
    @Override
    public Comando costruisciComando(String istruzione) {
        Scanner scannerDiParole = new Scanner(istruzione);
        String nomeComando = null;
        String parametro = null;
        Comando comando = null;

        if (scannerDiParole.hasNext())
            nomeComando = scannerDiParole.next(); // prima parola: nome del comando
        if (scannerDiParole.hasNext())
            parametro = scannerDiParole.next(); // seconda parola: eventuale parametro
        try {
            String nomeClasse = "it.uniroma3.diadia.comandi.Comando";
            nomeClasse += Character.toUpperCase(nomeComando.charAt(0));
            nomeClasse += nomeComando.substring(1);
            comando = (Comando) Class.forName(nomeClasse).newInstance();
            comando.setParametro(parametro);
        } catch (Exception e) {
            comando = new ComandoNonValido();
            this.io.mostraMessaggio("Comando inesistente");
        }
        return comando;
    }
}
```

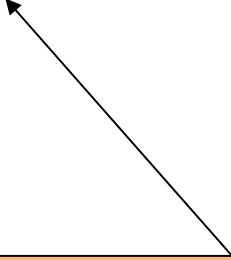
Propagazione al Chiamante (1)

- In alternativa, il metodo client a sua volta può dichiarare che *risolleva* l'eccezione lanciata dal metodo server di cui fa uso
- In effetti si limita a «propagarla» al *suo* metodo chiamante, non gestendolo direttamente
- Quando questo tipo di gestione è opportuna?
 - se esisteva quella clausola era proprio perché si consigliava/richiedeva una gestione esplicita
 - ✓ d'altra parte talvolta il chiamante *diretto* può essere in una posizione meno favorevole per una gestione corretta e per una diagnostica più efficace rispetto ad un chiamante *indiretto*

Propagazione al Chiamante (2)

```
class ClasseClient {  
    ClasseServer server;  
    public void metodoClient()  
        throws IllegalArgumentException {  
        int i;  
        i = server.metodoServer(0);  
    }  
}
```

Le eccezioni sollevate nel corpo
del metodo sono propagate al suo
metodo invocante, a livello più alto



```
class ClasseServer {  
    public int metodoServer(int arg)  
        throws IllegalArgumentException {  
        ...  
    }  
}
```

Propagazione al Chiamante (3)

- Quando questo tipo di gestione è opportuna?
 - se esisteva quella clausola era proprio perché si consigliava/richiedeva una gestione esplicita
 - ✓ d'altra parte talvolta il chiamante *diretto* può essere in una posizione meno favorevole per una gestione corretta e per una diagnostica più efficace rispetto ad un chiamante *indiretto*
- ✓ Si pensi ad un errore dovuto a un input errato che si manifesta in una chiamata di metodo molto “profonda”: può essere conveniente rileggere l'input ad un livello molto più alto

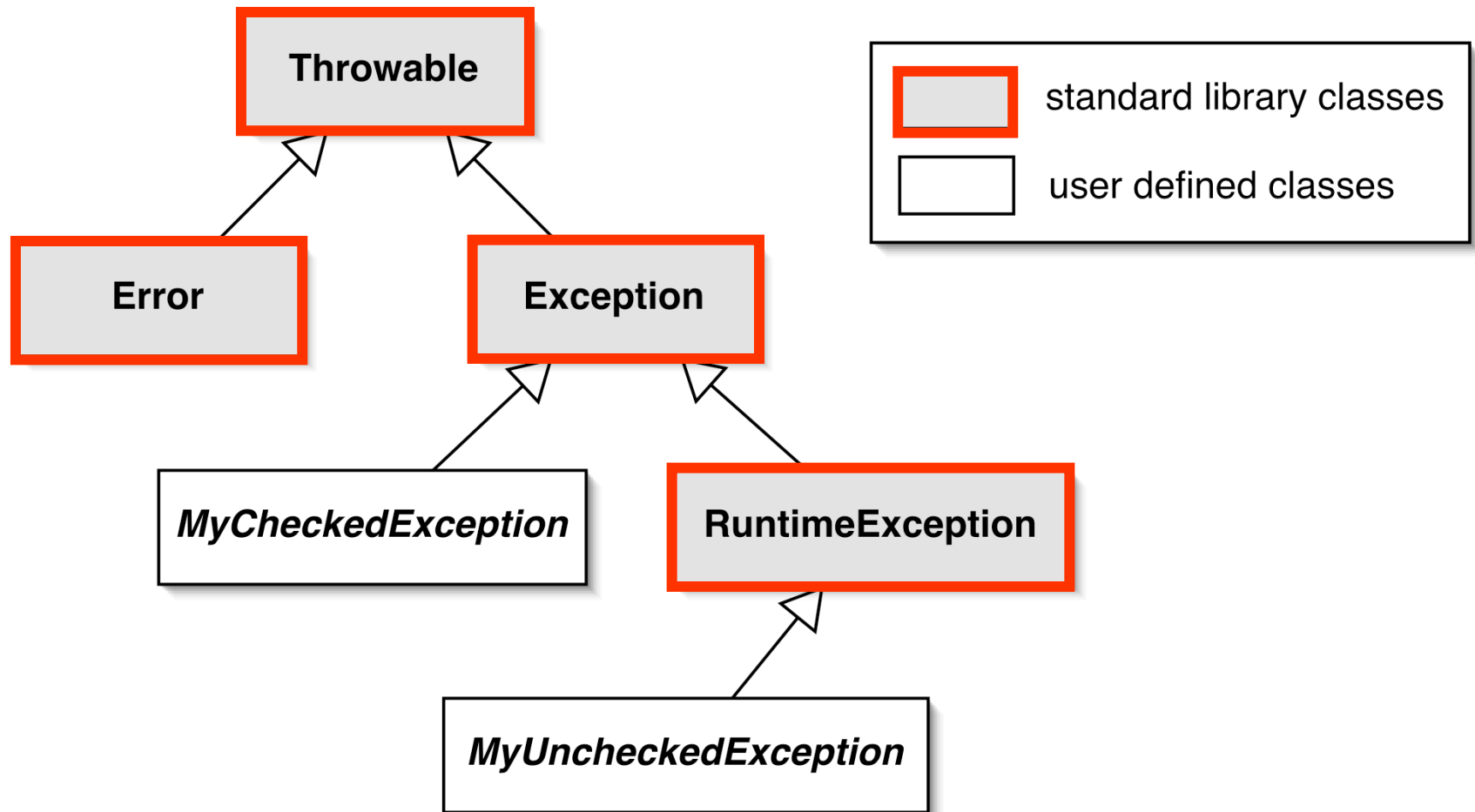
Definizione di Nuovi Tipi di Eccezione

- Parte integrante della normale attività di progettazione
- Fa parte della definizione dei *tipi del dominio*
 - ✓ forniscono al chiamante informazioni diagnostiche tenendo conto del dominio applicativo
 - ✓ chiariscono la libreria da cui si è originato il problema

```
public class DefLabirintoNotFoundException extends Exception {  
    public DefLabirintoNotFoundException(String message) {  
        super(message) ;  
    }  
}
```

- Classi che definiscono nuove tipologie di eccezione devono estendere `Exception` o `RuntimeException`, e quindi essere sottotipi di `java.lang.Throwable`

Gerarchia delle Eccezioni Java (1)



Gerarchia delle Eccezioni Java (2)

- *Error*
 - veri e propri errori, non recuperabili, dovuti a fattori esterni (ad es. esaurimento memoria)
- *Checked exception*
 - sottoclasse di `Exception`
 - il client **deve** gestire esplicitamente questo tipo di eccezioni: il compilatore altrimenti si rifiuta di compilare
 - si usa quando è possibile e conveniente «forzare» una gestione lato client
 - un esempio: `FileNotFoundException`
- *Unchecked exception*
 - sottoclasse di `RuntimeException`
 - client non obbligato (durante la compilazione) a gestirle
 - un esempio: `IllegalArgumentException`

Checked vs Unchecked Exception (1)

- Nel caso di *checked exception*, il client è obbligato a gestire l'eccezione
 - il programmatore **deve** specificare il blocco **try-catch** che cattura e gestisce l'eccezione, oppure propargarla esplicitamente
 - ✓ si forza la gestione lato client
 - in caso contrario il codice non compila
`Main.java:7: error: unreported exception
FileNotFoundException; must be caught or declared to
be thrown`
 - un esempio ben noto: `FileNotFoundException`
- Nel caso di una *unchecked exception*
 - il compilatore non esegue verifiche
 - se non gestite esplicitamente, sono automaticamente propagate al metodo chiamante
 - un ben noto esempio: `NullPointerException`

Checked vs Unchecked Exception (2)

- I metodi che lanciano una *checked* exception
 - **devono** dichiararlo esplicitamente
- I metodi che lanciano una *unchecked* exception
 - **possono** dichiararlo esplicitamente
- *Sintatticamente* si usa in entrambi i casi sempre la stessa clausola **throws** ad integrazione della dichiarazione della segnatura di un metodo
- ✓ *Semanticamente* solo le checked exception sono considerate parte integrante della segnatura
 - per rendersene conto, basta dichiararle nei metodi di una interfaccia
 - Solo le checked possono far fallire un override

Diatriba *Checked* vs *Unchecked* (1)

- Le checked exception hanno generato vere e proprie diatribe sulla loro opportunità di esistere, con analisi dei benefici e dei costi ben diverse, di volta in volta, ma sempre opinabili
- Sta di fatto che capita di scrivere:

```
public static final URL createURL(String url) {  
    try {  
        return new URL(url);  
    } catch (MalformedURLException e) {  
        throw new RuntimeException(e, "Cannot create URL from: " + url);  
    }  
}
```

...per incapsulare la *checked* `MalformedURLException` dentro ad una *unchecked* `RuntimeException` e non essere più obbligati alla gestione esplicita imposta dalla classe `java.net.URL` che fa ampio uso di checked exception

Diatriba *Checked* vs *Unchecked* (2)

- La più recente `java.net.URI` (da Java 4) recepisce la questione: lascia `java.net.URISyntaxException` checked ma allo stesso tempo offre un metodo factory statico per nasconderla dietro una unchecked:

```
public URI(String str) throws URISyntaxException
```

```
public static URI create(String str)  
Creates a URI by parsing the given string.
```

*This convenience factory method works as if by invoking the `URI(String)` constructor; **any `URISyntaxException` thrown by the constructor is caught and wrapped in a new `IllegalArgumentException` object, which is then thrown.***

This method is provided for use in situations where it is known that the given string is a legal URI, for example for `URI` constants declared within in a program, and so it would be considered a programming error for the string not to parse as such. The constructors, which throw `URISyntaxException` directly, should be used situations where a `URI` is being constructed from user input or from some other source that may be prone to errors.

Unchecked Exception: *Generate vs Programmatiche*

- `ArrayIndexOutOfBoundsException`
- `ClassCastException`
- `NullPointerException`
- Sono tutti esempi di ben note unchecked exception generate direttamente anche dalla JVM
- Non c'è modo (e/o interesse) a distinguerle da quelle generate programmaticamente, ad es.:
`throw new NullPointerException();`

try-catch Multipli

- Un metodo di un oggetto server potrebbe lanciare diversi tipi di eccezione
 - corrispondenti a diversi tipi di anomalie
- Il client può gestire separatamente queste situazioni

```
try {  
    server.metodoServer();  
}  
catch (ExType1 e) {  
    «Gestione eccezione in e di tipo ExType1»  
}  
...  
catch (ExTypeN e) {  
    «Gestione eccezione in e di tipo ExTypeN»  
}
```

try-catch Multipli: Esempio

- Se viene generata un'eccezione:
 - il primo (e solo il primo) blocco `catch` il cui argomento è associabile al tipo di eccezione sollevata viene attivato
 - le istruzioni del suo blocco `catch` sono eseguite
 - l'eccezione è considerata servita

```
try {  
    comando = (Comando)Class.forName(nomeClasseComando).newInstance();  
} catch (InstantiationException e) {  
    /* possibile causa: lo sviluppatore si è dimenticato di aggiun-  
    gere un costruttore no-args in una sottoclasse di Comando */  
} catch (IllegalAccessException e) {  
    /* possibile causa: lo sviluppatore si è dimenticato di rendere  
    pubblico il costruttore no-args di un sottoclasse di Comando */  
} catch (ClassNotFoundException e) {  
    /* possibile causa: comando ignoto - errore digitazione utente */  
}
```

Attenzione: P.d.S. & Clausole catch

- ✓ Quindi un supertipo finisce per «nascondere» i suoi sottotipi che lo seguono lungo nella catena di blocchi

catch

```
try {  
    server.metodoServer();  
}  
catch (Exception e) { ←  
    // <gestione di una generica exception>  
    ...  
}  
catch (IOException e) {  
    // <gestione generica I/O exception>  
    ...  
}  
catch (FileNotFoundException e) {  
    // <gestione di una file-not-found exception>  
    ...  
}
```

qualsiasi
eccezione
verrebbe
catturata
dal primo
catch!

**FileNotFoundException è sottotipo di
IOException!**

Ordinamento delle Clausole catch

- ✓ Ordinare sempre le clausole catch dal tipo più specifico catturato a quello meno specifico

```
try {  
    server.metodoServer();  
}  
catch (FileNotFoundException e) {  
    // <gestione di una file-not-found exception>  
    ...  
}  
catch (IOException e) {  
    // <gestione generica I/O exception>  
    ...  
}  
catch (Exception e) {  
    // <gestione di una generica exception>  
    ...  
}
```



Catch Disgiuntive

- In Java 7 furono introdotte sintassi abbreviate per alleviare l'eccessiva verbosità dei costrutti inerenti la gestione delle eccezioni:

```
try {  
    « codice che solleva diverse eccezioni »  
} catch (IllegalArgumentException |  
        IOException |  
        IllegalStateException e ) {  
    « codice comune per la gestione »  
    « di tutti i tipi di eccezione »  
}
```

- Qual'è il tipo statico di `e`?

try-with-resources Statement (1)

- Da Java 7 è stata introdotta anche una sintassi abbreviata per la gestione di «risorse»
- *Risorsa*: in questo contesto si intende un qualsiasi oggetto con protocollo di utilizzo che preveda il rilascio esplicito:
 - «richiesta oggetto/risorsa»
 - «utilizzo»
 - «rilascio oggetto/risorsa»
- Il rilascio prevede l'invocazione del metodo `close()`
 - Cfr. interface `java.io.Closeable`
- utili per interagire con risorse limitate e gestite dal S.O. che le API Java «nascondono» dietro un oggetto, ad. es. **File**

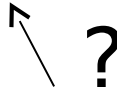
try-with-resources Statement (2)

- E' lecito scrivere (da Java 7+):

```
static String readFistLine(String path) throws IOException {  
    try (BufferedReader br =  
        new BufferedReader(new FileReader(path))) {  
        return br.readLine();  
    }  
}
```

- Al posto del più verboso e poco leggibile ma corretto (per quanto concerne la gestione delle risorse):

```
static String readFirstLine(String path) throws IOException {  
    BufferedReader br = null;  
    try {  
        br = new BufferedReader(new FileReader(path));  
        return br.readLine();  
    } finally {  
        if (br!=null) br.close();  
    }  
}
```

 ?

Clausola `finally`: Utilizzo

```
try {  
    ...  
}  
catch (Exception e) {  
    ...  
}  
finally {  
    « blocco di codice sempre eseguito »  
}
```


Clausola `finally`: Semantica

- Il meccanismo di gestione delle eccezioni è completato dal blocco `finally {...}`
- Il blocco `finally` viene eseguito *sempre*, anche se l'eccezione non è stata rilevata
 - ✓ Attenzione, *sempre* è *sempre*: anche se nel blocco `try` o `catch` c'è una istruzione `return` che viene eseguita
- Tipicamente serve a garantirsi il rilascio di risorse costose (come file, o connessioni ad un DBMS) *anche* in presenza di situazioni anomale
 - ✓ previene la “perdita di risorse” («resource leak»)
 - ✓ le eccezioni sollevate potrebbero impedire il corretto rilascio delle risorse già richieste

Linee Guida Gestione Eccezioni (1)

- ✓ Se il metodo incontra una condizione anomala che non sa gestire, allora dovrebbe lanciare un'eccezione
- ✓ Evitare l'uso di eccezioni solo fornire risultati
- ✓ Se un metodo scopre che il client ha violato il suo contratto (ad es. inviandogli argomenti errati), meglio che sollevi una unchecked exception, ad es. una `IllegalArgumentException`

Linee Guida Gestione Eccezioni (2)

- ✓ Se un metodo non riesce a rispettare il contratto, allora sollevi una checked o una unchecked exception
 - *Punto oggetto di accese discussioni: meglio usare solo eccezioni unchecked? (come in C# ???)*
- ✓ Se si lancia una eccezione per una anomalia, che si ritiene il programmatore del client *possa voler gestire*, allora si sollevi una checked exception
 - *(discussioni come sopra)*
- ✓ Si definisca una nuova classe exception (o si riusi una già esistente) per ciascun distinto tipo di condizione anomala che potrebbe spingere un metodo a lanciare eccezioni
 - Un progetto spesso possiede una «capostipite»

Eccezioni e Testing

- JUnit (4+) supporta direttamente la scrittura di test-case per verificare che un metodo sollevi eccezioni
- Utile ogni qualvolta si vuole verificare che il proprio codice sappia gestire correttamente situazioni «anomale»
- Il test ha successo se e solo se l'eccezione specificata nell'annotazione `@Test` viene sollevata:

```
@Test(expected=NoSuchElementException.class)
```

```
public void testMinOfEmptyCollectionNotDefined() {  
    final List<Comparable<Object>> empty =  
        Collections.emptyList();  
    Collections.min(empty);  
}
```

Consultare javadoc:
`Collections.min()` su
collezione vuota *DEVE* sollevare
`NoSuchElementException`

Conclusioni

- Java quando fu introdotto innovò (ad es. rispetto al C++) i meccanismi offerti per la gestione delle situazioni anomale
- In Java le anomalie sono gestite tramite oggetti particolari chiamati *eccezioni*
- Il fatto che dopo molti anni i nuovi linguaggi di programmazione non abbiano introdotto alternative valide, tutto sommato è una garanzia sulla bontà delle scelte fatte
 - le diatribe *checked vs unchecked* possono quasi essere viste anche come una “conferma”

Conclusioni

- La gestione delle anomalie risulta parte integrante, tutt'altro che trascurabile, della normale attività di programmazione
- La modellazione dei diversi tipi di anomalie è parte integrante della normale attività di modellazione del dominio
- I framework a supporto della scrittura di test di unità, come JUnit 4+, permettono di verificare il comportamento di metodi che sollevano eccezioni

Programmazione Orientata agli Oggetti

Classi Astratte e
Costanti Enumerative

Contenuti

- Classi astratte
 - Metodi astratti
 - Classi astratte o interface?
- Metodi e Classi finali
- Costanti enumerative
 - Pre-Java 5
 - Java 5 Enum
 - Quando usarli
 - Enum e Collezioni

Contenuti

- **Classi astratte**
 - Metodi astratti
 - Classi astratte o interface?
- Metodi e Classi finali
- Costanti enumerative
 - Pre-Java 5
 - Java 5 Enum
 - Quando usarli
 - Enum e Collezioni

Introduzione (1)

- Abbiamo visto come l'estensione può essere uno strumento utile per il riuso del codice
- Le classi estese ereditano anche l'implementazione della classe base
 - La classe estesa può avere variabili di istanza e metodi aggiuntivi a quelli ereditati dalla classe base
 - La classe estesa può ridefinire metodi della classe base
- La classe estesa è un sottotipo della classe base, quindi, vale il principio di sostituzione:
 - ✓ possiamo usare istanze di una classe estesa al posto delle istanze della classe base

Introduzione (2)

- Abbiamo visto che anche le interface favoriscono il riuso del codice
- Grazie al principio di sostituzione possiamo scrivere codice con metodi polimorfi
 - accettano come parametro formale un riferimento ad un tipo (statico) astratto
 - il parametro attuale sarà un riferimento ad un sottotipo (dinamico) concreto di tale tipo
- Talvolta, classi diverse che implementano una medesima interface possono voler condividere anche una significativa porzione dell'implementazione

Motivazioni

- Nella pratica, capita di voler definire classi base pensate **solo** per essere estese e non per essere direttamente istanziate
 - ✓ allo scopo di evitare duplicazioni nel codice
 - ✓ Per favorire la qualità *interna* (<<)
- Queste classi contengono una parziale implementazione da condividere con le sottoclassi
 - variabili di istanza
 - implementazione di alcuni metodi
 - cosiddetti metodi «concreti»
 - segnatura di altri metodi
 - cosiddetti metodi «astratti»
- I metodi che rimangono privi di implementazione nella classe base possono poi essere «completati» nelle classi derivate

Classi Astratte

- Una classe astratta contiene una definizione parziale della implementazione
- Una classe astratta non può essere istanziata, ma possono essere istanziate le classi (concrete) che la estendono
- Ovviamente vale la relazione sottotipo-supertipo, e quindi il principio di sostituzione
 - ✓ i riferimenti alle istanze delle classi che estendono una classe astratta possono essere usate quando è atteso un riferimento ad una istanza della classe base
- ✓ *Analogie* con le interface:
 - le interface non possono essere istanziate, ma possono essere istanziati oggetti di classi che le implementano
 - vale il principio di sostituzione
- ✓ *Differenze* con le interface:
 - le interface NON contengono implementazione (non più vero da Java 8+!!!)

Esempio (Caso di Studio)

- Supponiamo di voler introdurre nel nostro gioco dei personaggi (es. mostri, maghi, ecc.)
- I personaggi sono nelle stanze del gioco (per semplicità accontentiamoci di un singolo personaggio per stanza)

```
import ...
public class Stanza {
    private String nome;
    private Map<String, Stanza> uscite;
    private Map<String, Attrezzo> nome2attrezzo;

    private AbstractPersonaggio personaggio;
    ... ..
    public void setPersonaggio(AbstractPersonaggio personaggio) {
        this.personaggio = personaggio;
    }

    public AbstractPersonaggio getPersonaggio() {
        return this.personaggio;
    }
    ...
}
```

Esempio (Caso di Studio)

- I personaggi hanno un nome (una stringa) ed una descrizione (una stringa)
- I personaggi possono rispondere al saluto del giocatore
 - introduciamo il comando `saluta` per salutare il personaggio presente nella stanza
- I personaggi possono agire
 - introduciamo il comando `interagisci`: provoca l'interazione del giocatore con il personaggio presente

Esempio: i «Personaggi»

- Possiamo introdurre diverse tipologie di personaggi. Ad esempio potremmo avere:
 - *Mago*: possiede un attrezzo che può donare
 - *Strega*: se interagiamo con una strega questa ci trasferisce in una stanza tra quelle adiacenti. Siccome è permalosa:
 - se non l'abbiamo ancora salutata, ci «trasferisce» nella stanza adiacente che contiene meno attrezzi
 - altrimenti in quella che contiene più attrezzi
 - *Cane*: morde! Ogni morso diminuisce i CFU del protagonista

Esempio: Caratteristiche Generali

- Tutte le tipologie di personaggi condividono una parte della implementazione
 - le variabili per memorizzare nome e descrizione
 - metodi accessori
 - costruttori
 - il codice che gestisce la risposta al saluto
- Tutte le tipologie di personaggi hanno un metodo (astratto) per modellare l'azione:

```
abstract public String agisci(Partita partita) ;
```

- Tuttavia non ha senso definire sino al dettaglio il comportamento di un generico personaggio
- ✓ il comportamento dipende infatti dalle specificità di ogni particolare personaggio

Esempio (cont.)

- Ogni personaggio ha un comportamento specifico
 - il mago ci dona un attrezzo
 - la strega ci sposta in una stanza
 - il cane morde
- Il codice che implementa l'azione è specifico del personaggio: ogni tipologia di personaggio ha la propria, specifica, implementazione
- In altri termini, il personaggio è definibile prescindendo da alcuni suoi dettagli che lo differenziano da tutti gli altri
 - alcune sue proprietà possono essere «concrete» (fornite anche di una implementazione)
 - altre solo «astratte» (ovvero senza l'implementazione)

Esempio: la Classe Astratta Personaggio (1)

```
package it.uniroma3.personaggi;
import ...

public abstract class AbstractPersonaggio {
    private String nome;
    private String presentazione;
    private boolean haSalutato;

    public AbstractPersonaggio(String nome, String presentaz) {
        this.nome = nome;
        this.presentazione = presentaz;
        this.haSalutato = false;
    }

    public String getNome() {
        return this.nome;
    }

    public boolean haSalutato() {
        return this.haSalutato;
    }
}
```

Esempio: la Classe Astratta Personaggio (2)

```
public String saluta() {
    StringBuilder risposta =
        new StringBuilder("Ciao, io sono ");
    risposta.append(this.getNome()+".");
    if (!haSalutato)
        risposta.append(this.presentazione);
    else
        risposta.append("Ci siamo gia' presentati!");
    this.haSalutato = true;
    return risposta.toString();
}
```

```
abstract public String agisci(Partita partita);
```

```
@Override
public String toString() {
    return this.getNome();
}
```

```
}
```

La Classe ComandoInteragisci

```
package it.diadia.comandi;
import ...
public class ComandoInteragisci implements Comando {

    private static final String MESSAGGIO_CON_CHI =

                                "Con chi dovrei interagire?...";

    private String messaggio;
    private IO io;

    @Override
    public void esegui(Partita partita) {
        AbstractPersonaggio personaggio;
        personaggio = partita.getStanzaCorrente().getPersonaggio();
        if (personaggio!=null) {
            this.messaggio = personaggio.agisci(partita);
            io.mostraMessaggio(this.messaggio);
        } else io.mostraMessaggio(MESSAGGIO_CON_CHI);
    }

    public String getMessaggio() {
        return this.messaggio;
    }

    @Override
    public void setParametro(String parametro) {}
}
```

Esempio: la Classe Mago

```
public class Mago extends AbstractPersonaggio {

    private static final String MESSAGGIO_DONO = "Sei un vero simpaticone, " +
        "con una mia magica azione, troverai un nuovo oggetto " +
        "per il tuo borsone!";

    private static final String MESSAGGIO_SCUSE = "Mi spiace, ma non ho piu' nulla...";

    private Attrezzo attrezzo;

    public Mago(String nome, String presentazione, Attrezzo attrezzo) {
        super(nome, presentazione);
        this.attrezzo = attrezzo;
    }

    @Override
    public String agisci(Partita partita) {
        String msg;
        if (this.attrezzo!=null) {
            partita.getStanzaCorrente().addAttrezzo(this.attrezzo);
            this.attrezzo = null;
            msg = MESSAGGIO_DONO;
        }
        else {
            msg = MESSAGGIO_SCUSE;
        }
        return msg;
    }
}
```

Classi e Metodi Astratti

- Le classi astratte:
 - utilizzano il modificatore `abstract` nella definizione
 - non possono essere istanziate *direttamente* (sebbene devono avere, anche solo implicitamente, almeno un costruttore!)
 - definiscono implementazioni poi ereditate dalle sottoclassi
- I metodi astratti:
 - utilizzano `abstract` nella segnatura
 - non possiedono corpo
- La presenza anche di un solo metodo astratto rende necessaria la dichiarazione della classe come astratta
 - ma una classe può essere dichiarata astratta anche se non possiede alcun metodo astratto (e non potrà essere istanziata)
- Le sottoclassi, per essere concrete ed istanziabili, devono completare l'implementazione sovrascrivendo *tutti* i metodi astratti (oppure, a loro volta, devono dichiararsi astratte...)

Classi Astratte

- Servono a definire implementazioni parziali che verranno completate nelle classi concrete che le estendono
- Rispetto alle interface ?
 - come le interface non possono essere istanziate
 - diversamente dalle interface riportano una implementazione parziale e vengono estese
- Molto usate nei framework, ma la progettazione di framework va (ben!) oltre gli obiettivi formativi di questo corso...

Classi Astratte vs Interface

- Classe astratta
 - *pro:* permette di riutilizzare l'implementazione
 - *contro:* limita fortemente le possibilità di estensione
 - ✓ la gerarchia delle implementazioni Java è *lineare*
- Interface
 - *pro:* nessun limite di estensione
 - ✓ ereditarietà multipla delle interfacce
 - *contro:* nessun meccanismo di riutilizzo del codice
- Come scegliere?
 - preferire le interface, meno vincolanti
 - Valutare l'alternativa solo in presenza di codice e soprattutto logica da riutilizzare: per es. con i framework

Testing di Classi Astratte (1)

- Qual è l'obiettivo del testing di classi astratte?
 - ✓ testare i metodi implementati direttamente nella classe astratta
- Convienne definire una sua estensione concreta solo ai fini del testing
 - ✓ fornisce una implementazione concreta (e *minimale*) dei metodi astratti per permettere la creazione dell'oggetto da testare
 - ✓ ad es. implementa i metodi astratti restituendo costanti
- Si testano i metodi concreti della superclasse

Testing di Classi Astratte (2)

```
public class FakePersonaggio extends AbstractPersonaggio {  
  
    public FakePersonaggio(String nome, String presentazione) {  
        super(nome, presentazione);  
    }  
  
    @Override  
    public String agisci(Partita partita) {  
        return "done";  
    }  
  
}
```

- Scriviamo la classe di test **AbstractPersonaggioTest** che verifichi il comportamento dei metodi concreti ereditati da **AbstractPersonaggio**

Esercizi (1)

- Definire la classe **Cane**, che estende la classe **AbstractPersonaggio**: quando interagiamo con un cane, questi morde, togliendoci CFU!
- Definire il comando *interagisci*
- Introdurre **AbstractComando** per eliminare i metodi replicati nelle implementazioni dell'interface **Comando** e relativi alla gestione del nome e del parametro del comando
 - Utilizzarla anche per ospitare l'oggetto **IO** per la gestione dell'input/output. Come offrire l'accesso alle sottoclassi?
- ✓ N.B. quest'ultimo utilizzo delle classi astratte risulta utile per esercitarsi con l'uso dello strumento ma è in *nitido* contrasto con la raccomandazione di limitare il loro utilizzo al caso di sufficiente logica duplicata

Esercizi (2)

- Definire la classe **Strega** come riportato nella descrizione precedente
 - ✓ è anche un buon esercizio sulle collezioni...
- Definire ed organizzare i test-case per tutte le classi appena introdotte; rifattorizzare i test-case già prodotti per le classi modificate

Contenuti

- Classi astratte
 - Metodi astratti
 - Classi astratte o interface?
- **Metodi e Classi finali**
- Costanti enumerative
 - Pre-Java 5
 - Java 5 Enum
 - Quando usarli
 - Enum e Collezioni

Metodi e Classi final (1)

- Per evitare che un metodo possa essere ridefinito si usa il modificatore **final**

```
public class ClasseConMetodoFinale {  
    public final int nonMiPoteteSovrascrivere() {...}  
}
```

- In questo modo il metodo non può essere ridefinito nelle classi che estendono **ClasseConMetodoFinale**
 - viene sollevato un errore a tempo di compilazione

```
public class EstendeClasseConMetodoFinale  
    extends ClasseConMetodoFinale {  
    @Override  
    public int nonMiPoteteSovrascrivere() {...} //ERRORE  
}
```

Metodi e Classi final (2)

- E' possibile dichiarare **final** intere classi

```
public final class NonMiPoteteEstendere {  
    ...  
}
```

- La classe **NonMiPoteteEstendere** non può essere estesa

✓ anche in questo caso errore a tempo di compilazione

```
public class EstendeClasseFinale  
    extends NonMiPoteteEstendere { // ERRORE  
    ...  
}
```

- Molte classi della libreria standard sono dichiarate **final**, un esempio per tutti: **java.lang.String**

Contenuti

- Classi astratte
 - Metodi astratti
 - Classi astratte o interface?
- Metodi e Classi finali
- **Costanti enumerative**
 - Pre-Java 5
 - Java 5 Enum
 - Quando usarli
 - Enum e Collezioni

Java Enum e Costanti Enumerative

- I metodi e le classi **final** sono stati inizialmente motivati da questioni legate alla sicurezza
- Hanno conosciuto un altro diffuso utilizzo, ma forse meno noto perché nascosto da un consistente strato di zucchero sintattico
- *Costanti Enumerative*: valori costanti raccolti in insiemi finiti e stabili. Es.:
 - i 12 mesi dell'anno ed i 7 giorni della settimana
 - i 7 *tipi* di tetramino nel gioco Tetris!
 - i 4 punti cardinali (>>)
- I Java Enum (da Java 5) consentono di modellare efficacemente insiemi di «costanti enumerative» usando opportunamente
 - Classi astratte
 - Classi **final**
 - Costruttori privati

Enumerazioni (pre Java 5)

- Per capire il loro funzionamento, ed apprezzarne i vantaggi, conviene descrivere come venivano realizzati prima di Java 5, (versione in cui ne fu introdotto il supporto)
 - ✓ In effetti è la pratica tuttora utilizzata in C
- In assenza di un meccanismo dedicato, la modellazione di costanti enumerative ripiegava spesso nell'uso di generiche costanti di un tipo «preso in prestito»
- Ad esempio, mediante semplici numeri interi:

```
public interface Direzione {  
    // Da NORD in senso orario  
    static final public int NORD    = 0;  
    static final public int OVEST   = 1;  
    static final public int SUD     = 2;  
    static final public int EST     = 3;  
}
```

Enumerazioni (pre Java 5)

- Principale vantaggio: semplicità ed immediatezza
- Importanti conseguenze negative, fra tutte una tipizzazione piuttosto lasca:

```
int direzione = 5; // COMPILA! Ma non dovrebbe...
```

- Ne derivano molti svantaggi, ad es. Metodi con signature di non immediata interpretazione:

```
public int direzioneOpposta(int arg) { ... }
```

Enumerazioni Tipate (pre Java 5)

- Soluzioni più articolate prevedono infatti almeno la creazione di un tipo dedicato, per rendere i controlli a tempo statico e la tipizzazione più stringenti
- Una soluzione diffusamente utilizzata:
 - classi `final`
 - costruttori privati

```
public final class Direzione {  
    private int ordinal;  
    private Direzione(int index) { this.ordinal = index; }  
    public int getOrdinal() { return this.ordinal; }  
    static public final Direzione NORD = new Direzione(0);  
    static public final Direzione EST = new Direzione(1);  
    static public final Direzione SUD = new Direzione(2);  
    static public final Direzione OVEST = new Direzione(3);  
    static private final Direzione[] cardinali =  
                                                { NORD, EST, SUD, OVEST };  
    static public Direzione[] values() { return cardinali; }  
}
```

Enumerazioni Tipate (pre Java 5)

- Risolti i problemi legati alla tipizzazione lasca:

```
Direzione dir = new Direzione(5); // NON COMPILA  
public Direzione direzioneOpposta(Direzione arg) {...}
```

- Nuove possibilità, ad es. cicli for-each:

```
for(Direzione dir : Direzione.values()) { ... }
```

- Si può anche pensare di affiancare metodi (polimorfi) alle costanti. Ad es. per il calcolo della direzione `opposta()`:

```
import static Direzione.*;  
assertEquals(SUD, NORD.opposta());
```

- Si rende però necessario creare un sottotipo per ogni costante della stessa enumerazione>>

Enumerazioni Polimorfe (pre Java 5)

```
public abstract class Direzione {  
    private int ordinal;  
    private Direzione(int index) { this.ordinal = index; }  
    public int getOrdinal()      { return this.ordinal; }  
    abstract public Direzione opposta();  
  
    static public final Direzione NORD  = new NORD();  
    static public final Direzione EST   = new EST();  
    static public final Direzione SUD   = new SUD();  
    static public final Direzione OVEST = new OVEST();  
    static final class NORD extends Direzione {  
        private NORD() { super(0); }  
        @Override public Direzione opposta() { return SUD; }  
    }  
    // ...similmente per EST, SUD...  
    ...  
    static final class OVEST extends Direzione {...}  
}
```

Permette di dichiarare una nuova classe *interna* (>>) e di invocare il costruttore privato

Limitazioni dell'Enumerazioni pre Java 5: Perché i Java 5+ Enum

- Efficace: ma per N costanti è necessario creare N+1 classi...
 - ✓ Decisamente macchinoso, anche con l'aiuto di un IDE evoluto

- I Java Enum introdotti in Java 5 sostanzialmente adottano questa stessa soluzione, ma in aggiunta offrono:

- Una sintassi meno verbosa; ad es. per enum senza metodi aggiuntivi basta scrivere:

```
public enum Direzione {  
    NORD, EST, SUD, OVEST;  
}
```

- Una superclasse astratta `java.lang.Enum` condivisa da tutti i tipi enumerati
 - con metodi condivisibili da tutti i tipi enumerati (es. `ordinal()`)
- Un generoso aiuto da parte del compilatore
 - definisce alcuni metodi statici aggiuntivi (>>)

Java Enum: Sintassi

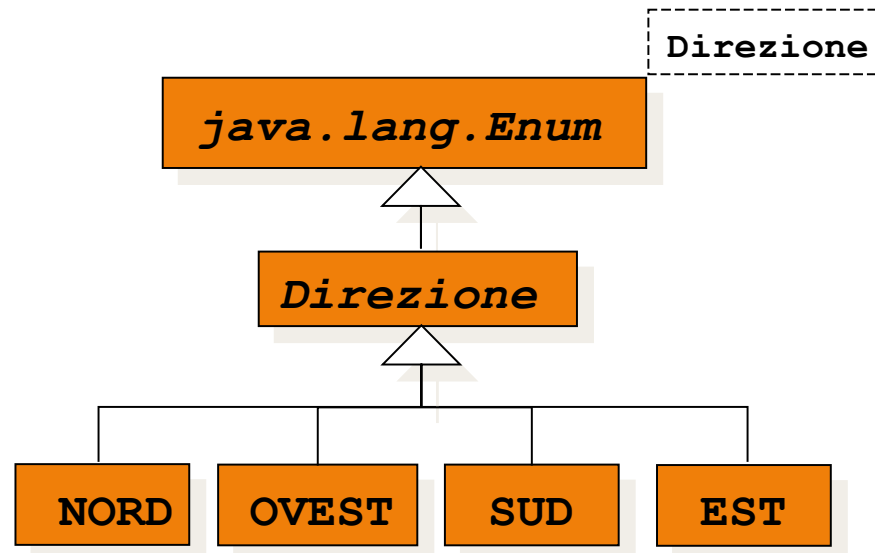
- In presenza di metodi aggiuntivi, la sintassi dei Java Enum in caso di enumerazioni polimorfe è più prolissa, ma comunque più compatta della soluzione “manuale”:

```
public enum Direction {  
    NORD() {  
        @Override public Direction opposta() {  
            return SUD;  
        }  
    },  
    ...  
    OVEST() {  
        @Override public Direction opposta() {  
            return EST;  
        }  
    };  
    public abstract Direction opposta();  
}
```

java.lang.Enum: Gerarchia

- Per ogni Enum dichiarato il compilatore genera
 - Una sottoclasse (ad es. `Direzione`) della classe astratta generica `java.lang.Enum` che modella il tipo enumerato
 - Una sua sottoclasse per ciascun valore costante (ad es. `NORD`)

```
public abstract class Enum<E extends Enum<E>>  
    implements Comparable<E>
```



java.lang.Enum: Costruttore

- La classe `java.lang.Enum` accentra funzionalità per tutti i tipi enumerati
 - Ad es. metodi per gestire i numeri ordinali (`ordinal()`) ed il nome del tipo enumerato (`name()`)
- Unico costruttore:

`Enum` **protected** `Enum(String name, int ordinal)`

Sole constructor. Programmers cannot invoke this constructor. It is for use by code emitted by the compiler in response to enum type declarations.

Parameters:

name - *The name of this enum constant, which is the identifier used to declare it.*

ordinal - *The ordinal of this enumeration constant (its position in the enum declaration, where the initial constant is assigned ordinal 0).*

Java Enum: Costruttori delle «Costanti Enumerative»

- I valori costanti possono avere uno o più costruttori ed uno stato
 - Si dichiara un costruttore *privato* solo nel tipo enumerato (es. `Direzione`)
 - Lo stato *deve* essere immutabile (>>)
 - ✓ è preferibile dichiarare tutte le variabili di istanza `final`

```
public enum Direzione {  
    NORD(0) {        @Override  
                    public Direzione opposta() { return SUD; }  
    }, ...,  
    OVEST(270) {      @Override  
                    public Direzione opposta() { return EST; }  
    };  
    private final int gradi;  
    private Direzione(int gradi) {  
        this.gradi = gradi;  
    }  
    public int getGradi() { return this.gradi; }  
    public abstract Direzione opposta();  
}
```

java.lang.Enum: Metodi (1)

- Documentiamo la semantica degli altri metodi di

`java.lang.Enum`

- tramite qualche test di unità, per es. sul tipo `Direzione`
- metodo `ordinal()`

```
import static org.junit.Assert.*;
import org.junit.Test;
import static Direzione.*;

public class EnumTest {

    @Test
    public void testOrdinal() {
        assertEquals(0, NORD.ordinal());
        assertEquals(1, EST.ordinal());
        assertEquals(2, SUD.ordinal());
        assertEquals(3, OVEST.ordinal());
    }

    ... // altri test-case a seguire
}
```

java.lang.Enum: Metodi (2)

- Ogni tipo enumerativo è associato ad una classe per l'intero tipo più un oggetto-singleton/classe per ciascuno dei valori costanti del tipo
- Le costanti ricordano la classe del loro supertipo (ed associato a tutta la collezione di costanti enumerative) mediante il metodo

Class<E> getDeclaringClass()

- *returns the Class object corresponding to this enum constant's enum type.*

```
public class EnumTest {...  
    @Test  
    public void testGetDeclaringClass() {  
        assertSame(Direzione.class, NORD.getDeclaringClass());  
        assertNotSame(Direzione.class, NORD.getClass());  
        assertNotSame(EST.getClass(), NORD.getClass());  
        ... // similamente per le altre direzioni  
    }  
...}
```

java.lang.Enum: Metodi (3)

- Ad ogni valore costante è associato un singolo oggetto (singleton)
- Evidente con un test sul metodo `valueOf()`, che consente di creare gli oggetti associati alle costanti di un tipo enumerato anche senza scomodare la più “pesante” API sull’introspezione (>>)

```
public class EnumTest {...  
    @Test  
    public void testTuttiSingleton() {  
        assertSame(NORD, Direzione.valueOf("NORD"));  
        final Direzione singleton = Direzione.valueOf("NORD");  
        assertSame(singleton, NORD);  
        ...  
        assertNotSame(EST, NORD);  
        ... // similamente per le altre direzioni  
    }  
}
```

Enumerazioni: Criterio di Equivalenza (1)

- Quindi riassumendo:
 - ogni tipo enumerato di N valori costanti genera $N+1$ classi
 - di queste N sono singleton (classi a singola istanza) che modellano i valori
 - ad ogni valore corrisponde quindi una sola classe che possiede una sola istanza
- Tutto questo si riflette sul criterio di equivalenza dei tipi enumerati che risulta essere quello più naturale
- Ciascuno dei valori costanti è un oggetto che
 - è equivalente solo a se stesso
 - non è equivalente a nessuno degli altri

Enumerazioni: Criterio di Equivalenza (2)

- `equals()` e l'operatore `==` finiscono per modellare lo stesso criterio di equivalenza
 - ✓ ATTENZIONE! NON è affatto vero in generale: vale solo per i tipi enumerativi!
- `hashCode()` ritorna il valore restituito dal metodo `ordinal()`
 - ✓ `HashSet` ed `HashMap` su tipi enumerativi possiedono prestazioni ottimali (non esistono conflitti!)

```
public class EnumTest {...
    @Test
    public void testCriterioDiEquivalenza() {
        assertEquals(NORD, NORD);
        assertNotEquals(NORD, EST);
        assertNotEquals(NORD, SUD); //...
        ... // similamente per le altre direzioni
    }...
}
```

Enumerazioni: Ordinamento

- E' direttamente quello basato sul valore ordinale
 - ✓ N.B. Induce anche un criterio di equivalenza basato sull'identità degli oggetti
 - ✓ Ovvero come l'operatore ==

```
public class EnumTest {...  
    @Test  
    public void testCompareTo() {  
        assertTrue(NORD.compareTo(EST)<0) ;  
        assertTrue(EST.compareTo(SUD)<0) ;  
        assertTrue(SUD.compareTo(OVEST)<0) ;  
        assertTrue(OVEST.compareTo(NORD)>0) ;  
    }  
...}
```

Enum.toString() / name()

- Non è necessario ridefinire un metodo `toString()` in quanto è naturalmente definito sulla base del nome stesso della costante

`String name()`

- *Returns the name of this enum constant, as declared in its enum declaration.*

`String toString()`

- *Returns the name of this enum constant, as contained in the declaration.*

```
public class EnumTest {...  
    @Test  
    public void testToStringAndName() {  
        assertEquals("NORD", NORD.toString());  
        assertEquals("NORD", NORD.name());  
        ... // similamente per le altre direzioni  
    }...  
}
```

Java Enum e Metodi «Fantasma» (1)

- I Java Enum introducono «zucchero sintattico»: la controparte nei sorgenti di codice oggetto generato non risulta osservabile
 - ✓ si pensi a boxing-unboxing (<<)
- Sorprendentemente, anche *alcuni* metodi appaiono, in particolare il metodo statico `values()` ed anche `valueOf()`
 - ✓ Metodi di cui non esiste il codice sorgente!
 - ✓ Eclipse non fornisce meta-informazioni! (e javadoc)
- Il compilatore genera automaticamente questi metodi adottando un comportamento particolare per tutte le classi che estendano `java.lang.Enum`:

```
public class EnumTest {...
    @Test
    public void testValues() {
        final Direzione[] expected = { NORD, EST, SUD, OVEST } ;
        assertArrayEquals(expected , Direzione.values() );
    }...
}
```

Java Enum e Metodi «Fantasma» (2)

- Per avere documentazione è necessario riferirsi direttamente alla [Java Language Specification](#)

```
/**
 * Returns an array containing the constants of this enum
 * type, in the order they're declared. This method may be
 * used to iterate over the constants as follows:
 *     for(E c : E.values())
 *         System.out.println(c);
 * @return an array containing the constants of
 * this enum type, in the order they're declared */
public static E[] values();
/**
 * Returns the enum constant of this type with the specified name.
 * The string must match exactly an identifier used to declare
 * an enum constant in this type. (Extraneous whitespace
 * characters are not permitted.)
 * @return the enum constant with the specified name
 * @throws IllegalArgumentException if this enum type has no
 * constant with the specified name */
public static E valueOf(String name);
```

Il metodo statico **values()** rende possibile usare cicli «for-each» sopra tipi enumerati

Java Enum e «Switch-Statement»:

- E' possibile usare tipi enumerati direttamente negli «switch statement»; in precedenza era possibile farlo solo per le costanti letterali

```
import static Direzione.*;

public class DirezioneUtils {
    public static Direzione opposta(Direzione direzione) {
        switch (direzione) {
            case NORD:    return SUD;
            case EST:     return OVEST;
            case SUD:     return NORD;
            case OVEST:   return EST;
            default:      return null;
        }
    }
}
```

Conclusioni:

Costanti Enumerative - Quando Usarle? (1)

- Quando è opportuno utilizzare costanti enumerative?
- Quando l'insieme di elementi da modellare:
 - ✓ è finito
 - ✓ tutti i suoi elementi siano già noti
 - ✓ è «stabile»: molto difficilmente cambierà nel tempo
- Quando per gli elementi ospitati:
 - ✓ ne conosciamo i nomi... «propri»
 - ad es. “NORD”, “GENNAIO”
 - ✓ possono anche avere uno stato, purché *immutabile*
 - inutile/impossibile crearne diverse «istanze»

Conclusioni:

Costanti Enumerative - Quando Usarle? (2)

- Ed inoltre:
 - Il dominio suggerisce
 - una tipizzazione forte (concetto di “primo ordine”)
 - un tipo dedicato alla collezione di costanti
 - Anche per l’esigenza di aggiungere metodi (polimorfi) alle costanti
 - Ad es. **Direzione.opposta()**

Esercizio (Studio di Caso)

- Introdurre l'enumerazione **Direzione** nello studio di caso per modellare con un tipo dedicato le direzioni degli spostamenti e delle adiacenze tra oggetti **Stanza**
 - ✓ Attualmente si «prende in prestito» il tipo **String**
 - ✓ Ma attenzione, è un sintomo di cattiva «modellazione»:

<http://c2.com/cgi/wiki?StringlyTyped>

Esercizio (Tetris)

- Studiare il codice dell'esercitazione «Tetris», in particolare nel package `tetris.tetramino`
- Perché si è deciso di separare la modellazione del tetramino (classe `tetris.tetramino.Tetramino`) da quella del suo tipo (`enum tetris.tetramino.Tipo`)?
- *Suggerimento:*
 - Da quale campi è composto lo stato degli oggetti istanza dei due tipi?
 - Quali sono o possono essere dichiarati `final`?
 - Quale dei due tipi possiede uno stato mutabile?
- Cosa accadrebbe se fondessimo le due classi addossando le responsabilità di `Tipo` alla classe `Tetramino`?

Enum e Collezioni

- Le costanti enumerative finiscono per costituire un tipo di collezione di caratteristiche particolari
- Sono possibili rappresentazioni ancora più compatte ed efficienti rispetto alle generiche collezioni (che già sono particolarmente efficienti sui tipi enumerativi)
- Dai javadoc di `java.util.EnumSet`
 - *Implementation note: All basic operations execute in constant time. They are likely (though not guaranteed) to be much faster than their `HashSet` counterparts. Even bulk operations execute in constant time if their argument is also an enum set.*
- Vedere anche `java.util.EnumMap`
 - *Specializzazione di `java.util.Map` che utilizza come chiavi un tipo enumerativo*

Conclusioni

- L'insieme dei meccanismi offerti da un linguaggio per la modellazione dei tipi è uno degli aspetti più caratterizzanti lo stesso
- Con le classi astratte abbiamo coperto l'insieme dei principali meccanismi per la definizione dei tipi:
 - Interface
 - Classi
 - Classi astratte
 - Generics
- Esistono ancora altri meccanismi:
 - Tipi enumerativi
 - Classi nidificate (>>)

meno importanti dei precedenti, ma che contribuiscono alla ricchezza del sistema dei tipi in Java

Programmazione Orientata agli Oggetti

Classi Nidificate

Sommario

- Introduzione
- Nidificazione di Classi (non statica)
 - Interne
 - Locali
 - Anonime
- Nidificazione Statica
 - Classi
 - Interfacce
- Conclusioni

Classi Nidificate (o «Interne»)

- Disponibili a partire da Java 1.1 (non da subito, quindi)
- All'epoca motivate dalla necessità di facilitare la scrittura di codice per la gestione di «eventi grafici»
 - Il fatto che questa esigenza si sia avvertita, forte, anche dopo Java 1.1, testimonia forse il non totale successo dell'iniziativa
 - Difatti anche quelle esigenze (e molte altre) hanno motivato successive iniziative, su tutte
 - le lambda function di Java 8
- Rimangono comunque diffuse, e ben motivate, non solo per gli eventi grafici, come ulteriore strumento per la definizione di tipi

Definizione di Classi Nidificate

- Una classe nidificata è una classe dichiarata all'interno del corpo della definizione di un'altra classe
 - la classe dichiarata all'interno è detta classe «interna»
 - la classe dichiarata all'esterno è detta classe «esterna»

```
public class ClasseEsterna {  
    ...  
    class ClasseInterna {  
        ...  
    }  
}
```


Quattro Varianti per «Nidificare»

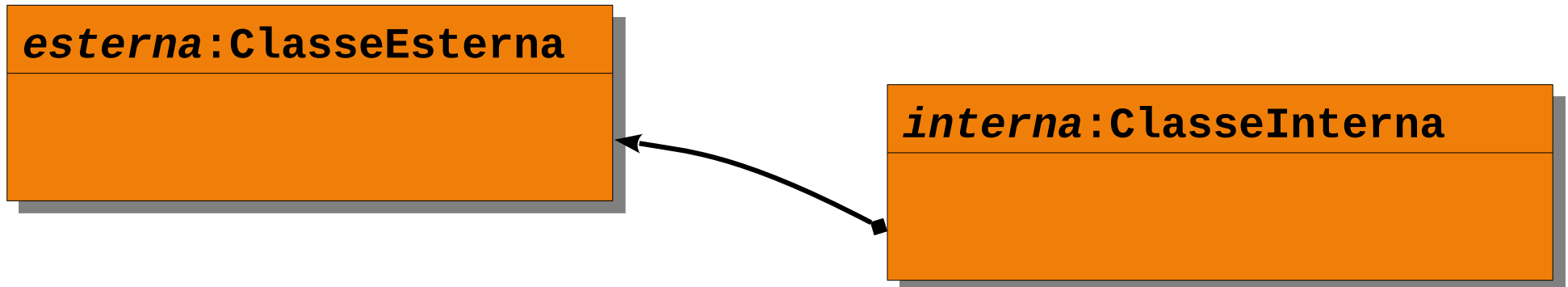
- Esistono quattro diverse varianti di *classi nidificate*:
 - *Interne*
 - ad es. usate per le classi degli iteratori dipendenti dal tipo di collezione all'interno della classe della collezione
 - *Locali*
 - dichiarate all'interno di un blocco di codice direttamente nel corpo di un metodo
 - ad es. un comparatore “usa e getta”
 - *Anonime*
 - consentono di *NON* specificare il nome della classe concreta
 - ad es. un comparatore “usa e getta”
- ...e per finire:
 - *Statiche*

Quattro Varianti; Statiche vs Non-Statiche (1)

- La semantica delle prime tre varianti è nettamente diversa rispetto a quella delle classi nidificate statiche
 - L'idea fondamentale alla loro base è che un'istanza della classe interna deve mantenere un *riferimento* all'istanza della classe esterna che l'ha creato
 - ✓ Questo *NON* vale solo per le classi nidificate statiche
- Le classi nidificate statiche si possono inquadrare, nella sostanza, quasi solo come ad un meccanismo di *namespacing*
- Rende esplicito ed evidente un legame esistente tra due classi, ma senza sostanziali differenze concettuali rispetto alle ordinarie classi non nidificate (>>)

Quattro Varianti; Statiche vs Non-Statiche (2)

- Invece per le tre prime varianti di classi nidificate, al legame di contenimento delle classi nel codice, corrisponde effettivamente un forte legame tra le istanze coinvolte a tempo dinamico
- Una istanza della classe interna è sempre associata ad una sola istanza della classe esterna
- L'istanza interna **non** può esistere senza una istanza della classe esterna a cui rimane permanentemente associata sin dalla creazione
- Per creare l'istanza interna serve quella esterna



Classe Nidificata Interna

- Una classe interna è una classe definita all'interno di un'altra classe (anche detta *esterna*)

```
public class Outer {  
    ...  
    public class Inner { ... }  
    ...  
}
```

- Le regole di visibilità sono simili a quelle dei metodi
 - Un oggetto della classe interna conserva sempre un riferimento all'oggetto della classe esterna a partire dal quale è stato istanziato
 - L'oggetto interno può accedere alle variabili di istanza e ai metodi della classe esterna, anche se privati
 - Se dichiarata privata non può essere invece utilizzata da altri, fuori della classe «esterna»

Classe Interna: Esempio

```
public class Outer {  
    private String outerName = "outer";  
  
    public class Inner {  
        private String innerName = "inner";  
        public void saluta() {  
            return innerName + " " + outerName;  
        }  
    }  
}
```

Classe Interna ed Esterna: Legame tra le Istanze (1)

- Ogni istanza di una classe interna è legata ad un singolo oggetto della classe esterna in cui viene dichiarata
 - Non può esistere un oggetto «interno» (ovvero istanza della classe interna) senza un oggetto «esterno» (istanza della classe esterna) che lo crei e verso cui conserva un riferimento sin dalla sua creazione (e fino alla sua deallocazione)
- Nell'esempio precedente un metodo della classe nidificata **Inner** accede alla variabile **outerName**
 - Questa è una variabile di istanza appartenente all'oggetto «esterno»

Classe Interna ed Esterna: Legame tra le Istanze (2)

- La sintassi da utilizzarsi per creare istanze della classe interna rende evidente l'esistenza di questo legame

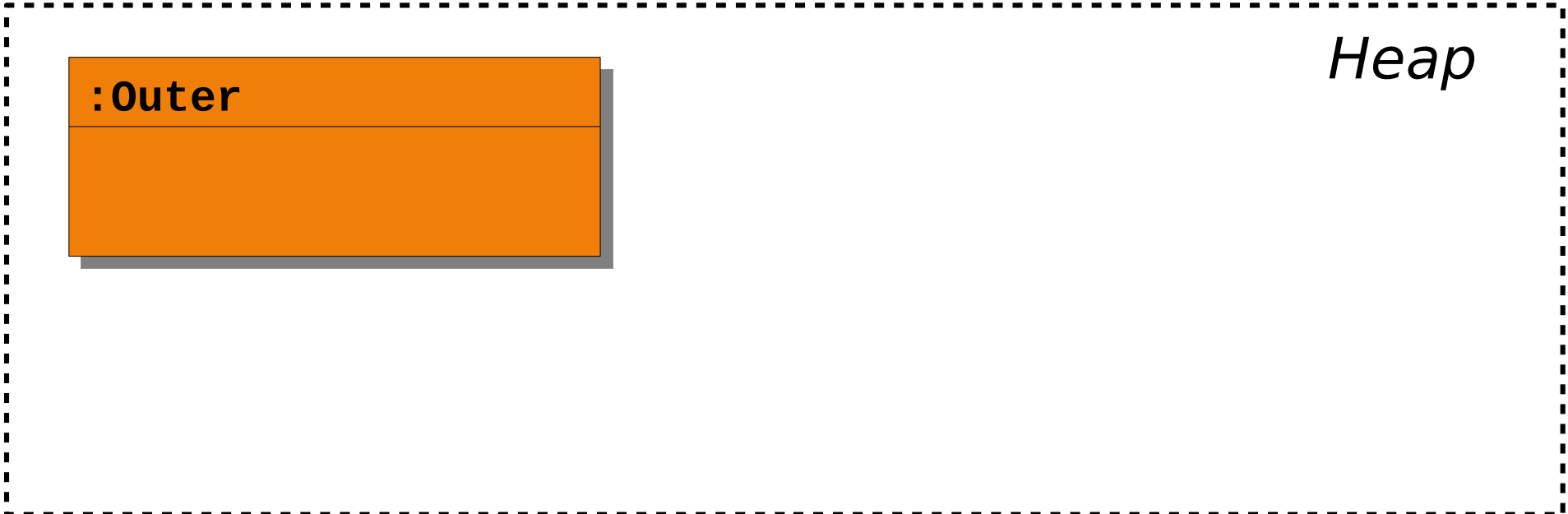
```
public class InnerTest {  
  
    @Test  
    public void testSaluta() {  
        Outer outer = new Outer();  
        Outer.Inner inner = outer.new Inner();  
        assertEquals("inner outer", inner.saluta());  
    }  
}
```

- Possibile solo se la classe interna *non* è dichiarata privata
- N.B.
 - In questo esempio l'oggetto «interno» viene creato dall'esterno della classe **Outer** che include al suo interno la definizione di **Inner**

Classi Interne (1)

```
@Test
public void testSaluta() {
    Outer outer = new Outer();
    ▶ Outer.Inner inner = outer.new Inner(); ◀

    assertEquals("inner outer", inner.saluta());
}
```



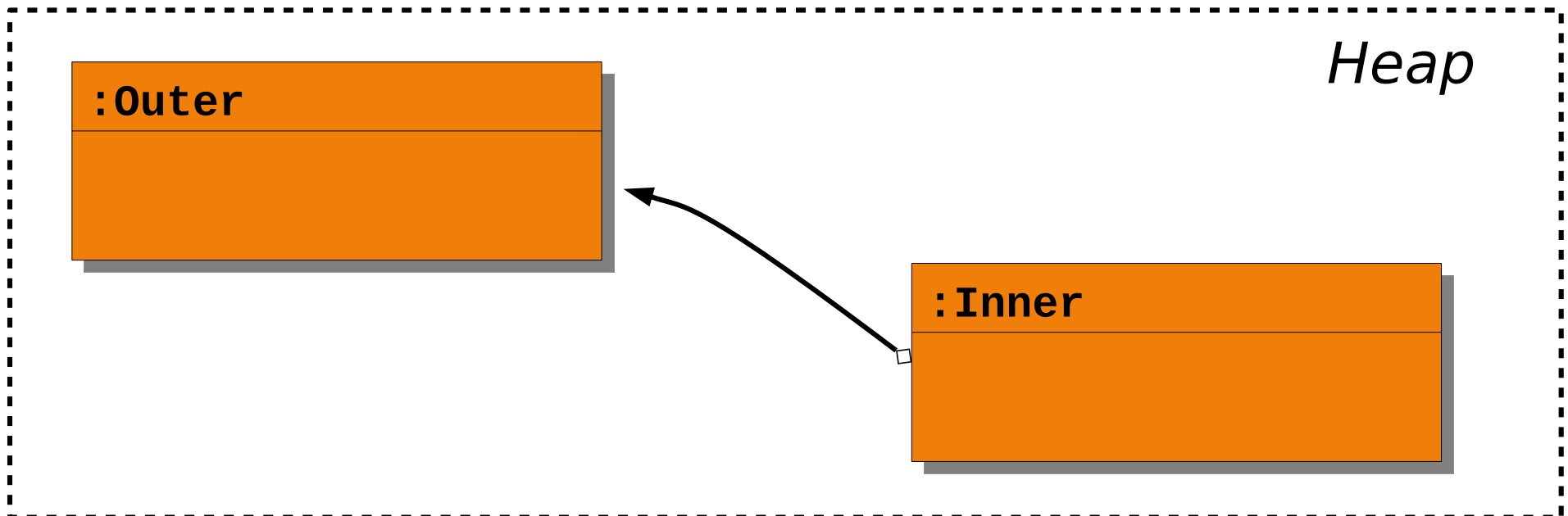
The diagram shows a dashed rectangular box representing the memory heap. Inside the box, on the left, is an orange rectangular object. The top portion of this object is labeled ':Outer'. To the right of the object, the word 'Heap' is written in a black, italicized font.

:Outer

Heap

Classi Interne (2)

```
@Test
public void testSaluta() {
    Outer outer = new Outer();
    Outer.Inner inner = outer.new Inner();
    
    assertEquals("inner outer", inner.saluta());
}
```



Creazioni di Istanze Nidificate dall'Interno

- All'interno di una classe si può creare un'istanza di una classe nidificata come se si trattasse di una qualsiasi classe ordinaria:

```
public class Outer {  
    ...  
    public void riproponiSaluto() {  
        System.out.println("La inner saluta con:");  
        Inner inner = new Inner();  
        inner.saluta();  
    }  
}
```

- In questo caso l'istanza della classe esterna **Outer** referenziata dalla nuova istanza della classe nidificata **Inner** sarà quella esterna che ha effettuato l'invocazione del costruttore della classe interna (ovvero l'istanza il cui riferimento, prima dell'invocazione, è **this**)

Accesso alle Variabili di Istanza nelle Classi Nidificate

- La classe interna può accedere alle variabili di istanza (anche se private) della classe esterna
 - **Inner** accede alla variabile di istanza **outerName** della classe **Outer**

```
public class Outer {  
    private String outerName = "outer";  
    public class Inner {  
        private String innerName = "inner";  
        public void saluta() {  
            System.out.println(innerName + " " +  
                                outerName);  
        }  
    }  
}
```

Shadowing e Classi Nidificate (1)

- Se la classe interna dichiara una variabile (di istanza o locale) con lo stesso nome di una variabile di istanza della classe esterna, quest'ultima risulterà *non* più accessibile:
 - la nuova variabile (di istanza o locale) ha uno scope più ristretto e “mette in ombra” la variabile di istanza della classe esterna
 - Per poter accedervi bisogna qualificare completamente il nome con la sintassi
`<Nome_OuterClass>.this.<Nome_Variabile>`
 - Ad esempio:
`Outer.this.name;`

Shadowing e Classi Nidificate (2)

```
public class Outer {  
    private String name = "outer";  
    public class Inner {  
        private String name = "inner";  
        public void saluta() {  
            System.out.println(name + " " +  
                                Outer.this.name);  
        }  
    }  
}
```

Accesso alle Variabili di istanza della Classe Esterna

- La sintassi
<Nome_OuterClass>.this.<Nome_Variabile>
sebbene decisamente prolissa, risulta efficace:
 - rende chiaro a quale variabile si fa riferimento
 - Somiglia all'uso di **this** accedere alle variabili di istanza

Esempio di Classi Nidificate Interne: `Iterator<E>` ed `ArrayList<E>`

- `ArrayList<E>` mette a disposizione un iteratore per enumerare gli elementi che contiene
- Un iteratore è un oggetto istanza di una classe sottotipo (concreto) di `Iterator<E>`
 - Per poter fornire gli elementi di una `ArrayList<E>`, l'iteratore deve poter accedere al sottostante array che li contiene
 - tale array è privato e quindi, per potervi accedere, risulta ben motivata l'implementazione dell'iteratore mediante una classe interna ad `ArrayList<E>`

ArrayList<E>, Vista dall'Interno (1)

- **ArrayList<E>** gestisce i dati al suo interno con un array di **Object**:

```
public class ArrayList<E> extends AbstractList<E> implements
    List<E>, RandomAccess, Cloneable, java.io.Serializable
{
    // Dati dell'ArrayList
    private transient Object[] elementData; // Gli elementi

    private int size; // Dimensione dell'array

    public ArrayList(int initialCapacity) {
        super();
        if (initialCapacity < 0)
            throw new IllegalArgumentException("Illegal Capacity: "+
                                             initialCapacity);
        this.elementData = new Object[initialCapacity];
    }
    ...
}
```


ArrayList<E>, Vista dall'Interno (2)

- Gli elementi sono conservati in un array di **Object**, tuttavia viene sempre effettuato un *cast* esplicito nei metodi **get** (dell'interfaccia **List<E>**)

```
public E get(int index) {  
    rangeCheck(index);    // index < size ?  
  
    return elementData(index);  
}
```

```
E elementData(int index) {  
    return (E) elementData[index];  
}
```

ArrayList<E>: Iterator<E>

- **ArrayList<E>** implementa (indirettamente) l'interfaccia **Iterable<E>** e possiede dunque un metodo

```
public Iterator<E> iterator()
```

che restituisce un iteratore, istanza della classe **Itr<E>**

- **Itr** è una classe interna di **ArrayList<E>**
- **Itr** implementa **Iterator<E>**

- Ecco il metodo **iterator()** che viene utilizzato per ottenere un iteratore:

```
public Iterator<E> iterator() {  
    return new Itr();  
}
```

La Classe Interna Itr (1)

- La classe **Itr** implementa i metodi di **Iterator<E>**:
 - **public E next()**
 - **public void remove()**
 - **public boolean hasNext()**

// All'interno di ArrayList ...

```
private class Itr implements Iterator<E> {  
    int cursor; // = 0  
    int lastRet = -1;  
    int expectedModCount = modCount;  
    ...
```

La Classe Interna Itr (2)

```
private class Itr implements Iterator<E> {  
    int cursor;  
    int lastRet = -1;  
    int expectedModCount = modCount;  
    ...  
}
```

- **int cursor**
 - L'indice del prossimo elemento che **next()** deve restituire
- **int lastRet**
 - L'indice dell'ultimo elemento restituito da **next()**

La Classe Interna Itr: next()

```
private class Itr implements Iterator<E> {  
  
    public E next() {  
        checkForComodification();  
        int i = cursor;  
        if (i >= size)  
            throw new NoSuchElementException();  
        Object[] elementData = ArrayList.this.elementData;  
  
        if (i >= elementData.length)  
            throw new ConcurrentModificationException();  
  
        cursor = i + 1;  
        return (E) elementData[lastRet = i];  
    }  
}
```

La Classe Interna `Itr`: Altri Metodi

- La classe `Itr` dispone di altri metodi
- `hasNext()`
 - Restituisce: `cursor != size`

`size` è una variabile di istanza privata di `ArrayList` a cui però è possibile accedere da una classe interna
- `remove()`
 - Rimuove l'ultimo elemento restituito dall'iteratore usando i metodi di `ArrayList` e `lastRet`

Vantaggi delle Classi Interne

- Usando una classe interna non è necessario che **ArrayList<E>** esponga la propria rappresentazione interna, e specifica, degli *iteratori*
- Questo permette di applicare più agevolmente i principi *dell'information hiding*:
 - In caso contrario **ArrayList** dovrebbe esporre un getter per ottenere l'accesso all'array alla base della sua rappr.
- Anche per la distribuzione delle responsabilità la scelta è opportuna
 - L'iteratore e la collezione possiedono responsabilità ben definite e distinte (sebbene strettamente correlate) ed è quindi desiderabile definirle in due classi distinte
- Per *economia di pensiero*: non è nemmeno necessario conoscere il nome del tipo dinamico degli iteratori delle collezioni, ma solo il supertipo astratto **Iterator<E>**

Utilizzi delle Classi Interne

- Raggruppare logicamente classi che vengono utilizzate assieme senza essere costretti a renderle tutte visibili esternamente
- Favorire l'incapsulamento: una classe interna può accedere anche ai dati privati della classe esterna
 - Non è quindi necessario avere getter e setter per esporre la rappresentazione dati nelle variabili di istanza private anche quando si vuole condividere tale rappresentazione
- Favorire la coesione: tenendo la dichiarazione di una classe interna ben vicina ai suoi unici utilizzi (da parte della classe esterna che la definisce), si favorisce la qualità del codice

Classi Nidificate Locali (1)

- Una classe locale è una classe definita all'interno di un blocco di codice (solitamente il corpo di un metodo)
- Il suo campo d'azione (scope), come per ogni variabile locale, è il blocco di codice all'interno del quale è viene dichiarata:

```
{  
    class LocalInner {  
        ...  
    }  
    LocalInner inner = new LocalInner();  
}
```

// Qui non si può usare LocalInner

Classi Nidificate Locali (2)

- Le classi locali possiedono le stesse caratteristiche delle classi interne
 - Possono essere create solo a partire da un'istanza di una classe esterna e mantengono sempre un riferimento a tale istanza tramite ***<Nome_OuterClass>.this***
 - L'istanza esterna è quella che sta eseguendo il blocco di codice contenente la dichiarazione della classe locale
- ✓ Le classi locali non possono mai essere istanziate dall'esterno (per costruzione!)

Classi Locali: Esempio (1)

- Le classi locali spesso servono ad utilizzi “usa-e-getta”, all'interno di un unico metodo
 - il codice risulta così più leggibile perché la definizione e l'unico utilizzo sono vicine
- Classico esempio: *comparatori*
 - E' veramente necessario creare una nuova classe (in un ennesimo file) per un comparatore mono-uso?
 - Ad es. perché serve un ordinamento particolare per fare una stampa ordinata in un solo punto del codice...

Classi Locali: Esempio (2)

```
class Aula {  
    private List<Studente> studenti;  
    ...  
    public List<Studente> ordinaStudentiPerEta() {  
        class ComparatorePerEta implements Comparator<Studente> {  
            @Override  
            public int compare(Studente s1, Studente s2){  
                return s1.getEta() - s2.getEta();  
            }  
        }  
        ComparatorePerEta comp = new ComparatorePerEta();  
        List<Studente> studentiOrdinati = new ArrayList<>(studenti);  
        Collections.sort(studentiOrdinati, comp);  
        return studentiOrdinati;  
    }  
}
```

Classi Locali & Variabili Locali

- Le classi locali possono accedere alle variabili locali del metodo (in generale, del blocco di codice) ove definite
 - È però richiesto che tali variabili locali siano dichiarate **final** (Java 7-) o che siano *effectively final* (Java 8+)

<https://stackoverflow.com/questions/4732544/why-are-only-final-variables-accessible-in-anonymous-class>

- Quando una classe locale fa riferimento ad una variabile locale, una copia del valore di tale variabile viene “catturata”
 - ✓ Subito: nel momento della creazione dell’istanza nidificata
 - ✓ ma potrà essere usata anche successivamente
 - ✓ anche se il ciclo di vita della copia *originale* termina
- Non potrà accedere (e tantomeno modificare) l’originale ma *solo* la copia
 - Il vincolo sintattico sulle variabili **final** rende tutto questo palese

Classi Locali & Variabili Locali: Esempio

```
class Canale {
    private List<Studente> studenti;
    ...
    public SortedSet<Studente> ordinaStudentiPerVoto(
        final Map<Studente, Integer> studente2voto) {
        class ComparatorePerVoto implements Comparator<Studente> {
            @Override
            public int compare(Studente s1, Studente s2){
                int votoS1 = studente2voto.get(s1);
                int votoS2 = studente2voto.get(s2);
                return ( votoS1 - votoS2 ) ;
            }
        }
        ComparatorePerVoto comp = new ComparatorePerVoto();
        SortedSet<Studente> studentiOrdinati = new TreeSet<>(comp);
        studentiOrdinati.addAll(studentiOrdinati);
        return studentiOrdinati;
    }
}
```

Classi Locali & Variabili Locali: Esempio (cont.)

- Nel precedente esempio una collezione di studenti viene ordinata sulla base del voto
 - Il metodo `ordinaPerVoto()` riceve una mappa che associa gli studenti al voto da loro conseguito
- Tale mappa può essere acceduta direttamente all'interno del comparatore
 - deve però essere esplicitamente dichiarata come **final**
 - Il riferimento è costante (il contenuto della mappa non necessariamente)

Classi Nidificate Anonime

- Le classi nidificate locali spesso sono classi usa-e-getta
 - unico utilizzo, o cmq utilizzi confinati all'interno del blocco contenente la definizione
 - comode se risulta eccessivo definire un'intera nuova classe solo per utilizzi "locali"
- Talvolta può risultare eccessivo anche solo dover scegliere un nome per una classe mono-uso
 - soprattutto se già si conosce il tipo astratto (l'interface) che dovrà implementare
- Le classi anonime permettono di definire ed istanziare una classe in un'unica espressione
 - risultano molto utili quando di tale classe è necessaria un'unica istanza per un unico utilizzo

Classi Anonime: Esempio

- Le classi anonime vengono definite sempre e comunque per estensione/implementazione di un supertipo già noto
 - Un'interfaccia (viene implementata)
 - Una classe astratta (viene estesa)
 - Una superclasse (viene estesa)

```
Comparator<Studente> comp = new Comparator<Studente>() {  
    @Override  
    public int compare(Studente s1, Studente s2) {  
        ...  
    }  
}; // ← N.B. ';' necessaria
```

Classi Anonime ed Estensione

- La creazione di una istanza di una classe anonima avviene sempre con **new** seguito dal nome di un'interfaccia, una classe o una classe astratta
- Come nelle ordinarie estensioni di classe/implementazioni di interfacce:
 - è possibile aggiungere nuove variabili di istanza e nuovi metodi
 - è possibile implementare metodi
 - tutti quelli previsti dall'interfaccia
 - tutti quelli abstract della superclasse astratta
 - è possibile sovrascrivere metodi
- La classe così definita è quindi, a tutti gli effetti, una nuova classe, senza nome (appunto, *anonima*)

Istanza di una Classe Anonima

- La definizione di una classe anonima e la creazione della sua unica istanza avviene contestualmente, mediante un'unica espressione

```
Comparator<Studente> comp = new Comparator<Studente> {  
    @Override  
    public int compare(Studente s1, Studente s2) {  
        ...  
    }  
}; // Notare ; → questa è un'espressione
```

- comp** conterrà un riferimento all'istanza della classe anonima appena definita
 - può esistere solo un'istanza, creata al momento stesso della definizione di classe

Classi Anonime vs Locali

- Le classi nidificate anonime sono sostanzialmente delle classi nidificate locali senza nome, possono quindi:
 - far riferimento all'oggetto della classe esterna dalla quale sono create mediante `<Nome_OuterClass>.this` e accedere ai suoi metodi e variabili di istanza private
 - accedere alle variabili visibili nel blocco di codice in cui vengono definite, purché siano dichiarate `final`

Classi Anonime: Costruttori

- Una classe anonima viene creata invocando uno dei costruttori della sua superclasse
 - direttamente il costruttore *no-args* di `java.lang.Object` (se non estende altre classi ma implementa una interface)
 - ✓ la classe viene definita ed istanziata contestualmente, non è quindi possibile definire nuovi costruttori e, contemporaneamente, invocarli
- È comunque possibile invocare i costruttori della superclasse (visibili) o catturare variabili locali `final`

Comparator<T> Anonimi

```
class Canale {  
    private List<Studente> studenti;  
  
    ...  
  
    public List<Studente> ordinaStudentiPerVoto(  
        final Map<Studente, Integer> studente2voto) {  
        List<Studente> studentiOrdinati = new ArrayList<>(studenti);  
        Collections.sort(studentiOrdinati, new Comparator<Studente>() {  
            @Override  
            public int compare(Studente s1, Studente s2) {  
                return studente2voto.get(s1) - studente2voto.get(s2);  
            }  
        });  
        return studentiOrdinati;  
    }  
}
```

Esempio Invocazione Costruttore

- Per testare il comportamento di una classe astratta (che non si può istanziare dirett.)

```
AbstractPersonaggio out =  
    new AbstractPersonaggio("fake") {  
        @Override  
        public String agisci(Partita partita) {  
            return "fake";  
        }  
    };  
};
```

- La definizione di classe (nidificata anonima locale) è contestuale al suo primo ed unico istanziamento

Codice Oggetto (.class) per Classi Nidificate

- Dopo la compilazione i file **.class** generati per le classi nidificate risultano separati da quelli associati alle classi che le contengono
 - le classi interne vengono salvate in file di nome **<NomeOuter>\$<NomeInner>.class**
 - per le classi anonime, si usa un numero progressivo **<NomeOuter>\$1.class, <NomeOuter>\$2.class ...**
 - Per le classi locali:
<NomeOuter>\$1<NomeLocal>.class,
<NomeOuter>\$2<NomeLocal>.class ...
- Quest'ultime possiedono un nome ma è comunque lecito dichiarare due classi locali omonime

Classi Nidificate e Riflessione

- I nomi dei file `.class` associati alle classi nidificate sono esattamente quelli (tranne l'estensione `.class`) ottenibili tramite l'introspezione:

```
public class Outer {  
    class Inner {}  
    public void inner() {  
        Inner inner = new Inner();           // Stampa:  
        System.out.println(inner.getClass().getName()); // 'Outer$Inner'  
    }  
  
    public void local() {  
        class Local {}  
        Local local = new Local();  
        System.out.println(local.getClass().getName()); // 'Outer$1Local'  
    }  
  
    public void anonymous() {  
        Object anonymous = new Object(){};  
        System.out.println(anonymous.getClass().getName()); // 'Outer$1'  
    }  
}
```

Classi Statiche Nidificate (1)

- Le classi nidificate statiche sono classi dichiarate all'interno di altre classi usando il modificatore **static**:

```
public class Outer {  
    ...  
    public static class Nested {  
        ...  
    }  
}
```

Classi Statiche Nidificate (2)

- Le classi statiche, sono diverse da tutte le altre tipologie di classi nidificate viste sinora
- ✓ Le sue istanze *NON* sono create a partire da un oggetto della classe esterna
- Sono create come le classi ordinarie
 - sia dall'esterno della classe stessa:
`Outer.Nested nested = new Outer.Nested();`
 - sia dall'interno della classe (ma il nome completamente qualificato non è necessario):
`Nested nested = new Nested();`

Classi Statiche Nidificate (3)

- Pertanto un'istanza di una classe statica nidificata non mantiene alcun legame con gli oggetti della classe in cui viene dichiarata:
 - semplicemente è una classe con un nome dipendente dal nome della classe esterna
- Quanto già discusso per le altre tre tipologie di classi nidificate *NON* si applica alle classi nidificate statiche
- Una classe statica può essere:
 - *privata*: non può essere usata all'esterno della classe in cui viene dichiarata
 - *pubblica*: può essere usata anche all'esterno della classe in cui viene dichiarata

Utilizzo delle Classi Statiche

- Possono essere usate quando non si ha la necessità di avere un riferimento ad un oggetto della classe esterna ma ha ancora senso mantenere le due classi (esterna vs interna) “vicine”
 - Ad es. perchè rappresentano concetti naturalmente ed imprescindibilmente molto legati
- Per un esempio semplice ma convincente, introduciamo anche le interfacce nidificate, già usate nel JCF
 - Rispetto al caso delle classi statiche nidificate, non sussistono differenze (almeno non specificatamente dovute alla nidificazione), tra classi ed interfacce

Classi Statiche: Esempio (1)

- Un utile esempio si trova nella classe `java.util.HashMap`
- La classe `HashMap` contiene al suo interno una tavola hash per la rappresentazione della mappa
- Quando si chiama il metodo `put()` la coppia passata al metodo (`<chiave>`, `<valore>`) deve essere memorizzata all'interno della tavola hash (per gestire le collisioni tra codici hash)

Classi Statiche: Esempio (2)

- La coppia viene memorizzata all'interno di un tipo nidificato statico **Entry<K, V>** in **HashMap<K, V>**
 - **Map.Entry<K, V>** è anche il nome di una interfaccia nidificata nell'interfaccia **Map<K, V>**
 - **Map**, in quanto *capostipite* della gerarchia di mappe in Java, include anche una rappresentazione astratta delle coppie chiave-valore ivi contenute
- La *classe* **Entry<K, V>** in **HashMap<K, V>** implementa l'*interface* **Entry<K, V>** in **Map<K, V>**

Classi Statiche: Esempio (3)

- E' possibile iterare su una mappa usando

```
Map<String, Integer> nome2eta = new HashMap<>();
```

```
...
```

```
for (Entry<String, Integer> e : nome2eta.entrySet())  
    System.out.println(e.getKey() + " " + e.getValue());
```

- ✓ Si sta facendo riferimento all'interface **Entry** e non alla classe statica nidificata di **HashMap**
 - si itera allo stesso modo sulle coppie chiave-valore senza dover conoscere il tipo dinamico concreto della mappa in cui tali coppie sono effettivamente contenute

Classi Statiche: Esempio (4)

- La coppia viene memorizzata all'interno di una classe nidificata statica chiamata **Entry**

```
public class HashMap<K,V> ... {  
    Entry[] table; // tavola hash: un'array di Entry  
  
    static class Entry<K,V> implements Map.Entry<K,V> {  
        final K key;  
        V value;  
        Entry<K,V> next; // lista di trabocco  
        ...  
    }  
}
```

Classi Statiche: Esempio (5)

- L'utilizzo di una classe statica nidificata per la classe **Entry** è ben motivata:
 - Una mappa e le coppie ivi contenute sono concetti ben distinti ma strettamente legati
 - Tuttavia le istanze della classe **Entry** non hanno bisogno di un rif. all'oggetto di tipo **Map** che la ospita
 - Una volta create possiedono tutto quello di cui necessitano per offrire i loro servizi (cfr. con le istanze di **Iterator**)
 - Ha quindi senso definirle assieme alla classe **HashMap<K, V>**, come semplice classe nidificata statica

Esercizio: LabirintoBuilder

- Alcuni tools consentono la generazione automatica di classi *builder* per classi il cui stato sia complesso: *Lombok*
 - 1) Discutere dell'utilizzo delle classi nidificate per la specifica di classi *builder* e perché può essere opportuno meno
 - 2) Quale tipologia di classi nidificate conviene utilizzare e perché?
 - 3) Scrivere **LabirintoBuilder** di conseguenza
 - 4) Studiare la soluzione adottata da *Lombok*

Conclusioni

- Le classi nidificate completano il quadro dei meccanismi offerti dal linguaggio Java per la definizione di tipi
 - N.B. Le lambda function di Java 8+ non introducono nuovi meccanismi di definizione di tipo
- Anche se sempre rimpiazzabili con le classi ordinarie, consentono una modellazione dei tipi più efficace per molti casi
 - si vuole precisare una relazione più stretta tra due tipi
- Il concetto di classe nidificata è totalmente sconosciuto alla JVM: il compilatore si occupa di “tradurre” le classi nidificate in classi ordinarie, ed aggiunge il campo che mantiene il rif. Interna → Esterna
- Fu forse il primo significativo passo di un processo di allontanamento della JVM dal linguaggio Java: processo che via via si è poi consolidato
- La JVM intrapreso un percorso sempre più indipendente da Java
- Oggi la JVM viene utilizzata per molti altri linguaggi:
 - Anche i più critici del linguaggio Java non esitano a riconoscere l'enorme successo della JVM sottostante

Programmazione Orientata agli Oggetti

Java Input/Output (Stream)

Sommario

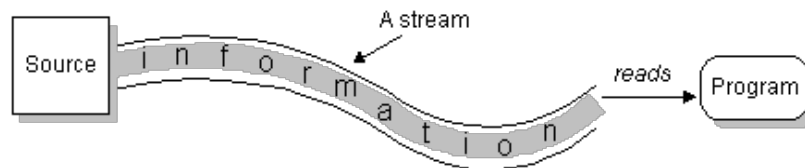
- Gestione Input/Output in Java
- Introduzione agli stream
 - caratteri vs byte : charset ed encoding
 - **InputStreamReader**
 - Gerarchie per la gestione I/O in **java.io**:
 - **Reader/Writer**
 - **InputStream/OutputStream**
- Esternalizzazione delle costanti
 - **java.util.Properties**
- Accesso alle risorse esterne
 - Risoluzione dei nomi logici di risorse
 - **Class** vs **ClassLoader**

Gestione dell'I/O in Java

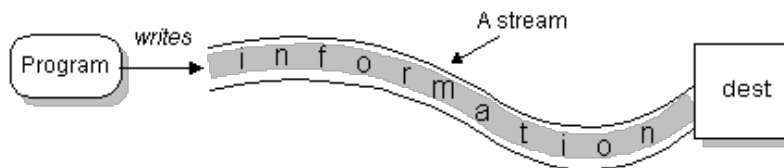
- Il package `java.io.*` di Java
 - datato, alcuni storici limiti, ma tuttora popolare (e necessario...)
- La più importante astrazione per la manipolazione dell'I/O è quella di *stream* (*flusso*)
 - «Stream»: collezione di dati a cui è possibile accedere sequenzialmente, accedendo ad un dato solo dopo aver acceduto al precedente nella sequenza
- N.B. Parliamo di stream per la gestione dell'I/O
 - ✓ da non confondere con gli *Stream* di Java 8 nel JCF

Stream di Input / Output

- Uno stream può essere di
 - *Input*: se permette di leggere i dati prodotti da una «sorgente» (stringa, file, connessione di rete...)



- *Output*: se permette scrivere dati diretti ad una «destinazione» (di nuovo: stringa, file, connessione ...)



Stream: Caratteri vs Byte

- Uno stream di byte può essere interpretato in molti modi
 - Stream di caratteri (testi)
 - Stream binari (es. immagini, eseguibili, .class, ecc. ecc.)
- Una delle possibili e più frequenti interpretazioni di uno stream di byte è come stream di caratteri, che per importanza meritano una gestione specializzata
- Per accedere agli stream esistono classi che agiscono da «lettori» e «scrittori» estendendo queste classi astratte
 - Stream di caratteri: ***Reader*** e ***Writer***
 - Stream di byte: ***InputStream***, ***OutputStream***
- Molti tipi di sorgenti e destinazioni:
 - Stringhe: ***StringReader***, ***StringWriter***
 - File: ***FileReader***, ***FileWriter***, ***FileInputStream***

Reader : *InputStream* = char : byte

- Per convertire uno stream di byte in uno di caratteri è necessario:
 - fissare un insieme di caratteri disponibili («charset»)
 - definire un algoritmo di conversione (codifica/«encoding») dello stream di byte in uno stream di caratteri
- Java internamente gestisce una *variante* della codifica UTF-16 per il charset *unicode* (a 16 bit) per permettere al tipo primitivo **char** di coprire gli alfabeti più diffusi
 - Vedi [javadoc](#) [java.lang.Character](#)
- Nonostante tutto questo fosse decisamente innovativo per l'epoca in cui Java è nato e si è diffuso, l'avvento del Web ha reso indispensabile il supporto di ancora altre codifiche

Stesso Charset Diversi Encoding

- Lo stesso charset unicode è rappresentato in diverse codifiche supportate anche da Java
 - UTF-8
 - ✓ Forte diffusione perché codifica i primi 127 caratteri come la codifica ASCII, con un solo byte, ed è quindi sia compatta sia retrocompatibile

– UTF-16

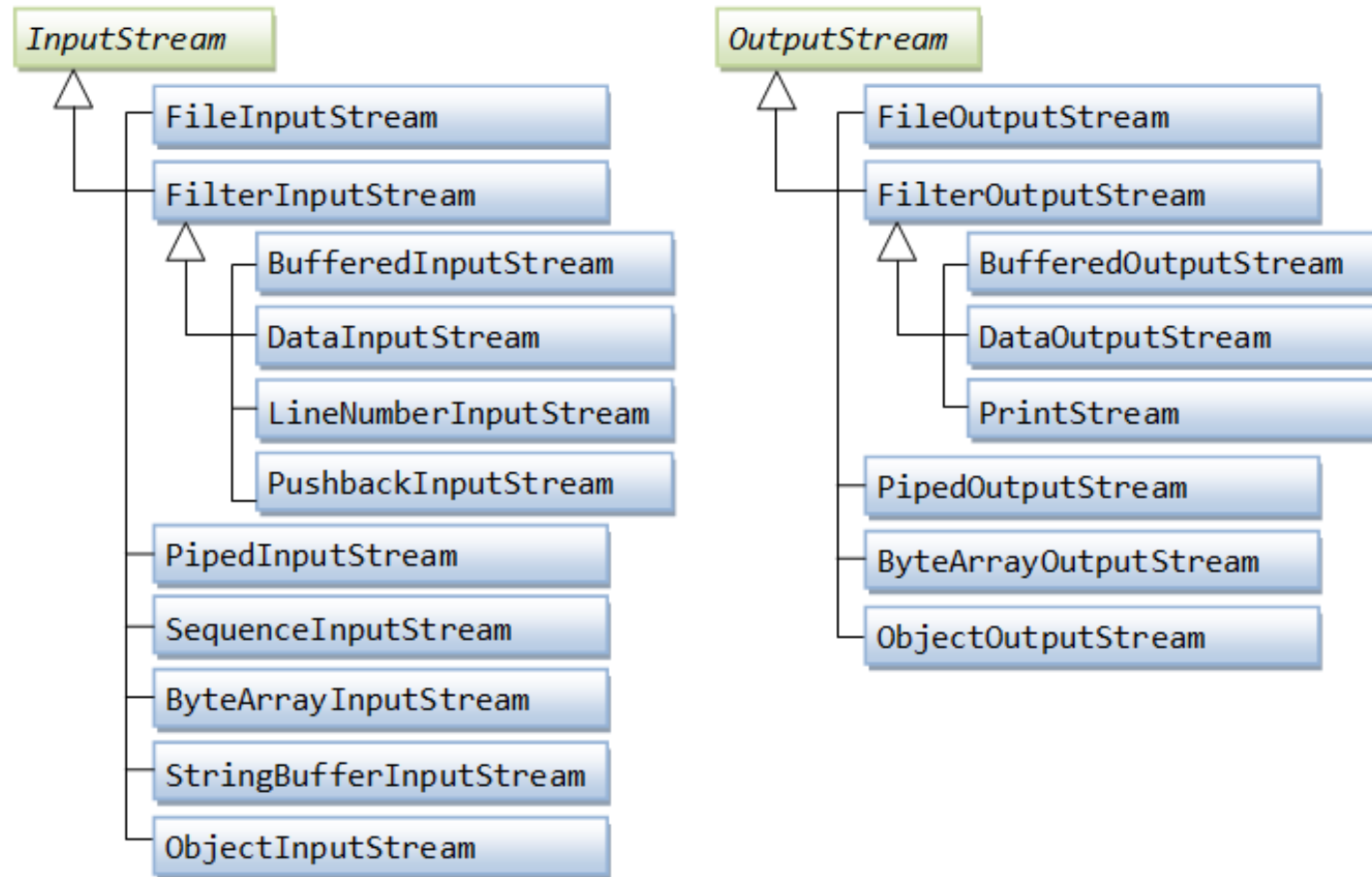
– UTF-32

– ...

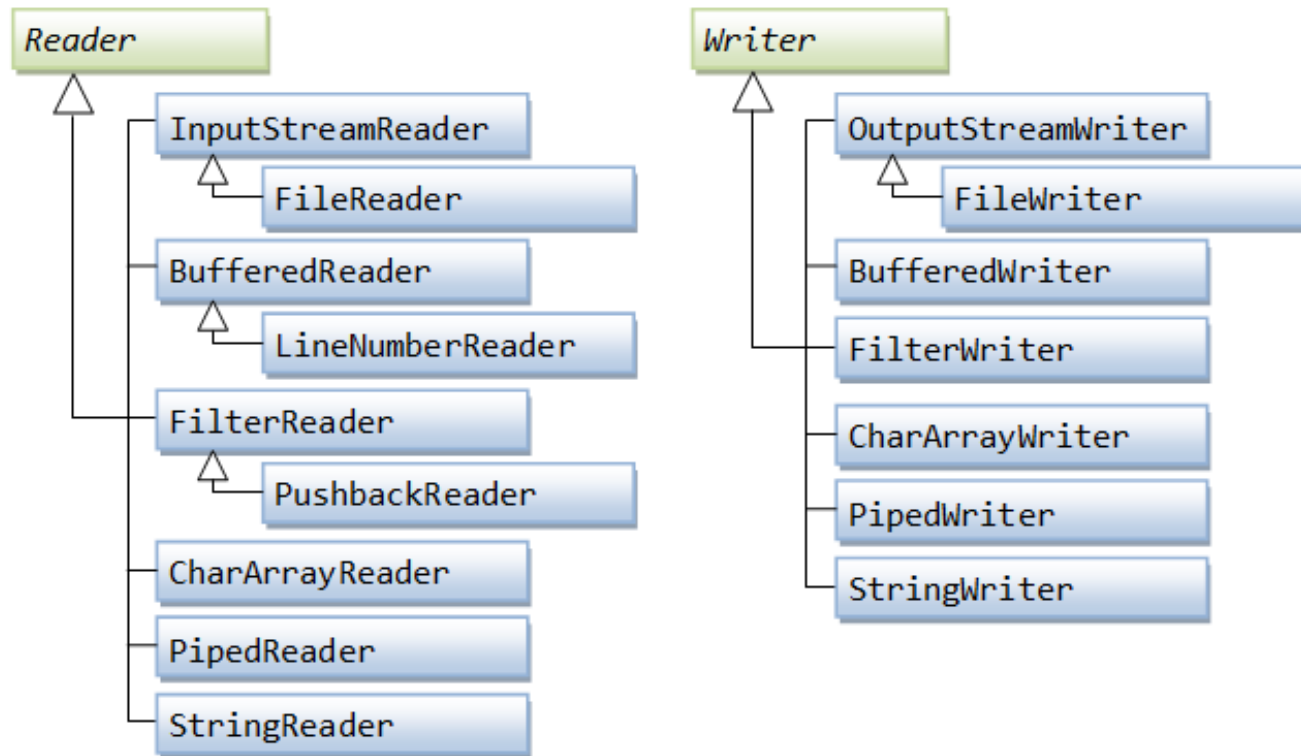
	A	ᄀ	好	丕
Code point	U+0041	U+05D0	U+597D	U+233B4
UTF-8	41	D7 90	E5 A5 BD	F0 A3 8E B4
UTF-16	00 41	05 D0	59 7D	D8 4C DF B4
UTF-32	00 00 00 41	00 00 05 D0	00 00 59 7D	00 02 33 B4

<https://www.w3.org/International/articles/definitions-characters/>

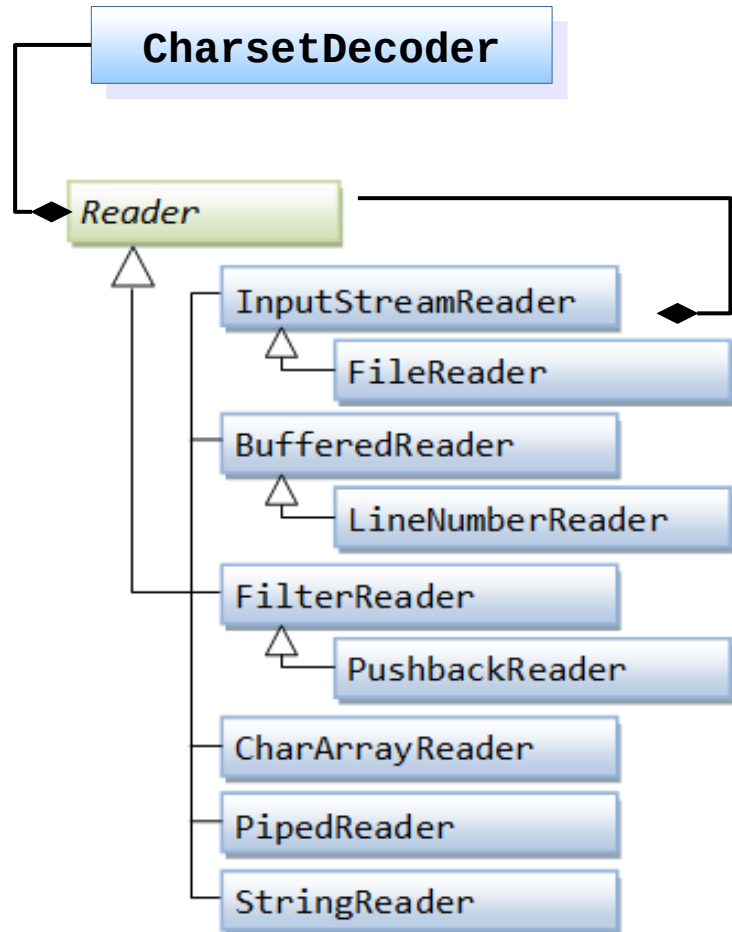
Gerarchia InputStream/OutputStream



Gerarchia Reader/Writer



Reader/Writer vs InputStream/OutputStream: InputStreamReader



- Uno stream di caratteri presuppone un *encoding* per interpretare il sottostante stream di byte
 - prima specificato esplicitamente oppure utilizzando un default fissato della piattaforma
 - da Java 18+ si usa UTF-8 come default

<https://openjdk.java.net/jeps/400>

 - ✓ per questo motivo spesso si lavora con stream di caratteri anche senza precisare l'encoding
- Il legame tra le due gerarchie di stream si spiega bene osservando una classe che fa da "ponte" in particolare: **InputStreamReader**
- Possiede un costruttore che riceve **InputStream** + encoding
 - interpreta lo stream di byte secondo l'encoding per convertirlo in caratteri **Reader**:

```
Reader in = new InputStreamReader(  
    new FileInputStream("test.txt"), "UTF-8")  
);
```

Principali Metodi di Reader/Writer

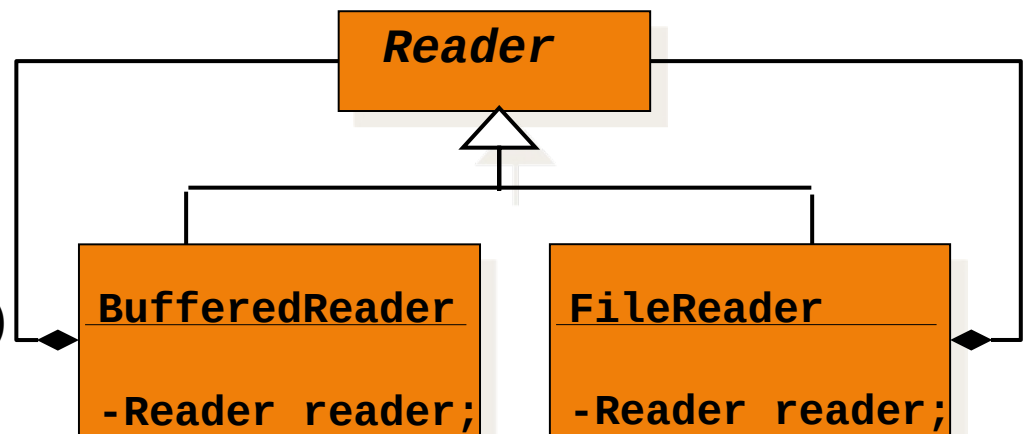
- I principali metodi di **Reader**
 - `int read()`
 - `int read(char cbuf[])`
 - `int read(char cbuf[], int offset, int length)`
- E i principali metodi di **Writer**
 - `int write(int c)`
 - `int write(char cbuf[])`
 - `int write(char cbuf[], int offset, int length)`
- Tutti questi metodi lanciano una checked exception: **`java.io.IOException`**
- Per ottenere le signature dei corrispondenti metodi in **`InputStream`** ed **`OutputStream`** basta sostituire:
`char` → `byte`

Composizione per Decorazione

- Le funzionalità di base di un **Reader** (**Writer**) risultano piuttosto limitate/primitive
- **java.io** offre diverse implementazioni per arricchirle
- Le funzionalità risultano così componibili tramite un meccanismo spesso detto di *decorazione*
 - i **Reader** concreti ospitano un riferimento ad un altro **Reader** (*decorato*) di cui adornano le funzionalità
 - Il reader decorato è “iniettato” tramite costruttore nel «decoratore»

• Ad es.

```
Reader br =  
    new BufferedReader(  
        new FileReader("file.txt")  
    );
```



Lettura/Scrittura da/verso Stream

- Il protocollo di utilizzo degli *stream* è scontatamente il seguente
 - ✓ tranne per l'esplicita chiusura finale (necessaria perché richiesto dal protocollo adottato dal S.O. per la gestione delle sottostanti *risorse*), ricorda l'uso degli iteratori
 - ✓ in definitiva, si enumerano gli elementi dello stream

Reading

open a stream
while **more information**
 read information
close the stream

Writing

open a stream
while **more information**
 write information
close the stream

Esempio di Utilizzo java.io.File

- **FileReader** e **FileWriter** permettono di gestire stream da/verso file. Il seguente programma copia un file:

```
import java.io.*;
public class FileCopier {
    public static void main(String[] args) {
        FileReader in = null;
        FileWriter out = null;
        try {
            in = new FileReader(args[0]);
        } catch (FileNotFoundException e) {
            e.printStackTrace();
            return;
        }
        try {
            out = new FileWriter(args[1]);
        } catch (IOException e) {
            e.printStackTrace();
            return;
        }
    }
}
```

...

Esempio di Utilizzo java.io.File

```
boolean eof = false;
int c;
try {
    while (!eof) {
        c = in.read();
        if (c != -1)
            out.write(c);
        else
            eof = true;
    }
} catch (IOException e) {
    e.printStackTrace();
    return;
}
finally {
    try {
        if (in != null)
            in.close();
        if (out != null)
            out.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
} }
```

Altro Esempio: Contatore di Parole

```
public Map<String, Integer> countWordOccurrences(String fileName)
                                                    throws IOException {
    Map<String, Integer> word2count = new HashMap<>();
    BufferedReader reader =
        new BufferedReader(new FileReader(fileName));
    String line;
    while ((line = reader.readLine()) != null) {
        Scanner scannerLinea = new Scanner(line);
        while (scannerLinea.hasNext()) {
            String word = scannerLinea.next();
            int count = 0;
            if (word2count.containsKey(word))
                count = word2count.get(word);
            count = count + 1;
            word2count.put(word, count);
        }
    }
    reader.close();
    return word2count;
}
```

java.nio.*

- **java.io** datata; alcuni scelte progettuali discutibili
 - ✓ su tutte: **java.io.File** modella sia il concetto di percorso logico in un file-system che quello di file (come risorsa del S.O. che ospita la JVM)
- La distribuzione standard è stata progressivamente arricchita (da Java 4) del package **java.nio**
 - che in effetti introduce **java.nio.file.Path** per risolvere il problema di modellazione di **java.io.File**
- Alcune operazioni prima macchinose ora sono realizzabili con singoli metodi statici di utilità in **java.nio.file.Files**

Copiare File con java.nio

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;

public class FileCopyTest {
    public static void main(String[] args)
                                throws IOException {
        Path source = Paths.get("src.txt");
        Path destination = Paths.get("copied.txt");
        Files.copy(source, destination);
    }
}
```

Esercizio: Eccezioni e File nello Studio di Caso (1)

- Scrivere i dati necessari a creare un labirinto in un file di testo che viene caricato all'inizio del gioco
- Il file di testo segue una semplice (ma rigida) sintassi:

Stanze: biblioteca, N10, N11

Inizio: N10

Vincente: N11

Attrezzi: martello 10 biblioteca, pinza 2 N10

Uscite: biblioteca nord N10, biblioteca sud N11

Esercizio: Eccezioni e File nello Studio di Caso (2)

- Aggiungere un costruttore alla classe **Labirinto**
 - Inizializza il labirinto caricandolo da un file di testo
 - L'operazione di caricamento dei dati la deleghiamo ad una opportuna classe che incapsula e nasconde tutta la logica di parsing del file di testo
 - Si avvale di **LabirintoBuilder** (vedi HW3)
 - Ideare una opportuna logica di gestione delle eventuali anomalie introducendo anche «eccezioni di dominio»

```
public Labirinto(String nomeFile) {  
    CaricatoreLabirinto c =  
        new CaricatoreLabirinto(nomeFile);  
    c.carica();  
    this.stanzaIniziale = c.getStanzaIniziale();  
    this.stanzaVincente = c.getStanzaVincente();  
}
```


Esercizio: Eccezioni e File nello Studio di Caso (3)

- Implementare la logica necessaria per gestire diversi livelli del gioco,
 - Livelli di difficoltà via via crescenti
- Se si perde una partita, bisogna cominciare una nuova partita dal primo livello
- Ciascun labirinto è specificato da un file diverso di nome progressivo siffatto:
 - `labirinto1.txt`
 - `labirinto2.txt`
 - ...

Sviluppo nel Ciclo di Vita del Software

- Lo sviluppo, per quanto sia importante, non è poi tutto...
- Bisogna saper distinguere e tener a mente ben separati:
 - l'ambiente di sviluppo da quello di produzione in cui l'applicativo verrà eseguito
 - le operazioni che possono svolgere, con riferimento ad una certa applicazione, le diverse figure coinvolte nel suo ciclo di vita:
 - sviluppatore
 - amministratore
 - utente finale
- Un corso incentrato esclusivamente sullo sviluppo, dove si usa sempre e soltanto l'ambiente di sviluppo e mai quello di produzione, può indurre molti a confondere ruoli ed ambienti!

Sviluppatore vs Amministratore

- Per questo motivo spesso è opportuno «spostare» informazioni fuori dal codice sorgente in file esterni al codice stesso
- Il primo è accessibile solo allo sviluppatore, il secondo anche agli amministratori
 - ✓ Le informazioni di configurazione devono poter essere cambiate da un amministratore (quindi NON dallo sviluppatore) che conosce i dettagli dell'ambiente di produzione ma non conosce affatto l'ambiente di sviluppo
 - ✓ Viceversa, lo sviluppatore deve evitare di cablare nel codice dipendenze verso l'ambiente di sviluppo che sarà ben diverso dall'ambiente di produzione finale

«Eternalizzazione» delle Costanti

- Anche il nostro semplice gioco di ruolo, per quanto artatamente semplice, già manifesta questi problemi
- Problematica ben circostanziata tecnicamente:
 - ✓ *l'esternalizzazione* delle costanti e dei parametri di configurazione
- Abbiamo già osservato come anche solo dichiarare apposite costanti **final static** migliora sia la leggibilità che la coesione del codice
 - Esempi di costanti già usate nello studio di caso:
 - il numero di CFU iniziali
 - il peso max della borsa
 - Altre informazioni di configurazione che si possono aggiungere: la specifica dei labirinti, magari con vari livelli di difficoltà (<<)

«Eternalizzazione» delle Costanti

- L'utilizzo di costanti ci ricorda esigenze delle altre figure coinvolte nel ciclo di vita di un applicativo
 - modificare il valore di una costante richiede la ricompilazione
 - operazione sgradita a tutti tranne che allo sviluppatore
 - ✓ il codice potrebbe anche non essere distribuito!
 - in molti casi non è nemmeno possibile pensare di riavviare un'applicazione
- ✓ Le costanti cablate nel codice possono essere utilizzate per fissare i valori di default (ovvero da usare solo in caso di non reperibilità della config. >>)

java.util.Properties

- Esigenze sufficientemente radicate da motivare classi dedicate alla gestione di informazioni persistenti (ad es. di configurazione) nella distribuzione standard sin Java 1.0 (N.B. quindi prima del JCF e `java.util.Map`)
- `java.util.Properties` è un sottotipo di `Map<Object, Object>` che gestisce una collezione di proprietà («property»), ovvero coppie di stringhe nome/valore
 - ✓ N.B. nonostante la tipizzazione, sia nome che valore sono di tipo `java.lang.String`
- Offre tre principali tipologie di servizi:
 - Metodi di accesso ai valori delle proprietà per nome
 - Gestione dei valori di default delle proprietà
 - Servizi di lettura/scrittura da file testuali/XML
- ✓ Esistono librerie decisamente più moderne; i principi restano immutati
 - [https://en.wikipedia.org/wiki/Jackson_\(API\)](https://en.wikipedia.org/wiki/Jackson_(API))
 - <http://commons.apache.org/proper/commons-configuration/>

java.util.Properties: Accesso alle Proprietà

Metodi di accesso alla proprietà

String getProperty(String key)

- *Searches for the property with the specified key in this property list. If the key is not found in this property list, the default property list, and its defaults, recursively, are then checked. The method returns null if the property is not found.*

Object setProperty(String key, String value)

- *Calls the Hashtable method put. Provided for parallelism with the getProperty method. Enforces use of strings for property keys and values. The value returned is the result of the Hashtable call to put.*

java.util.Properties: Gestione Valori di Default

E' prevista la gestione di valori di default per le proprietà di valore non esplicitamente inizializzato. In particolare si può fornire un altro oggetto **Properties** contenente diversi valori di default al costruttore:

Properties()

- *Creates an empty property list with no default values.*

Properties(Properties defaults)

- *Creates an empty property list with the specified defaults.*

Inoltre il metodo **getProperty()** offre una versione sovraccarica:

String getProperty(String key, String defaultValue)

- *Searches for the property with the specified key in this property list. If the key is not found in this property list, the default property list, and its defaults, recursively, are then checked. The method returns null if the property is not found.*

java.util.Properties: Lettura/Scrittura da Testo/XML

void load(InputStream inStream)

– *reads a property list (key and element pairs) from the input byte stream.*

void load(Reader reader)

– *Reads a property list (key and element pairs) from the input character stream in a simple line-oriented format.*

void loadFromXML(InputStream in)

– *Loads all of the properties represented by the XML document on the specified input stream into this properties table.*

void store(OutputStream out, String comments)

– *writes this property list (key and element pairs) in this Properties table to the output stream in a format suitable for loading into a Properties table using the load(InputStream) method*

void store(Writer writer, String comments)

void storeToXML(OutputStream os, String comment)

void storeToXML(OutputStream os, String comment, String encoding)

java.util.Properties: Formato Testuale/XML (1)

```
import java.io.*;
import java.util.Properties;

public class EsempioProperties {
    public static void main(String[] args)
        throws IOException {
        Properties prop = new Properties();
        prop.setProperty("cfu_iniziali", "10");
        prop.setProperty("peso_max_borsa", "20");
        prop.store(new FileWriter("diadia.properties"),
            "Configurazione del gioco DIADIA");
        prop.storeToXML(new FileOutputStream("diadia.xml"),
            "Configurazione del gioco DIADIA");
    }
}
```

java.util.Properties: Formato Testuale/XML (2)

diadia.properties:

```
#Configurazione del gioco DIADIA
#Tue May 17 12:30:11 CEST 2016
peso_max_borsa=20
cfu_iniziali=10
```

vedere i javadoc di
[**java.util.Properties**](#)
per i dettagli sui
formati

diadia.xml:

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<!DOCTYPE properties SYSTEM
    "http://java.sun.com/dtd/properties.dtd">
<properties>
<comment>Configurazione del gioco DIADIA</comment>
<entry key="peso_max_borsa">20</entry>
<entry key="cfu_iniziali">10</entry>
</properties>
```

Dipendenze da «Risorse»

- «Spostando» le informazioni di configurazione in file esterni al codice, si solleva un nuovo problema...
- Come sviluppare codice dipendente da «risorse» esterne senza che l'accesso dipenda dall'ubicazione del codice stesso?
 - *Risorsa*: qualsiasi contenuto (immagini, testi...) fuori dal sorgente ed in appositi file, ad es. ***diadia.properties***.
- Una NON-soluzione da EVITARE accuratamente:
 - `new FileReader("/home/rossi/diadia/diadia.properties")`
 - cabla nel codice compilato una assunzione sulla ubicazione di una risorsa che con tutta probabilità sarà violata al primo *deployment*
 - ✓ Se anche non fosse ancora disponibile l'ambiente di produzione finale, almeno bisognerebbe provare l'esecuzione in ambienti diversi da quello di sviluppo
- ✓ In generale è da evitare *sempre* l'utilizzo di percorsi *assoluti* (ovvero che dipendono dall'ambiente di sviluppo)

Nomi Logici delle Risorse

- La piattaforma Java supporta l'assegnazione di *nomi-logici* gerarchici alle risorse proprio per gestire queste problematiche
- Si basa sugli stessi meccanismi&metodi utilizzati per la risoluzione dei nomi delle classi
- Per questi motivi, i servizi necessari sono offerti da due classi
 - **java.lang.Class**
 - **java.lang.ClassLoader** (responsabile caricamento classi)... legate alla gestione delle classi
- *Entrambe* offrono questa coppia di metodi (ma non implementano alcuna interfaccia comune e la semantica differisce! >>)
 - URL getResource(*String name*)**
 - *Finds a resource with a given name*
 - InputStream getResourceAsStream(*String name*)**
 - *Returns an input stream for reading the specified resource*

Risoluzione Nomi Logici Risorse (1)

- Per la risoluzione dei nomi logici in risorse concrete, valgono le stesse regole per la risoluzione del *nome completamente qualificato* di una classe nella posizione del corrispondente file **.class**
 - **it.uniroma3.dia.comandi.ComandoVai** è il *nome completamente qualificato* della classe **ComandoVai** nel package **it.uniroma3.dia.comandi**
 - **ComandoVai.class** deve essere ubicata dentro una cartella di nome **comandi**, dentro una cartella di nome **dia**, ... , dentro una cartella di nome **it**.
 - La cartella di nome **it** deve essere raggiungibile a partire dal classpath noto alla JVM a tempo di esecuzione

Risoluzione Nomi Logici Risorse (2)

- Dato il nome logico di una risorsa, questa deve essere ubicata in un cartella posizionata seguendo lo schema già seguito per le classi
- Esistono alcune sottili differenze: a differenza dei nomi delle classi, i nomi logici delle risorse usano `'/'` come separatore, e possono essere:
 - *Relativi*: NON cominciano per `'/'`
ad es. `'diadia.properties'`
 - *Assoluti*: cominciano per `'/'`
ad es. `'/resources/diadia.properties'`

Class vs ClassLoader

- **java.lang.Class** risolve i percorsi
 - *relativi* a partire dalla posizione del file **.class** corrispondente all'oggetto **Class** a cui è associato
 - *assoluti* a partire dal classpath
- **java.lang.ClassLoader** risolve i percorsi
 - *assoluti*: non sono risolti!
 - ✓ quindi mai usarla con nomi logici che cominciano per '/'
 - *relativi* sono risolti a partire dal classpath (come fossero assoluti!)

ClassLoader: responsabile caricamento classi

- ✓ Ottenibile invocando il metodo di istanza:

Class.getClassLoader() (>>)

Esempi di Risoluzione (1)

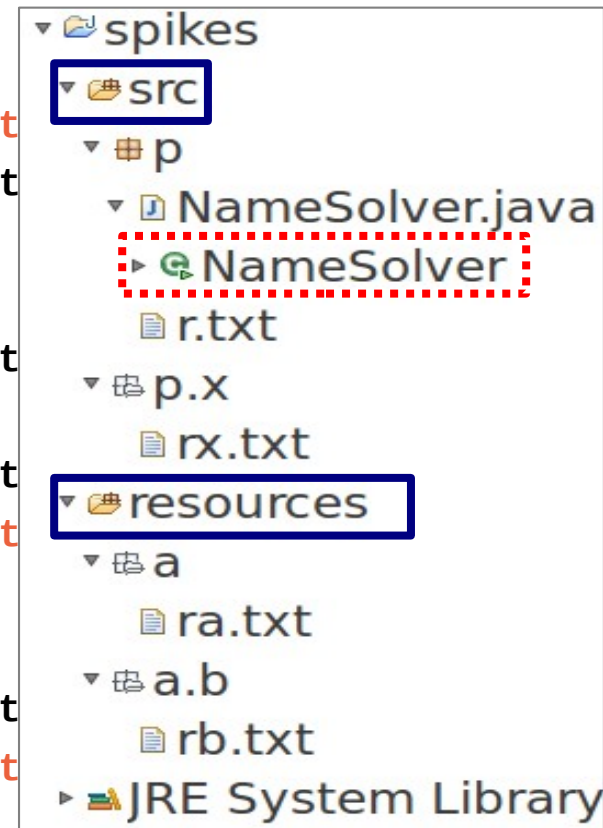
```
package p;
public class NameSolver {
    private static final String RNS_R = "p/r.txt",
    private static final String ABS_R = "/p/r.txt";
    private static final String REL_R = "r.txt";
    ...// e molte altre ancora (>>)
    private void testResourceSolver() {
        Class<?> clazz = this.getClass();
        ClassLoader loader = clazz.getClassLoader();
        System.out.println("ClassLoader.getResource(\""+ABS_R+"")=\t"+
                           loader.getResource(ABS_R));
        System.out.println("    Class.getResource(\""+ABS_R+"")=\t"+
                           clazz.getResource(ABS_R));
        System.out.println("ClassLoader.getResource(\""+RNS_R+"")=\t"+
                           loader.getResource(RNS_R));
        ...
        ... // e molte altre ancora (>>)
    }
    public static void main(String[] args) {
        new NameSolver().testResourceSolver();
    }
}
```

Esempi di Risoluzione (2)

<code>ClassLoader.getResource("/p/r.txt")=</code>	<code>null</code>
<code>Class.getResource("/p/r.txt")=</code>	<code>file:///.../p/r.txt</code>
<code>ClassLoader.getResource("p/r.txt")=</code>	<code>file:///.../p/r.txt</code>
<code>Class.getResource("p/r.txt")=</code>	<code>null</code>
<code>ClassLoader.getResource("r.txt")=</code>	<code>null</code>
<code>Class.getResource("r.txt")=</code>	<code>file:///.../p/r.txt</code>
<code>ClassLoader.getResource("/p/x/rx.txt")=</code>	<code>null</code>
<code>Class.getResource("/p/x/rx.txt")=</code>	<code>file:///.../p/x/rx.t</code>
<code>ClassLoader.getResource("p/x/rx.txt")=</code>	<code>file:///.../p/x/rx.t</code>
<code>Class.getResource("p/x/rx.txt")=</code>	<code>null</code>
<code>ClassLoader.getResource("x/rx.txt")=</code>	<code>null</code>
<code>Class.getResource("x/rx.txt")=</code>	<code>file:///.../p/x/rx.t</code>
<code>ClassLoader.getResource("/a/ra.txt")=</code>	<code>null</code>
<code>Class.getResource("/a/ra.txt")=</code>	<code>file:///.../a/ra.txt</code>
<code>ClassLoader.getResource("a/ra.txt")=</code>	<code>file:///.../a/ra.txt</code>
<code>Class.getResource("a/ra.txt")=</code>	<code>null</code>
<code>ClassLoader.getResource("/a/b/rb.txt")=</code>	<code>null</code>
<code>Class.getResource("/a/b/rb.txt")=</code>	<code>file:///.../a/b/rb.t</code>
<code>ClassLoader.getResource("a/b/rb.txt")=</code>	<code>file:///.../a/b/rb.t</code>
<code>Class.getResource("a/b/rb.txt")=</code>	<code>null</code>

Cartella nel classpath

Classe in esecuzione



Risorse Esterne nei Progetti Eclipse

- Per tenere separate le risorse dal codice sorgente è buona prassi creare una cartella apposita per contenerle
 - ✓ spesso chiamata **resources**
- E' però importante ricordarsi di includerla nel *classpath*
 - ✓ ovvero, nel *buildpath* di Eclipseper permettere la risoluzione a tempo di esecuzione dei nomi logici in risorse concrete
- Nell'esempio precedente la cartella **resources** svolge esattamente questo ruolo

Verifica «Preliminare» sulla Mancanza di Dipendenze Verso l'Ambiente di Sviluppo

- Creare un «jar eseguibile» (da Eclipse) contenente tutto il necessario all'esecuzione, salvarlo fuori dall'ambiente di sviluppo, ed eseguirlo:
 1. File → Export... → Runnable JAR file
 2. selezionare una «launch configuration» che specifichi la classe contenente il **main()** (ad es. **Gioco** oppure **DiaDia** a seconda del nome della classe che contiene il metodo)
 3. selezionare il nome del file .jar, ad es. **diadia.jar**
 4. per eseguire (da console): **java -jar diadia.jar**
- *Avvertimento.* Attenzione ad un frequente motivo di molti mal di testa:
 - Il codice eseguibile nel **.jar** utilizza il metodo **URL getResource()** per ottenere un riferimento alla risorsa a partire dal suo nome logico
 - poi cerca di aprirla come se fosse un qualsiasi file del file-system
 - tuttavia un file impacchettato e compresso dentro un archivio non è accessibile come fosse un file ordinario!
- ✓ Soluzione: usare invece **InputStream getResourceAsStream()** che permette di ottenere un accesso alla risorsa (come stream)
 - sia nel caso sia «interna» all'archivio **.jar**
 - sia nel caso sia presente come file nel file-system

Esercizio (Studio di Caso)

- Rivedere lo studio di caso al fine di rendere configurabile (mediante un file **diadia.properties** da distribuire assieme al codice stesso) il gioco senza ricompilarlo. Ad es. almeno:
 - Il numero di CFU iniziali
 - Il peso max della borsa
- N.B. Molte altre informazioni di configurazione potrebbero essere «rimaste nascoste» nel codice sottoforma di costanti letterali
- Rifattorizzare il codice affinché tutte le dipendenze verso le risorse (incluso quelle verso il file contenente la specifica del labirinto) non dipendano dall'ambiente di sviluppo
- Esportare l'applicazione in formato .jar e verificare che continui a funzionare nonostante la dipendenza verso le risorse “esterne”
- Provare quindi esecuzioni con diverse configurazioni (senza ricompilare il codice ma solo cambiando **diadia.properties**)

P PER GLI ESERCIZI TECNICI

SE CHIAMA UN METODO:

- PRIMA GUARDO SE IL TIPO STATICO HA QUEL METODO

E SUCCESSIVAMENTE GUARDO IL TIPO DINAMICO

SE È TRA PARENTESI:

- GUARDO IL TIPO STATICO